

Biology-Informed Multi-Omics Integration

A working fork of the SIB Training from a wet-lab scientist

Sébastien Wieckowski

2026-09-28

Table of contents

1	Biology-Informed Multi-Omics: A Wet-Lab Scientist's Study Fork	8
1.1	What You Will Find in This Fork	8
1.1.1	Key Code Optimizations in this Fork	8
1.2	Analytical Workflow Covered	9
1.3	Prerequisites & Scope	9
1.3.1	Looking to build foundational skills or explore more topics?	10
1.4	Setup & Running the Notebooks	10
1.4.1	1. Clone the repository	11
1.4.2	2. Restore the R environment	11
1.4.3	3. Running in VS Code	11
1.5	Connect & Discuss	12
1.6	Credits & Upstream Authors	12
I	I. Theoretical Reference & Notes	13
2	Epigenetic Analysis Reference Guide	14
2.1	Definitions	14
2.1.1	ATAC-seq	14
2.1.2	ChIP-seq	14
2.1.3	WGBS	15
2.2	Data Interpretation	15
2.2.1	Key Abbreviations	15
2.2.2	Legend	16
2.2.3	Table 1: Basic Enhancer Analysis	16
2.2.4	Table 2: Enhancer States with H3K27ac	16
2.2.5	Table 3: Enhancer vs Promoter Comparison	17
2.2.6	Table 4: Promoter-Focused Analysis	18
2.2.7	Table 5: Simplified Three-Factor Analysis	19
2.2.8	Table 6: Comprehensive Multi-Mark Analysis	20
2.2.9	Table 7: Polycomb Repression Analysis	21
2.2.10	Table 8: Complete Seven-Factor Analysis	22
2.3	Summary	24

3	Normalization models	25
3.1	Averaging Models for Normalization	25
3.2	Understanding Windows and Genomic Signals	25
3.3	Application to Specific Assays	25
3.4	RNA-seq Data Analysis	27
3.4.1	1. Within-Sample Normalization (Relative Abundance)	27
3.4.2	2. Between-Sample Normalization (Differential Expression)	28
3.4.3	Method Comparison	28
3.5	EnrichedHeatmap	28
3.5.1	Core Structure	29
3.5.2	Preparing the Data	29
3.5.3	Reading the Data	29
4	Functional Enrichment Analysis: Multi-Omics Integration	31
4.1	Overview of Enrichment Analysis	31
4.2	Database Challenges & Semantic Similarity	32
4.3	Cis-Regulatory Regions & GREAT	32
4.4	Connection to the Practical Exercise	33
II	II. Data Structures & Visualization	34
5	Datasets, SummarizedExperiment, and GRanges	35
5.1	Learning outcomes	35
5.2	Environment and data setup	35
5.2.1	Datasets for the course	35
5.2.2	Downloading and extracting data	36
5.2.3	Loading libraries	37
5.3	The SummarizedExperiment (SE) object	37
5.3.1	Loading SE objects	37
5.3.2	Inspecting the RNA SE object	37
5.3.3	Accessing colData, rowRanges and assays	38
5.3.4	Subsetting data	46
5.3.5	Boxplot of expression for a few genes	46
5.4	PRACTICE & EXERCISES	47
5.4.1	Question 1	47
5.4.2	Question 2	49
5.4.3	Question 3	52
5.5	The GRanges object	54
5.5.1	Extracting and inspecting GRanges from ATAC-seq	54
5.5.2	Subsetting regions on chr1 with width > 1000 bp	56
5.5.3	Plotting the width distribution of genomic ranges	58

5.6	Visualizing genomic data	59
5.6.1	Heatmap of H3K27ac signal	59
5.6.2	Question 4	61
6	Enriched Heatmaps and identification of regions with different activities	63
6.1	Learning objectives	63
6.2	Loading libraries	63
6.3	Preparing the ATAC SummarizedExperiment	63
6.3.1	ATAC peaks information	64
6.3.2	Statistical results	66
6.3.3	From BAM files to differential peaks	66
6.3.4	Filtering the dataset	67
6.3.5	ATAC-seq peak widths	69
6.3.6	Binning signal across a 2 kb window	70
6.3.7	Finding peak centers with <code>resize()</code>	71
6.4	Preparing the continuous bigWig data	73
6.4.1	Loading data from bigWig	73
6.4.2	Normalization	76
6.5	Building the EnrichedHeatmap	79
6.5.1	Basic EnrichedHeatmap visualization	79
6.5.2	Changing color and size	80
6.5.3	Color based on quantile	81
6.5.4	Customizing heatmap aesthetics for publication	83
6.6	Splitting and clustering	85
6.6.1	Stratifying the heatmap by direction of change	85
6.6.2	Setting categorical levels with <code>factor()</code>	87
6.7	Functional programming	87
6.7.1	Creating a reusable heatmap function	87
6.7.2	Making EnrichedHeatmaps for both ATAC samples	89
6.7.3	Resetting the canvas for larger plots	89
6.8	Combining multiple heatmaps	90
6.8.1	Generating the unified heatmaps and annotations	90
6.8.2	Named vectors vs. simple vectors for colors	92
6.9	Question 1	93
6.10	Refining the custom heatmap function	97
6.11	Question 2	100
6.12	Question 3	102
6.12.1	The “Mini-Heatmap” method	102
6.12.2	The <code>rowAnnotation</code> method	106
6.12.3	The power of native annotations	108
7	ChIP and RNA sequencing data plots	110
7.1	Learning objectives	110

7.2	Loading libraries	110
7.3	Preparing the ChIP-seq SE object	110
7.3.1	Loading the RDS files	110
7.3.2	Exploratory data analysis	113
7.3.3	Interpreting ChIP-seq peak widths	115
7.4	Question 1	116
7.4.1	Proposed strategy	116
7.4.2	Defining anchor regions from differentially accessible chromatin	117
7.4.3	Extracting and normalizing continuous ChIP-seq signal	119
7.4.4	Constructing and combining the heatmaps	120
7.4.5	Analyzing histone mark distributions across genomic features	126
7.5	Analysis of the RNA-seq data	126
7.5.1	Loading the RNA-seq SE object	127
7.5.2	Heatmap of Log-normalized counts	127
7.5.3	Changing visualization to relative expression	128
7.5.4	Adding column annotations	130
7.6	Question 2	132
7.6.1	A note on visualizing expression changes	134

III III. Multi-Omics Integration 135

8	Overlapping all the datasets	136
8.1	Learning objectives	136
8.2	Initial setup and data loading	136
8.2.1	Loading required libraries	136
8.2.2	Loading the SummarizedExperiment objects	137
8.2.3	Defining a custom function for multi-omics integration	137
8.3	Constructing the multi-omics Overlap Matrix	138
8.3.1	Processing and merging datasets	139
8.3.2	Finalizing and saving the integrated object	142
8.3.3	Inspecting the integrated object	142
8.3.4	Processing ATAC and ChIP-seq continuous signals	144
8.3.5	Verifying our dataset's peak widths	144
8.3.6	Unifying the coordinate system (centering on ATAC)	145
8.3.7	Extracting and normalizing bigWig signals	146
8.4	Processing RNA-seq discrete signals	148
8.5	Processing WGBS (DNA Methylation) signals	150
8.5.1	Reading the WGBS coverage files	150
8.5.2	Handling high-resolution WGBS data	151
8.5.3	Question 1	153
8.5.4	Biological interpretation of DNA methylation	156

9	Integrating ATAC and RNA sequencing data	158
9.1	Learning objectives	158
9.2	Defining functional genomic states	158
9.3	Initial setup and data loading	160
9.3.1	Importing processed datasets	160
9.4	Categorizing genomic regions by multi-omics behavior	161
9.5	Visualizing multi-omics integration	172
9.5.1	Recycling our custom plotting function	172
9.5.2	ATAC-seq signal heatmaps	173
9.5.3	Interpreting ATAC-seq accessibility clusters	175
9.5.4	RNA-seq expression heatmap	176
9.5.5	RNA-seq expression patterns	178
9.5.6	Constructing the composite multi-omics heatmap	178
9.5.7	Reordering biological categories	179
9.5.8	Global vsualization settings	180
9.5.9	Decoding the Multi-Omics landscape	181
10	Integrating ATAC, ChIP, and RNA sequencing data	183
10.1	Learning objectives	183
10.2	Defining multi-omics regulatory states	183
10.3	Loading packages and pre-computed data	185
10.4	Categorizing genomic regions by multi-omics behavior	186
10.4.1	Mathematical cleanup (Handling missing values)	187
10.4.2	Defining strict biological states	188
10.4.3	The reality of multi-omics strictness	190
10.4.4	Hierarchy collapsing for visualization	190
10.5	Visualizing the complete multi-omics landscape	192
10.5.1	Preparing the continuous ATAC, ChIP and WGBS block	192
10.5.2	Preparing the discrete RNA-seq block	195
10.5.3	Preparing the hierarchical row splits	196
10.5.4	Annotation bar	196
10.5.5	Rendering the composite figure	198
10.5.6	Exporting the composite figure	199
10.6	Biological interpretation	201
10.6.1	Validating “Active Promoters”	201
10.6.2	Validating “Repressed Promoters”	201
10.6.3	Validating “Lost Enhancers”	202
10.6.4	Validating the “Silenced” state (Hypermethylation)	202
10.6.5	The Mystery of “Incongruent” elements	202
11	Functional enrichment of genomic regions	204
11.1	Learning objectives	204

11.2	Environment setup & data loading	204
11.2.1	Loading the required libraries	204
11.2.2	Importing the multi-omics overlap data	205
11.2.3	Exploring the “Silenced” Regions	206
11.3	Genomic region enrichment via rGREAT	206
11.3.1	Subsetting the Genomic Ranges	206
11.3.2	Executing the GREAT algorithm	216
11.3.3	QA/QC: verifying the GREAT mapping	218
11.4	Visualizing and interpreting enrichment results	219
11.4.1	Statistical significance: the Volcano plot	219
11.4.2	Physical mapping: region-gene associations	220
11.4.3	Exploring the raw data	222
11.4.4	Downstream filtering of biological pathways	223
11.4.5	Clustering redundant pathways (semantic similarity)	224
11.4.6	Exploring the “Silenced” regions	228

1 Biology-Informed Multi-Omics: A Wet-Lab Scientist's Study Fork

Welcome! This repository is an annotated, restructured study fork of the [SIB Swiss Institute of Bioinformatics Multi-Omics Training Course](#).

I come from a **wet-lab background** and am investing significant effort into transitioning toward computational biology. To make the learning curve manageable and fully focus on the code and biology, **I simplified the project architecture**: I removed the Docker containers and Quarto website infrastructure in favor of standalone **Jupyter Notebooks (.ipynb)** and **clean Markdown files**.

1.1 What You Will Find in This Fork

- **Streamlined Interactive Format**: Converted from the original .qmd files into pure Jupyter Notebooks (.ipynb) (via `quarto convert`, configured to use the R kernel via `jupyter: ir` in the YAML front matter) for direct, interactive execution.
- **Wet-Lab Context & Concept Deconstruction**: Line-by-line notes translating abstract computational steps into biological meaning.
- **Troubleshooting & Technical Deep Dives**: Detailed reflections and extra research on tricky parameters, Bioconductor quirks, and statistical assumptions.
- **Refactored & Annotated Code**: Code cells with additional comments explaining the *why* behind data transformations.
- **Beginner-Friendly Focus**: Written specifically for researchers without formal bioinformatics degrees who want to understand multi-omics integration step by step.

1.1.1 Key Code Optimizations in this Fork

Over the course of the notebooks, I adapted the original base R code to implement modern data science and tidyverse workflows. If you are learning R, you will find highly optimized, real-world solutions for:

- **Modern Biological Logic (`dplyr::case_when`):** Transitioned classic boolean matrix subsetting into elegant, readable `case_when()` pipelines. This includes safely coercing NA values in `logFC` columns to 0 to build robust logical filtering for genomic categories.
 - **Genomic Range Wrangling:** Seamlessly manipulating `GRanges` metadata columns directly with `dplyr` (`mcols(gr) <- as.data.frame(mcols(gr)) %>% mutate(...)`) to keep genomic data structures perfectly aligned.
 - **Efficient I/O & Memory Management:** Refactored how large multi-omics matrices are grouped into named lists and saved via `saveRDS`, significantly streamlining downstream loading and keeping the global environment clean.
 - **Parameterizing Shared Scripts & Local Wrappers:** Updated shared plotting functions (`custom_enrichedheatmap_plotting.R`) to accept flexible aesthetic defaults (fonts, line widths), ensuring backward compatibility across all notebooks. Paired this with **local function wrappers** (e.g., `make_EH()`) in notebooks to abstract away dimensions when generating massive 14-column plots without repetitive copy-pasting.
-

1.2 Analytical Workflow Covered

1. **Multi-Layer Integration:** Importing and processing ATAC-seq, RNA-seq, ChIP-seq, and WGBS datasets in R/Bioconductor.
 2. **Overlap Matrices:** Quantifying co-occurrence across layers to detect epigenetic and transcriptional co-regulation.
 3. **Downstream Analysis:** Functional annotation and pathway enrichment.
 4. **Data Visualization:** Creating interpretable plots for multi-layer data.
-

1.3 Prerequisites & Scope

Important Scope Note:

This repository focuses specifically on **downstream multi-omics integration and cross-layer biological interpretation**. It does **not** cover raw single-omics preprocessing (e.g., FASTQ quality control, read alignment, transcript quantification) or general Exploratory Data Analysis (EDA) and feature engineering from scratch.

To get the most out of these notebooks, familiarity with the following concepts is recommended:

- **Basic R & Bioconductor:** Working with common data structures (`data.frame`, matrices) and core Bioconductor classes (`GRanges`, `SummarizedExperiment`).
- **Single-Omics Foundations:** Understanding basic quantification and differential analysis concepts in RNA-seq, ChIP-seq, and ATAC-seq.
- **Exploratory Data Analysis (EDA):** Experience diagnosing batch effects, sample clustering (PCA/UMAP), and data normalization before multi-layer joining.

1.3.1 Looking to build foundational skills or explore more topics?

- **Statistical Foundations & Coding with Python:** For those wanting to master coding from the ground up using biostatistics as the vehicle, check out my book [Intuitive Biostatistics with Python](#) (Oxford University Press). It uses Python's simulation and visualization capabilities to demystify confusing core concepts (null hypotheses, bootstrapping, non-parametrics) through hands-on coding and publication-ready graphs.
- **End-to-End RNA-seq & Preprocessing:** Check out my repository [sbwiecko/RNAseq_UPenn](#), rewritten alongside the excellent [UPenn DIY Transcriptomics course](#) for a ground-up journey through raw data, quality control, alignment, and differential expression.
- **SIB Training Catalog:** Explore the full lineup of world-class courses designed by the exceptional SIB Training Team: [SIB Swiss Institute of Bioinformatics Courses & Training](#)
- **Bioconductor Workflows:** [RNA-seq analysis is easy as 1-2-3 with limma, Glimma and edgeR](#)
- **EDA & Statistical Exploration:** [Modern Statistics for Modern Biology](#) by Susan Holmes and Wolfgang Huber
- **Free Bioinformatics Resources:** Discover a curated collection of free bioinformatics training materials and resources at [GLITTR](#)

1.4 Setup & Running the Notebooks

Prerequisite: You must have **R** installed on your system.

No Docker or Quarto setup is required. The project relies on **renv** to manage dependencies. Since these are standard Jupyter Notebooks (`.ipynb`) with an R kernel, you can run them locally using **VS Code**.

1.4.1 1. Clone the repository

```
git clone https://github.com/sbwiecko/biology-informed-multiomics-training.git
cd biology-informed-multiomics-training
```

1.4.2 2. Restore the R environment

Open R in your terminal, console, or IDE to restore dependencies via **renv**:

```
install.packages("renv")
renv::restore()
```

1.4.3 3. Running in VS Code

1. **Configure VS Code Working Directory (Crucial):** To guarantee that Jupyter always activates `.Rprofile` and locates your **renv** library, force VS Code to run kernels from the project root.

- Open Settings (Ctrl + , or Cmd + ,).
- Search for: Jupyter Notebook File Root.
- Set the value to: `${workspaceFolder}`.

2. **Register the Project Kernel:**

```
renv::install("IRkernel")
IRkernel::installspec(name = "ir_multiomics", displayname = "R (multiomics-renv)")
```

3. Open any `.ipynb` notebook.
4. Click **Select Kernel** in the top-right -> **Jupyter Kernel...** -> **R (multiomics-renv)**.

Optional check: You can always run `.libPaths()` in a notebook's first cell to verify that packages are loading from the project's local `./renv/library`.

1.5 Connect & Discuss

If you are also bridging the gap between wet-lab biology and bioinformatics, feel free to explore the notebooks, open an issue, or ask a question. Feedback, corrections, and discussions are always welcome!

1.6 Credits & Upstream Authors

This repository is built upon the open course materials designed by the **SIB Swiss Institute of Bioinformatics**:

- **Original Course Site:** <https://sib-swiss.github.io/biology-informed-multiomics-training/>
- **Original Repository:** [sib-swiss/biology-informed-multiomics-training](https://github.com/sib-swiss/biology-informed-multiomics-training)
- **Authors:** Deepak Tanwar, Geert van Geest, Patricia Palagi
- **Helper:** Joana Carlevaro-Fita
- **Original DOI:** [10.5281/zenodo.5703106](https://doi.org/10.5281/zenodo.5703106)

Part I

I. Theoretical Reference & Notes

2 Epigenetic Analysis Reference Guide

This document provides comprehensive reference tables for interpreting chromatin accessibility (ATAC-seq), histone modifications, RNA expression, and DNA methylation patterns in epigenetic analysis. Each table represents different combinations of assays and their biological interpretations.

2.1 Definitions

2.1.1 ATAC-seq

Assay for Transposase-Accessible Chromatin using sequencing (ATAC-seq) is a rapid molecular biology method used to map open, accessible regions of DNA across the genome. How ATAC-seq works:

1. **Tagmentation:** A hyperactive mutant enzyme called Tn5 transposase is loaded with sequencing adapters. It cuts DNA and inserts these adapters simultaneously only into open, uncompacted chromatin regions.
2. **Amplification & Sequencing:** The tagged DNA fragments are purified, amplified using PCR, and read using next-generation sequencing.
3. **Peak Analysis:** Bioinformatics tools map the resulting sequences back to the genome. Clusters of reads form “peaks” that show where the chromatin was open, revealing active gene promoters, enhancers, and transcription factor binding sites.

2.1.2 ChIP-seq

Chromatin Immunoprecipitation Sequencing (ChIP-seq) is a powerful laboratory technique used to find where specific proteins bind to DNA across the entire genome. How ChIP-seq works:

1. **Cross-linking:** Treat cells with a chemical like formaldehyde to freeze and bind proteins to their current spots on the DNA.
2. **Fragmentation:** Break the chromatin into smaller, manageable pieces using sound waves or enzymes (sonication).

3. Immunoprecipitation: Add a specific antibody that attaches only to your protein of interest, allowing you to pull down that protein and its stuck DNA fragment.
4. Reversal and Purification: Remove the chemical cross-links, get rid of the protein, and isolate the remaining bound DNA pieces.
5. Sequencing and Alignment: Send the purified DNA fragments for high-throughput sequencing, then use computers to map the reads back to a reference genome.

2.1.3 WGBS

Whole-genome bisulfite sequencing (WGBS) is the gold-standard next-generation sequencing method used to map DNA methylation at single-base resolution across an entire genome. By treating genomic DNA with sodium bisulfite, researchers can differentiate between methylated and unmethylated cytosines. How WGBS works:

1. Denaturation: Double-stranded genomic DNA is separated into single strands using heat.
2. Bisulfite Treatment: The DNA is treated with sodium bisulfite.
3. Unmethylated cytosines undergo hydrolytic deamination and convert into uracil (U).
4. Methylated cytosines (5mC) are chemically protected and remain unchanged (C).
5. PCR Amplification: During PCR, the converted uracils are amplified as thymines (T), while the protected methylated cytosines are amplified as cytosines (C).
6. Sequencing & Alignment: The final library is sequenced using a high-throughput platform. Computational pipelines (like Bismark) align the C-to-T altered reads back to a reference genome to determine the exact methylation ratio at every single site.

2.2 Data Interpretation

2.2.1 Key Abbreviations

- **ATAC-seq**: Assay for Transposase-Accessible Chromatin using sequencing
- **H3K4me1**: Histone H3 lysine 4 monomethylation (enhancer mark)
- **H3K4me3**: Histone H3 lysine 4 trimethylation (active promoter mark)
- **H3K27ac**: Histone H3 lysine 27 acetylation (active enhancer/promoter mark)
- **H3K27me3**: Histone H3 lysine 27 trimethylation (Polycomb repressive mark)
- **RNA-seq**: RNA sequencing (gene expression)
- **DNAme**: DNA methylation (CpG methylation)
- **TSS**: Transcription Start Site

2.2.2 Legend

- ↑: Increased signal/activity
- ↓: Decreased signal/activity
- ~: Low level or no significant change
- N/A: Not applicable
- -: No change or not detected

2.2.3 Table 1: Basic Enhancer Analysis

Analysis of enhancer states using **ATAC-seq**, **H3K4me1**, **RNA-seq**, and **DNAme**.

Category	ATAC-seq	H3K4me1	RNA-seq	DNAme (CpG)	Dist. to TSS	Interpretation
Active Enhancer	↑	↑	Target Gene ↑	↓	>2500 bp	Open chromatin, marked by both H3K4me1 and H3K27ac (see next table), driving expression of target gene. Typically distal. Hypomethylated.
Poised/Inactive Enhancer	↑	↑	Target Gene ~	↓	>2500 bp	Open chromatin, marked by H3K4me1 but lacking H3K27ac, indicating an accessible but not fully active enhancer. Target gene not expressed. Typically distal.
Silent Region	↓	↓	~ No expression	↑	>2500 bp / Any	Closed chromatin, H3K4me1 absent, indicating no enhancer activity. Often hypermethylated. Can be distal or other.
Promoter (Contrast)	↑	↓	↑	↓	<2500 bp	While H3K4me1 can be found at some promoters, it's generally low compared to H3K4me3, which is the primary promoter mark.

2.2.4 Table 2: Enhancer States with H3K27ac

Comprehensive analysis of enhancer states including **H3K27ac** for activity determination.

Cate- gory	ATAC- seq	H3K4me1	H3K4me2	H3K27ac	RNA- seq (Target Region)	DNAme (Enhancer Region)	Dist. to TSS	Interpretation
Active Enhancer	↑	↑	↑	↑	↓		>2500 bp	Open chromatin, both H3K4me1 and H3K27ac present. This is the classic signature of an active enhancer, strongly correlated with increased expression of its target gene(s). Typically distal to the TSS. Hypomethylated.
Poised/Primed Enhancer	↑	↑	~	~	↓	Low/No	>2500 bp	Open chromatin, H3K4me1 present but H3K27ac absent or very low. This indicates an accessible enhancer element that has potential but is not currently active. It's "primed" for activation under specific conditions. Typically distal to the TSS. Hypomethylated.
Inactive Enhancer (Accessible)	↑	↓	↓	~	↓ /	Low/Vari- able	>2500 bp	Open chromatin, but lacks both H3K4me1 and H3K27ac. This state is less common for enhancer definitions, but could represent a broadly accessible region without specific enhancer marks, or a very transient state. Typically distal to the TSS.
Inactive Enhancer (Closed)	↓	↓	↓	~	↑	Low/No	>2500 bp	Closed chromatin, both H3K4me1 and H3K27ac absent. This represents a silent or repressed enhancer element that is not accessible and not active. Often hypermethylated in the enhancer region. Typically distal to the TSS.
Weak/Dormant Enhancer	↑	~	~	↓			>2500 bp	A variant of the poised enhancer, where H3K27ac might be present but at very low levels, suggesting minimal or transient activity, or a region with enhancer potential that is not fully "on." Typically distal to the TSS.

2.2.5 Table 3: Enhancer vs Promoter Comparison

Direct comparison of enhancer and promoter states using **H3K27ac** as the key discriminator.

Category	H3K4me1		H3K27ac		Dist.		Interpretation
	ATAC-seq	Con-text	RNA-seq	DNAse (CpG)	TSS		
Active Enhancer	↑	↑	↑	Target Gene ↑	↓	>2500 bp	Open chromatin at a distal element, co-occurrence of H3K4me1 and H3K27ac, strong H3K27ac signal, often correlates with increased expression of a target gene. Hypomethylated.
Active Promoter	↑	~ Low/N/A	↑	↑	↓	<2500 bp	Open chromatin directly at the promoter, strong H3K27ac signal, high gene expression. Hypomethylated. (Often co-occurs with H3K4me3 here). Proximal to TSS.
Primed/Dormant Enhancer	↑	~ Low/N/A	~	Target Gene ~	↓	>2500 bp	Open chromatin, H3K4me1 present but H3K27ac is low or absent, suggesting the enhancer is accessible but not fully active. Target gene not expressed. Typically distal.
Inactive Enhancer	↓	↓	↓	↓	↑	<2500 bp /	Closed chromatin, H3K27ac absent, element is not active and associated gene is repressed.
Enhancer/Promoter						>2500 bp	Often hypermethylated. Can be proximal or distal.

2.2.6 Table 4: Promoter-Focused Analysis

Analysis specifically focused on promoter regions using **H3K4me3** as the primary marker.

Category	ATAC-seq		H3K4me3		Dist.		Interpretation
	ATAC-seq	Con-text	RNA-seq	DNAse (CpG)	TSS		
Active Promoter	↑	↑	↑	↓		<2500 bp	Open chromatin, active promoter, high gene expression, typically unmethylated. Proximal to the gene's TSS.
Poised Promoter	↑	↑	~ Low	↓ No		<2500 bp	Open chromatin, H3K4me3 present, but gene is not actively expressed (e.g., awaiting developmental cue). Proximal to TSS.
Inactive Promoter	↓	↓	↓	↑		<2500 bp	Closed chromatin, H3K4me3 absent, gene repressed, often hypermethylated. Proximal to TSS.

Category	ATAC-seq	DNAm (CpG)		Dist. to TSS	Interpretation
		H3K4me3	RNA-Pro-moter)		
Pro-moter Flanks	↑	~ Low/No	N/A ↓	<2500 bp	Open chromatin around active promoter, but H3K4me3 typically peaks precisely at the TSS.

2.2.7 Table 5: Simplified Three-Factor Analysis

Basic analysis using just **ATAC-seq**, **RNA-seq**, and **DNAm**.

Category	ATAC-seq		RNA-seq		DNAm (CpG)		Dist. to TSS	Interpretation
	seq	seq	seq	seq	moter)	moter)		
Active Gene/Element	↑	↑	↓				<2500 bp /	Open chromatin, high gene expression, and low DNA methylation. This is the classic signature of an actively transcribed gene (often proximal) or a highly active regulatory element (can be distal).
Repressed Gene/Element	↓	↓	↑				<2500 bp /	Closed chromatin, low/no gene expression, and high DNA methylation. This indicates a stably silenced gene or an inactive regulatory element. Can be proximal or distal.
Poised/Primed Region			↓				<2500 bp /	Open chromatin, but low/no gene expression, and low DNA methylation. The region is accessible, and DNA is unmethylated, suggesting it's "ready" for activation but not currently expressing. Can be proximal (poised promoter) or distal (poised enhancer).
Incongruent/Complex	↓	↑			Variable		<2500 bp /	Closed chromatin, but high gene expression. This is an unusual state. It might suggest very efficient transcription from a less accessible promoter, or a regulatory mechanism (e.g., enhancer) acting from a distance which is not reflected by local accessibility. Requires deeper investigation.
Silent/Inactive Region	↓	~ No Ex-pression	↑				<2500 bp /	Closed chromatin, no gene expression, and high DNA methylation. Represents a fully repressed and inaccessible genomic region, such as silent heterochromatin. Can be proximal or distal.

Category	DNAm (CpG Dist.				Interpretation
	ATAC-seq	RNA-Pro-seq	to	TSS	
Lost Accessibility, Unchanged RNA	↓	~	Vari- No Change	<2500 bp / >2500 bp	Chromatin becomes less accessible, but gene expression remains unchanged. This could indicate a shift in regulatory mechanisms (e.g., reliance on distal elements), or that the lost accessibility at a specific site does not immediately impact overall transcript levels.
Gained Accessibility, Unchanged RNA	↑	~	↓ No Change	<2500 bp / >2500 bp	Chromatin becomes more accessible, but gene expression remains unchanged. This might indicate a “priming” event where the region is opened up in preparation for future activation, or that the accessibility change alone isn’t sufficient to drive expression.

2.2.8 Table 6: Comprehensive Multi-Mark Analysis

Complete analysis including all major histone modifications (**H3K4me3**, **H3K4me1**, **H3K27ac**).

Category	RNA-seq						Dist.	Interpretation
	ATAC-seq	H3K4me3 (Pro-moter)	H3K4me1 (Enhancer)	H3K27ac (Active Gene)	H3K27me3 (Repressed)	DNAm (CpG)		
Active Promoter	↑	↑	~ Low/No	↑	↑	↓	<2500 bp	Open chromatin, strong H3K4me3 and H3K27ac. This is the hallmark of a highly active transcription start site (TSS), leading to high gene expression. H3K4me1 is typically low at active promoters. Proximal to TSS. Usually unmethylated.
Active Enhancer	↑	~ Low/No	↑	↑	↑	↓	>2500 bp	Open chromatin, strong H3K4me1 and H3K27ac. This combination defines an active enhancer, contributing to increased expression of its target gene(s), which may be proximal or distal. H3K4me3 is typically low at enhancers. Typically distal to TSS. Usually unmethylated.

Category	RNA-seq						Dist. bp	Interpretation
	ATAC-seq	H3K4me1	H3K4me3	H3K27ac	H3K27me3	DNase-seq (CpG)		
Poised/Enhancer	↑	Low/No	~	~	~	↓	>2500 bp	Open chromatin, H3K4me1 present but H3K27ac low or absent. This signifies an enhancer that is accessible and has potential, but is not currently actively driving gene expression. It's "poised" for activation. Typically distal to TSS. Usually unmethylated.
Poised Promoter (Bivalent)	↑	↑	~	~	~	↓	<2500 bp	Open chromatin, H3K4me3 present, but H3K27ac low/absent. (Note: If H3K27me3 is also present, it's a "bivalent" promoter, signaling developmental plasticity). The gene is prepared for activation but not highly expressed yet. Proximal to TSS. Usually unmethylated.
Inactive/Silent Region	↓	↓	↓	↓	↓ / ~	↑	<2500 bp	Closed chromatin, all three marks are low or absent. This indicates a transcriptionally inactive or repressed genomic region, including inactive promoters and enhancers. Often associated with DNA hypermethylation and/or repressive histone marks (like H3K27me3).
Weakly Active Enhancer	↑	~	↑	~	~	↓	>2500 bp	Open chromatin, H3K4me1 present, but H3K27ac at low levels. This could represent an enhancer with minimal activity or an enhancer that is just beginning to become active. Typically distal to TSS.

2.2.9 Table 7: Polycomb Repression Analysis

Analysis focusing on **H3K27me3**-mediated Polycomb repression.

Category	Dist. bp					Interpretation
	ATAC-seq	H3K27ac	H3K27me3	DNase-seq (CpG)	TSS	
Polycomb-Repressed Gene	↓	↑	↓	↓ / ~	<2500 bp	Closed chromatin due to Polycomb, gene is repressed, but often retains an unmethylated CpG island at the promoter, allowing for potential reactivation. Proximal to TSS.

Category	Dist.					Interpretation
	ATAC-seq	H3K27me3	RNA-seq	DNAme (CpG)	TSS	
Bivalent Promoter	↑	↑ (+ H3K4me3) ↑	~ Low/No	↓	<2500 bp	Open chromatin, but marked by both H3K4me3 (active) and H3K27me3 (repressive), indicating a poised state, common in stem cells for developmental genes. Proximal to TSS.
Inactive Region (Polycomb-driven)	↓	↑	~ No expression	↓ / Variable	>2500 bp / Any	Large genomic regions silenced by Polycomb, often in a lineage-specific manner. Chromatin is closed, but not necessarily hypermethylated. Can be distal or other.
Active Gene (H3K27me3 removed)	↑	↓	↑	↓	<2500 bp	H3K27me3 has been removed, allowing chromatin to open and gene expression to activate (e.g., during differentiation). Proximal to TSS.

2.2.10 Table 8: Complete Seven-Factor Analysis

Most comprehensive analysis including all major epigenetic marks and Polycomb repression.

Category	RNA-seq							Dist.	Interpretation
	ATAC-seq	H3K4me1	H3K4me3	H3K27ac	H3K27me3	Pro-moter	En-hancer		
Active Promoter	↑	↑	~ Low/No	↑	↓	↑	↓	<2500 bp	Open chromatin at TSS, strong active promoter marks (H3K4me3, H3K27ac), leading to high gene expression. Typically unmethylated.
Active Enhancer	↑	~ Low/No	↑	↑	↓	↑	↓	>2500 bp	Open chromatin at a distal element, strong active enhancer marks (H3K4me1, H3K27ac), correlated with increased expression of a target gene. Typically unmethylated.
Poised Promoter (Bivalent)	↑	↑	~ Low/No	~ Low/No	↑	~ Low/No	↓	<2500 bp	Open chromatin at TSS, marked by both active (H3K4me3) and repressive (H3K27me3) marks, indicating a repressed but ready-to-activate state (common in stem cells). Gene expression is low/off. Unmethylated promoter.

Category	RNA-seq							Dist.	Interpretation
	ATAC-seq	H3K4me1	H3K4me3	H3K27me1	H3K27me3	H3K27ac	H3K9me3		
Poised/Enhancer	Low/No	↑	~	~	~	↓	>2500 bp	Open chromatin at a distal element, H3K4me1 present but H3K27ac absent, indicating an accessible enhancer element that is not currently active but “primed” for future activation. Target gene not expressed. Unmethylated.	
Polycomb-Repressed Region	↓	↓	↓	↑	↓ / ~	↓ / No	Any	Closed chromatin, marked by Polycomb-mediated repression (H3K27me3), leading to reversible gene silencing. Often affects developmental genes. DNA methylation can be low or variable.	
Repressed Gene (by Methylation)	↓	↓	↓	↓	↓	↓	↑	<2500 bp	Closed chromatin at promoter, lack of active marks, reduced gene expression, primarily driven by high DNA methylation at the promoter.
Incongruent (Closed but Expressed)	↓	Vari-able	Vari-able	Vari-able	Vari-able	↑	Vari-able	Any	An unusual state where chromatin appears closed (low ATAC-seq), but the gene is expressed. This might suggest very efficient transcription, or regulation from a highly distant and active element not locally detectable, or limitations of the assay resolution. Requires careful validation.
Silent/Inactive Region	↓	↓	↓	↓	~	No Ex-pression	Vari-able	Any	Chromatin is closed, no active histone marks, no expression. A general category for regions that are simply not active, regardless of specific repressive mechanism (e.g., DNAm, polycomb).

2.3 Summary

These tables provide a comprehensive framework for interpreting multi-dimensional epigenetic data. The key principles to remember are:

1. **Active regions** typically show: ↑ ATAC-seq, ↑ Active histone marks, ↑ RNA-seq, ↓ DNA methylation
2. **Poised regions** show: ↑ ATAC-seq, ↑ Some histone marks, ~ Low RNA-seq, ↓ DNA methylation
3. **Repressed regions** show: ↓ ATAC-seq, ↓ Active marks, ↓ RNA-seq, ↑ DNA methylation
4. **Enhancers** are characterized by H3K4me1 and are typically >2500 bp from TSS
5. **Promoters** are characterized by H3K4me3 and are typically <2500 bp from TSS
6. **H3K27ac** distinguishes active from poised elements
7. **H3K27me3** indicates Polycomb-mediated repression
8. **Bivalent** promoters have both H3K4me3 and H3K27me3, common in stem cells

This reference guide should help in the systematic interpretation of complex epigenetic datasets across different experimental conditions and cell types.

3 Normalization models

3.1 Averaging Models for Normalization

The slide deck outlines four distinct mathematical models for summarizing signals.

- **Absolute (absolute)**: Calculates the mean value from all signal regions regardless of their width. The formula is $v_a = \frac{\sum_{i=1}^n x_i}{n}$.
- **Weighted (weighted)**: Calculates the mean value from all signal regions, specifically weighted by the width of their intersections. The formula is $v_w = \frac{\sum_{i=1}^n x_i w_i}{\sum_{i=1}^n w_i}$.
- **W0 (w0)**: Calculates the weighted mean between both the intersected and un-intersected parts of the window. The formula is $v_{w0} = \frac{\sum_{i=1}^n x_i w_i}{W + W'}$.
- **Coverage (coverage)**: Calculates the mean signal averaged by the width of the window itself, represented by L . The formula is $v_c = \frac{\sum_{i=1}^n x_i w_i}{L}$.

3.2 Understanding Windows and Genomic Signals

To understand how these methods are applied, it helps to define the core structural components shown in the visual diagrams:

- **Window**: This represents a designated segment in the target regions or in the flanking regions when we are normalizing genomic signals.
- **Genomic signal regions**: These are the actual data segments (like mapped reads or peaks) that overlap with the given window.

3.3 Application to Specific Assays

The choice of averaging method depends heavily on the biological assay and how the signal is represented.

- **RNA-seq**: We are physically sequencing the transcribed RNA. Therefore, the coordinates in the `rowRanges` represent the actual, literal boundaries of the gene or mRNA transcript on the genome. The “feature” we are counting is the gene itself.

- **ATAC-seq, ChIP-seq, and WGBS:** We are measuring the physical **state** of the DNA (is it open? is a protein bound to it? is it methylated?). These events often happen outside of genes—in distant enhancers, promoters, or vast intergenic deserts. The **rowRanges** define the exact coordinates of that regulatory event (the peak or CpG site). To make biological sense of that peak, the software calculates its distance to the nearest Transcription Start Site and annotates it with that closest gene’s name (which is why we see the **Symbol** and **distanceTSS** columns in the ATAC/ChIP tables).

Assay	Signal Representation	Recommended Method	Key Considerations
RNA-seq	Coverage (numeric)	w0	Averages signal intensity (read depth), weighted by overlap. Used when plotting continuous RNA-seq BigWig tracks around specific genomic features (like the TSS or splice sites) in EnrichedHeatmap.
RNA-seq	Gene/Transcript Counts	Median of Ratios or TPM	Used for standard expression analysis rather than window averaging. TPM normalizes within-sample abundance; Median of Ratios (DESeq2) normalizes across samples for differential expression.
ATAC-seq & ChIP-seq	Peaks (binary)	coverage	Shows the fraction of the window covered by peaks.
ATAC-seq & ChIP-seq	Coverage (numeric)	w0	Averages signal intensity, weighted by overlap. This is recommended for BigWig data.

Assay	Signal Representation	Recommended Method	Key Considerations
WGBS	CpG methylation (numeric)	absolute	Averages methylation values of individual CpG sites within the window.

In short: RNA-seq counts the genes themselves, while epigenetic assays count regulatory regions and then tag them with the closest gene to hypothesize what they might be regulating.

3.4 RNA-seq Data Analysis

RNA-seq normalization is a massive topic. The core problem we have to solve is that raw read counts are biased by two main factors: **sequencing depth** (some samples just get sequenced more deeply than others) and **gene length** (longer genes naturally accumulate more reads, even if they aren't more highly expressed).

Here is a breakdown of the standard normalization practices used in the field today.

3.4.1 1. Within-Sample Normalization (Relative Abundance)

These methods are used when we want to compare the expression of *Gene A* to *Gene B* within the exact same sample.

- **TPM (Transcripts Per Million):** This is the current gold standard for within-sample comparisons. It normalizes for gene length first, and then for sequencing depth. Because every sample sums to exactly one million TPM, it is much easier to compare relative proportions across samples. The mathematical model for calculating TPM for a transcript i is:

$$TPM_i = \left(\frac{q_i/l_i}{\sum_j q_j/l_j} \right) \times 10^6$$

(Where q_i is the read count and l_i is the length of the transcript). * **FPKM / RPKM (Fragments / Reads Per Kilobase per Million):** We will see these in older papers. They normalize for depth first, then length. The major flaw is that the total sum of normalized reads differs between samples, making direct cross-sample comparisons mathematically inconsistent.

3.4.2 2. Between-Sample Normalization (Differential Expression)

When our goal is to compare *Gene A* in a control group versus *Gene A* in a treatment group, within-sample methods like TPM fail to account for **RNA composition bias** (where a few highly expressed genes “hog” all the reads).

- **Median of Ratios (used by DESeq2):** This method calculates a scaling factor for each sample. It takes the ratio of each gene’s read count to its geometric mean across all samples, and then uses the median of these ratios to scale the library. It brilliantly ignores extreme outliers, ensuring that housekeeping genes stabilize the normalization.
- **TMM (Trimmed Mean of M-values, used by edgeR):** Very similar in philosophy to DESeq2. It trims away genes that have extreme fold-changes or extremely high/low absolute expression before calculating a scaling factor.

3.4.3 Method Comparison

Normalization Method	Primary Use Case	Accounts for Gene Length?	Accounts for Composition Bias?
TPM	Comparing genes within a sample (Abundance)	Yes	No
Median of Ratios (DESeq2)	Differential Gene Expression (DGE)	No	Yes
TMM (edgeR)	Differential Gene Expression (DGE)	No	Yes

When performing Differential Gene Expression (DGE) analysis, tools like DESeq2 and edgeR expect *raw, unnormalized counts* as input because their statistical models (negative binomial distributions) need to estimate variance accurately, which is destroyed if we feed them pre-normalized metrics like TPM.

3.5 EnrichedHeatmap

An EnrichedHeatmap is a specialized visualization used in bioinformatics to show how genomic signals (like read coverage or accessibility) are distributed *around* specific target regions (like transcription start sites or peaks).

Here is a breakdown of what we are looking at, component by component:

3.5.1 Core Structure

- **The Grid (Matrix & Rows):** The main part of the image consists of four rectangular blocks of color. Every single horizontal line (row) within those blocks represents a specific genomic region.
- **The X-Axis (Axis):** The bottom labels show `-1kb`, `mid`, and `1kb`. This indicates that for every row, we are looking at a 2-kilobase window centered on a specific target (`mid`). If the mid point represents a Transcription Start Site (TSS), regulatory elements like promoters often span a few hundred to a couple of thousand base pairs around that site. A 2 kb window (1000 bp on either side) is usually wide enough to capture the entire regulatory landscape for that feature.
- **The Columns (Columns title):** The data is split into two experimental conditions or timepoints, labeled `E11.5` and `E15.5`. The note mentions that biological replicates are generally merged to show average differences in these columns.

3.5.2 Preparing the Data

The `EnrichedHeatmap` itself doesn't calculate the "increase" or "decrease"—it just visualizes regions we have already grouped.

Before plotting, we run a **differential accessibility analysis** (using tools similar to `DESeq2` or `edgeR`). We calculate the Log_2 fold-change between the `E15.5` and `E11.5` read counts for every target region.

- Regions with a significantly positive fold-change are grouped as "Increased."
- Regions with a significantly negative fold-change are grouped as "Decreased."

These predefined groupings are then fed into the `EnrichedHeatmap` package, which organizes the rows and adds the corresponding row annotations (the red and blue bars).

3.5.3 Reading the Data

- **The Heatmap (Color & Color scale):** The intensity of the red color represents the strength of the signal (e.g., chromatin accessibility). White means no signal; deep red means a strong signal.
- **The Groupings (Rows annotation & Rows order):** Notice the thick vertical bar on the far left. It is split into a red block and a blue block. According to the legend, the top group of rows (red bar) represents regions that have "Increased accessibility" over time, while the bottom group (blue bar) represents regions with "Decreased accessibility".
- If we look at the top group, the center (`mid`) is much darker red in `E15.5` than in `E11.5`.

- If we look at the bottom group, the center is bright red in E11.5 but fades significantly in E15.5.
- **The Summary Lines (Profile plot):** The line graphs at the very top summarize the heatmap data. The red line shows the average signal for all the “Increased accessibility” regions, and the blue line shows the average for the “Decreased accessibility” regions. They peak at the mid point, matching the visual concentration of red in the heatmaps below.

4 Functional Enrichment Analysis: Multi-Omics Integration

This document summarizes the core concepts of functional enrichment analysis for multi-omics data, bridging the theoretical foundations from the lecture slides with the practical implementation in the accompanying R/Jupyter notebook.

4.1 Overview of Enrichment Analysis

The primary goal of functional enrichment analysis is to extract biologically meaningful insights from long lists of genes. Instead of analyzing genes in isolation, these methods evaluate sets of genes to identify overactive or suppressed biological pathways.

The three primary methodologies include:

- **Over-Representation Analysis (ORA):** Evaluates the fraction of genes in a specific pathway found among a set of differentially expressed (DE) genes.
 - It relies on a strict significance threshold (e.g., $FDR = 0.05$) to select the input list.
 - Significance is commonly calculated using Fisher's exact test, hypergeometric, chi-square, or binomial distributions.
 - **Limitations:** It requires arbitrary cutoffs, treats all genes in the list equally regardless of expression fold-change, and assumes that all genes and pathways act independently.
- **Functional Class Scoring (FCS / GSEA):** Evaluates whether weaker but coordinated changes in sets of related genes have significant effects.
 - Instead of a cutoff, it ranks all genes (often by $\log_2(FC) \times t\text{-value}$) and calculates a running enrichment score using statistics like the Kolmogorov-Smirnov test.
 - **Limitations:** Like ORA, it still assumes that genes and pathways are independent of one another.
- **Pathway Topology (PT):** Incorporates structural biological context to assess pathway impact.
 - It factors in the specific number of reactions, the biological position of the gene, and the type of reaction taking place.

4.2 Database Challenges & Semantic Similarity

Enrichment relies heavily on curated databases like Gene Ontology (GO), KEGG, and Reactome.

- **The Resolution Problem:** The GO database (split into Biological Process, Molecular Function, and Cellular Component) contains highly similar or overlapping terms, such as “cell cycle” and “mitosis”.
- **Semantic Similarity:** To reduce redundancy, analysis often measures the semantic similarity of GO terms based on “exclusively inherited” shared information. This filters out broad, unrelated common ancestors to group terms by their true unique functions.

Here is the expanded section, integrating the details of how GREAT calculates domains, maps peaks, and runs its statistical tests:

4.3 Cis-Regulatory Regions & GREAT

Standard enrichment tools fail when analyzing non-coding regions or distal regulatory elements.

- **GREAT (Genomic Regions Enrichment of Annotations Tool):** Designed to accurately link cis-regulatory regions (like enhancers or ATAC-seq peaks) to the specific biological pathways they control. It solves the distal regulation problem in three distinct steps:
 - **Defining Domains (Coordinate Math):** GREAT calculates a “regulatory domain” for every known gene by starting at the Transcription Start Site (TSS). It first assigns a **Basal Domain** (typically 5kb upstream to 1kb downstream). It then calculates a **Distal Extension**, expanding outward along the chromosome in both directions until it hits the nearest neighboring gene’s basal domain (capped at a strict maximum of 1,000 kb).
 - **Mapping Peaks (Interval Intersection):** It compares the genomic coordinates of the input regions against this newly built catalog of regulatory domains. Any physical overlap assigns that distal DNA to the corresponding gene.
 - **Linking to Pathways (Binomial Test):** To evaluate pathway significance, GREAT measures the total genomic footprint (in base pairs) of all regulatory domains belonging to a specific pathway. It then runs a binomial over-representation test to determine if the peaks landed inside that pathway’s footprint significantly more often than would happen by random chance.
- It accounts for long-range interactions identified by chromatin conformation (Hi-C) or epigenetic markers (H3K27ac).

- **rGREAT Integration:** The rGREAT R/Bioconductor package allows this functional enrichment to be executed directly on genomic regions.

4.4 Connection to the Practical Exercise

The accompanying Jupyter notebook perfectly mirrors this methodology by applying rGREAT to multi-omics overlap data.

- It isolates a specific genomic cluster (“Active Promoters”) from an ATAC/RNA overlap matrix.
- It visualizes the region-gene associations and uses the **simplifyEnrichment** package to calculate semantic similarity, clustering redundant GO terms into functional groups like “ion transport” and “calcium signaling”.

Part II

II. Data Structures & Visualization

5 Datasets, SummarizedExperiment, and GRanges

5.1 Learning outcomes

- Understand the structure of `SummarizedExperiment` and `GRanges` objects in R.
- Access and manipulate metadata and assays within `SummarizedExperiment`.
- Visualize and subset genomic intervals using `GRanges`.
- Recognize how even RNA expression data can be stored as genomic ranges.

5.2 Environment and data setup

5.2.1 Datasets for the course

In this course, we are working with **multi-omics datasets** derived from the developing **mouse neural tube** at two developmental stages: **E11.5** and **E15.5**. These stages represent key transitions in central nervous system development. These datasets are downloaded and processed from the ENCODE project: [Mouse Development Matrix](#). For the sake of time and efficient computational usage, datasets are analyzed for only chromosomes 1 and 2.

The following chromatin and transcriptional profiling datasets are included, each with **two biological replicates per time point**:

Assay	Description	E11.5	E15.5
ATAC-seq	Chromatin accessibility profiling	2	2
H3K4me3	Histone mark at active promoters	2	2
H3K4me1	Histone mark at enhancers and poised regulatory elements	2	2
H3K27me3	Repressive histone modification (Polycomb repression)	2	2
H3K27ac	Active enhancer and promoter histone mark	2	2
RNA-seq	Gene expression quantification via transcriptome sequencing	2	2

All datasets are preprocessed and stored in **SummarizedExperiment (SE)** objects:

- **Assays** contain count matrices (e.g., raw counts, logCPM).

- **RowRanges** describe genomic coordinates (**GRanges**) for each feature (gene, peak, region).
- **ColData** holds metadata for each sample (e.g., stage, replicate).

The use of **SE** objects enables consistent handling of multi-omics data, facilitates downstream integration, and supports interoperability with Bioconductor tools.

- **SummarizedExperiment** enables structured storage of high-dimensional omics data and metadata.
- **GRanges** makes it easy to manipulate and visualize genomic intervals across datasets.
- Most analyses — from clustering to visualization — can be performed directly on these containers.

5.2.2 Downloading and extracting data

Note: run the following once for the entire course.

```
# Download and unpack data only if not already present
if (!dir.exists("data")) {
  dir.create("data")

  # Increase timeout for large file transfer
  options(timeout = 1000)

  message("Downloading course datasets...")
  download.file(
    url = "https://biology-informed-multiomics.s3.eu-central-1.amazonaws.com/biology-informed-multiomics.tar.gz",
    destfile = "data/data.tar",
    mode = "wb" # Binary mode required for Windows
  )

  message("Extracting archive...")
  untar(tarfile = "data/data.tar", exdir = "data")
  file.remove("data/data.tar")
  message("Done! Data ready in ./data")
} else {
  message("Data directory already exists. Skipping download.")
}
```

Data directory already exists. Skipping download.

5.2.3 Loading libraries

```
# suppress warnings and messages
suppressPackageStartupMessages(suppressWarnings({
  library(SummarizedExperiment)
  library(GenomicRanges)
  library(ggplot2)
  library(tidyr)
  library(ComplexHeatmap)
}))
```

5.3 The SummarizedExperiment (SE) object

5.3.1 Loading SE objects

The function `readRDS()` is not part of the SummarizedExperiment package. It is a standard R function used to read a single R object that was previously saved using `saveRDS()`. Because SummarizedExperiment objects are just specialized R objects, they can be saved and loaded using these standard base R functions.

```
rna <- readRDS("data/rna_se.rds")
atac <- readRDS("data/atac_se.rds")
h3k4me3 <- readRDS("data/h3k4me3_se.rds")
h3k4me1 <- readRDS("data/h3k4me1_se.rds")
h3k27me3 <- readRDS("data/h3k27me3_se.rds")
h3k27ac <- readRDS("data/h3k27ac_se.rds")
```

5.3.2 Inspecting the RNA SE object

When we use `readRDS("data/rna_se.rds")`, it doesn't just read raw text or numbers (like `read.csv` would), but it completely restores the R object from the file exactly as it was when it was saved.

```
rna
```

```
class: RangedSummarizedExperiment
dim: 2216 4
metadata(0):
assays(2): counts logCPM
```

```

rownames(2216): ENSMUSG000000000194 ENSMUSG000000000247 ...
               ENSMUSG000000103948 ENSMUSG000000104257
rowData names(18): Row.names gene_id ... PValue qvalue
colnames(4): RNA_11half_1.tsv.gz RNA_11half_2.tsv.gz
              RNA_15half_1.tsv.gz RNA_15half_2.tsv.gz
colData names(2): sample group

```

The object is technically a `RangedSummarizedExperiment`, which is a specialized version of a `SummarizedExperiment` that explicitly links every row (gene/peak) to genomic coordinates (a `GRanges` object).

5.3.3 Accessing colData, rowRanges and assays

There is a [nice tutorial](#) from the authors of the SE library. Sample meta-data (*condition*) describing the samples can be accessed using `colData()`, and is a `DataFrame` that can store any number of descriptive columns for each sample row.

```
colData(rna)
```

DataFrame with 4 rows and 2 columns

	sample	group
	<character>	<factor>
RNA_11half_1.tsv.gz	RNA_11half_1.tsv.gz	11half
RNA_11half_2.tsv.gz	RNA_11half_2.tsv.gz	11half
RNA_15half_1.tsv.gz	RNA_15half_1.tsv.gz	15half
RNA_15half_2.tsv.gz	RNA_15half_2.tsv.gz	15half

Notice that the group column uses 11half and 15half, which correspond to the E11.5 and E15.5 developmental stages.

The `rowRanges()` accessor is used to view the range information for a `RangedSummarizedExperiment`. The data are stored in a `GRangesList` (*Genomic Ranges*) object, where each list element corresponds to one gene transcript (*feature*) and the ranges in each `GRanges` correspond to the exons in the transcript.

```
rowRanges(rna)
```

GRanges object with 2216 ranges and 18 metadata columns:

	seqnames	ranges	strand	Row.names
	<Rle>	<IRanges>	<Rle>	<character>
ENSMUSG000000000194	chr2	31152316-31218775	+	ENSMUSG000000000194

ENSMUSG00000000247	chr2	38339281-38369733	+		ENSMUSG00000000247
ENSMUSG00000000305	chr2	179442431-179899373	+		ENSMUSG00000000305
ENSMUSG00000000308	chr2	121357714-121363737	+		ENSMUSG00000000308
ENSMUSG00000000359	chr2	152626951-152635198	+		ENSMUSG00000000359
...	...	...	...	.	...
ENSMUSG00000102976	chr1	133619862-133661380	-		ENSMUSG00000102976
ENSMUSG00000103711	chr1	171418872-171420352	+		ENSMUSG00000103711
ENSMUSG00000103746	chr1	33669824-33670712	+		ENSMUSG00000103746
ENSMUSG00000103948	chr1	37485851-37488982	-		ENSMUSG00000103948
ENSMUSG00000104257	chr1	25276404-25277954	-		ENSMUSG00000104257
	gene_id	SYMBOL			GENENAME
	<character>	<character>			<character>
ENSMUSG00000000194	277463	Gpr107	G	protein-coupled re..	
ENSMUSG00000000247	16870	Lhx2	LIM	homeobox protein 2	
ENSMUSG00000000305	12561	Cdh4		cadherin 4	
ENSMUSG00000000308	12716	Ckmt1	creatine	kinase, mit..	
ENSMUSG00000000359	19700	Rem1	rad	and gem related ..	
...	...	...			...
ENSMUSG00000102976	70579	Zc3h11a	zinc	finger CCCH typ..	
ENSMUSG00000103711	226654	Tstd1	thiosulfate	sulfurtr..	
ENSMUSG00000103746	67503	1700001G17Rik	RIKEN	cDNA 1700001G1..	
ENSMUSG00000103948	77633	4930594C11Rik	G1	to S phase transi..	
ENSMUSG00000104257	100504323	Gm20172	predicted	gene, 20172	
		ENSEMBL	geneLength	geneChr	geneStart
	<character>	<integer>	<character>	<integer>	
ENSMUSG00000000194	ENSMUSG00000000194	66460	chr2	31152316	
ENSMUSG00000000247	ENSMUSG00000000247	30453	chr2	38339281	
ENSMUSG00000000305	ENSMUSG00000000305	456943	chr2	179442431	
ENSMUSG00000000308	ENSMUSG00000000308	6024	chr2	121357714	
ENSMUSG00000000359	ENSMUSG00000000359	8248	chr2	152626951	
...	...	...	...	...	
ENSMUSG00000102976	ENSMUSG00000102976	41519	chr1	133619862	
ENSMUSG00000103711	ENSMUSG00000103711	1481	chr1	171418872	
ENSMUSG00000103746	ENSMUSG00000103746	889	chr1	33669824	
ENSMUSG00000103948	ENSMUSG00000103948	3132	chr1	37485851	
ENSMUSG00000104257	ENSMUSG00000104257	1551	chr1	25276404	
	geneEnd	geneStrand	geneId		transcriptId
	<integer>	<character>	<character>		<character>
ENSMUSG00000000194	31218775	+	277463	ENSMUST000000056433.6	
ENSMUSG00000000247	38369733	+	16870	ENSMUST00000143783.8	
ENSMUSG00000000305	179899373	+	12561	ENSMUST00000000314.12	
ENSMUSG00000000308	121363737	+	12716	ENSMUST00000125812.7	
ENSMUSG00000000359	152635198	+	19700	ENSMUST00000000369.3	

```

...      ...      ...      ...      ...
ENSMUSG000000102976 133661380      -      70579  ENSMUST000000191896.5
ENSMUSG000000103711 171420352      +      226654  ENSMUST000000193060.1
ENSMUSG000000103746 33670712      +      67503  ENSMUST000000191779.1
ENSMUSG000000103948 37488982      -      77633  ENSMUST000000192316.1
ENSMUSG000000104257 25277954      -      100504323  ENSMUST000000193138.1

      anno      logFC      logCPM      F      PValue
<character> <numeric> <numeric> <numeric> <numeric>
ENSMUSG000000000194      Gene      1.12372      8.74812 137.26552 8.72516e-05
ENSMUSG000000000247      Gene     -1.60859      5.40640 11.86742 1.87646e-02
ENSMUSG000000000305      Gene      2.11822      9.25294 504.51658 3.72434e-06
ENSMUSG000000000308      Gene      3.79826      8.63364 420.96960 5.79582e-06
ENSMUSG000000000359      Gene      1.90855      3.12253      2.67341 1.64139e-01

...      ...      ...      ...      ...      ...
ENSMUSG000000102976      Gene     -0.856315 10.09628 97.72448 0.000195722
ENSMUSG000000103711      Gene      0.499469      3.20636      2.31857 0.189210840
ENSMUSG000000103746      Gene      0.563047      2.57430      1.09798 0.343492016
ENSMUSG000000103948      Gene      2.760559      2.65971 12.70432 0.016224604
ENSMUSG000000104257      Gene      1.965857      2.42225      9.13281 0.030057179

      qvalue
<numeric>
ENSMUSG000000000194 0.000834921
ENSMUSG000000000247 0.033322641
ENSMUSG000000000305 0.000490337
ENSMUSG000000000308 0.000490337
ENSMUSG000000000359 0.211786009

...      ...
ENSMUSG000000102976 0.00126563
ENSMUSG000000103711 0.23858422
ENSMUSG000000103746 0.39804874
ENSMUSG000000103948 0.02952106
ENSMUSG000000104257 0.04873306

```

```
-----
seqinfo: 66 sequences from an unspecified genome; no seqlengths
```

We can isolate the **GRanges** (or sometimes a **GRangesList**) object. This object will contain the **seqnames** (chromosome), **ranges** (start and end positions), and **strand** (+ or -) for every single gene in the RNA-seq dataset, perfectly matching the order of the rows in the count matrix.

```
gr <- rowRanges(rna)

length(gr)
```

2216

```
gr[1]
```

GRanges object with 1 range and 18 metadata columns:

	seqnames	ranges	strand	Row.names
	<Rle>	<IRanges>	<Rle>	<character>
ENSMUSG000000000194	chr2	31152316-31218775	+	ENSMUSG000000000194
	gene_id	SYMBOL		GENENAME
	<character>	<character>		<character>
ENSMUSG000000000194	277463	Gpr107	G protein-coupled re..	
		ENSEMBL	geneLength	geneChr geneStart
		<character>	<integer>	<character> <integer>
ENSMUSG000000000194	ENSMUSG000000000194	66460	chr2	31152316
	geneEnd	geneStrand	geneId	transcriptId
	<integer>	<character>	<character>	<character>
ENSMUSG000000000194	31218775	+	277463	ENSMUST000000056433.6
	anno	logFC	logCPM	F PValue
	<character>	<numeric>	<numeric>	<numeric> <numeric>
ENSMUSG000000000194	Gene	1.12372	8.74812	137.266 8.72516e-05
	qvalue			
	<numeric>			
ENSMUSG000000000194	0.000834921			

seqinfo: 66 sequences from an unspecified genome; no seqlengths

```
seqnames(gr)
```

factor-Rle of length 2216 with 528 runs

Lengths: 11 3 1 1 20 1 7 ... 3 7 1 5 1 5
Values : chr2 chr1 chr2 chr1 chr2 chr1 chr2 ... chr2 chr1 chr2 chr1 chr2 chr1
Levels(66): chr1 chr2 chr3 ... chrUn_GL456394 chrUn_GL456396 chrUn_JH584304

```
strand(gr)
```

factor-Rle of length 2216 with 1124 runs

Lengths: 5 2 2 1 2 2 1 2 1 1 10 ... 1 1 2 2 1 2 3 6 2 2
Values : + - + - + - + - + - + ... + - + - + - + - + -
Levels(3): + - *

In a GRanges object, the core information is strictly the **genomic coordinates**. However, we almost always have extra information associated with those genomic intervals. For example:

- For RNA-seq, we might want to store the Gene Symbol, Ensembl ID, or Biotype for each gene
- For ATAC-seq/ChIP-seq, we might want to store the peak score, fold-change, or p-value for each peak

`mcols(gr)` (which stands for metadata cols) is how we access or modify this extra information. It returns a DataFrame where each row perfectly matches a genomic range in our GRanges object, and the columns contain whatever extra annotations we've attached to those ranges.

```
mcols(gr)
```

DataFrame with 2216 rows and 18 columns

	Row.names	gene_id	SYMBOL		
	<character>	<character>	<character>		
ENSMUSG00000000194	ENSMUSG00000000194	277463	Gpr107		
ENSMUSG00000000247	ENSMUSG00000000247	16870	Lhx2		
ENSMUSG00000000305	ENSMUSG00000000305	12561	Cdh4		
ENSMUSG00000000308	ENSMUSG00000000308	12716	Ckmt1		
ENSMUSG00000000359	ENSMUSG00000000359	19700	Rem1		
...	...	...	...		
ENSMUSG00000102976	ENSMUSG00000102976	70579	Zc3h11a		
ENSMUSG00000103711	ENSMUSG00000103711	226654	Tstd1		
ENSMUSG00000103746	ENSMUSG00000103746	67503	1700001G17Rik		
ENSMUSG00000103948	ENSMUSG00000103948	77633	4930594C11Rik		
ENSMUSG00000104257	ENSMUSG00000104257	100504323	Gm20172		
	GENENAME	ENSEMBL	geneLength		
	<character>	<character>	<integer>		
ENSMUSG00000000194	G protein-coupled re..	ENSMUSG00000000194	66460		
ENSMUSG00000000247	LIM homeobox protein 2	ENSMUSG00000000247	30453		
ENSMUSG00000000305	cadherin 4	ENSMUSG00000000305	456943		
ENSMUSG00000000308	creatine kinase, mit..	ENSMUSG00000000308	6024		
ENSMUSG00000000359	rad and gem related ..	ENSMUSG00000000359	8248		
...	...	...	...		
ENSMUSG00000102976	zinc finger CCCH typ..	ENSMUSG00000102976	41519		
ENSMUSG00000103711	thiosulfate sulfurtr..	ENSMUSG00000103711	1481		
ENSMUSG00000103746	RIKEN cDNA 1700001G1..	ENSMUSG00000103746	889		
ENSMUSG00000103948	G1 to S phase transi..	ENSMUSG00000103948	3132		
ENSMUSG00000104257	predicted gene, 20172	ENSMUSG00000104257	1551		
	geneChr	geneStart	geneEnd	geneStrand	geneId
	<character>	<integer>	<integer>	<character>	<character>

ENSMUSG00000000194	chr2	31152316	31218775	+	277463
ENSMUSG00000000247	chr2	38339281	38369733	+	16870
ENSMUSG00000000305	chr2	179442431	179899373	+	12561
ENSMUSG00000000308	chr2	121357714	121363737	+	12716
ENSMUSG00000000359	chr2	152626951	152635198	+	19700
...	...	...	...	...	...
ENSMUSG00000102976	chr1	133619862	133661380	-	70579
ENSMUSG00000103711	chr1	171418872	171420352	+	226654
ENSMUSG00000103746	chr1	33669824	33670712	+	67503
ENSMUSG00000103948	chr1	37485851	37488982	-	77633
ENSMUSG00000104257	chr1	25276404	25277954	-	100504323
	transcriptId		anno	logFC	logCPM
	<character>	<character>	<numeric>	<numeric>	
ENSMUSG00000000194	ENSMUST00000056433.6	Gene	1.12372	8.74812	
ENSMUSG00000000247	ENSMUST00000143783.8	Gene	-1.60859	5.40640	
ENSMUSG00000000305	ENSMUST00000000314.12	Gene	2.11822	9.25294	
ENSMUSG00000000308	ENSMUST00000125812.7	Gene	3.79826	8.63364	
ENSMUSG00000000359	ENSMUST00000000369.3	Gene	1.90855	3.12253	
...	...	...	...	...	...
ENSMUSG00000102976	ENSMUST00000191896.5	Gene	-0.856315	10.09628	
ENSMUSG00000103711	ENSMUST00000193060.1	Gene	0.499469	3.20636	
ENSMUSG00000103746	ENSMUST00000191779.1	Gene	0.563047	2.57430	
ENSMUSG00000103948	ENSMUST00000192316.1	Gene	2.760559	2.65971	
ENSMUSG00000104257	ENSMUST00000193138.1	Gene	1.965857	2.42225	
	F	PValue	qvalue		
	<numeric>	<numeric>	<numeric>		
ENSMUSG00000000194	137.26552	8.72516e-05	0.000834921		
ENSMUSG00000000247	11.86742	1.87646e-02	0.033322641		
ENSMUSG00000000305	504.51658	3.72434e-06	0.000490337		
ENSMUSG00000000308	420.96960	5.79582e-06	0.000490337		
ENSMUSG00000000359	2.67341	1.64139e-01	0.211786009		
...	...	...	...		
ENSMUSG00000102976	97.72448	0.000195722	0.00126563		
ENSMUSG00000103711	2.31857	0.189210840	0.23858422		
ENSMUSG00000103746	1.09798	0.343492016	0.39804874		
ENSMUSG00000103948	12.70432	0.016224604	0.02952106		
ENSMUSG00000104257	9.13281	0.030057179	0.04873306		

In a `RangedSummarizedExperiment`, `rowData()` is just a shortcut. Under the hood, `rowData(rna)` is programmed to reach into the `rowRanges` and pull out its `mcols()`. They return the exact same `DataFrame` of gene annotations.

Note: `rowData()` works on all `SE` objects. However, if this were a standard `SummarizedExperi-`

ment without genomic coordinates, we would only be able to use `rowData()`, as `rowRanges()` would not exist.

```
head(rowData(rna), 3)
```

DataFrame with 3 rows and 18 columns

	Row.names	gene_id	SYMBOL
	<character>	<character>	<character>
ENSMUSG00000000194	ENSMUSG00000000194	277463	Gpr107
ENSMUSG00000000247	ENSMUSG00000000247	16870	Lhx2
ENSMUSG00000000305	ENSMUSG00000000305	12561	Cdh4

	GENENAME	ENSEMBL	geneLength
	<character>	<character>	<integer>
ENSMUSG00000000194	G protein-coupled re..	ENSMUSG00000000194	66460
ENSMUSG00000000247	LIM homeobox protein 2	ENSMUSG00000000247	30453
ENSMUSG00000000305	cadherin 4	ENSMUSG00000000305	456943

	geneChr	geneStart	geneEnd	geneStrand	geneId
	<character>	<integer>	<integer>	<character>	<character>
ENSMUSG00000000194	chr2	31152316	31218775	+	277463
ENSMUSG00000000247	chr2	38339281	38369733	+	16870
ENSMUSG00000000305	chr2	179442431	179899373	+	12561

	transcriptId	anno	logFC	logCPM
	<character>	<character>	<numeric>	<numeric>
ENSMUSG00000000194	ENSMUST00000056433.6	Gene	1.12372	8.74812
ENSMUSG00000000247	ENSMUST00000143783.8	Gene	-1.60859	5.40640
ENSMUSG00000000305	ENSMUST00000000314.12	Gene	2.11822	9.25294

	F	PValue	qvalue
	<numeric>	<numeric>	<numeric>
ENSMUSG00000000194	137.2655	8.72516e-05	0.000834921
ENSMUSG00000000247	11.8674	1.87646e-02	0.033322641
ENSMUSG00000000305	504.5166	3.72434e-06	0.000490337

Here we are seeing start, end, or seqnames as columns inside our `rowData()` DataFrame, it simply means the person who created the dataset left redundant data in there.

By default, the *core spatial coordinates* are not part of the metadata table. Even though the `rowData` table has columns conveniently named start and end, Bioconductor tools do not know what they are.

A `SummarizedExperiment` object can hold multiple assay datasets (e.g., raw counts and normalized values). We can view the list of available assays using `assays(rna)` and access a specific one using the `$` operator on that list (for example, `assays(rna)$counts`). In these

assay matrices, each row represents a gene/transcript and each column represents a single sample.

```
assays(rna)
```

```
List of length 2
names(2): counts logCPM
```

```
assays(rna)$counts[1:5,]
```

		RNA_11half_1.tsv.gz	RNA_11half_2.tsv.gz	RNA_15half_
		<dbl>	<dbl>	<dbl>
A data.frame: 5 × 4	ENSMUSG00000000194	848	893.35	2619.55
	ENSMUSG00000000247	169	246.00	51.00
	ENSMUSG00000000305	694	778.00	4431.00
	ENSMUSG00000000308	148	195.00	3011.00
	ENSMUSG00000000359	8	13.00	101.00

The `assay()` function (singular) extracts a single matrix from the object. Because we didn't specify a name, it defaults to returning the very first assay in the object. Depending on how `rna` was created, this might happen to be “counts”, but it could be “logCPM”.

```
assay(rna)[1:5,] # same as assay(rna, 'counts')[1:5,] or assay(rna, 1)[1:5,]
```

		RNA_11half_1.tsv.gz	RNA_11half_2.tsv.gz	RNA_15half_
		<dbl>	<dbl>	<dbl>
A data.frame: 5 × 4	ENSMUSG00000000194	848	893.35	2619.55
	ENSMUSG00000000247	169	246.00	51.00
	ENSMUSG00000000305	694	778.00	4431.00
	ENSMUSG00000000308	148	195.00	3011.00
	ENSMUSG00000000359	8	13.00	101.00

Note: While `assays(rna)$counts` works perfectly, the common and preferred Bioconductor method we can use is the singular function: `assay(rna, "counts")`.

```
assay(rna, "counts")[1:5,]
```

		RNA_11half_1.tsv.gz	RNA_11half_2.tsv.gz	RNA_15half_
		<dbl>	<dbl>	<dbl>
A data.frame: 5 × 4	ENSMUSG00000000194	848	893.35	2619.55
	ENSMUSG00000000247	169	246.00	51.00
	ENSMUSG00000000305	694	778.00	4431.00
	ENSMUSG00000000308	148	195.00	3011.00
	ENSMUSG00000000359	8	13.00	101.00

It is often helpful to revisit the core principles of RNA-seq analysis. A great starting point is the [UPenn course material](#).

Meta-data describing the experimental methods and publication references can be accessed using `metadata()`.

```
print(metadata(rna))
```

```
list()
```

Note that `metadata()` is just a simple list, so it is appropriate for any experiment wide metadata the user wishes to save, such as storing model formulas.

5.3.4 Subsetting data

The sample metadata columns (which are stored in `colData()`) can be accessed directly on the object using the `$` accessor. This makes it easy to **subset** the entire object by a given phenotype or treatment group.

```
rna[, rna$group == "11half"]
```

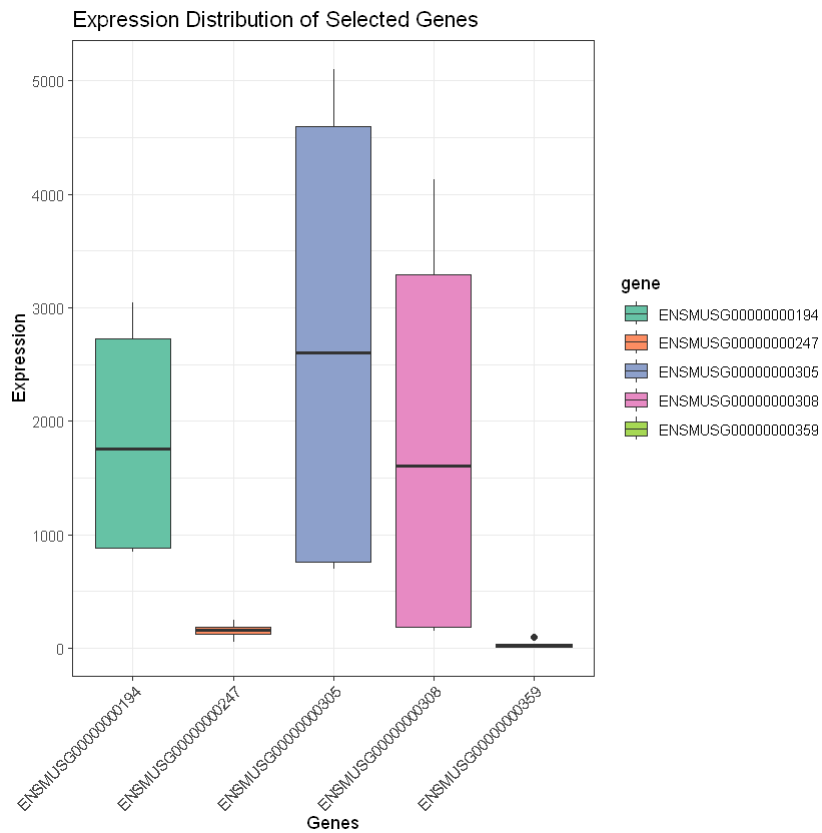
```
class: RangedSummarizedExperiment
dim: 2216 2
metadata(0):
assays(2): counts logCPM
rownames(2216): ENSMUSG000000000194 ENSMUSG000000000247 ...
               ENSMUSG000000103948 ENSMUSG000000104257
rowData names(18): Row.names gene_id ... PValue qvalue
colnames(2): RNA_11half_1.tsv.gz RNA_11half_2.tsv.gz
colData names(2): sample group
```

5.3.5 Boxplot of expression for a few genes

Let's pick-up the first 5 genes and plot their expression in the 4 samples tested.

```
df <- as.data.frame(t(assay(rna)[1:5, ]))
df$sample <- rownames(df)
df_long <- pivot_longer(
  df, -sample, names_to = "gene", values_to = "expression")
# -sample tells to pivot every column in the dataframe EXCEPT the sample column
```

```
ggplot(df_long, aes(x = gene, y = expression)) +
  geom_boxplot(aes(fill = gene)) +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust=1)) +
  labs(
    title = "Expression Distribution of Selected Genes",
    y = "Expression", x = "Genes") +
  scale_fill_brewer(palette = "Set2")
```



5.4 PRACTICE & EXERCISES

5.4.1 Question 1

Make a boxplot of samples, before and after normalization.

Hint: Check assays of rna object. Raw counts should be converted to log2 to make a comparison.

```
# Extract assays
raw_counts <- assay(rna, "counts")
logcpm <- assay(rna, "logCPM")

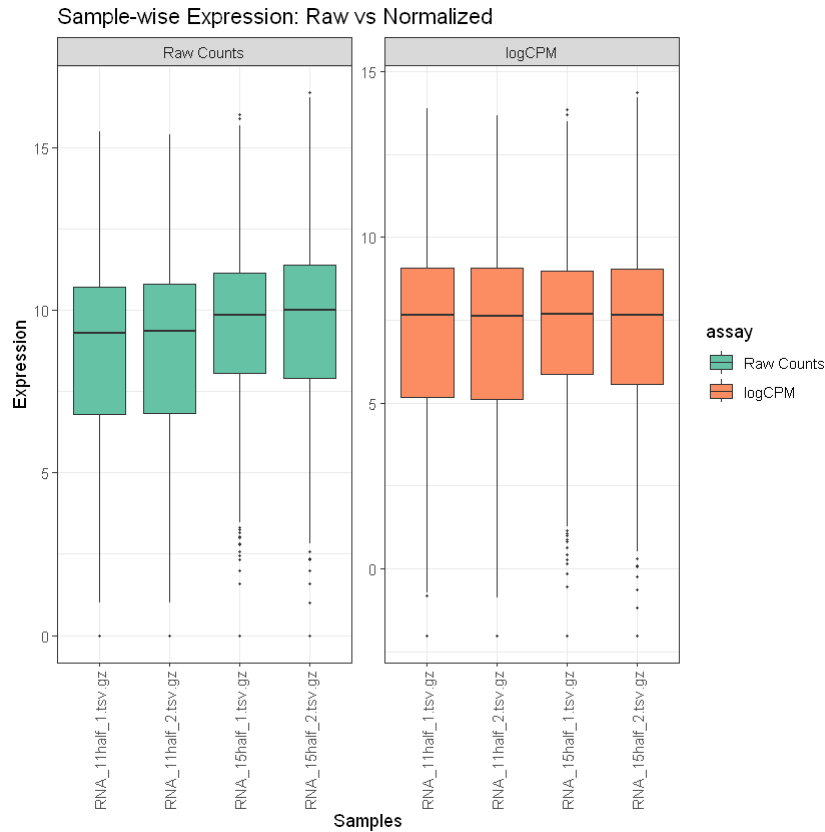
# Convert to long data frames for ggplot
df_raw <- as.data.frame(raw_counts)
df_raw$gene <- rownames(df_raw)
df_raw_long <- pivot_longer(
  df_raw, ~gene, names_to = "sample", values_to = "expression")
df_raw_long$assay <- "Raw Counts"
df_raw_long$expression <- log2(df_raw_long$expression + 1)

df_log <- as.data.frame(logcpm)
df_log$gene <- rownames(df_log)
df_log_long <- pivot_longer(
  df_log, ~gene, names_to = "sample", values_to = "expression")
df_log_long$assay <- "logCPM"

# Combine both
df_combined <- rbind(df_raw_long, df_log_long)

# Set factor for names to make Raw counts to be plotted first
df_combined$assay <- factor(
  df_combined$assay,
  levels = unique(df_combined$assay))

# Plot boxplot across samples
ggplot(df_combined, aes(x = sample, y = expression, fill = assay)) +
  geom_boxplot(outlier.size = 0.5, lwd = 0.3) +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust=1)) +
  facet_wrap(~ assay, scales = "free_y") +
  labs(
    title = "Sample-wise Expression: Raw vs Normalized",
    x = "Samples", y = "Expression") +
  scale_fill_brewer(palette = "Set2")
```



Important Note: While doing $\log_2(\text{counts} + 1)$ puts the raw counts on a similar visual scale as logCPM, it is not a true normalization! logCPM (Counts Per Million) explicitly scales the data by the total sequencing depth (library size) of each sample before taking the log. This is why normalization is essential before comparing samples.

5.4.2 Question 2

Make a new `SummarizedExperiment` object by sub-setting `rna`.

Take only first 2 samples and first 10 genes. Check the `assays`, row and column metadata.

```
rna_subset <- rna[1:10, 1:2]
rna_subset

assays(rna_subset)
```

```
class: RangedSummarizedExperiment
dim: 10 2
```

```

metadata(0):
assays(2): counts logCPM
rownames(10): ENSMUSG000000000194 ENSMUSG000000000247 ...
      ENSMUSG000000000827 ENSMUSG000000000876
rowData names(18): Row.names gene_id ... PValue qvalue
colnames(2): RNA_11half_1.tsv.gz RNA_11half_2.tsv.gz
colData names(2): sample group

```

```

List of length 2
names(2): counts logCPM

```

```
colData(rna_subset)
```

DataFrame with 2 rows and 2 columns

	sample	group
	<character>	<factor>
RNA_11half_1.tsv.gz	RNA_11half_1.tsv.gz	11half
RNA_11half_2.tsv.gz	RNA_11half_2.tsv.gz	11half

```
rowRanges(rna_subset)
```

GRanges object with 10 ranges and 18 metadata columns:

	seqnames	ranges	strand	Row.names
	<Rle>	<IRanges>	<Rle>	<character>
ENSMUSG000000000194	chr2	31152316-31218775	+	ENSMUSG000000000194
ENSMUSG000000000247	chr2	38339281-38369733	+	ENSMUSG000000000247
ENSMUSG000000000305	chr2	179442431-179899373	+	ENSMUSG000000000305
ENSMUSG000000000308	chr2	121357714-121363737	+	ENSMUSG000000000308
ENSMUSG000000000359	chr2	152626951-152635198	+	ENSMUSG000000000359
ENSMUSG000000000392	chr2	62500943-62574075	-	ENSMUSG000000000392
ENSMUSG000000000823	chr2	181582103-181592803	-	ENSMUSG000000000823
ENSMUSG000000000826	chr2	181520485-181555133	+	ENSMUSG000000000826
ENSMUSG000000000827	chr2	181497142-181517966	+	ENSMUSG000000000827
ENSMUSG000000000876	chr2	154585758-154603708	-	ENSMUSG000000000876
	gene_id	SYMBOL		GENENAME
	<character>	<character>		<character>
ENSMUSG000000000194	277463	Gpr107	G protein-coupled re..	
ENSMUSG000000000247	16870	Lhx2	LIM homeobox protein 2	
ENSMUSG000000000305	12561	Cdh4	cadherin 4	
ENSMUSG000000000308	12716	Ckmt1	creatine kinase, mit..	
ENSMUSG000000000359	19700	Rem1	rad and gem related ..	

ENSMUSG00000000392	14089	Fap	fibroblast	activatio..
ENSMUSG00000000823	269401	Zfp512b	zinc finger	protein ..
ENSMUSG00000000826	13002	Dnajc5	DnaJ	heat shock prot..
ENSMUSG00000000827	66314	Tpd52l2	tumor protein	D52-li..
ENSMUSG00000000876	59038	Pxmp4	peroxisomal	membrane..
	ENSEMBL	geneLength	geneChr	geneStart
	<character>	<integer>	<character>	<integer>
ENSMUSG00000000194	ENSMUSG00000000194	66460	chr2	31152316
ENSMUSG00000000247	ENSMUSG00000000247	30453	chr2	38339281
ENSMUSG00000000305	ENSMUSG00000000305	456943	chr2	179442431
ENSMUSG00000000308	ENSMUSG00000000308	6024	chr2	121357714
ENSMUSG00000000359	ENSMUSG00000000359	8248	chr2	152626951
ENSMUSG00000000392	ENSMUSG00000000392	73133	chr2	62500943
ENSMUSG00000000823	ENSMUSG00000000823	10701	chr2	181582103
ENSMUSG00000000826	ENSMUSG00000000826	34649	chr2	181520485
ENSMUSG00000000827	ENSMUSG00000000827	20825	chr2	181497142
ENSMUSG00000000876	ENSMUSG00000000876	17951	chr2	154585758
	geneEnd	geneStrand	geneId	transcriptId
	<integer>	<character>	<character>	<character>
ENSMUSG00000000194	31218775	+	277463	ENSMUST000000056433.6
ENSMUSG00000000247	38369733	+	16870	ENSMUST000000143783.8
ENSMUSG00000000305	179899373	+	12561	ENSMUST000000000314.12
ENSMUSG00000000308	121363737	+	12716	ENSMUST000000125812.7
ENSMUSG00000000359	152635198	+	19700	ENSMUST000000000369.3
ENSMUSG00000000392	62574075	-	14089	ENSMUST000000102732.9
ENSMUSG00000000823	181592803	-	269401	ENSMUST000000108789.8
ENSMUSG00000000826	181555133	+	13002	ENSMUST000000072334.11
ENSMUSG00000000827	181517966	+	66314	ENSMUST000000184588.7
ENSMUSG00000000876	154603708	-	59038	ENSMUST000000000896.10
	anno	logFC	logCPM	F
	<character>	<numeric>	<numeric>	<numeric>
ENSMUSG00000000194	Gene	1.123717	8.74812	137.26552
ENSMUSG00000000247	Gene	-1.608592	5.40640	11.86742
ENSMUSG00000000305	Gene	2.118219	9.25294	504.51658
ENSMUSG00000000308	Gene	3.798256	8.63364	420.96960
ENSMUSG00000000359	Gene	1.908545	3.12253	2.67341
ENSMUSG00000000392	Gene	-2.102269	6.02595	82.37588
ENSMUSG00000000823	Gene	0.776790	9.69040	21.64298
ENSMUSG00000000826	Gene	0.991148	11.54446	77.71581
ENSMUSG00000000827	Gene	-0.557763	9.55700	46.64975
ENSMUSG00000000876	Gene	0.304488	7.07357	4.36632
	qvalue			
	<numeric>			

```

ENSMUSG000000000194 0.000834921
ENSMUSG000000000247 0.033322641
ENSMUSG000000000305 0.000490337
ENSMUSG000000000308 0.000490337
ENSMUSG000000000359 0.211786009
ENSMUSG000000000392 0.001579555
ENSMUSG000000000823 0.012874947
ENSMUSG000000000826 0.001683051
ENSMUSG000000000827 0.003701948
ENSMUSG000000000876 0.127276923

```

seqinfo: 66 sequences from an unspecified genome; no seqlengths

```
length(rowRanges(rna_subset))
```

10

```
assay(rna_subset, "counts")
```

		RNA_11half_1.tsv.gz	RNA_11half_2.tsv.gz
		<dbl>	<dbl>
A data.frame: 10 × 2	ENSMUSG000000000194	848.00	893.35
	ENSMUSG000000000247	169.00	246.00
	ENSMUSG000000000305	694.00	778.00
	ENSMUSG000000000308	148.00	195.00
	ENSMUSG000000000359	8.00	13.00
	ENSMUSG000000000392	349.00	333.00
	ENSMUSG000000000823	1851.87	2077.29
	ENSMUSG000000000826	5908.00	7011.00
	ENSMUSG000000000827	2788.23	2998.49
	ENSMUSG000000000876	374.00	400.00

5.4.3 Question 3

Try to run all other commands from the following image on other SE objects as well.

```
h3k27ac
```

```
colData(h3k27ac)
```

```
assays(h3k27ac)
```

```

class: RangedSummarizedExperiment
dim: 24231 4
metadata(2): final.ext param
assays(2): counts logCPM
rownames: NULL
rowData names(18): annotation geneChr ... PValue qvalue
colnames: NULL
colData names(6): bam.files totals ... group norm.factors

```

DataFrame with 4 rows and 6 columns

	bam.files	totals	ext	rlen	group	norm.factors
	<character>	<integer>	<integer>	<integer>	<character>	<numeric>
1	../data/H3K27ac_11ha..	38907314	100	36	11half	0.916676
2	../data/H3K27ac_11ha..	38213305	100	36	11half	0.893381
3	../data/H3K27ac_15ha..	83981374	100	49	15half	1.096524
4	../data/H3K27ac_15ha..	94077476	100	49	15half	1.113600

List of length 2

```
names(2): counts logCPM
```

```
head(assay(h3k27ac, "logCPM"))
```

A matrix: 6 × 4 of type dbl

	4.157836	-1.072886	1.135676	1.540448
	3.659635	2.072572	4.200268	3.435317
	3.326437	2.808303	4.145176	3.884800
	3.052044	2.072572	4.353945	4.382375
	4.226394	3.944796	5.805263	5.680163
	2.507844	3.655477	1.135676	2.417469

```
metadata(h3k27ac)
```

Le chargement a nécessité le package : csaw

```
$final.ext
```

```
[1] NA
```

```
$param
```

```
An object of class "readParam"
```

```
Slot "pe":
```

```
[1] "none"
```

```

Slot "max.frag":
[1] 500

Slot "dedup":
[1] FALSE

Slot "minq":
[1] NA

Slot "forward":
[1] NA

Slot "restrict":
character(0)

Slot "discard":
GRanges object with 0 ranges and 0 metadata columns:
      seqnames      ranges strand
      <Rle> <IRanges>  <Rle>
-----
      seqinfo: no sequences

Slot "processed.discard":
named list()

```

5.5 The GRanges object

5.5.1 Extracting and inspecting GRanges from ATAC-seq

```

gr <- rowRanges(atac)
gr

```

```

GRanges object with 3656 ranges and 18 metadata columns:
      seqnames      ranges strand |      annotation      geneChr
      <Rle>      <IRanges> <Rle> | <character> <integer>
[1]   chr1      3670547-3672665   * | Promoter (<=1kb)         1
[2]   chr1      4332510-4332710   * | Intron (ENSMUST000000..   1
[3]   chr1      4491755-4492573   * | Promoter (1-2kb)         1
[4]   chr1      4571186-4572423   * | Distal Intergenic        1
[5]   chr1      4785062-4786325   * | Promoter (<=1kb)         1

```

```

...      ...      ...      ...      ...
[3652]    chr2 181764664-181764891    * |    Promoter (1-2kb)    2
[3653]    chr2 181766881-181767875    * |    Promoter (<=1kb)    2
[3654]    chr2 181837244-181838655    * |    Promoter (<=1kb)    2
[3655]    chr2 181863853-181865066    * |    Promoter (<=1kb)    2
[3656]    chr2 181918525-181918853    * |    Distal Intergenic    2

      geneStart    geneEnd    geneLength    geneStrand    geneId
      <integer>    <integer>    <integer>    <integer>    <character>
[1]      3214482      3671498      457017      2      497097
[2]      4344146      4360314      16169      2      19888
[3]      4492465      4493735      1271      2      20671
[4]      4491390      4497354      5965      2      20671
[5]      4773206      4785710      12505      2      27395

...      ...      ...      ...      ...
[3652] 181763332 181827797      64466      1      17932
[3653] 181767029 181795892      28864      1      17932
[3654] 181837854 181857461      19608      1      245867
[3655] 181864337 181870830      6494      1      67005
[3656] 181864360 181866209      1850      1      67005

      transcriptId    distanceToTSS    ENSEMBL    SYMBOL
      <character>    <numeric>    <character>    <character>
[1]    ENSMUST00000070533.4      0    ENSMUSG00000051951    Xkr4
[2]    ENSMUST00000027032.5      27604    ENSMUSG00000025900    Rp1
[3]    ENSMUST000000191939.1      1162    ENSMUSG00000025902    Sox17
[4]    ENSMUST000000192650.5     -73832    ENSMUSG00000025902    Sox17
[5]    ENSMUST000000130201.7      0    ENSMUSG00000033845    Mrpl15

...      ...      ...      ...      ...
[3652] ENSMUST00000081125.10      1332    ENSMUSG00000010505    Myt1
[3653] ENSMUST000000156190.7      0    ENSMUSG00000010505    Myt1
[3654] ENSMUST00000029116.13      0    ENSMUSG00000027589    Pcmt2d2
[3655] ENSMUST00000039551.8      0    ENSMUSG00000038628    Polr3k
[3656] ENSMUST000000153214.1     54165    ENSMUSG00000038628    Polr3k

      GENENAME      anno      logFC      logCPM      F
      <character>    <character>    <numeric>    <numeric>    <numeric>
[1] X-linked Kx blood gr..    Promoter    0.602984    10.45378    41.78446
[2] retinitis pigmentosa..    Intron      0.855748    5.18424    3.68371
[3] SRY (sex determining..    Promoter    -0.620926    7.13469    9.14522
[4] SRY (sex determining..    Intergenic  -0.511046    8.90885    15.15688
[5] mitochondrial riboso..    Promoter    0.540908    8.63469    15.08892

...      ...      ...      ...      ...
[3652] myelin transcription..    Promoter    -1.275275    5.54135    10.44866
[3653] myelin transcription..    Promoter    0.477658    7.56969    6.03877
[3654] protein-L-isoasparta..    Promoter    0.689247    7.92175    16.67881

```

```

[3655] polymerase (RNA) III.. Promoter 0.241693 9.05746 4.06676
[3656] polymerase (RNA) III.. Intergenic 0.394577 6.84745 2.73331
      PValue      qvalue
      <numeric> <numeric>
[1] 6.58453e-08 4.81983e-07
[2] 6.13517e-02 9.28785e-02
[3] 4.12368e-03 8.38540e-03
[4] 3.27158e-04 8.67115e-04
[5] 3.36058e-04 8.82634e-04
...
[3652] 0.002309221 0.00497558
[3653] 0.017945118 0.03116850
[3654] 0.000181039 0.00051468
[3655] 0.049773014 0.07677364
[3656] 0.105289776 0.15077925
-----

```

seqinfo: 21 sequences from an unspecified genome; no seqlengths

```
seqlevels(gr)
```

```

1. 'chr1' 2. 'chr2' 3. 'chr3' 4. 'chr4' 5. 'chr5' 6. 'chr6' 7. 'chr7' 8. 'chr8' 9. 'chr9' 10. 'chr10'
11. 'chr11' 12. 'chr12' 13. 'chr13' 14. 'chr14' 15. 'chr15' 16. 'chr16' 17. 'chr17' 18. 'chr18'
19. 'chr19' 20. 'chrX' 21. 'chrY'

```

5.5.2 Subsetting regions on *chr1* with width > 1000 bp

`width` (along with `seqnames`, `start`, `end`, and `strand`) is not in the metadata columns (`mcols`). It is a core spatial property of the `GRanges` object itself. It calculates width simply as `end - start + 1`. For a gene, this represents the entire genomic locus on the chromosome, spanning from the start of the first exon all the way to the end of the last exon, including all the introns.

```

subset_gr <- gr[seqnames(gr) == "chr1" & width(gr) > 1000]
length(subset_gr)

```

838

Bioconductor has a specialized `subset` method for `GRanges` that allows us to drop the repetitive `gr` calls, making it look much cleaner:

```
subset(gr, seqnames == "chr1" & width > 1000)
```

GRanges object with 838 ranges and 18 metadata columns:

	seqnames	ranges	strand	annotation	geneChr
	<Rle>	<IRanges>	<Rle>	<character>	<integer>
[1]	chr1	3670547-3672665	*	Promoter (<=1kb)	1
[2]	chr1	4571186-4572423	*	Distal Intergenic	1
[3]	chr1	4785062-4786325	*	Promoter (<=1kb)	1
[4]	chr1	4807268-4808558	*	Promoter (<=1kb)	1
[5]	chr1	4857303-4858480	*	Promoter (<=1kb)	1
...	...	...	...	...	...
[834]	chr1	193968000-193969035	*	Distal Intergenic	1
[835]	chr1	194160689-194161978	*	Distal Intergenic	1
[836]	chr1	194619386-194621007	*	Promoter (<=1kb)	1
[837]	chr1	194879295-194880523	*	Distal Intergenic	1
[838]	chr1	194976000-194977461	*	Promoter (<=1kb)	1

	geneStart	geneEnd	geneLength	geneStrand	geneId
	<integer>	<integer>	<integer>	<integer>	<character>
[1]	3214482	3671498	457017	2	497097
[2]	4491390	4497354	5965	2	20671
[3]	4773206	4785710	12505	2	27395
[4]	4807788	4848410	40623	1	18777
[5]	4857814	4897905	40092	1	21399
...	...	...	...	...	...
[834]	194212187	194222759	10573	2	74942
[835]	194212187	194222759	10573	2	74942
[836]	194619825	194816869	197045	1	18845
[837]	194822001	194822064	64	1	100628615
[838]	194955759	194976960	21202	2	320400

	transcriptId	distanceToTSS	ENSEMBL	SYMBOL
	<character>	<numeric>	<character>	<character>
[1]	ENSMUST00000070533.4	0	ENSMUSG000000051951	Xkr4
[2]	ENSMUST000000192650.5	-73832	ENSMUSG000000025902	Sox17
[3]	ENSMUST000000130201.7	0	ENSMUSG000000033845	Mrpl15
[4]	ENSMUST000000134384.7	0	ENSMUSG000000025903	Lypla1
[5]	ENSMUST000000081551.13	0	ENSMUSG000000033813	Tcea1
...	...	...	...	...
[834]	ENSMUST000000193395.1	253724	ENSMUSG000000103692	4930503007Rik
[835]	ENSMUST000000193395.1	60781	ENSMUSG000000103692	4930503007Rik
[836]	ENSMUST000000027952.11	0	ENSMUSG000000026640	Plxna2
[837]	ENSMUST000000174892.1	57294	ENSMUSG000000092633	Mir3962
[838]	ENSMUST000000180780.2	0	ENSMUSG000000097325	Gm16897

	GENENAME	anno	logFC	logCPM	F
	<character>	<character>	<numeric>	<numeric>	<numeric>
[1]	X-linked Kx blood gr..	Promoter	0.6029841	10.45378	41.784460

```

[2] SRY (sex determining.. Intergenic -0.5110459 8.90885 15.156883
[3] mitochondrial riboso.. Promoter 0.5409079 8.63469 15.088922
[4] lysophospholipase 1 Promoter 0.0485083 9.02241 0.158018
[5] transcription elonga.. Promoter 0.4092797 9.92134 14.681270
...
[834] RIKEN cDNA 493050300.. Intergenic 1.789029 6.37400 36.41420
[835] RIKEN cDNA 493050300.. Intergenic -2.621150 7.53898 143.60529
[836] plexin A2 Promoter -0.444863 9.24715 14.79019
[837] microRNA 3962 Intergenic -1.195438 6.95062 28.66158
[838] predicted gene, 16897 Promoter -0.246053 8.75503 3.60378
      PValue      qvalue
      <numeric> <numeric>
[1] 6.58453e-08 4.81983e-07
[2] 3.27158e-04 8.67115e-04
[3] 3.36058e-04 8.82634e-04
[4] 6.92882e-01 7.45491e-01
[5] 3.95063e-04 1.01572e-03
...
[834] 2.85224e-07 1.73797e-06
[835] 1.50833e-15 1.77316e-13
[836] 3.78302e-04 9.79514e-04
[837] 2.86565e-06 1.28866e-05
[838] 6.41228e-02 9.63951e-02
-----

```

seqinfo: 21 sequences from an unspecified genome; no seqlengths

5.5.3 Plotting the width distribution of genomic ranges

`mcols(gr)` (Metadata only) returns only the extra annotations (the “fact sheet”). It returns a `DataFrame` of things like Gene Name, P-value, or Score. It does not include the actual genomic coordinates (`seqnames`, `start`, `end`, `width`, `strand`). `as.data.frame(gr)` (Everything) flattens the entire `GRanges` object into a standard R dataframe. The resulting table will have `seqnames`, `start`, `end`, `width`, and `strand` as its first five columns, and then it will attach all of the `mcols` right next to them.

We must use `as.data.frame(gr)` here. If we used `mcols(gr)`, the resulting table wouldn’t have the **width** column.

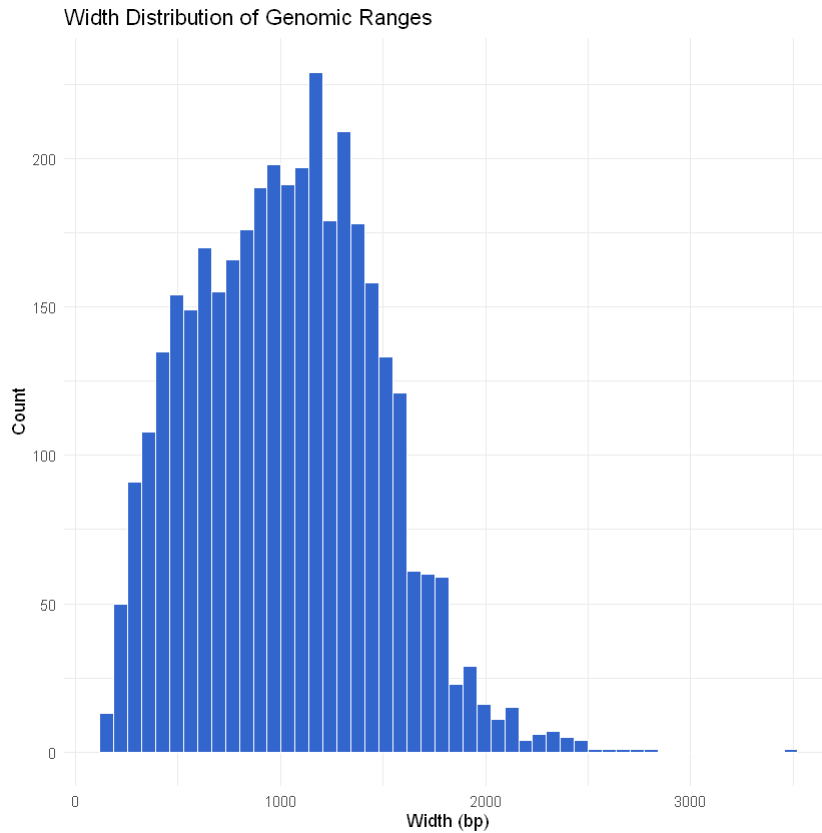
```

df_gr <- as.data.frame(gr)

ggplot(df_gr, aes(x = width)) +
  geom_histogram(bins = 50, fill = "#3366cc", color = "white") +

```

```
theme_minimal() +
labs(
  title = "Width Distribution of Genomic Ranges",
  x = "Width (bp)", y = "Count")
```



The *geneLength* column in the metadata is usually the effective length of the mature mRNA transcript. It is calculated by summing up only the lengths of the exons (ignoring the introns). This is the number used to normalize counts into TPM (Transcripts Per Million).

5.6 Visualizing genomic data

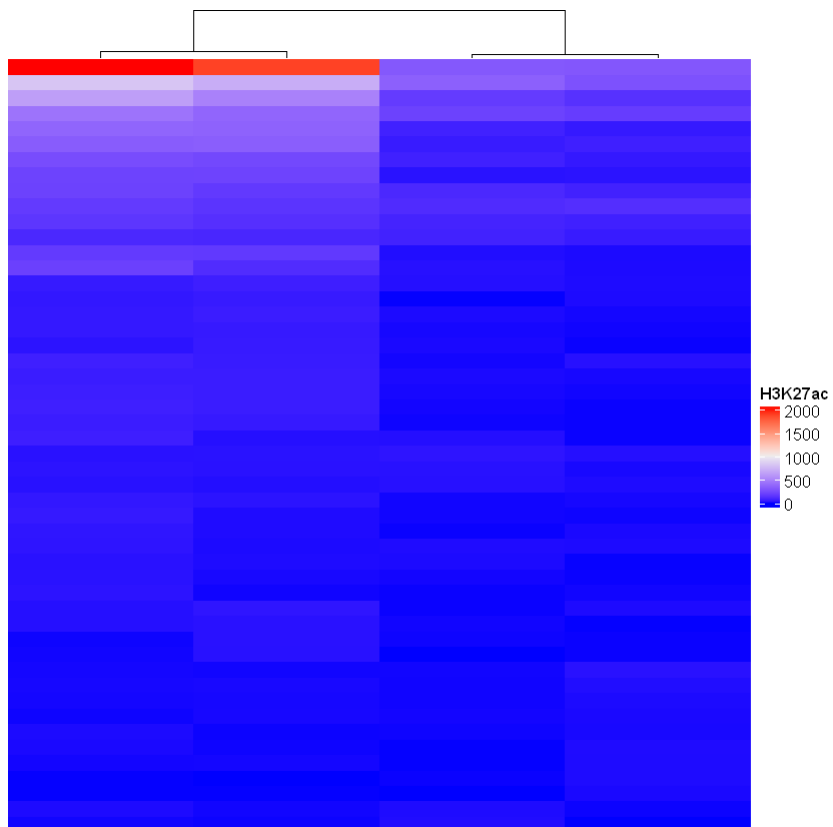
5.6.1 Heatmap of H3K27ac signal

```
mat <- assay(h3k27ac)
head(mat)
```

A matrix: 6×4 of type int

20	0	5	8
14	4	52	34
11	7	50	47
9	4	58	67
21	16	161	167
6	13	5	16

```
Heatmap(
  mat[1:50,],
  name = "H3K27ac",
  show_row_names = FALSE,
  show_column_names = TRUE,
  cluster_rows = TRUE,
  show_row_dend = FALSE, # hide row dendrogram
  cluster_columns = TRUE)
```



5.6.2 Question 4

Try to run all other commands from the following image on other `GRanges` from SE objects as well.

```
gr <- rowRanges(h3k4me1)
head(gr)
```

`GRanges` object with 6 ranges and 18 metadata columns:

	seqnames	ranges	strand	annotation	geneChr
	<Rle>	<IRanges>	<Rle>	<character>	<integer>
[1]	chr1	3120410-3120664	*	Distal Intergenic	1
[2]	chr1	3254884-3255166	*	Intron (ENSMUST000000..	1
[3]	chr1	3366339-3366557	*	Exon (ENSMUST00000019..	1
[4]	chr1	3368377-3368571	*	Exon (ENSMUST00000019..	1
[5]	chr1	3398970-3399433	*	Intron (ENSMUST000000..	1
[6]	chr1	3399520-3400407	*	Intron (ENSMUST000000..	1

	geneStart	geneEnd	geneLength	geneStrand	geneId
	<integer>	<integer>	<integer>	<integer>	<character>
[1]	3214482	3671498	457017	2	497097
[2]	3214482	3671498	457017	2	497097
[3]	3214482	3671498	457017	2	497097
[4]	3214482	3671498	457017	2	497097
[5]	3214482	3671498	457017	2	497097
[6]	3214482	3671498	457017	2	497097

	transcriptId	distanceToTSS	ENSEMBL	SYMBOL
	<character>	<numeric>	<character>	<character>
[1]	ENSMUST00000070533.4	550834	ENSMUSG000000051951	Xkr4
[2]	ENSMUST00000070533.4	416332	ENSMUSG000000051951	Xkr4
[3]	ENSMUST00000070533.4	304941	ENSMUSG000000051951	Xkr4
[4]	ENSMUST00000070533.4	302927	ENSMUSG000000051951	Xkr4
[5]	ENSMUST00000070533.4	272065	ENSMUSG000000051951	Xkr4
[6]	ENSMUST00000070533.4	271091	ENSMUSG000000051951	Xkr4

	GENENAME	anno	logFC	logCPM	F
	<character>	<character>	<numeric>	<numeric>	<numeric>
[1]	X-linked Kx blood gr..	Intergenic	2.530271	3.05733	8.022852
[2]	X-linked Kx blood gr..	Intron	1.330709	3.35988	4.865120
[3]	X-linked Kx blood gr..	Exon	1.904164	3.05805	5.515882
[4]	X-linked Kx blood gr..	Exon	0.245368	3.04948	0.131943
[5]	X-linked Kx blood gr..	Intron	1.088826	3.98058	6.467585
[6]	X-linked Kx blood gr..	Intron	1.345539	5.22784	26.869684

	PValue	qvalue
[1]		
[2]		
[3]		
[4]		
[5]		
[6]		

```

      <numeric>    <numeric>
[1] 4.63050e-03 4.08230e-02
[2] 2.74329e-02 1.34633e-01
[3] 1.88685e-02 1.06593e-01
[4] 7.16436e-01 8.48565e-01
[5] 1.10044e-02 7.42101e-02
[6] 2.22912e-07 1.53039e-05
-----

```

```
seqinfo: 24 sequences from an unspecified genome; no seqlengths
```

```
length(gr)
```

```
41330
```

In the context of ChIP-seq (or ATAC-seq), the core geometry of our `GRanges` object represents the peaks (the specific, relatively short regions where a protein bound or chromatin is open, often just 200–1000 bp).

If we have annotated those peaks to nearby genes, the genes themselves have their own “start” and “end” coordinates, which can span tens or hundreds of thousands of base pairs.

```
head(width(gr)) # length between IRanges bounds, not between start and end of gene!
```

```
1. 255 2. 283 3. 219 4. 195 5. 464 6. 888
```

```
head(gr$geneLength) # head(mcols(gr)$geneLength)
```

```
1. 457017 2. 457017 3. 457017 4. 457017 5. 457017 6. 457017
```

Understanding how different omics layers are represented in `SE` and `GRanges` objects is essential for multi-omics integration, visualization, and downstream analysis.

6 Enriched Heatmaps and identification of regions with different activities

6.1 Learning objectives

- Distinguish between when to use `Heatmap()` and `HeatmapAnnotation()`
- Generate row split annotations for enriched heatmaps
- Add multiple categorical annotations with separate color legends
- Avoid common data handling errors in heatmap plotting

6.2 Loading libraries

In this exercise, we shall use methods from the `gUtils` library. It provides a set of utility functions for use with `GenomicRanges`. We need to install it properly in the current environment [following the author's instructions](#).

```
suppressPackageStartupMessages(suppressWarnings({  
  library(SummarizedExperiment)  
  library(GenomicRanges)  
  library(gUtils)  
  library(rtracklayer)  
  library(EnrichedHeatmap)  
  library(ComplexHeatmap)  
  library(circlize)  
}))
```

6.3 Preparing the ATAC SummarizedExperiment

Here we load the `SummarizedExperiment` for the ATAC-seq data

```
atac <- readRDS("data/atac_se.rds")
```

6.3.1 ATAC peaks information

Each peak region represent the activity in a genomic region. These peaks are annotated for various features, including their genomic annotations and the results for differential accessibility analysis is also added there.

```
rd_atac <- rowRanges(atac)
```

```
rd_atac
```

GRanges object with 3656 ranges and 18 metadata columns:

	seqnames		ranges	strand		annotation	geneChr
	<Rle>		<IRanges>	<Rle>		<character>	<integer>
[1]	chr1		3670547-3672665	*		Promoter (<=1kb)	1
[2]	chr1		4332510-4332710	*		Intron (ENSMUST000000..	1
[3]	chr1		4491755-4492573	*		Promoter (1-2kb)	1
[4]	chr1		4571186-4572423	*		Distal Intergenic	1
[5]	chr1		4785062-4786325	*		Promoter (<=1kb)	1
...	...		...	...	.	...	...
[3652]	chr2	181764664-181764891		*		Promoter (1-2kb)	2
[3653]	chr2	181766881-181767875		*		Promoter (<=1kb)	2
[3654]	chr2	181837244-181838655		*		Promoter (<=1kb)	2
[3655]	chr2	181863853-181865066		*		Promoter (<=1kb)	2
[3656]	chr2	181918525-181918853		*		Distal Intergenic	2
	geneStart	geneEnd	geneLength	geneStrand		geneId	
	<integer>	<integer>	<integer>	<integer>		<character>	
[1]	3214482	3671498	457017	2		497097	
[2]	4344146	4360314	16169	2		19888	
[3]	4492465	4493735	1271	2		20671	
[4]	4491390	4497354	5965	2		20671	
[5]	4773206	4785710	12505	2		27395	
...	...	...	...	...		...	
[3652]	181763332	181827797	64466	1		17932	
[3653]	181767029	181795892	28864	1		17932	
[3654]	181837854	181857461	19608	1		245867	
[3655]	181864337	181870830	6494	1		67005	
[3656]	181864360	181866209	1850	1		67005	
	transcriptId	distanceToTSS		ENSEMBL		SYMBOL	
	<character>	<numeric>		<character>		<character>	
[1]	ENSMUST00000070533.4	0		ENSMUSG00000051951		Xkr4	
[2]	ENSMUST00000027032.5	27604		ENSMUSG00000025900		Rp1	
[3]	ENSMUST000000191939.1	1162		ENSMUSG00000025902		Sox17	

```

[4] ENSMUST00000192650.5      -73832 ENSMUSG00000025902      Sox17
[5] ENSMUST00000130201.7              0 ENSMUSG00000033845      Mrpl15
...
[3652] ENSMUST00000081125.10      1332 ENSMUSG00000010505      Myt1
[3653] ENSMUST00000156190.7              0 ENSMUSG00000010505      Myt1
[3654] ENSMUST00000029116.13              0 ENSMUSG00000027589      Pcmt2d
[3655] ENSMUST00000039551.8              0 ENSMUSG00000038628      Polr3k
[3656] ENSMUST00000153214.1      54165 ENSMUSG00000038628      Polr3k
      GENENAME      anno      logFC      logCPM      F
      <character> <character> <numeric> <numeric> <numeric>
[1] X-linked Kx blood gr.. Promoter 0.602984 10.45378 41.78446
[2] retinitis pigmentosa.. Intron 0.855748 5.18424 3.68371
[3] SRY (sex determining.. Promoter -0.620926 7.13469 9.14522
[4] SRY (sex determining.. Intergenic -0.511046 8.90885 15.15688
[5] mitochondrial riboso.. Promoter 0.540908 8.63469 15.08892
...
[3652] myelin transcription.. Promoter -1.275275 5.54135 10.44866
[3653] myelin transcription.. Promoter 0.477658 7.56969 6.03877
[3654] protein-L-isoasparta.. Promoter 0.689247 7.92175 16.67881
[3655] polymerase (RNA) III.. Promoter 0.241693 9.05746 4.06676
[3656] polymerase (RNA) III.. Intergenic 0.394577 6.84745 2.73331
      PValue      qvalue
      <numeric> <numeric>
[1] 6.58453e-08 4.81983e-07
[2] 6.13517e-02 9.28785e-02
[3] 4.12368e-03 8.38540e-03
[4] 3.27158e-04 8.67115e-04
[5] 3.36058e-04 8.82634e-04
...
[3652] 0.002309221 0.00497558
[3653] 0.017945118 0.03116850
[3654] 0.000181039 0.00051468
[3655] 0.049773014 0.07677364
[3656] 0.105289776 0.15077925
-----

```

seqinfo: 21 sequences from an unspecified genome; no seqlengths

Once we have the short sequencing reads in an ATAC-seq experiment, bioinformatics algorithms take over to recreate the biological landscape of the genomic accessibility.

- **Alignment:** We take every read and match it to a reference genome to find exactly where it originated.

- **Generating Coverage:** When thousands of reads map to the exact same open genomic location, they form a “pile-up.” This is the black, mountainous signal we see in the example snapshot of chromatin accessibility.
- **Peak Calling:** We use statistical algorithms (like *MACS2*) to distinguish true biological pile-ups from random background noise, artifacts and duplicates. The algorithm defines the precise boundaries of these pile-ups, creating discrete peak regions (the shaded boxes on the example snapshot of chromatin accessibility) with specific coordinates (e.g., `chr1`, `start: 100`, `end: 400`).

6.3.2 Statistical results

Now that we have genomic coordinates for our peaks, we can extract the specific information:

- **Genomic annotation:** we compare our peak coordinates against a known database of genes. This tells us which gene is nearby (e.g., `Gene1`) and how far the peak is from the Transcription Start Site (`distanceTSS`).
- **Differential accessibility:** to compare conditions, we count exactly how many sequencing reads fall within a specific peak region for each experimental group (e.g., `Group1` vs `Group2`). Statistical software (like *DESeq2*) then calculates the **fold-change (FC)** between these counts to determine if a region *significantly* opened or closed in response to a treatment.

6.3.3 From BAM files to differential peaks

Here is the step-by-step bioinformatics pipeline used to generate those exact values from the raw data:

1. Creating a consensus peak set: to compare the samples, bioinformatics tools merge all the individual peak files across all replicates and conditions into a single “master list” of non-overlapping genomic windows. This ensures we are comparing the exact same spatial coordinates across the board.
2. Generating the count matrix: once the master list of regions is defined, we go back to the original aligned sequence alignments (the BAM files). Using tools like *featureCounts* or *bedtools*, we count exactly how many reads fall into each consensus region for every single sample. This generates a large matrix where the rows are the genomic regions and the columns are the individual samples.
3. Statistical modeling: this raw count matrix is then fed into differential expression software, most commonly *DESeq2* or *edgeR*.
 - **Normalization:** the software mathematically adjusts for differences in sequencing depth between the samples.

- **Testing:** it fits a negative binomial model to test if the difference in read counts between groups (e.g., Group 1 vs. Group 2) is statistically significant.
 - **Output:** the algorithm outputs a results table containing the Log fold-change (logFC), p-values, and adjusted p-values for every single region.
4. Constructing the **rowRanges**: finally, the data is assembled for visualization tools like **EnrichedHeatmap**, which relies on the Bioconductor ecosystem. The genomic coordinates (**chr**, **start**, **end**) form the core of the **rowRanges** object, and the statistical results (like the logFC and p-values) are tacked on as metadata columns. This creates a unified object where the visualization tool instantly knows both *where* the region is located and *how significantly* its accessibility changed.

6.3.4 Filtering the dataset

Based on standard bioinformatics practices, we want to filter for peaks that have a significant statistical change (*qvalue*) and a meaningful magnitude of change (*logFC*).

```
# qvalue is the FDR adjusted P value
#rd_atac <- rd_atac[abs(rd_atac$logFC) >= 0.5 & rd_atac$qvalue <= 0.1]
rd_atac_filtered = subset(rd_atac, abs(logFC) >= .5 & qvalue <= .1)

# Add an "ATAC_" prefix to all the metadata columns in rd_atac_filtered
colnames(elementMetadata(rd_atac_filtered)) <- paste(
  "ATAC", colnames(elementMetadata(rd_atac_filtered)), sep = "_")
```

Using the absolute value (**abs()**) is crucial here. It ensures we are keeping peaks that *gained* accessibility (logFC > 0.5) AND peaks that *lost* accessibility (logFC < -0.5). If we didn't use **abs()**, we would accidentally throw away all the regions that closed down!

```
rd_atac_filtered
```

GRanges object with 1933 ranges and 18 metadata columns:

	seqnames	ranges	strand	ATAC_annotation
	<Rle>	<IRanges>	<Rle>	<character>
[1]	chr1	3670547-3672665	*	Promoter (<=1kb)
[2]	chr1	4332510-4332710	*	Intron (ENSMUST00000..
[3]	chr1	4491755-4492573	*	Promoter (1-2kb)
[4]	chr1	4571186-4572423	*	Distal Intergenic
[5]	chr1	4785062-4786325	*	Promoter (<=1kb)
...	...	...	...	...
[1929]	chr2	181713633-181714134	*	Promoter (<=1kb)

[1930]	chr2	181714476-181715931	*		Promoter (<=1kb)
[1931]	chr2	181762779-181764180	*		Promoter (<=1kb)
[1932]	chr2	181764664-181764891	*		Promoter (1-2kb)
[1933]	chr2	181837244-181838655	*		Promoter (<=1kb)

	ATAC_geneChr	ATAC_geneStart	ATAC_geneEnd	ATAC_geneLength
	<integer>	<integer>	<integer>	<integer>
[1]	1	3214482	3671498	457017
[2]	1	4344146	4360314	16169
[3]	1	4492465	4493735	1271
[4]	1	4491390	4497354	5965
[5]	1	4773206	4785710	12505
...	...	...	...	...
[1929]	2	181715016	181720985	5970
[1930]	2	181715016	181720985	5970
[1931]	2	181763332	181827797	64466
[1932]	2	181763332	181827797	64466
[1933]	2	181837854	181857461	19608

	ATAC_geneStrand	ATAC_geneId	ATAC_transcriptId	ATAC_distanceToTSS
	<integer>	<character>	<character>	<numeric>
[1]	2	497097	ENSMUST00000070533.4	0
[2]	2	19888	ENSMUST00000027032.5	27604
[3]	2	20671	ENSMUST000000191939.1	1162
[4]	2	20671	ENSMUST000000192650.5	-73832
[5]	2	27395	ENSMUST000000130201.7	0
...	...	...	...	...
[1929]	1	18389	ENSMUST000000071585.9	-882
[1930]	1	18389	ENSMUST000000071585.9	0
[1931]	1	17932	ENSMUST000000081125.10	0
[1932]	1	17932	ENSMUST000000081125.10	1332
[1933]	1	245867	ENSMUST000000029116.13	0

	ATAC_ENSEMBL	ATAC_SYMBOL	ATAC_GENENAME	ATAC_anno
	<character>	<character>	<character>	<character>
[1]	ENSMUSG000000051951	Xkr4	X-linked Kx blood gr..	Promoter
[2]	ENSMUSG000000025900	Rp1	retinitis pigmentosa..	Intron
[3]	ENSMUSG000000025902	Sox17	SRY (sex determining..	Promoter
[4]	ENSMUSG000000025902	Sox17	SRY (sex determining..	Intergenic
[5]	ENSMUSG000000033845	Mrpl15	mitochondrial riboso..	Promoter
...	...	...	...	...
[1929]	ENSMUSG000000027584	Oprl1	opioid receptor-like 1	Promoter
[1930]	ENSMUSG000000027584	Oprl1	opioid receptor-like 1	Promoter
[1931]	ENSMUSG000000010505	Myt1	myelin transcription..	Promoter
[1932]	ENSMUSG000000010505	Myt1	myelin transcription..	Promoter
[1933]	ENSMUSG000000027589	Pcmt2	protein-L-isoasparta..	Promoter

	ATAC_logFC	ATAC_logCPM	ATAC_F	ATAC_PValue	ATAC_qvalue
	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
[1]	0.602984	10.45378	41.78446	6.58453e-08	4.81983e-07
[2]	0.855748	5.18424	3.68371	6.13517e-02	9.28785e-02
[3]	-0.620926	7.13469	9.14522	4.12368e-03	8.38540e-03
[4]	-0.511046	8.90885	15.15688	3.27158e-04	8.67115e-04
[5]	0.540908	8.63469	15.08892	3.36058e-04	8.82634e-04
...	...	...	...	...	...
[1929]	1.922716	6.37607	40.6570	8.88771e-08	6.28500e-07
[1930]	1.213349	7.47748	37.2944	2.22741e-07	1.40677e-06
[1931]	-2.192918	7.94745	146.4501	1.07660e-15	1.40573e-13
[1932]	-1.275275	5.54135	10.4487	2.30922e-03	4.97558e-03
[1933]	0.689247	7.92175	16.6788	1.81039e-04	5.14680e-04

seqinfo: 21 sequences from an unspecified genome; no seqlengths

6.3.5 ATAC-seq peak widths

Most of the peaks are around 1000 bp wide. We can check the distribution with a histogram.

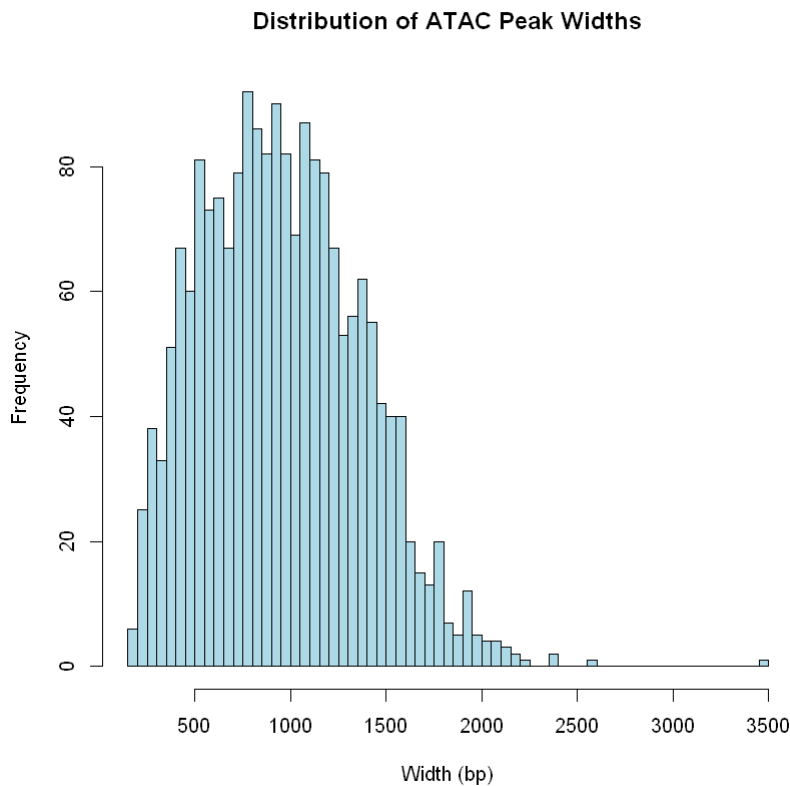
```
# median(lengths(rd_atac_filtered)) # `lengths` (plural) is an alias for `width`
cat(
  "Median peak width in the filtered dataset =",
  median(width(rd_atac_filtered)),
  '\n')

cat(
  "99% percentile in the filtered dataset =",
  quantile(width(rd_atac_filtered), .99))

hist(
  width(rd_atac_filtered),
  breaks = 50,
  main = "Distribution of ATAC Peak Widths",
  xlab = "Width (bp)",
  col = "lightblue",
  border = "black")
```

Median peak width in the filtered dataset = 932

99% percentile in the filtered dataset = 1985.88



The rows in the `SummarizedExperiment` represent actual biological features—specifically, regions where the chromatin was physically open (peaks). Indeed, an open chromatin region (like a promoter or enhancer) is naturally a few hundred base pairs wide. However, to compare E11.5 and E15.5, we need to merge the peaks from all samples together to create a unified “master list” (a consensus peak set). When we merge overlapping peaks from multiple different samples, the resulting master regions naturally stretch out, averaging around ~1,000 bp to ensure they cover the entire open zone for all samples.

6.3.6 Binning signal across a 2 kb window

For plotting the data, we can hence consider 1000 bp around the **peak-mid** that we find using `gr.mid()`, which returns the 1-bp center of each peak. It directly overwrites the data inside the `ranges` column.

```
# Finding mid of all peaks
mid_peaks <- gr.mid(rd_atac_filtered)

head(mid_peaks, 3)
```

GRanges object with 3 ranges and 18 metadata columns:

	seqnames	ranges	strand	ATAC_annotation	ATAC_geneChr
	<Rle>	<IRanges>	<Rle>	<character>	<integer>
[1]	chr1	3671606	*	Promoter (<=1kb)	1
[2]	chr1	4332610	*	Intron (ENSMUST000000..	1
[3]	chr1	4492164	*	Promoter (1-2kb)	1

	ATAC_geneStart	ATAC_geneEnd	ATAC_geneLength	ATAC_geneStrand	ATAC_geneId
	<integer>	<integer>	<integer>	<integer>	<character>
[1]	3214482	3671498	457017	2	497097
[2]	4344146	4360314	16169	2	19888
[3]	4492465	4493735	1271	2	20671

	ATAC_transcriptId	ATAC_distanceToTSS	ATAC_ENSEMBL	ATAC_SYMBOL
	<character>	<numeric>	<character>	<character>
[1]	ENSMUST000000070533.4	0	ENSMUSG000000051951	Xkr4
[2]	ENSMUST000000027032.5	27604	ENSMUSG000000025900	Rp1
[3]	ENSMUST000000191939.1	1162	ENSMUSG000000025902	Sox17

	ATAC_GENENAME	ATAC_anno	ATAC_logFC	ATAC_logCPM	ATAC_F
	<character>	<character>	<numeric>	<numeric>	<numeric>
[1]	X-linked Kx blood gr..	Promoter	0.602984	10.45378	41.78446
[2]	retinitis pigmentosa..	Intron	0.855748	5.18424	3.68371
[3]	SRY (sex determining..	Promoter	-0.620926	7.13469	9.14522

	ATAC_PValue	ATAC_qvalue
	<numeric>	<numeric>
[1]	6.58453e-08	4.81983e-07
[2]	6.13517e-02	9.28785e-02
[3]	4.12368e-03	8.38540e-03

seqinfo: 21 sequences from an unspecified genome; no seqlengths

6.3.7 Finding peak centers with `resize()`

There actually is a native way to do this in standard `GenomicRanges`, but it doesn't have a dedicated *midpoint* name. Instead, Bioconductor relies on its highly versatile `resize()` function.

```
# Finding mid of all peaks
mid_peaks <- resize(rd_atac_filtered, width = 1, fix = "center")

# adding names to peaks to give them identity
names(mid_peaks) <- paste("peak", 1:length(mid_peaks), sep = "_")

head(mid_peaks, 3)
```

GRanges object with 3 ranges and 18 metadata columns:

	seqnames	ranges	strand	ATAC_annotation	ATAC_geneChr
	<Rle>	<IRanges>	<Rle>	<character>	<integer>
peak_1	chr1	3671606	*	Promoter (<=1kb)	1
peak_2	chr1	4332610	*	Intron (ENSMUST000000..	1
peak_3	chr1	4492164	*	Promoter (1-2kb)	1
	ATAC_geneStart	ATAC_geneEnd	ATAC_geneLength	ATAC_geneStrand	
	<integer>	<integer>	<integer>	<integer>	
peak_1	3214482	3671498	457017	2	
peak_2	4344146	4360314	16169	2	
peak_3	4492465	4493735	1271	2	
	ATAC_geneId	ATAC_transcriptId	ATAC_distanceToTSS	ATAC_ENSEMBL	
	<character>	<character>	<numeric>	<character>	
peak_1	497097	ENSMUST00000070533.4	0	ENSMUSG00000051951	
peak_2	19888	ENSMUST00000027032.5	27604	ENSMUSG00000025900	
peak_3	20671	ENSMUST000000191939.1	1162	ENSMUSG00000025902	
	ATAC_SYMBOL	ATAC_GENENAME	ATAC_anno	ATAC_logFC	ATAC_logCPM
	<character>	<character>	<character>	<numeric>	<numeric>
peak_1	Xkr4	X-linked Kx blood gr..	Promoter	0.602984	10.45378
peak_2	Rp1	retinitis pigmentosa..	Intron	0.855748	5.18424
peak_3	Sox17	SRY (sex determining..	Promoter	-0.620926	7.13469
	ATAC_F	ATAC_PValue	ATAC_qvalue		
	<numeric>	<numeric>	<numeric>		
peak_1	41.78446	6.58453e-08	4.81983e-07		
peak_2	3.68371	6.13517e-02	9.28785e-02		
peak_3	9.14522	4.12368e-03	8.38540e-03		

seqinfo: 21 sequences from an unspecified genome; no seqlengths

As we saw in the previous notebook, a GRanges object is not a standard data frame. It strictly separates its data into two distinct parts: **Core Geometry** and **Metadata**. Because GRanges is an object-oriented structure (S4), we have to use specific “accessor” functions to extract the core geometry.

```
# Returns an IRanges object with start, end, and width
ranges(mid_peaks)
```

IRanges object with 1933 ranges and 0 metadata columns:

	start	end	width
	<integer>	<integer>	<integer>
peak_1	3671606	3671606	1
peak_2	4332610	4332610	1

peak_3	4492164	4492164	1
peak_4	4571804	4571804	1
peak_5	4785693	4785693	1
...	...	...	...
peak_1929	181713883	181713883	1
peak_1930	181715203	181715203	1
peak_1931	181763479	181763479	1
peak_1932	181764777	181764777	1
peak_1933	181837949	181837949	1

*Note: Because **GRanges** is an S4 object, its core architecture is rigidly locked. we physically cannot add a custom geometry column (like **midpeak**) next to **start** and **end**. The rules of the object state that a **GRanges** row can only represent exactly one genomic interval at a time.*

6.4 Preparing the continuous bigWig data

6.4.1 Loading data from bigWig

The SE aggregates the data into one single number per peak. All spatial information is destroyed. It cannot tell us if those 500 reads were tightly piled up exactly in the middle, or spread out evenly across the region.

A bigWig file contains the continuous, base-pair-by-base-pair signal coverage across the entire genome. To draw an **EnrichedHeatmap**, we don't want just one flat color for the whole peak. We want to see the actual biological shape of the chromatin accessibility (e.g., a sharp spike at the center, tapering off 1000 bp away).

bigWig files are compressed, indexed, binary format used for efficiently displaying continuous data, like genomic signal data, in genome browsers. Here, we read in ATAC-seq bigWig files, **filters** the data to specific chromosomes, normalizes signal intensity around genomic regions of interest (peak centers), and saves the resulting matrices for downstream visualization.

```
# Find all files in the "data" folder that contain "ATAC" in their name.
# full.names = TRUE ensures we get the full file path (e.g., "data/ATAC_11half.bw")
atac_files <- list.files("data", pattern = "ATAC", full.names = TRUE)

# Clean up the filenames to use as names for our list; basename() extracts just the filename
# from the path; gsub() replaces the ".bw" extension with nothing (""), leaving a clean name
names(atac_files) <- gsub(pattern = "\\..bw", replacement = "", x = basename(atac_files))

# Loop over every file in our list using lapply()
atac_bw <- lapply(atac_files, function(x){
```

```

# Read the bigWig file from the hard drive into R as a GRanges object
a <- rtracklayer::import(x)

# Filter the GRanges object to ONLY keep data mapping to chr1 or chr2
a <- a[seqnames(a) %in% c("chr1", "chr2")]

# Return the filtered GRanges object to be saved in the atac_bw list
return(a)
})

```

Here, we load the entire .bw file into memory using `rtracklayer::import(x)` and then filter it down to *chr1* and *chr2*. However, real-world whole-genome bigWig files can be massive and will crash a standard computer's RAM. To do this efficiently, we can pass a `Ranges` object to the `which=` argument of the `import()` function, for example, `import(x, which = my_regions)`. This forces R to dynamically extract only the data for those specific coordinates directly from the hard drive, saving huge amounts of memory and time.

```

atac_bw <- lapply(atac_files, function(x){

  # Peek at the file's index to get the lengths of the chromosomes
  bw_file <- rtracklayer::BigWigFile(x)
  genome_info <- seqinfo(bw_file)

  # Create a GRanges object representing the full length of chr1 and chr2
  # as() converts the Seqinfo object directly into a GRanges object spanning the chromosomes
  target_chrs <- as(genome_info[c("chr1", "chr2")], "GRanges")

  # Read ONLY chr1 and chr2 from the hard drive
  a <- rtracklayer::import(x, which = target_chrs)

  return(a)
})

```

Since the only reason we are loading this data is to draw a heatmap spanning 1000 bp up/downstream of our `mid_peaks`, we don't even need the rest of *chr1* and *chr2*. We can use our `mid_peaks` object to fetch only the exact 2000 bp windows we need, reducing the memory footprint from millions of rows per sample down to just a tiny fraction of that.

```

# Define the actual 2000 bp windows we need for the heatmap
heatmap_windows <- resize(mid_peaks, width = 2000, fix = "center")

```

```

atac_bw <- lapply(atac_files, function(x){

  # Read ONLY the 2000 bp windows around our peaks from the hard drive
  a <- rtracklayer::import(x, which = heatmap_windows)

  return(a)
})

```

When imported into R, each individual bigWig file is represented as a **GRanges** object. Because we loaded multiple files at once, our `atac_bw` variable is a list of **GRanges** objects.

```
atac_bw
```

```
$ATAC_11half
```

```
GRanges object with 72531 ranges and 1 metadata column:
```

	seqnames	ranges	strand	score
	<Rle>	<IRanges>	<Rle>	<numeric>
[1]	chr1	3670606-3670650	*	0.195004
[2]	chr1	3670651-3670700	*	0.176254
[3]	chr1	3670701-3670750	*	0.168754
[4]	chr1	3670751-3670800	*	0.303756
[5]	chr1	3670801-3670850	*	0.382508
...	...	...	...	...
[72527]	chr2	181838651-181838700	*	0.0262506
[72528]	chr2	181838701-181838750	*	0.0000000
[72529]	chr2	181838751-181838800	*	0.0412509
[72530]	chr2	181838801-181838850	*	0.0450010
[72531]	chr2	181838851-181838948	*	0.0112502

```
-----
seqinfo: 66 sequences from an unspecified genome
```

```
$ATAC_15half
```

```
GRanges object with 72127 ranges and 1 metadata column:
```

	seqnames	ranges	strand	score
	<Rle>	<IRanges>	<Rle>	<numeric>
[1]	chr1	3670606-3670650	*	0.853376
[2]	chr1	3670651-3670700	*	0.861825
[3]	chr1	3670701-3670750	*	0.828028
[4]	chr1	3670751-3670800	*	1.064610
[5]	chr1	3670801-3670850	*	1.030810
...	...	...	...	...

```

[72123]      chr2 181838701-181838750      * | 0.00844927
[72124]      chr2 181838751-181838800      * | 0.07604340
[72125]      chr2 181838801-181838850      * | 0.14363800
[72126]      chr2 181838851-181838900      * | 0.12673900
[72127]      chr2 181838901-181838948      * | 0.08449270
-----

```

seqinfo: 66 sequences from an unspecified genome

```

cat(
  "Median width = ",
  median(width(atac_bw$ATAC_11half))
)

```

Median width = 50

Unlike the SE peaks, the rows in a bigWig file are artificial mathematical bins, not biological features.

To compress the file, the genome is artificially chopped into an invisible grid of 50 bp windows, likely using a tool like deepTools **bamCoverage** with a command like `--binSize 50`. The software calculates the average signal for that 50 bp window and saves it as a single chunk.

Note that there are some exceptions. bigWig uses a compression trick called “run-length encoding.” If two consecutive 50 bp bins happen to have the exact same signal score, the bigWig file merges them into a single 100 bp row. And if a bin is completely empty (zero reads), it is completely deleted from the file to save space, which can cause the surrounding bins to look chopped or offset.

6.4.2 Normalization

Next, we calculate the normalized signals into the area of interest.

In **EnrichedHeatmap**, the genomic region is chopped up into tiny windows (bins). The algorithm needs to calculate a single “average” signal for each bin. Because bigWig files are highly compressed, they completely omit regions that have zero reads.

For example, if a 20 bp bin has a signal of 10 covering 5 base pairs, and the bigWig file is “missing” data for the other 15 base pairs, `w0` correctly assumes the missing data is zero. The `mean_mode = "w0"` calculates the weighted average as: $(10 \times 5\text{bp}) + (0 \times 15\text{bp}) / 20\text{bp} = 2.5$.

If we used a different mode, the algorithm might ignore the missing data and claim the average signal is 10, artificially inflating the signal!

And because the 99th percentile of our ATAC peak widths is approximately 2000 bp, we extend our analysis window by 1 kb upstream and 1 kb downstream (`extend = 1000`) from the peak center. This 2 kb total window ensures that the complete biological signal for 99% of our regions is fully captured in the heatmap without clipping the edges.

Please check `?normalizeToMatrix` or [the official documentation](#) for details of this function.

```
mat_AS <- lapply(atac_bw, FUN = function(x) {
  normalizeToMatrix(
    x, mid_peaks, # bigWig data, and target "anchor" (point 0)
    mean_mode = "w0",
    extend = 1000,
    value_column = "score",
    include_target = TRUE, # includes the 1 bp anchor itself
    w = 20, # window size
    smooth = T, # LOESS smoothing curve
    background = 0
  )
})
```

Warning message:

"Width of `target` are all 1, `include\_target` is set to `FALSE`."

Warning message:

"Width of `target` are all 1, `include\_target` is set to `FALSE`."

We imported two bigWig files, "ATAC_11half.bw" (E11.5) and "ATAC_15half.bw" (E15.5), therefore the list returned by `lapply()` will have exactly two matrix elements.

```
str(mat_AS)
```

List of 2

```
$ ATAC_11half: 'normalizedMatrix' num [1:1933, 1:100] 0.2039 0.1108 0.0247 0.0473 0.052 ...
..- attr(*, "upstream_index")= int [1:50] 1 2 3 4 5 6 7 8 9 10 ...
..- attr(*, "target_index")= int(0)
..- attr(*, "downstream_index")= int [1:50] 51 52 53 54 55 56 57 58 59 60 ...
..- attr(*, "extend")= num [1:2] 1000 1000
..- attr(*, "smooth")= logi TRUE
..- attr(*, "signal_name")= chr "x"
..- attr(*, "target_name")= chr "mid_peaks"
..- attr(*, "target_is_single_point")= logi TRUE
..- attr(*, "background")= num 0
..- attr(*, "signal_is_categorical")= logi FALSE
```

```

..- attr(*, "dimnames")=List of 2
.. ..$ : chr [1:1933] "peak_1" "peak_2" "peak_3" "peak_4" ...
.. ..$ : chr [1:100] "u1" "u2" "u3" "u4" ...
$ ATAC_15half: 'normalizedMatrix' num [1:1933, 1:100] 0.8644 0 0.0242 0.0978 0.2013 ...
..- attr(*, "upstream_index")= int [1:50] 1 2 3 4 5 6 7 8 9 10 ...
..- attr(*, "target_index")= int(0)
..- attr(*, "downstream_index")= int [1:50] 51 52 53 54 55 56 57 58 59 60 ...
..- attr(*, "extend")= num [1:2] 1000 1000
..- attr(*, "smooth")= logi TRUE
..- attr(*, "signal_name")= chr "x"
..- attr(*, "target_name")= chr "mid_peaks"
..- attr(*, "target_is_single_point")= logi TRUE
..- attr(*, "background")= num 0
..- attr(*, "signal_is_categorical")= logi FALSE
..- attr(*, "dimnames")=List of 2
.. ..$ : chr [1:1933] "peak_1" "peak_2" "peak_3" "peak_4" ...
.. ..$ : chr [1:100] "u1" "u2" "u3" "u4" ...

```

Inside each slot of that list is a special matrix (officially called a `normalizedMatrix` object by the `EnrichedHeatmap` package).

If we look at one of these matrices, its structure represents the physical heatmap grid:

- The Rows: Every row is one of the ATAC peaks (there will be thousands of rows).
- The Columns: Every column represents one of the 20 bp bins from our 2000 bp extended window (so there will be ~100 columns; 50 upstream and 50 downstream).
- The Cells (Values): The actual numbers inside the matrix are the `w0` weighted average signal scores for that specific peak at that specific 20 bp window.

```

cat(
  "Number of columns in the matrix =",
  ncol(mat_AS[[1]]))

mat_AS[[1]][1:5,]

```

Number of columns in the matrix = 100

	u1	u2	u3	u4	u5
peak_1	0.20391640	0.18583284	0.17312667	0.16737224	0.1694
peak_2	0.11075053	0.11270090	0.11794239	0.12587906	0.1370
peak_3	0.02466337	0.02390869	0.02419351	0.02498987	0.0267
peak_4	0.04730565	0.04142725	0.03701785	0.03425858	0.0330
peak_5	0.05196178	0.04576406	0.04102784	0.03792034	0.0369

A normalizedMatrix: 5 × 100 of type dbl

In conclusion, the process started with a BAM file which massive file contains every single/**individual DNA fragment** the sequencer read, mapped to exact coordinates on the genome.

To make the data manageable, bioinformatics tools “flatten” the BAM file as a **continuous pile-up of fragments**. Instead of tracking every single fragment, they just walk down the genome and count: “How many fragments are piled up at base 1? At base 2?”

We can’t plot the entire genome in one heatmap. So, the `normalizeToMatrix()` function acts like a cookie-cutter. It goes into the bigWig file, cuts out the 2000 bp region around every single ATAC peak, and aligns them all perfectly by their center point. Then, it chops those regions into 20-bp bins to create a clean **mathematical grid**.

6.5 Building the EnrichedHeatmap

6.5.1 Basic EnrichedHeatmap visualization

[Enriched heatmap](#) is a special type of heatmap which visualizes the *enrichment of genomic signals* over specific target regions.

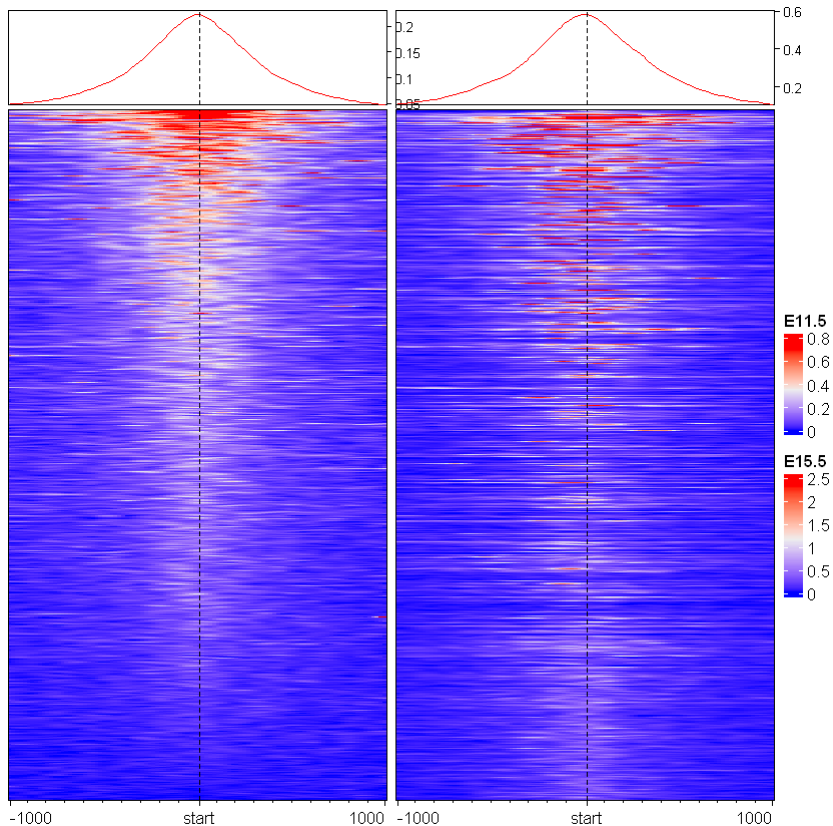
```
(  
  EnrichedHeatmap(mat = mat_AS$ATAC_11half, name = "E11.5")  
+ # joining of different EnrichedHeatmaps  
  EnrichedHeatmap(mat = mat_AS$ATAC_15half, name = "E15.5")  
)
```

The automatically generated colors map from the 1st and 99th of the values in the matrix. There are outliers in the matrix whose patterns might be hidden by this color mapping. You can manually set the color to ``col`` argument.

Use ``suppressMessages()`` to turn off this message.

The automatically generated colors map from the 1st and 99th of the values in the matrix. There are outliers in the matrix whose patterns might be hidden by this color mapping. You can manually set the color to ``col`` argument.

Use ``suppressMessages()`` to turn off this message.

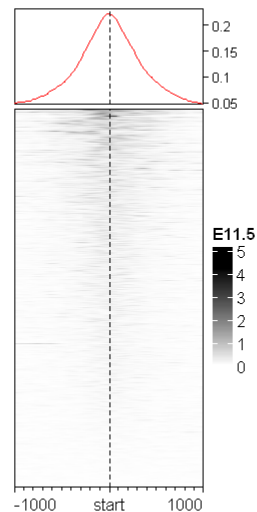


6.5.2 Changing color and size

Notice the warning message above about the 1st and 99th percentiles? Most normal ATAC peaks might have a “coverage score” of around 20 to 100 reads. However, the genome contains a few “extreme” regions (often repeats, artifacts, or super-enhancers) that pull in thousands of reads.

By default, `EnrichedHeatmap` assigns the darkest red color to the absolute highest number in our entire matrix.

```
EnrichedHeatmap(
  mat = mat_AS$ATAC_11half,      # normalized matrix
  name = "E11.5",                # Name for the plot
  # col = c("yellow", "black"),  # Colors for low to high values
  col = c("#ffffff", "black"),   # Colors for low to high values
  width = unit(4, "cm"),         # Width of the heatmap
  height = unit(8, "cm")        # Height of the heatmap
)
```



6.5.3 Color based on quantile

Because that maximum number is so ridiculously high, the color scale is stretched too far. All of our normal, biologically real peaks (the 20-100 values) get squashed into the bottom 1% of the color scale, making the entire plot look washed out and white.

We can fix it by **capping** the color scale at the 99th percentile. Instead of letting a few extreme peaks with 5,000 reads wash out the color scale, we cap the darkest red at 100 reads. Any region with more than 100 reads is just colored dark red, revealing the contrast in our normal peaks.

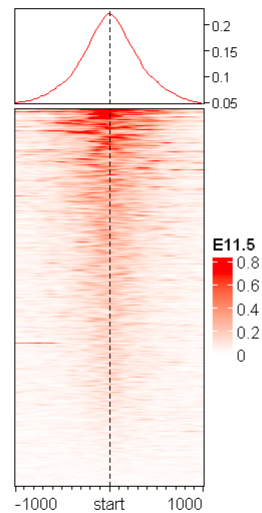
```
# Taking data between 1 and 99 percentile
col_fun <- colorRamp2(
  quantile(mat_AS$ATAC_11half, c(0.01, 0.99)),
  c("white", "red"))

EnrichedHeatmap(
  mat = mat_AS$ATAC_11half,
```

```

name = "E11.5",
col = col_fun,
width = unit(4, "cm"),
height = unit(8, "cm")
)

```



There are two main reasons why the dark red signal is shifted to the left (upstream) and isn't perfectly anchored at the mid line:

1. Earlier we used `gr.mid()` to find the exact mathematical center of the peaks. However, `rd_atac` is a consensus peak list (meaning peaks from E11.5 and E15.5 were merged together to make a unified map). If we merge two overlapping peaks, the mathematical midpoint of that new giant region is usually not the same as the biological summit (the spot with the highest read pile-up) for our specific sample. For the E11.5 sample, the actual "hottest" part of the open chromatin just happened to be on the left side of that merged mathematical window.
2. Chromatin accessibility is rarely a perfect, symmetrical bell curve. If a peak is at a promoter, the transcription machinery (like RNA Polymerase) and the +1 nucleosome

push the open DNA into an asymmetrical shape. But if a Transcription Factor (TF) is bound to the DNA, the ATAC-seq enzyme (Tn5) cannot cut exactly where the protein is sitting. It has to cut on the sides of it. If the TF is bound slightly off-center, we will see a heavy red signal shifted to one side.

6.5.4 Customizing heatmap aesthetics for publication

Now that we have fixed our color scale, we can upgrade the rest of the plot from a default visual into a publication-ready figure. In the code block below, we are applying several aesthetic tweaks to make the data much easier to read:

- Cleaning up the legend (`heatmap_legend_param`): instead of letting R guess the legend ticks, we mathematically calculate the 1st percentile, the 99th percentile, and the exact midpoint. We then force the legend to display only these three clean, rounded numbers.
- Biological x-axis (`axis_name`): we replace the default mathematical bin coordinates with clear biological labels (`-1kb`, `mid`, `1kb`).
- Adding a summary profile (`top_annotation`): we add a line plot to the very top of the heatmap using `anno_enriched()`. This incredibly useful annotation displays the *average* signal across all peaks, giving us a quick bird’s-eye view of the chromatin accessibility profile.
- General polishing: we lock in the exact height and width of the heatmap, adjust font sizes so they aren’t visually overwhelming, and add a subtle background color to our column title.

Note that when creating complex figures like heatmaps in a Jupyter Notebook, we might notice that the output sometimes looks squashed, the text overlaps, or the legends get cut off at the edges. This happens because Jupyter has a default, invisible “canvas” size (usually 7×7 inches) that it uses to draw all plots. Because ComplexHeatmap is an advanced package designed for publication, it tries to draw elements using strict physical dimensions (e.g., forcing a column to be exactly 3 mm wide). If the total size of our heatmap doesn’t match Jupyter’s default 7x7 canvas, the final image gets distorted to make it fit. To fix this, we can explicitly tell the notebook’s plotting engine to change the size of the “paper” it is drawing on to perfectly frame our heatmap.

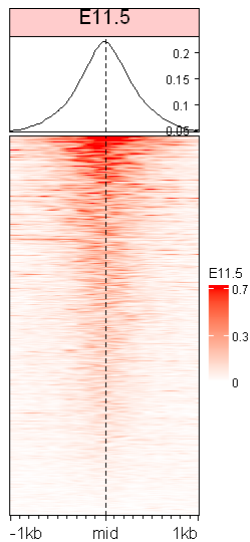
```
# Change Jupyter's global canvas size (measured in inches)
options(repr.plot.width = 3.5, repr.plot.height = 4.5)
```

```
# We first change the color legend to display only 3 values
vmin <- as.numeric(quantile(mat_AS$ATAC_11half, c(0.01)))
vmax <- as.numeric(quantile(mat_AS$ATAC_11half, c(0.99)))
vmid <- (vmin + vmax) / 2
legend_ticks <- c(vmin, vmid, vmax)
```

```

EnrichedHeatmap(
  mat = mat_AS$ATAC_11half,
  name = "E11.5",
  col = col_fun, # re-use the previous custom colormap
  width = unit(4, "cm"),
  height = unit(8, "cm"),
  column_title = "E11.5",
  column_title_gp = gpar( # 'gp' and `gpar` for graphic parameter
    fontsize = 12,
    fill = "#ffcccc"),
  axis_name = c("-1kb", "mid", "1kb"),
  heatmap_legend_param = list(
    at = legend_ticks,
    labels = round(legend_ticks, digits = 1),
    title_gp = gpar(fontsize = 8),
    labels_gp = gpar(fontsize = 7)
  ),
  top_annotation = HeatmapAnnotation(
    lines = anno_enriched(
      height = unit(2, "cm"),
      gp = gpar(
        lwd = 0.7,
        fontsize = 5),
      axis_param = list(
        side = "right",
        facing = "inside",
        gp = gpar(
          fontsize = 7,
          col = "black",
          lwd = 0.4
        )
      )
    )
  )
)

```



6.6 Splitting and clustering

6.6.1 Stratifying the heatmap by direction of change

Right now, all of our ATAC peaks are bundled together. While we can clearly see a general peak of accessibility at the center, this single block hides the dynamic changes occurring between our experimental conditions.

To get a better understanding of the biology, we will use our `logFC` values to split the heatmap into two distinct clusters: regions that *gained accessibility* and regions that *lost accessibility*. This allows us to visually confirm our statistical results and track how different parts of the genome behave.

```
split_change <- ifelse(
  mid_peaks$ATAC_logFC > 0,
  yes = "Increased accessibility", # TRUE
  no  = "Decreased accessibility")

names(split_change) <- names(mid_peaks)

str(split_change)
```

```
Named chr [1:1933] "Increased accessibility" "Increased accessibility" ...
- attr(*, "names")= chr [1:1933] "peak_1" "peak_2" "peak_3" "peak_4" ...
```

```

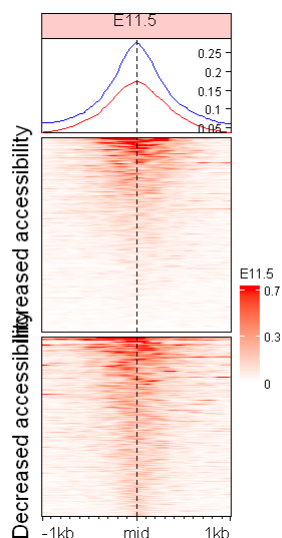
# Define cluster colors
cluster_colors <- c(
  "Increased accessibility" = "red",
  "Decreased accessibility" = "blue")

# Make sure split_change has levels matching the color names
split_change <- factor(
  split_change,
  levels = names(cluster_colors))

EnrichedHeatmap(
  mat = mat_AS$ATAC_11half,
  name = "E11.5",
  row_split = split_change,
  col = col_fun,
  width = unit(4, "cm"),
  height = unit(8, "cm"),
  column_title = "E11.5",
  column_title_gp = gpar(
    fontsize = 10,
    fill = "#ffcccc"),
  axis_name = c("-1kb", "mid", "1kb"),
  heatmap_legend_param = list(
    at = legend_ticks,
    labels = round(legend_ticks, digits = 1),
    title_gp = gpar(fontsize = 8),
    labels_gp = gpar(fontsize = 7)
  ),
  top_annotation = HeatmapAnnotation(
    lines = anno_enriched(
      gp = gpar(col = cluster_colors),
      height = unit(2, "cm"),
      axis_param = list(
        side = "right",
        facing = "inside",
        gp = gpar(
          fontsize = 7,
          lwd = 0.4
        )
      )
    )
  )
)

```

)



6.6.2 Setting categorical levels with factor()

`ComplexHeatmap` is very smart, but if the data order doesn't match our color order, the package can sometimes get confused when building the row annotations and legends.

By default, if we give R raw text, it will almost always sort it alphabetically. Because “Decreased” starts with D and “Increased” starts with I, R's default behavior is to put “Decreased accessibility” at the very top of the heatmap and legend.

The `factor()` function takes raw text and converts it into a strict **categorical** variable. If we simply passed the raw text, R would automatically do this conversion for us under the hood, but it would blindly sort the categories alphabetically (putting ‘Decreased’ before ‘Increased’)

By manually setting `levels = names(cluster_colors)`, we prevent R from guessing the layout. We are telling R that the strict, *permanent order* of these categories is exactly the order we defined in our color list. We explicitly used `factor()` to override the alphabet and force ‘Increased’ to the top of our heatmap.

6.7 Functional programming

6.7.1 Creating a reusable heatmap function

Because we have multiple samples to analyze, copying and pasting that massive block of plotting code is inefficient. Instead, it is best practice to wrap our code into a custom **function** so we

can easily generate matching heatmaps for any dataset.

```
make_EH <- function(
  norm_mat,
  hm_name,
  split_rows = NULL,
  heatmap_cols = c("white", "red"),
  col_fill = "#ffcccc",
  line_colors = cluster_colors) {

  # 1. Define color scale based on 1st and 99th percentiles
  col_fun <- colorRamp2(
    quantile(norm_mat, c(0.01, 0.99)),
    heatmap_cols)

  # 2. Calculate ticks for the legend
  vmin <- as.numeric(quantile(norm_mat, c(0.01)))
  vmax <- as.numeric(quantile(norm_mat, c(0.99)))
  vmid <- (vmin + vmax) / 2
  legend_ticks <- c(vmin, vmid, vmax)

  # 3. Generate and return the heatmap
  EnrichedHeatmap(
    mat = norm_mat,
    name = hm_name,
    row_split = split_rows,
    col = col_fun,
    width = unit(4, "cm"),
    height = unit(8, "cm"),
    column_title = hm_name,
    column_title_gp = gpar(fontsize = 10, fill = col_fill),
    axis_name = c("-1kb", "mid", "1kb"),
    heatmap_legend_param = list(
      at = legend_ticks,
      labels = round(legend_ticks, digits = 1),
      title_gp = gpar(fontsize = 8),
      labels_gp = gpar(fontsize = 7)
    ),
    top_annotation = HeatmapAnnotation(
      lines = anno_enriched(
        gp = gpar(col = line_colors),
        height = unit(2, "cm"),
        axis_param = list(
```

```

        side = "right",
        facing = "inside",
        gp = gpar(
            fontsize = 7,
            lwd = 0.4
        )
    )
)
)
)
}

```

6.7.2 Making EnrichedHeatmaps for both ATAC samples

Importantly, this allows us to generate independent `EnrichedHeatmap` objects for each sample. Once we have those individual objects, we can easily concatenate them and use the `draw()` function to plot them beautifully side-by-side.

```

eh_11h <- make_EH(
  norm_mat = mat_AS$ATAC_11half,
  split_rows = split_change,
  hm_name = "E11.5")

eh_15h <- make_EH(
  norm_mat = mat_AS$ATAC_15half,
  split_rows = split_change,
  hm_name = "E15.5",
  col_fill = "#87cefa")

```

6.7.3 Resetting the canvas for larger plots

Remember the `options()` command we used earlier to shrink our Jupyter canvas? Because that was a global setting, our notebook is currently stuck drawing on a tiny 3.5 x 4.5 inch piece of “paper”.

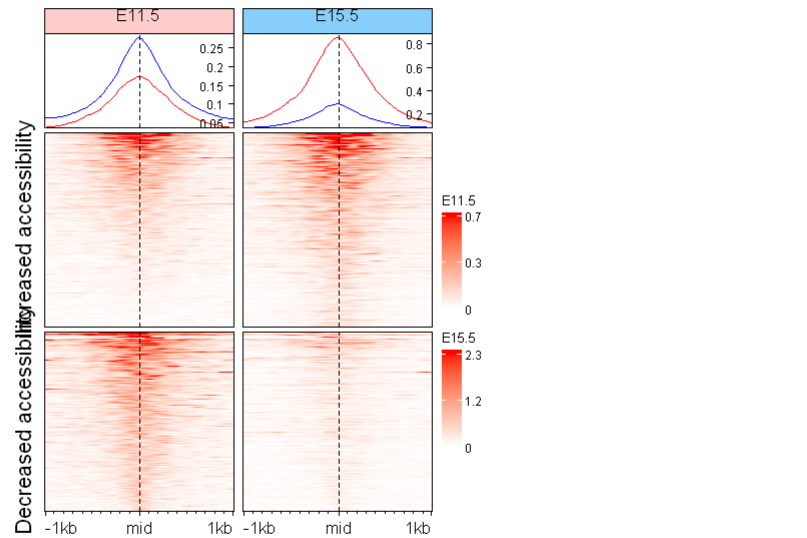
Now that we are combining multiple heatmaps and annotations into one large, unified figure, that tiny canvas is far too small. Before we generate this larger figure, we must reset the canvas back to its default.

```

# Reset Jupyter plot dimensions to the standard defaults (7x7 inches)
options(repr.plot.width = 7, repr.plot.height = 7)

```

```
draw(eh_11h + eh_15h)
```



By default, `ComplexHeatmap` appoints the first heatmap in the list as the “Main Heatmap”. Because `eh_11h` is the Main Heatmap here, it calculates the row order based on its signal intensity, and `eh_15h` is automatically forced to obey that exact order.

6.8 Combining multiple heatmaps

6.8.1 Generating the unified heatmaps and annotations

First, we will use our new `make_EH()` function to generate the heatmaps for both our E11.5 and E15.5 samples. Notice that we are intentionally leaving out the `split_rows` argument (letting it default to `NULL`). By removing this split, the heatmaps will initially generate as simple, continuous blocks rather than breaking into separate clusters.

Next, we want to *label* which peaks belong to which cluster. To make this visually intuitive, we will build a tiny, 3-millimeter wide “*mini-heatmap*” to act as a colored side-bar, making sure to reuse our exact `cluster_colors` palette to maintain visual consistency.

Finally, we can glue our samples and this new color bar together into a single figure. But how do we get our biological clusters back? Instead of trying to split each heatmap individually (which can cause frustrating alignment errors), we use a function of the `ComplexHeatmap` package: the `draw()` function. By passing our `split_change` groups directly into `draw()`, it acts as a global master controller, seamlessly splitting the entire combined figure in one shot.

```
eh_11h <- make_EH(  
  norm_mat = mat_AS$ATAC_11half,  
  # split_rows = NULL,  
  hm_name = "E11.5")  
  
eh_15h <- make_EH(  
  norm_mat = mat_AS$ATAC_15half,  
  hm_name = "E15.5",  
  col_fill = "#e6fff2")  
  
row_order_eh <- row_order(eh_11h)  
  
anno_hm <- Heatmap(  
  split_change,  
  # col = c("red", "blue"),  
  col = cluster_colors, # key-to-value color mapping  
  name = "Change",  
  show_row_names = FALSE,  
  show_column_names = FALSE,  
  width = unit(3, "mm"),  
  height = unit(8, "cm"),  
  row_order = row_order_eh,  
  # row_title_gp = gpar(fontsize = 0), # still showing 1-px text  
  row_title = NULL  
)
```

Warning message:

"The heatmap has not been initialized. You might have different results if you repeatedly execute this function, e.g. when `row_km/column_km` was set. It is more suggested to do as ``ht = draw(ht); row_order(ht)``."

6.8.2 Named vectors vs. simple vectors for colors

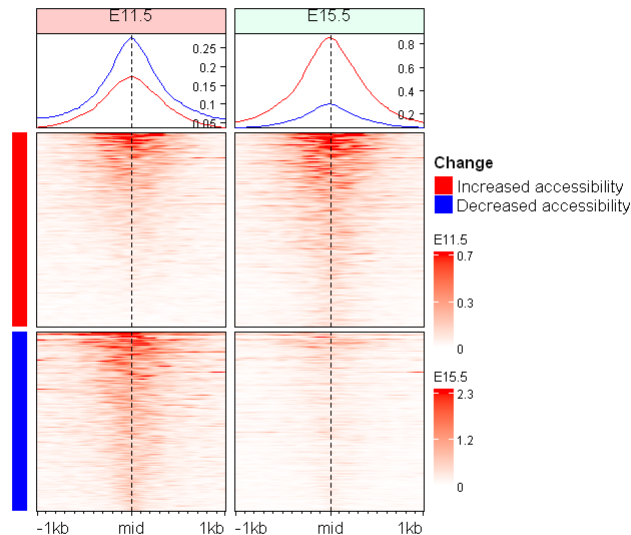
Because our side-bar annotation represents **discrete values** (a peak is strictly either “Increased” or “Decreased”), we must be careful about how we assign colors. Rather than passing a simple list of colors like `c("red", "blue")`, we pass our `cluster_colors` variable, which is a **named vector** (a dictionary).

According to the [official ComplexHeatmap documentation](#):

“If the matrix contains discrete values (either numeric or character), colors should be specified as a named vector to make it possible for the mapping from discrete values to colors. If there is no name for the color vector, the order of colors corresponds to the order of `unique(mat)`.”

By using our named vector, we mathematically lock the exact text keys to their specific colors, ensuring that “Increased” is always red and “Decreased” is always blue.

```
draw(  
  anno_hm + eh_11h + eh_15h,  
  split = split_change)
```



Standard `Heatmap()` always try to cluster (sort) their data by default. When we attached `anno_hm` to the plot, it started trying to sort the rows based on the text. By explicitly passing `row_order_eh` to the mini-heatmap, we “locked” it, forcing it to stop trying to sort itself and to just perfectly mirror the Main Heatmap, i.e. `eh_11h`.

Note that we can also just pass `cluster_rows = FALSE` into `anno_hm` and remove `row_order = row_order_eh`, and then use `draw(anno_hm + eh_11h + eh_15h, main_heatmap = "E11.5")`, or use `rowAnnotation()` instead (discussed at the end of the notebook) to achieve the exact same result.

6.9 Question 1

Can you make split the Enriched Heatmap based on the annotations?

Hint: `mid_peaks$ATAC_anno` contain annotations for the regions.

```
mid_peaks
```

GRanges object with 1933 ranges and 18 metadata columns:

	seqnames	ranges	strand	ATAC_annotation	ATAC_geneChr
	<Rle>	<IRanges>	<Rle>	<character>	<integer>
peak_1	chr1	3671606	*	Promoter (<=1kb)	1
peak_2	chr1	4332610	*	Intron (ENSMUST000000..	1
peak_3	chr1	4492164	*	Promoter (1-2kb)	1
peak_4	chr1	4571804	*	Distal Intergenic	1
peak_5	chr1	4785693	*	Promoter (<=1kb)	1
...	...	...	...	...	...
peak_1929	chr2	181713883	*	Promoter (<=1kb)	2
peak_1930	chr2	181715203	*	Promoter (<=1kb)	2
peak_1931	chr2	181763479	*	Promoter (<=1kb)	2
peak_1932	chr2	181764777	*	Promoter (1-2kb)	2
peak_1933	chr2	181837949	*	Promoter (<=1kb)	2
	ATAC_geneStart	ATAC_geneEnd	ATAC_geneLength	ATAC_geneStrand	
	<integer>	<integer>	<integer>	<integer>	
peak_1	3214482	3671498	457017	2	
peak_2	4344146	4360314	16169	2	
peak_3	4492465	4493735	1271	2	
peak_4	4491390	4497354	5965	2	
peak_5	4773206	4785710	12505	2	
...	...	...	...	...	
peak_1929	181715016	181720985	5970	1	
peak_1930	181715016	181720985	5970	1	

peak_1931	181763332	181827797	64466	1
peak_1932	181763332	181827797	64466	1
peak_1933	181837854	181857461	19608	1
	ATAC_geneId	ATAC_transcriptId	ATAC_distanceToTSS	
	<character>	<character>	<numeric>	
peak_1	497097	ENSMUST00000070533.4	0	
peak_2	19888	ENSMUST00000027032.5	27604	
peak_3	20671	ENSMUST00000191939.1	1162	
peak_4	20671	ENSMUST00000192650.5	-73832	
peak_5	27395	ENSMUST00000130201.7	0	
...	...	...	...	
peak_1929	18389	ENSMUST00000071585.9	-882	
peak_1930	18389	ENSMUST00000071585.9	0	
peak_1931	17932	ENSMUST00000081125.10	0	
peak_1932	17932	ENSMUST00000081125.10	1332	
peak_1933	245867	ENSMUST00000029116.13	0	
	ATAC_ENSEMBL	ATAC_SYMBOL	ATAC_GENENAME	ATAC_anno
	<character>	<character>	<character>	<character>
peak_1	ENSMUSG00000051951	Xkr4 X-linked Kx blood gr..	Promoter	
peak_2	ENSMUSG00000025900	Rp1 retinitis pigmentosa..	Intron	
peak_3	ENSMUSG00000025902	Sox17 SRY (sex determining..	Promoter	
peak_4	ENSMUSG00000025902	Sox17 SRY (sex determining..	Intergenic	
peak_5	ENSMUSG00000033845	Mrpl15 mitochondrial riboso..	Promoter	
...	...	...	...	...
peak_1929	ENSMUSG00000027584	Oprl1 opioid receptor-like 1	Promoter	
peak_1930	ENSMUSG00000027584	Oprl1 opioid receptor-like 1	Promoter	
peak_1931	ENSMUSG00000010505	Myt1 myelin transcription..	Promoter	
peak_1932	ENSMUSG00000010505	Myt1 myelin transcription..	Promoter	
peak_1933	ENSMUSG00000027589	Pcmdt2 protein-L-isoasparta..	Promoter	
	ATAC_logFC	ATAC_logCPM	ATAC_F	ATAC_PValue
	<numeric>	<numeric>	<numeric>	<numeric>
peak_1	0.602984	10.45378	41.78446	6.58453e-08
peak_2	0.855748	5.18424	3.68371	6.13517e-02
peak_3	-0.620926	7.13469	9.14522	4.12368e-03
peak_4	-0.511046	8.90885	15.15688	3.27158e-04
peak_5	0.540908	8.63469	15.08892	3.36058e-04
...	...	...	...	...
peak_1929	1.922716	6.37607	40.6570	8.88771e-08
peak_1930	1.213349	7.47748	37.2944	2.22741e-07
peak_1931	-2.192918	7.94745	146.4501	1.07660e-15
peak_1932	-1.275275	5.54135	10.4487	2.30922e-03
peak_1933	0.689247	7.92175	16.6788	1.81039e-04

seqinfo: 21 sequences from an unspecified genome; no seqlengths

```
head(mid_peaks$ATAC_anno, 10)
```

```
1. 'Promoter' 2. 'Intron' 3. 'Promoter' 4. 'Intergenic' 5. 'Promoter' 6. 'Intergenic' 7. 'Promoter'
8. 'Promoter' 9. 'Promoter' 10. 'Intergenic'
```

```
# Extract the genomic annotations (e.g., "Promoter", "Intron", "Intergenic")
# from the peak metadata and save it as a standalone vector.
```

```
split_anno <- mid_peaks$ATAC_anno
```

```
# Attach the original peak IDs as names to this new vector.
# This ensures every annotation label stays permanently linked to its peak
names(split_anno) <- names(mid_peaks)
```

```
head(split_anno)
```

```
peak\_1 'Promoter' peak\_2 'Intron' peak\_3 'Promoter' peak\_4 'Intergenic' peak\_5
'Promoter' peak\_6 'Intergenic'
```

Instead of building a new dataframe from scratch, we may simply extract the `GRanges` object's metadata table (`mcols`) as a standard dataframe and slice out the exact column we want. Because we are slicing the existing table, R automatically brings the peak IDs along as our row names.

*Note: `mcols()` returns a **DataFrame** (with a capital D and F), which is a highly complex, specialized object built by Bioconductor to hold biological data types (like raw DNA sequences) that a normal table cannot hold. However, visualization packages expect R's standard, universally accepted **data.frame**. Therefore, we must use `as.data.frame()` here to safely convert our heavy genomic object into a lightweight table that the heatmap can easily understand.*

```
# Extract the metadata as a dataframe, and slice the specific column
# (drop = FALSE ensures it stays a 2D dataframe instead of collapsing)
```

```
split_anno_df <- as.data.frame(
  mcols(mid_peaks))[, "ATAC_anno", drop = FALSE]
```

```
head(split_anno_df)
```

		ATAC_anno <chr>
A data.frame: 6 × 1	peak_1	Promoter
	peak_2	Intron
	peak_3	Promoter
	peak_4	Intergenic
	peak_5	Promoter
	peak_6	Intergenic

However, in base R, when we pull a column out of a data frame or a `GRanges` object (`split_anno_df$ATAC_anno`), R stubbornly strips off the row names and just hands us a flat, nameless vector. To build a “Series” in base R, we have to manually map the index back onto the vector...

```
head(split_anno_df$ATAC_anno)
```

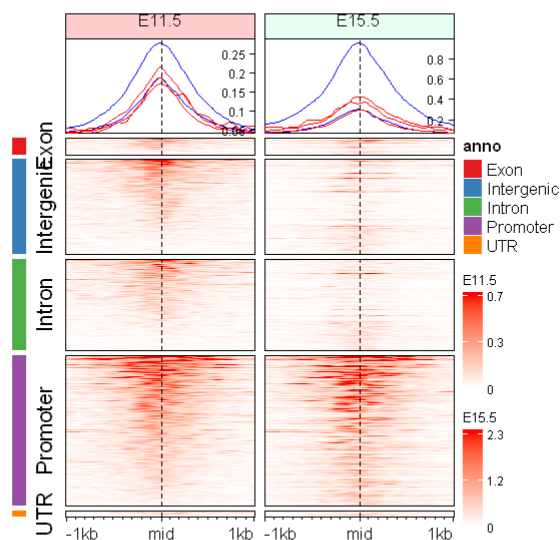
1. 'Promoter' 2. 'Intron' 3. 'Promoter' 4. 'Intergenic' 5. 'Promoter' 6. 'Intergenic'

```
# Dynamically generate a distinct color palette for all the distinct categories
anno_colors <- RColorBrewer::brewer.pal(n = length(unique(split_anno)), name = "Set1")
```

```
names(anno_colors) <- sort(unique(split_anno))
```

```
anno_an <- Heatmap(
  split_anno,
  col = anno_colors,
  name = "anno",
  show_row_names = FALSE,
  show_column_names = FALSE,
  width = unit(3, "mm"),
  height = unit(8, "cm"),
  cluster_rows = FALSE,
  row_title = NULL
)
```

```
draw(anno_an + eh_11h + eh_15h, split = split_anno, main_heatmap = "E11.5")
```



6.10 Refining the custom heatmap function

If we look closely at the summary profile at the top of our previous heatmaps, it only displayed two lines (Red and Blue), which perfectly represented the aggregate signal for our “Increased” and “Decreased” clusters.

However, by splitting our heatmap by Direction and/or Genomic Region, we create multiple distinct sub-clusters. Consequently, the top `anno_enriched()` profile will now attempt to calculate and display overlapping lines for every specific sub-cluster combination.

Because our `make_EH()` function explicitly passes `gp = gpar(col = cluster_colors)` to the top annotation, `EnrichedHeatmap` will be forced to **recycle colors**, trying to draw many lines using only our two predefined colors. To fix this, we will update our custom function to make the `line_colors` argument **optional** (defaulting to `NULL`). This gives us the flexibility to dynamically pass a *custom colormap* that perfectly matches the exact number of clusters we are analyzing.

Additionally, we must address a frustrating quirk in how the package handles text labels. When we pass our `split_anno_df` to the global `draw()` function, it automatically generates text labels for every single cluster combination. It then forcefully hands these text labels down to our Main Heatmap (`eh_11h`) and tells it to print them.

Putting `row_title = NULL` inside `draw()` only hides the *global* title for the entire figure; it does nothing to hide these individual slice titles. To truly remove this text clutter and let our color bars do the talking, we must tell the heatmap object itself to reject them. We will do this by adding `row_title = NULL` directly into our `make_EH()` function, and then we will quickly regenerate our heatmaps.

```
# Upgraded version of the original custom function
make_EH2 <- function(
  norm_mat,
  hm_name,
  split_rows = NULL,
  heatmap_cols = c("white", "red"),
  col_fill = "#ffcccc",
  line_colors = NULL # make it optional for more flexible splits
) {

  # Define color scale based on 1st and 99th percentiles
  col_fun <- colorRamp2(
    quantile(norm_mat, c(0.01, 0.99)),
    heatmap_cols)

  # Calculate ticks for the legend
  vmin <- as.numeric(quantile(norm_mat, c(0.01)))
  vmax <- as.numeric(quantile(norm_mat, c(0.99)))
  vmid <- (vmin + vmax) / 2
  legend_ticks <- c(vmin, vmid, vmax)

  # Generate and return the heatmap
  EnrichedHeatmap(
    mat = norm_mat,
    name = hm_name,
    row_split = split_rows,
    row_title = NULL, # remove cluster row text
    col = col_fun,
    width = unit(4, "cm"),
    height = unit(8, "cm"),
    column_title = hm_name,
    column_title_gp = gpar(fontsize = 10, fill = col_fill),
```

```

axis_name = c("-1kb", "mid", "1kb"),
heatmap_legend_param = list(
  at = legend_ticks,
  labels = round(legend_ticks, digits = 1),
  title_gp = gpar(fontsize = 8),
  labels_gp = gpar(fontsize = 7)
),
top_annotation = HeatmapAnnotation(
  lines = anno_enriched(
    gp = gpar(col = line_colors),
    height = unit(2, "cm"),
    axis_param = list(
      side = "right",
      facing = "inside",
      gp = gpar(
        fontsize = 7,
        lwd = 0.4
      )
    )
  )
)
)
)
)
}

```

```

# Create two new ComplexHeatmap objects with the updated custom function
eh_11h2 <- make_EH2(
  norm_mat = mat_AS$ATAC_11half,
  line_colors = anno_colors,
  hm_name = "E11.5")

```

```

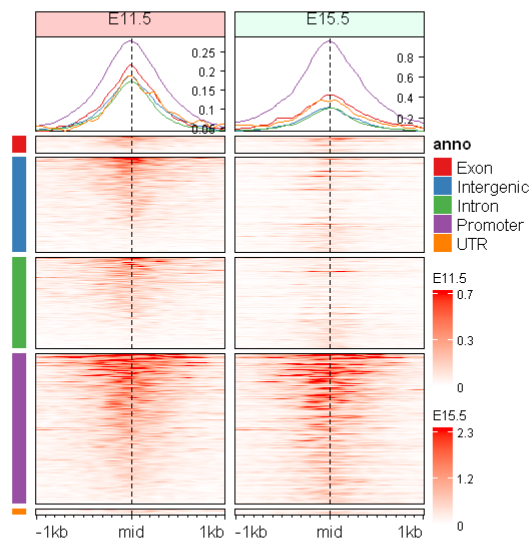
eh_15h2 <- make_EH2(
  norm_mat = mat_AS$ATAC_15half,
  line_colors = anno_colors,
  hm_name = "E15.5",
  col_fill = "#e6fff2")

```

```

draw(anno_an + eh_11h2 + eh_15h2, split = split_anno, main_heatmap = "E11.5")

```



6.11 Question 2

Can you make split the Enriched Heatmap based on the annotations and change in direction?

Hint: `mid_peaks$ATAC_anno` contain annotations for the regions. `mid_peaks$ATAC_logFC` contain sign of change.

```
split_anno_dir <- paste(
  mid_peaks$ATAC_anno, ifelse(
    mid_peaks$ATAC_logFC > 0,
    yes = "Inc",
    no = "Dec"))
```

paste() destroys the factor() levels we built earlier. Moreover, we create 6 brand new
categories ("Promoter Inc", "Promoter Dec", etc.). If we keep them separate, we can just
stitch the two existing 3mm color bars side-by-side, as shown in the next question.

```
names(split_anno_dir) <- names(mid_peaks)
```

```
head(split_anno_dir)
```

```
peak\_1 'Promoter Inc' peak\_2 'Intron Inc' peak\_3 'Promoter Dec' peak\_4 'Intergenic  
Dec' peak\_5 'Promoter Inc' peak\_6 'Intergenic Dec'
```

```
cols_an <- RColorBrewer::brewer.pal(  
  n = length(unique(split_anno_dir)),  
  name = "Paired")
```

```
# Create two new ComplexHeatmap objects with the updated custom function
```

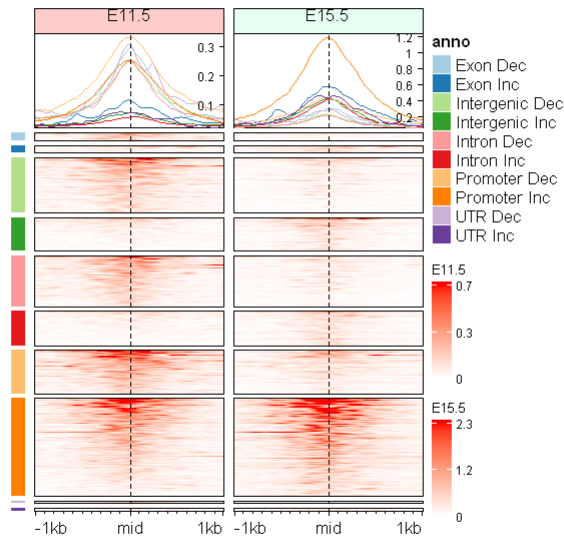
```
eh_11h3 <- make_EH2(  
  norm_mat = mat_AS$ATAC_11half,  
  line_colors = cols_an,  
  hm_name = "E11.5")
```

```
eh_15h3 <- make_EH2(  
  norm_mat = mat_AS$ATAC_15half,  
  line_colors = cols_an,  
  hm_name = "E15.5",  
  col_fill = "#e6fff2")
```

```
anno_an_dir <- Heatmap(  
  split_anno_dir,  
  col = cols_an,  
  name = "anno",  
  show_row_names = FALSE,  
  show_column_names = FALSE,  
  width = unit(3, "mm"),  
  height = unit(8, "cm"),  
  row_order = row_order_eh,  
  row_title = NULL  
)
```

There are 10 unique colors in the vector `col` and 10 unique values in `matrix`. `Heatmap()` will treat it as an exact discrete one-to-one mapping. If this is not what you want, slightly change the number of colors, e.g. by adding one more color or removing a color.

```
draw(anno_an_dir + eh_11h3 + eh_15h3, split = split_anno_dir)
```



6.12 Question 3

Can you make split the Enriched Heatmap based on the annotations and change in direction with separate color bars for annotation and direction of change?

6.12.1 The “Mini-Heatmap” method

To make this two-level split work while keeping our top profile lines colored strictly by Direction, we must generate brand new ATAC heatmap objects using the updated version of our custom function. Creating new objects allows us to pass the correct color dictionary and ensure no messy auto-generated slice text appears on the plot. Furthermore, we must explicitly set the `main_heatmap` argument inside our `draw()` function to prevent the mini-heatmaps from hijacking the layout and causing a sorting “tug-of-war.”

```
# Combine our two existing variables into a single data frame
split_anno_df <- data.frame(
  Annotation = mid_peaks$ATAC_anno,
  Direction = ifelse(mid_peaks$ATAC_logFC > 0, yes = "Inc", no = "Dec")
)

head(split_anno_df)
```

A data.frame: 6 × 2

	Annotation <chr>	Direction <chr>
1	Promoter	Inc
2	Intron	Inc
3	Promoter	Dec
4	Intergenic	Dec
5	Promoter	Inc
6	Intergenic	Dec

```
annot_colors <- c(
  "red", "blue", RColorBrewer::brewer.pal(
    n = length(unique(split_anno_df$Annotation)), name = "Set1"))

names(annot_colors) <- c(unique(split_anno_df$Direction), unique(split_anno_df$Annotation))

annot_colors
```

Inc 'red' **Dec** 'blue' **Promoter** '#E41A1C' **Intron** '#377EB8' **Intergenic** '#4DAF4A' **Exon** '#984EA3' **UTR** '#FF7F00'

```
# Define cluster colors
cluster_colors_short <- c(
  "Inc" = "red",
  "Dec" = "blue")

# Another heatmaps with top line colored by Direction
eh_11h2_dir <- make_EH2(
  norm_mat = mat_AS$ATAC_11half,
  line_colors = cluster_colors_short,
  hm_name = "E11.5")

eh_15h2_dir <- make_EH2(
  norm_mat = mat_AS$ATAC_15half,
  line_colors = cluster_colors_short,
```

```

hm_name = "E15.5",
col_fill = "#e6fff2")

anno_an_df <- Heatmap(
  split_anno_df,
  name = "anno",
  col = annot_colors,
  show_row_names = FALSE,
  show_column_names = FALSE,
  width = unit(3, "mm"),
  height = unit(8, "cm"),
  cluster_rows = FALSE,
  row_title = NULL
)

```

Warning message:

"The input is a data frame-like object, convert it to a matrix."

Warning message:

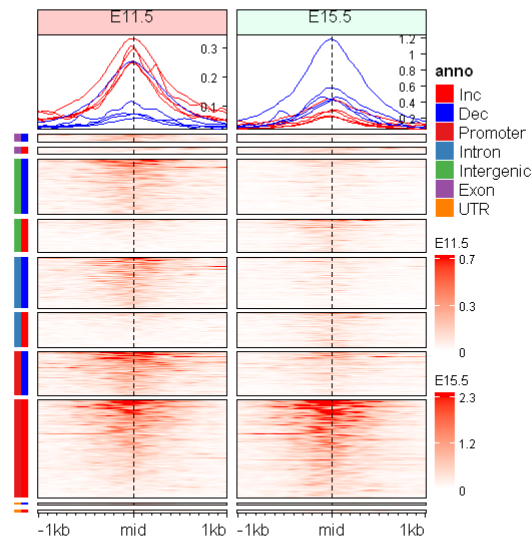
"Note: not all columns in the data frame are numeric. The data frame will be converted into a character matrix."

```

ht <- draw(
  anno_an_df + eh_11h2_dir + eh_15h2_dir,
  main_heatmap = "E11.5",
  split = split_anno_df)

ht

```



If we look closely at the E11.5 heatmap (left), the highest signal is clearly in the “Dec” (Blue) clusters. But the top profile draws those highest peaks in Red! Conversely, E15.5 has intense signal in the “Inc” (Red) clusters, but its top profile draws the highest peaks in Blue. It doesn’t correspond to the profiles observed in the previous plots.

Remember `cluster_colors_short` only contains two names (“Inc” and “Dec”). But because our final `draw()` function is going to split the heatmap by two variables (using the dataframe), the top profile is going to ask the dictionary for the color of the squashed string: “Inc, Promoter”. Because “Inc, Promoter” is not inside `cluster_colors_short`, the heatmap blindly recycles the red and blue colors, scrambling the lines.

```
# Extract the exact slice names
print(names(row_order(ht)))
```

```
[1] "Exon,Dec"      "Exon,Inc"      "Intergenic,Dec" "Intergenic,Inc"
[5] "Intron,Dec"    "Intron,Inc"    "Promoter,Dec"   "Promoter,Inc"
[9] "UTR,Dec"       "UTR,Inc"
```

6.12.2 The rowAnnotation method

Instead of forcing tiny heatmaps to act as labels, **ComplexHeatmap** provides a dedicated `rowAnnotation()` function. This is the cleanest, most professional way to annotate complex plots.

By passing our metadata as a `data.frame` along with a list of color dictionaries, the package handles the mapping for multiple variables simultaneously. Best of all, because it is structurally recognized as an “annotation” rather than a heatmap, it never fights for the Main Heatmap role. This keeps our code clean and guarantees our biological sorting remains perfectly intact.

Note: the visual order of the color bars is dictated entirely by the order of the columns inside the dataframe.

```
# Similarly using previously created vectors
split_anno_change_df <- data.frame(
  Annotation = split_anno,
  Change = split_change
)

head(split_anno_change_df)
```

	Annotation <chr>	Change <fct>
A data.frame: 6 × 2	peak_1 Promoter	Increased accessibility
	peak_2 Intron	Increased accessibility
	peak_3 Promoter	Decreased accessibility
	peak_4 Intergenic	Decreased accessibility
	peak_5 Promoter	Increased accessibility
	peak_6 Intergenic	Decreased accessibility

```
# Let ComplexHeatmap handle the mapping
my_annotations <- rowAnnotation(
  df = split_anno_change_df,
  col = list( # follow the order of the columns in the dataframe
    Annotation = anno_colors,
    Change = cluster_colors
  ),
  show_annotation_name = FALSE
)
```

We now pass the `draw()` function the `split_anno_change_df` dataframe with two columns. **ComplexHeatmap** goes row by row and groups the peaks by *every unique combination* of those

two variables (e.g., all peaks that are both “Increased” AND “Promoter” get grouped together). It therefore preserves our biological sorting (factors), perfectly syncs with the Color Bars, and calculates the aggregate signal for the top `anno_enriched()` lines much more accurately.

However, because this multi-variable split generates new combined cluster names (like "Inc, Promoter"), our original 2-color dictionary is no longer sufficient. To prevent the top profile from blindly recycling colors, we still need to build a custom color dictionary that explicitly maps our red and blue colors to these exact combination strings!

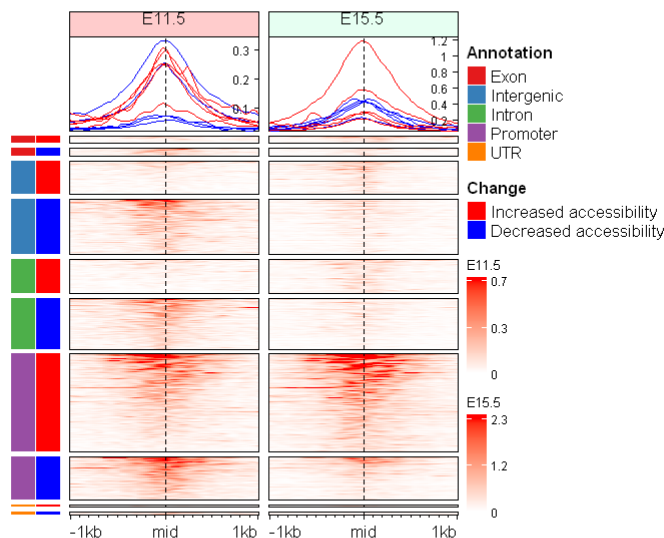
```
# Generate the exact slice names that ComplexHeatmap will create
# (By default, ComplexHeatmap pastes dataframe columns together separated by ", ")
slice_names <- unique(
  paste(split_anno_change_df$Annotation, split_anno_change_df$Change, sep = ", ")

# Build the TRUE, locked color dictionary
# If the slice name contains "Inc", it gets red. Otherwise, it gets blue.
true_line_colors <- ifelse(
  grepl("Inc", slice_names), "red", "blue")
names(true_line_colors) <- slice_names

# Generate the heatmaps using the foolproof dictionary
eh_11h3_dir <- make_EH2(
  norm_mat = mat_AS$ATAC_11half,
  line_colors = true_line_colors,
  hm_name = "E11.5"
)

eh_15h3_dir <- make_EH2(
  norm_mat = mat_AS$ATAC_15half,
  line_colors = true_line_colors,
  hm_name = "E15.5",
  col_fill = "#e6fff2"
)

draw(
  my_annotations + eh_11h3_dir + eh_15h3_dir,
  split = split_anno_change_df)
```



6.12.3 The power of native annotations

In our earlier attempts to add a color bar for the direction of change, we used what we can call the “Mini-Heatmap” hack. We created an entirely separate heatmap and forcefully squashed it into a tiny column by hardcoding `width = unit(3, "mm")` inside our code. Because we were forcing exact dimensions, we also had to tightly micromanage the global Jupyter canvas size just to make everything fit.

However, as we just saw, utilizing `ComplexHeatmap`’s native `rowAnnotation()` function changes everything. `rowAnnotation()` is specifically designed for this exact purpose. It natively understands how to size itself, how to align with the main heatmap rows, and how to format its own legends without us having to write hacky dimensional constraints.

What does this mean for our code moving forward? It means we can go back to our custom `make_EH()` function and remove the hardcoded width and height definitions entirely. By letting `rowAnnotation()` handle the side-bars, and letting `ComplexHeatmap`’s layout engine dynamically calculate the sizes of the main plotting areas, our code becomes significantly more flexible. Instead of rigid, hardcoded plots, we now have a highly adaptable pipeline that will

automatically scale and look beautiful, whether we are plotting a tiny subset of 100 peaks or a massive multi-omics integration of 50,000 regions.

7 ChIP and RNA sequencing data plots

7.1 Learning objectives

- Load and explore `SummarizedExperiment` objects for ChIP-seq and RNA-seq data.
- Visualize ChIP-seq signal enrichment across genomic regions using the `EnrichedHeatmap` package.
- Generate and interpret RNA-seq expression heatmaps using the `ComplexHeatmap` package.
- Transform RNA-seq data to highlight differential expression using log fold-change.
- Annotate heatmaps with sample metadata to enhance interpretability.
- Split and visualize RNA-seq data based on gene regulation status (up- or down-regulated).
- Compare and reflect on the similarities and differences in visualization techniques between ChIP-seq and RNA-seq data.

7.2 Loading libraries

```
suppressPackageStartupMessages(suppressWarnings({  
  library(SummarizedExperiment)  
  library(GenomicRanges)  
  library(EnrichedHeatmap)  
  library(ComplexHeatmap)  
  library(gUtils)  
  library(rtracklayer)  
  library(circlize)  
}))
```

7.3 Preparing the ChIP-seq SE object

7.3.1 Loading the RDS files

As in the first notebook, we load the `SummarizedExperiment` for the ChIP-seq data.

```
h3k4me3 <- readRDS("data/h3k4me3_se.rds")
h3k4me1 <- readRDS("data/h3k4me1_se.rds")
h3k27me3 <- readRDS("data/h3k27me3_se.rds")
h3k27ac <- readRDS("data/h3k27ac_se.rds")
```

We check the structure of one of these objects.

```
h3k4me3
```

```
class: RangedSummarizedExperiment
dim: 4936 4
metadata(2): final.ext param
assays(2): counts logCPM
rownames: NULL
rowData names(18): annotation geneChr ... PValue qvalue
colnames: NULL
colData names(6): bam.files totals ... group norm.factors
```

```
colData(h3k4me3)
```

DataFrame with 4 rows and 6 columns

	bam.files	totals	ext	rlen	group	norm.factors
	<character>	<integer>	<integer>	<integer>	<character>	<numeric>
1	../data/H3K4me3_11ha..	38077388	100	36	11half	0.922457
2	../data/H3K4me3_11ha..	33020628	100	36	11half	0.852379
3	../data/H3K4me3_15ha..	78731363	100	49	15half	1.119011
4	../data/H3K4me3_15ha..	91317562	100	49	15half	1.136546

```
rd_h3k4me3 <- rowRanges(h3k4me3)
rd_h3k4me3
```

GRanges object with 4936 ranges and 18 metadata columns:

	seqnames	ranges	strand	annotation	geneChr
	<Rle>	<IRanges>	<Rle>	<character>	<integer>
[1]	chr1	3667266-3667503	*	Intron (ENSMUST00000..	1
[2]	chr1	3668238-3668743	*	Intron (ENSMUST00000..	1
[3]	chr1	3669041-3672787	*	Promoter (<=1kb)	1
[4]	chr1	4491514-4493868	*	Promoter (<=1kb)	1
[5]	chr1	4496194-4496402	*	Promoter (<=1kb)	1
...	...	...	...	...	...

[4932]	chr2	181765975-181766393	*		Promoter (<=1kb)	2
[4933]	chr2	181766549-181770566	*		Promoter (<=1kb)	2
[4934]	chr2	181837343-181839141	*		Promoter (<=1kb)	2
[4935]	chr2	181863316-181865849	*		Promoter (<=1kb)	2
[4936]	chr2	181953101-181953592	*		Distal Intergenic	2
	geneStart	geneEnd	geneLength	geneStrand	geneId	
	<integer>	<integer>	<integer>	<integer>	<character>	
[1]	3214482	3671498	457017	2	497097	
[2]	3214482	3671498	457017	2	497097	
[3]	3214482	3671498	457017	2	497097	
[4]	4492465	4493735	1271	2	20671	
[5]	4491713	4496363	4651	2	20671	
...	...	...	...	...	...	
[4932]	181767029	181795892	28864	1	17932	
[4933]	181767029	181795892	28864	1	17932	
[4934]	181837854	181857461	19608	1	245867	
[4935]	181864337	181870830	6494	1	67005	
[4936]	181991129	182003861	12733	1	672125	
	transcriptId	distanceToTSS	ENSEMBL	SYMBOL		
	<character>	<numeric>	<character>	<character>		
[1]	ENSMUST00000070533.4	3995	ENSMUSG00000051951	Xkr4		
[2]	ENSMUST00000070533.4	2755	ENSMUSG00000051951	Xkr4		
[3]	ENSMUST00000070533.4	0	ENSMUSG00000051951	Xkr4		
[4]	ENSMUST00000191939.1	0	ENSMUSG00000025902	Sox17		
[5]	ENSMUST00000116652.7	0	ENSMUSG00000025902	Sox17		
...	...	...	...	...		
[4932]	ENSMUST00000156190.7	-636	ENSMUSG00000010505	Myt1		
[4933]	ENSMUST00000156190.7	0	ENSMUSG00000010505	Myt1		
[4934]	ENSMUST00000029116.13	0	ENSMUSG00000027589	Pcmt2		
[4935]	ENSMUST00000039551.8	0	ENSMUSG00000038628	Polr3k		
[4936]	ENSMUST00000071760.7	-37537	ENSMUSG00000098505	Gm14496		
	GENENAME	anno	logFC	logCPM	F	
	<character>	<character>	<numeric>	<numeric>	<numeric>	
[1]	X-linked Kx blood gr..	Intron	-0.373770	2.28064	0.2530847	
[2]	X-linked Kx blood gr..	Intron	1.861338	3.79682	16.3418021	
[3]	X-linked Kx blood gr..	Promoter	0.607114	9.26749	135.4429255	
[4]	SRY (sex determining..	Promoter	-0.259414	6.39021	3.9624216	
[5]	SRY (sex determining..	Promoter	-0.195908	2.51613	0.0873602	
...	...	...	...	...	...	
[4932]	myelin transcription..	Promoter	-1.42147743	3.08745	10.2690521	
[4933]	myelin transcription..	Promoter	0.50942696	8.25885	50.8903963	
[4934]	protein-L-isoasparta..	Promoter	-0.00811134	7.80433	0.0102742	
[4935]	polymerase (RNA) III..	Promoter	-0.13758781	8.78019	5.6852857	

```

[4936] predicted gene 14496 Intergenic 0.40807614 4.43807 1.9921177
      PValue      qvalue
      <numeric> <numeric>
[1] 6.14925e-01 7.15697e-01
[2] 5.33720e-05 2.50660e-04
[3] 4.65079e-31 1.28968e-29
[4] 4.65601e-02 9.65633e-02
[5] 7.67568e-01 8.34518e-01
...      ...      ...
[4932] 1.35802e-03 4.81764e-03
[4933] 1.06052e-12 1.19242e-11
[4934] 9.19266e-01 9.43150e-01
[4935] 1.71305e-02 4.32734e-02
[4936] 1.58158e-01 2.56714e-01
-----
seqinfo: 33 sequences from an unspecified genome; no seqlengths

```

7.3.2 Exploratory data analysis

Histone methylation (especially active marks like H3K4me3 at promoters) can have very sharp, well-defined boundaries compared to the broader regions of accessibility we see in ATAC-seq or repressive marks.

```

cat("Summary statistics of H3K4me3 peak width:")
summary(width(rd_h3k4me3))

```

Summary statistics of H3K4me3 peak width:

```

      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
191.0    279.8    749.0   1283.9   2009.0  18723.0

```

We can actually directly compare the peak width distributions and “sharpness” for all the ChIP-seq datasets using histograms.

```

# Create a named list of your 4 SE objects
se_list <- list(
  "H3K4me3" = h3k4me3,
  "H3K4me1" = h3k4me1,
  "H3K27me3" = h3k27me3,
  "H3K27ac" = h3k27ac)

```

```

# Set the plotting layout to 2 rows by 2 columns
par(mfrow = c(2, 2))

# Loop through the list
for (mark_name in names(se_list)) {

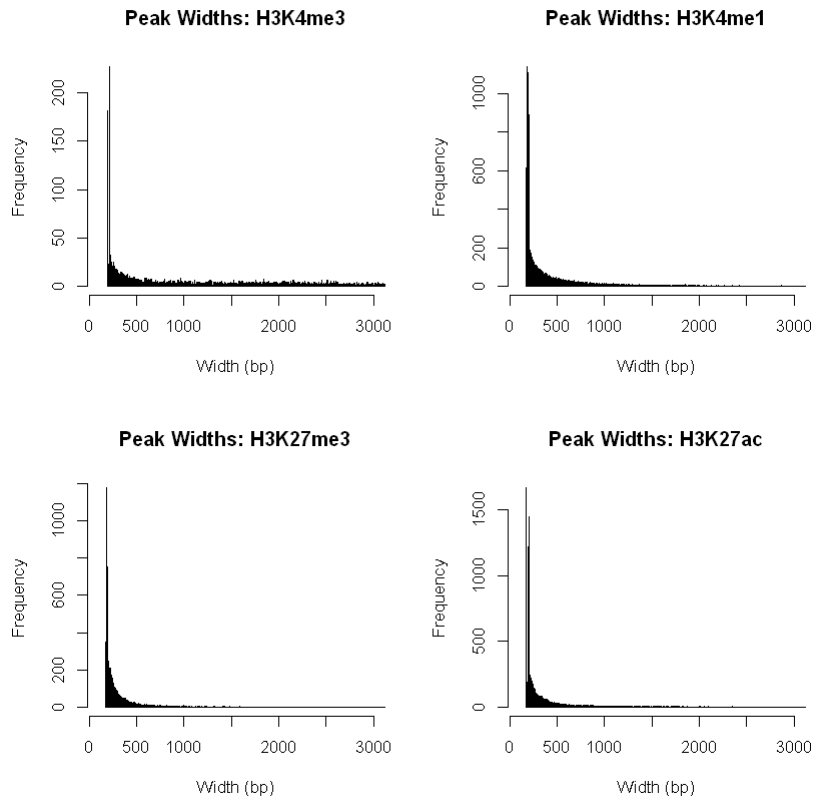
  # Extract the specific SE object
  se <- se_list[[mark_name]]

  # Extract the genomic ranges
  gr <- rowRanges(se)

  # Plot the histogram
  hist(
    width(gr),
    breaks = 10000,
    xlim = c(100, 3000),    # Zoom in on widths up to 3000bp
    main = paste("Peak Widths:", mark_name),
    xlab = "Width (bp)"
  )
}

# Reset the layout back to a standard 1x1 plot when done
par(mfrow = c(1, 1))

```



7.3.3 Interpreting ChIP-seq peak widths

Histone modifications (like H3K4me3 or H3K27ac) are physically attached to the histone proteins that make up a nucleosome. A single nucleosome wraps roughly **147 base pairs** of DNA. Because the modification sits on the nucleosome, the absolute minimum biological “footprint” that can be captured is the size of one nucleosome (around ~150 bp), like we observe on the histograms.

Moreover, during the ChIP-seq protocol, the DNA is physically sheared (via sonication or enzymatic digestion) before sequencing. We typically aim to shear the DNA into fragments of about 200 to 500 base pairs. Because the sequence reads come from the ends of these fragments, it is virtually impossible for the sequencing to resolve a peak that is narrower than the size of the fragments we physically cut the DNA into.

Furthermore, the software used to detect these peaks (usually *MACS2*) has built-in parameters to filter out noise. *MACS2* models the average fragment length (which it often estimates to be ~150-250 bp) and deliberately ignores or merges signal that is smaller than that expected

fragment size, because a 30 bp “peak” is almost certainly just random sequencing noise rather than a true modified nucleosome.

While H3K4me3 is typically a sharp mark found at promoters (~200–1000 bp), we observe a subset of peaks extending well beyond 3,000 bp. These **broad H3K4me3 domains** are a well-documented biological phenomenon that typically mark highly transcribed, lineage-defining genes with large CpG islands or multiple transcription start sites.

7.4 Question 1

Similar to ATAC-seq data, please create `EnrichedHeatmap` for all ChIP-seq datasets and observe difference in genome activity at different genomic regions.

7.4.1 Proposed strategy

The goal of this exercise is to visualize and compare the spatial distribution of four different histone modifications (H3K4me3, H3K4me1, H3K27ac, H3K27me3) across specific genomic regions to infer chromatin state and genome activity.

To accurately compare the histone marks, we need a common set of genomic anchor regions. Since ATAC-seq peaks represent all open chromatin (both promoters and enhancers), we can use the **ATAC-seq peak centers** (generated in the [previous notebook](#)) as our universal anchor points. Alternatively, we could combine the peak ranges from all 4 ChIP-seq objects into a consensus set, but using the ATAC peaks is conceptually cleaner as it defines “all potentially active regulatory regions.” (And I have the intuition that we would finally combine all of this data at the end of the course ;-)

7.4.1.1 H3K27me3 and the chromatin states

Because ATAC-seq captures open chromatin, using it as our “universal anchor” creates a massive bias. It works beautifully for H3K4me3, H3K4me1, and H3K27ac because those marks actively push the chromatin open. But what about H3K27me3 (the repressive mark)? If a genomic region is heavily repressed by H3K27me3, the chromatin there is tightly closed (inactive). Because it is closed, the ATAC-seq machinery can’t get in, meaning there is no ATAC-seq peak there.

If we force H3K27me3 to be plotted using ATAC-seq anchors, we will only see H3K27me3 at weird, transitioning, or “bivalent” regions that happen to be slightly open. We will completely blind ourselves to the deeply repressed, true heterochromatin regions where H3K27me3 is actually the strongest.

7.4.1.2 Expected final figure layout

The final visualization will be a highly informative, multi-panel composite heatmap generated via the `EnrichedHeatmap` and `ComplexHeatmap` packages, as we learned throughout the previous notebook:

- Horizontal panels: 8 side-by-side heatmaps representing the continuous bigWig signal (4 histone marks $\times$ 2 developmental stages: E11.5 and E15.5)
- Row splitting (vertical): the rows (genomic regions) will be split by Genomic Annotation (e.g., Promoters vs. Distal Intergenic/Intron, etc.). This allows us to clearly observe how H3K4me3 specifically enriches at promoters, while H3K4me1 enriches at distal enhancers
- Top annotations: above each heatmap, a summarized meta-profile (*line plot*) will show the average signal intensity across the 2kb window, all colored in black (default)
- Ordering: rows will be ordered by the overall intensity of the ATAC-seq signal or the H3K27ac signal to smoothly transition from highly active to less active regions

7.4.2 Defining anchor regions from differentially accessible chromatin

We will rely on the ATAC-seq data processed in the previous notebook, specifically extracting the `rowRanges` from the ATAC-seq `SummarizedExperiment` and filtering for **significantly differentially accessible regions** (using `qvalue` and `logFC` thresholds) exclusively on chromosomes 1 and 2.

Also remind that once we have isolated these significant ATAC-seq peaks, we used the `resize(width = 1, fix = "center")` function to collapse each peak down to its exact 1-bp midpoint. These midpoints will now serve as the universal anchors for our `EnrichedHeatmap` windows, ensuring that all 4 ChIP-seq datasets are visualized and compared across the exact same spatial coordinates.

```
# Loading the ATAC-seq dataset
atac <- readRDS("data/atac_se.rds")

# Extracting the Genomic Ranges
rd_atac <- rowRanges(atac)

# Filtering by logFC, qvalue and seqnames (chromosomes)
rd_atac_filtered <- subset(
  rd_atac,
  abs(logFC) >= 0.5 &
  qvalue <= 0.1 &
  seqnames %in% c("chr1", "chr2"))

# Add an "ATAC_" prefix to all the metadata columns in rd_atac_filtered
```

```

# (!GOOD PRACTICE preventing multi-omics collisions)
colnames(elementMetadata(rd_atac_filtered)) <- paste(
  "ATAC", colnames(elementMetadata(rd_atac_filtered)), sep = "_"
)

# Finding peak centers
mid_peaks <- resize(rd_atac_filtered, width = 1, fix = "center")

# Identify the ATAC peaks
# (!GOOD PRACTICE for traceability)
names(mid_peaks) <- paste("peak", 1:length(mid_peaks), sep = "_")

head(mid_peaks, 3)

```

GRanges object with 3 ranges and 18 metadata columns:

	seqnames	ranges	strand	ATAC_annotation	ATAC_geneChr
	<Rle>	<IRanges>	<Rle>	<character>	<integer>
peak_1	chr1	3671606	*	Promoter (<=1kb)	1
peak_2	chr1	4332610	*	Intron (ENSMUST000000..	1
peak_3	chr1	4492164	*	Promoter (1-2kb)	1
	ATAC_geneStart	ATAC_geneEnd	ATAC_geneLength	ATAC_geneStrand	
	<integer>	<integer>	<integer>	<integer>	
peak_1	3214482	3671498	457017	2	
peak_2	4344146	4360314	16169	2	
peak_3	4492465	4493735	1271	2	
	ATAC_geneId	ATAC_transcriptId	ATAC_distanceToTSS	ATAC_ENSEMBL	
	<character>	<character>	<numeric>	<character>	
peak_1	497097	ENSMUST00000070533.4	0	ENSMUSG00000051951	
peak_2	19888	ENSMUST00000027032.5	27604	ENSMUSG00000025900	
peak_3	20671	ENSMUST00000019139.1	1162	ENSMUSG00000025902	
	ATAC_SYMBOL	ATAC_GENENAME	ATAC_anno	ATAC_logFC	ATAC_logCPM
	<character>	<character>	<character>	<numeric>	<numeric>
peak_1	Xkr4	X-linked Kx blood gr..	Promoter	0.602984	10.45378
peak_2	Rp1	retinitis pigmentosa..	Intron	0.855748	5.18424
peak_3	Sox17	SRY (sex determining..	Promoter	-0.620926	7.13469
	ATAC_F	ATAC_PValue	ATAC_qvalue		
	<numeric>	<numeric>	<numeric>		
peak_1	41.78446	6.58453e-08	4.81983e-07		
peak_2	3.68371	6.13517e-02	9.28785e-02		
peak_3	9.14522	4.12368e-03	8.38540e-03		

seqinfo: 21 sequences from an unspecified genome; no seqlengths

7.4.3 Extracting and normalizing continuous ChIP-seq signal

To extract the continuous ChIP-seq signal, we first retrieve the paths for the 8 ChIP-seq .bw (bigWig) files from the `data/` directory. Next, we define our target extraction regions, or `heatmap_windows`, by expanding our ATAC `mid_peaks` to the desired viewing window size (for example, using `resize(mid_peaks, width = 2000, fix = "center")`, just as we did in the previous notebook).

Because there is exactly one bigWig file per condition (4 histone marks $\times$ 2 developmental stages), the most elegant and “R-like” approach is to grab all 8 files at once and process them using a single `lapply()` function, rather than writing complicated nested loops. Within this single loop, we use `rtracklayer::import()` and pass our specific windows to the `which` argument. This forces R to dynamically read the continuous signal coverage only within our exact targeted regions directly from the hard drive, massively saving RAM and computation time.

Once the targeted signal is read, the exact same loop immediately feeds it into `normalizeToMatrix()`, outputting a clean list of 8 normalized matrices ready for `EnrichedHeatmap`.

```
# Get all 8 ChIP-seq bigWig files at once using the regex:
#   ^H3K   -> The file name MUST start with "H3K" (ignores the ATAC files)
#   .*     -> Followed by any sequence of characters (e.g., "4me3_11half")
#   \\.bw$ -> And MUST end exactly with the literal ".bw" extension
chip_bw_files <- list.files("data", pattern = "^H3K.*\\.bw$", full.names = TRUE)

# Give them clean names for the list (e.g., "H3K4me3_11half")
names(chip_bw_files) <- gsub("\\.bw$", "", basename(chip_bw_files))

# Define the actual 2000 bp windows we need for the heatmap
heatmap_windows <- resize(mid_peaks, width = 2000, fix = "center")

# Use a single lapply loop to process all 8 files
chip_matrices <- lapply(chip_bw_files, function(file_path) {

  # Import ONLY the regions overlapping our heatmap_windows
  signal <- rtracklayer::import(file_path, which = heatmap_windows)

  # Convert the signal to a normalized matrix for EnrichedHeatmap
  mat <- normalizeToMatrix(
    signal,          # bigWig data
    target = mid_peaks, # anchor ("point 0")
    value_column = "score", # read coverage values
```

```

    extend = 1000,      # 2 kb-wide window
    w = 20,             # binning size of 20 bp
    smooth = TRUE,      # LOESS smoothing
    background = 0      # noise floor
)

return(mat)
})

```

```

# View of few columns and peaks from the first matrix
chip_matrices[[1]][1:5, 45:50]

```

A matrix: 5 × 6 of type dbl

	u45	u46	u47	u48	u49	u50
peak_1	0.41785877	0.36655732	0.34175880	0.35650114	0.41155756	0.4662
peak_2	0.01211658	0.02286782	0.03310115	0.03916861	0.04122399	0.0393
peak_3	0.11026432	0.07951743	0.05068564	0.03356038	0.02665345	0.0250
peak_4	0.69515939	0.68179722	0.65382211	0.61544447	0.56406654	0.5190
peak_5	0.12519382	0.12562346	0.13146672	0.14713624	0.17450900	0.2011

7.4.4 Constructing and combining the heatmaps

With our list of 8 normalized matrices prepared, we can now translate this numerical data into a visual map of the chromatin landscape. We will generate an individual **EnrichedHeatmap** object for each matrix. To streamline this process and ensure perfect formatting consistency, we will utilize an upgraded custom wrapper function (**make_EH_custom**). Rather than building each heatmap from scratch, this function automates several advanced features: 1. **Dynamic Color Scaling:** It automatically calculates a responsive color scale based on the 1st and 99th percentiles of each specific matrix (using **circlize::colorRamp2**), and perfectly aligns the legend ticks. 2. **Proportional Layout Engine:** By defaulting to the **grid** graphic's flexible **width = unit(1, "null")** unit, it ensures that when we stitch multiple heatmaps together, they will dynamically stretch and divide the page space perfectly equally! Here is the perfectly tweaked bullet point to clarify that your hardcoded default precisely mirrors the native software! 3. **Architectural Control:** It exposes a **top_anno_height** parameter (which we safely defaulted to 1.5 cm to precisely mirror the software's native default behavior), giving us the ability to elegantly squeeze or expand the vertical size of the top summary meta-profile plots on the fly without complex math.

*Note: This custom function has pre-defined x-axis labels hard-coded as **c("-1kb", "mid", "1kb")**. In the future, if the genomic window being analyzed is different from the current 2000 bp window centered on the ATAC-seq peaks, be sure to make the appropriate changes to the **axis_name** argument.*

To highlight how these marks behave differently across distinct regulatory elements, we will pass our ATAC peak annotations to the `split_rows` argument (separating, “Promoters” from “Exon”, “Intron” and “Intergenic” regions).

Finally, leveraging the power of the `ComplexHeatmap` ecosystem, we can seamlessly concatenate all 8 resulting heatmaps side-by-side using the simple `+` operator. Just as we did before, we will also append a `rowAnnotation` block to the side of the figure—visually tagging the rows with their associated metadata—to create a highly informative, unified composite figure.

```
# Upgraded version of the original custom function
make_EH_custom <- function(
  norm_mat, hm_name, split_rows = NULL,
  width = unit(1, "null"), height = NULL, top_anno_height = grid::unit(1.5, "cm"),
  heatmap_cols = c("white", "red"), col_fill = "white",
  line_colors = NULL) {

  # Calculate ticks for the legend
  vmin <- as.numeric(quantile(norm_mat, c(0.01)))
  vmax <- as.numeric(quantile(norm_mat, c(0.99)))
  vmid <- (vmin + vmax) / 2
  legend_ticks <- c(vmin, vmid, vmax)

  # Define color scale based on 1st and 99th percentiles (vmin and vmax)
  col_fun <- colorRamp2(c(vmin, vmax), heatmap_cols)

  # Generate and return the heatmap
  EnrichedHeatmap(
    mat = norm_mat, name = hm_name, row_split = split_rows,
    row_title = NULL, col = col_fun, column_title = hm_name,
    column_title_gp = gpar(fontsize = 10, fill = col_fill),
    width = width, height = height,
    axis_name = c("-1kb", "mid", "1kb"),
    heatmap_legend_param = list(
      at = legend_ticks, labels = round(legend_ticks, digits = 1),
      title_gp = gpar(fontsize = 8), labels_gp = gpar(fontsize = 7)),
    top_annotation = HeatmapAnnotation(
      lines = anno_enriched(
        height = top_anno_height,
        axis_param = list(
          # gp = gpar(fontsize = 7, lwd = 0.4),
          side = "right", facing = "inside"),
        gp = gpar(col = line_colors))))}
```

Let's now create the individual heatmaps objects, using colors similar to what is shown on the course logo.

```
names(chip_matrices)
```

1. 'H3K27ac_11half' 2. 'H3K27ac_15half' 3. 'H3K27me3_11half' 4. 'H3K27me3_15half'
5. 'H3K4me1_11half' 6. 'H3K4me1_15half' 7. 'H3K4me3_11half' 8. 'H3K4me3_15half'

```
# Heatmaps for the H3K4me1 data
# REMINDER - `[ ]` returns a list, while `$` (or `[[ ]]`)
# returns the actual object inside the list.
eh_h3k4me1_11h <- make_EH_custom(
  norm_mat = chip_matrices$H3K4me1_11half,
  hm_name = 'K4me1-E11.5',
  col_fill = 'pink',
  # width = unit(2, "cm"), height = unit(8, "cm"),
  heatmap_cols = c('white', 'darkblue'))

eh_h3k4me1_15h <- make_EH_custom(
  norm_mat = chip_matrices$H3K4me1_15half,
  hm_name = 'K4me1-E15.5',
  col_fill = 'paleturquoise',
  # width = unit(2, "cm"), height = unit(8, "cm"),
  heatmap_cols = c('white', 'darkblue'))
```

```
# Heatmaps for the H3K4me3 data
eh_h3k4me3_11h <- make_EH_custom(
  norm_mat = chip_matrices$H3K4me3_11half,
  hm_name = 'K4me3-E11.5',
  col_fill = 'pink',
  heatmap_cols = c('white', 'darkgreen'))

eh_h3k4me3_15h <- make_EH_custom(
  norm_mat = chip_matrices$H3K4me3_15half,
  hm_name = 'K4me3-E15.5',
  col_fill = 'paleturquoise',
  heatmap_cols = c('white', 'darkgreen'))
```

```
# Heatmaps for the H3K27me3 data
eh_h3k27me3_11h <- make_EH_custom(
  norm_mat = chip_matrices$H3K27me3_11half,
  hm_name = 'K27me3-E11.5',
```

```

col_fill = 'pink',
heatmap_cols = c('white', 'darkorchid'))

eh_h3k27me3_15h <- make_EH_custom(
  norm_mat = chip_matrices$H3K27me3_15half,
  hm_name = 'K27me3-E15.5',
  col_fill = 'paleturquoise',
  heatmap_cols = c('white', 'darkorchid'))

```

```

# Heatmaps for the H3K27ac data
eh_h3k27ac_11h <- make_EH_custom(
  norm_mat = chip_matrices$H3K27ac_11half,
  hm_name = 'K27ac-E11.5',
  col_fill = 'pink',
  heatmap_cols = c('white', 'darkred'))

eh_h3k27ac_15h <- make_EH_custom(
  norm_mat = chip_matrices$H3K27ac_15half,
  hm_name = 'K27ac-E15.5',
  col_fill = 'paleturquoise',
  heatmap_cols = c('white', 'darkred'))

```

```

# Retrieve the genomic annotations (e.g., "Promoter", "Intron", "Intergenic")
split_anno <- mid_peaks$ATAC_anno

# Attach the original peak IDs as names to this new vector
names(split_anno) <- names(mid_peaks)

anno_colors <- RColorBrewer::brewer.pal(
  n = length(unique(split_anno)), name = "Set1")

names(anno_colors) <- sort(unique(split_anno))

# Prepare the annotations
eh_annotations <- rowAnnotation(
  df = data.frame(Annotation = split_anno),
  col = list(Annotation = anno_colors),
  show_annotation_name = FALSE
)

```

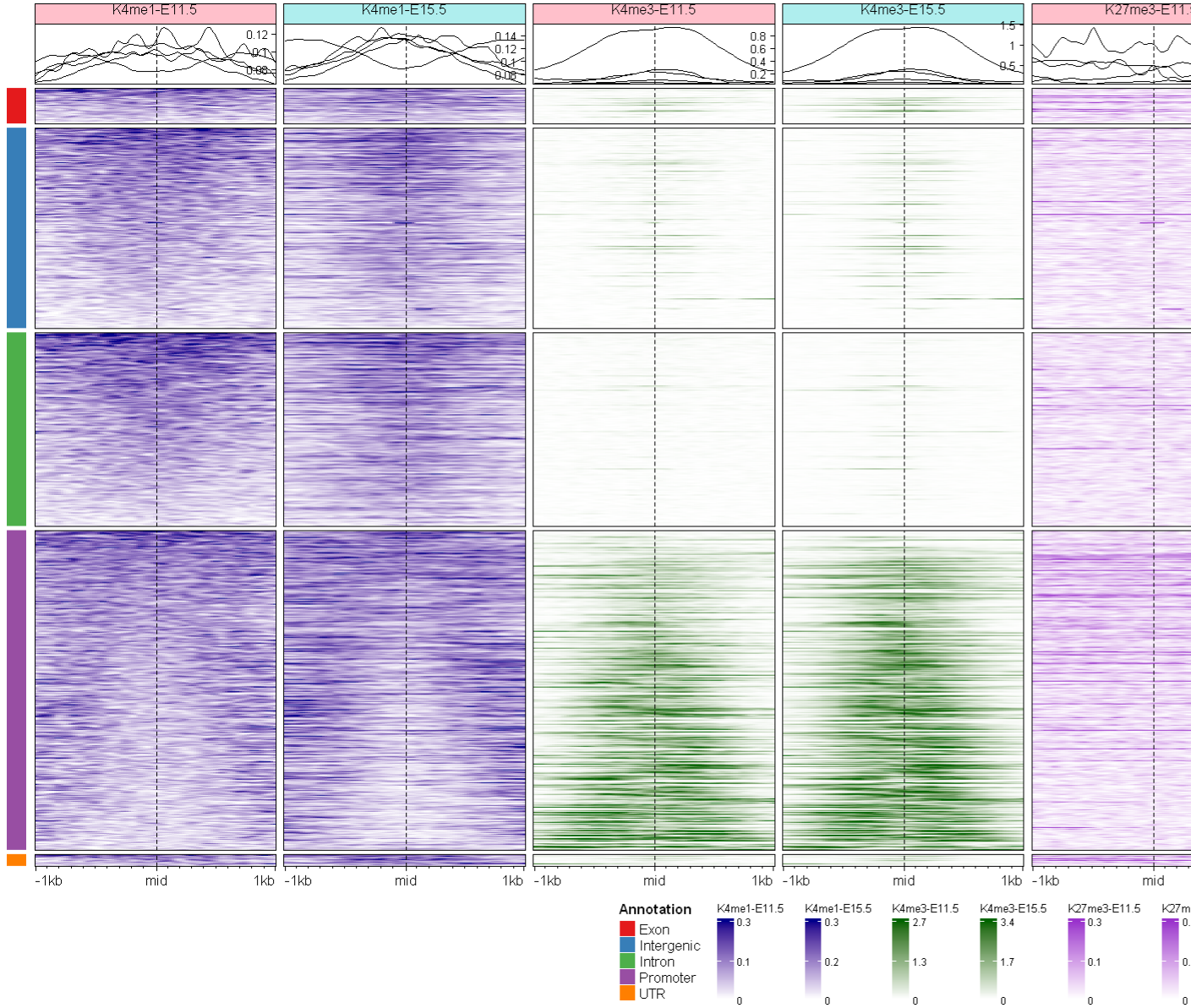
In the **ComplexHeatmap** ecosystem, when we combine heatmaps using `+`, they are merged together into a single master object called a **HeatmapList**. Because the rows are strictly locked

together across all 8 heatmaps, we don't need to split them individually.

By passing `split = split_anno` directly into the `draw()` function, we are applying the split globally to the entire composite figure at the very end. We don't have to pass `split_rows` 8 separate times in our `make_EH_custom` function, which guarantees that every single heatmap is forced to split in the exact same way simultaneously. Moreover, R only has to calculate the splitting logic once for the master figure, rather than recalculating it 8 times!

```
options(repr.plot.width = 20, repr.plot.height = 10)

draw(
  eh_annotiations
  + eh_h3k4me1_11h + eh_h3k4me1_15h
  + eh_h3k4me3_11h + eh_h3k4me3_15h
  + eh_h3k27me3_11h + eh_h3k27me3_15h
  + eh_h3k27ac_11h + eh_h3k27ac_15h,
  split = split_anno,
  # merge_legend = TRUE,
  heatmap_legend_side = "bottom")
```



Note: Because we allowed each histone mark to dynamically scale its colors based on its own signal strength (i.e., the color scale is dynamically calculated based on the 1st and 99th percentiles of each individual matrix), we must have separate legends. This means the actual numeric scale is different for every single heatmap. Moreover, `merge_legend` only combines legends that share the exact same title. Therefore, the `merge_legend` argument has no effect here.

7.4.5 Analyzing histone mark distributions across genomic features

In the composite heatmap, the **H3K4me3** signal is entirely restricted to the violet (*Promoter*) block and absent everywhere else. As the canonical “**Active Promoter**” mark, H3K4me3 is dynamically placed directly around the Transcription Start Site (TSS) by specific enzymes (like MLL/COMPASS complexes) that associate with the transcription machinery. Seeing this signal cleanly restricted to the promoter block confirms the high quality of the data and strictly adheres to expected biological principles.

Unlike H3K4me3, **H3K27ac** marks **both** active promoters and active enhancers, which perfectly explains its strong presence across the promoter, intergenic, and intronic blocks. Interestingly, a temporal shift is visible: H3K27ac signal is higher in distal regions (intergenic/introns) at E11.5, but becomes relatively stronger at promoters by E15.5. This represents a classic developmental signature. At E11.5, cells are still highly plastic and maintain massive networks of active distal enhancers. As they mature to E15.5, many of these broad enhancer networks shut down, while specific terminal genes become aggressively transcribed.

A striking feature of the visualization is the distinct lack of signal (*white space*) directly in the middle of the promoters for **H3K4me1** and, to a lesser extent, **H3K27me3**. Because these heatmaps are centered precisely on *ATAC-seq peaks*, the exact middle (*mid*) represents open chromatin where nucleosomes have been physically displaced to allow transcription factors to bind. Since histone marks require a histone protein to attach to, the signal completely drops to zero in this physical “hole,” resulting in a characteristic **bimodal (twin-peak) signal** on the flanking nucleosomes. Furthermore, because H3K4me1 is traditionally an enhancer mark, the central nucleosomes at active promoters get heavily decorated with H3K4me3 instead, physically displacing H3K4me1 to the outer edges and explaining the wide central “dip.”

Overall, the visualization successfully captures the known epigenetic landscape: H3K4me3 strictly identifies active promoters, H3K27ac identifies both promoters and enhancers while highlighting developmental transitions, and the fine spatial resolution accurately maps nucleosome depletion directly at the ATAC-seq peak centers.

7.5 Analysis of the RNA-seq data

Having deeply explored the spatial distribution of chromatin accessibility (ATAC-seq; previous notebook) and histone modifications (ChIP-seq) across regulatory elements, we now shift our focus to the functional output of these epigenetic changes: **gene expression**.

Unlike ChIP-seq, where we used **EnrichedHeatmap** to visualize the continuous spatial footprint of modifications across 2kb genomic windows, RNA-seq data is fundamentally different. Here, we are working with discrete transcription levels quantified as **normalized read counts per gene**. Therefore, the heatmaps we generate in this section will be standard expression heatmaps (where rows represent individual genes, columns represent experimental conditions, and color

intensity reflects the expression level). This will allow us to identify clusters of co-regulated genes and track how expression profiles shift between E11.5 and E15.5.

7.5.1 Loading the RNA-seq SE object

To begin, we will load the `SummarizedExperiment` object containing our processed RNA-seq count matrices and associated metadata.

```
rna <- readRDS("data/rna_se.rds")

rna

class: RangedSummarizedExperiment
dim: 2216 4
metadata(0):
assays(2): counts logCPM
rownames(2216): ENSMUSG000000000194 ENSMUSG000000000247 ...
               ENSMUSG000000103948 ENSMUSG000000104257
rowData names(18): Row.names gene_id ... PValue qvalue
colnames(4): RNA_11half_1.tsv.gz RNA_11half_2.tsv.gz
             RNA_15half_1.tsv.gz RNA_15half_2.tsv.gz
colData names(2): sample group
```

In a typical RNA-seq experiment, finding 1,000 to 4,000 significant Differentially Expressed Genes (DEGs) is a very normal, healthy result

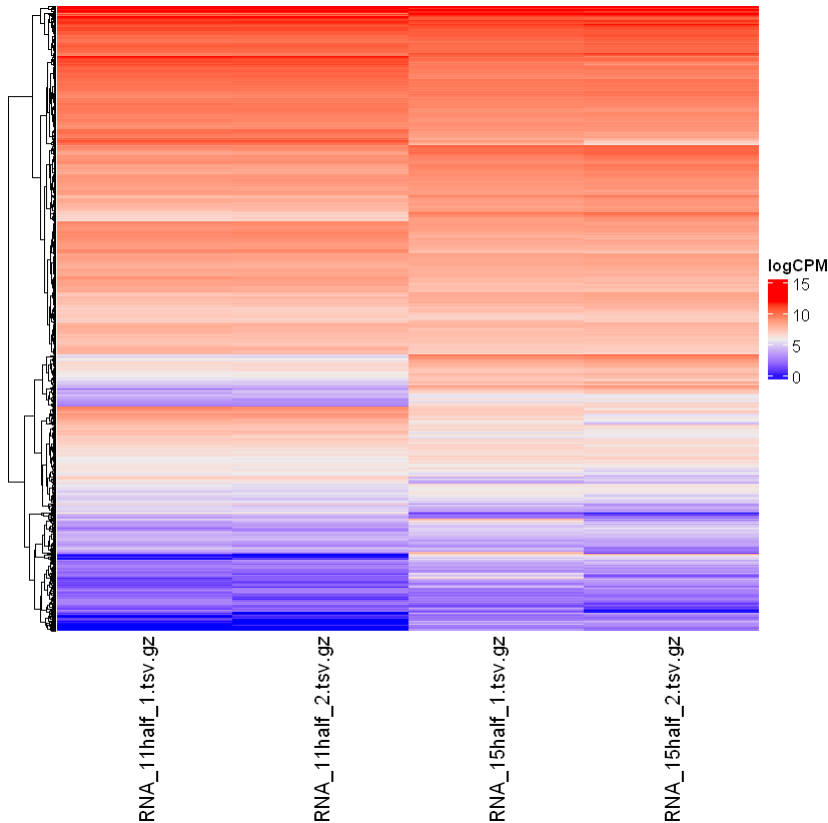
7.5.2 Heatmap of Log-normalized counts

To visualize the global gene expression profile, we will switch from `EnrichedHeatmap` to the `Heatmap()` function from the `ComplexHeatmap` package (different from the `heatmap()` function from base R).

Instead of plotting raw read counts, which are heavily skewed by total sequencing depth and extreme outliers, we extract the `logCPM` (*log2 Counts Per Million*) matrix directly from the `SummarizedExperiment` object. The `logCPM` transformation normalizes the data for library size and compresses the massive dynamic range of RNA expression into a scale that is much better suited for visual clustering.

```
# Back to standard Jupyter Notebook plotting options
options(repr.plot.width = 7, repr.plot.height = 7)
```

```
Heatmap(
  matrix = as.matrix(assay(rna, "logCPM")),
  name = "logCPM", # internal name of the heatmap; also used for the legend title
  cluster_columns = FALSE, # prevents automatic sample rearrangement
  show_row_names = FALSE
)
```



7.5.3 Changing visualization to relative expression

Plotting absolute logCPM is useful for seeing overall expression levels, but it can make it difficult to quickly spot dynamic *changes* between the developmental stages. A gene that is constantly highly expressed will visually overwhelm a lowly expressed gene that dramatically doubles in transcription.

To make the developmental shifts immediately obvious, we can transform our absolute expression matrix into a **relative expression matrix**. Because our data is already on a log2 scale, we

can easily *approximate* the log-fold change simply by subtracting the baseline expression (the mean of the E11.5 samples) from every sample in the matrix.

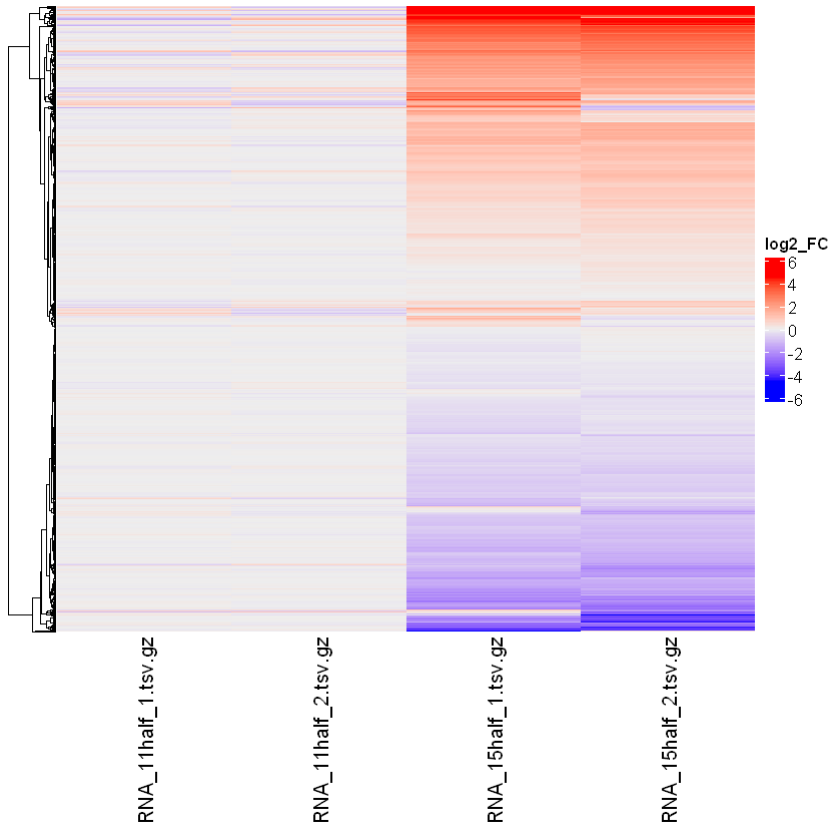
By centering the data on E11.5, the E11.5 columns will become effectively zero (our baseline). The E15.5 columns will then reflect the relative change: positive values will indicate genes that are **up-regulated** as development proceeds, while negative values will indicate genes that are **down-regulated**.

```
# Extract the data and immediately cast to a true numeric matrix
log_cpm <- as.matrix(assay(rna, "logCPM"))

# Identify the columns belonging to the E11.5 baseline group
idx_11half <- grep(
  pattern = "11half",
  x = colnames(log_cpm))

# Subtract the E11.5 baseline mean (row-by-row)
counts_fc <- log_cpm - rowMeans(log_cpm[, idx_11half])
```

```
Heatmap(
  matrix = counts_fc,
  name = "log2_FC", # no parenthesis allowed in the name!
  cluster_columns = FALSE,
  show_row_names = FALSE
)
```



Note: while we are manually calculating these relative fold-changes across the matrix columns to generate a centered heatmap here, we will extract and analyze the rigorously calculated statistical $\log_{2}FC$ from the differential analysis table later in the notebook.

7.5.4 Adding column annotations

Now that our heatmap matrix is structured correctly, we can improve the aesthetics by adding a color-coded annotation bar to the top of our columns. This visually identifies which samples belong to E11.5 and E15.5 at a single glance, without the reader needing to squint at the text labels.

To achieve this, we extract the sample metadata directly from the `rna` object using `colData()`. In addition, we take advantage of the `scales` package: its `pretty_breaks` function dynamically looks at the minimum and maximum values of our matrix to calculate exactly 3 “pretty” (round) numbers for our legend ticks. Finally, we adjust the width and height slightly to make the heatmap look more professional.

Tip: The `drop = FALSE` argument prevents a single-column `data.frame` from collapsing into a nameless vector. This ensures `HeatmapAnnotation` receives a true dataframe (always check the

documentation...) and can automatically pull the correct label name (like “Group” or “Stage”).

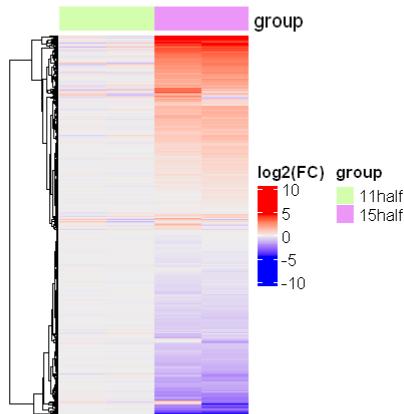
```
colData(rna)
```

DataFrame with 4 rows and 2 columns

	sample	group
	<character>	<factor>
RNA_11half_1.tsv.gz	RNA_11half_1.tsv.gz	11half
RNA_11half_2.tsv.gz	RNA_11half_2.tsv.gz	11half
RNA_15half_1.tsv.gz	RNA_15half_1.tsv.gz	15half
RNA_15half_2.tsv.gz	RNA_15half_2.tsv.gz	15half

```
set.seed(100) # Forces R to always pick the exact same "random" colors
```

```
Heatmap(  
  matrix = counts_fc,  
  # name = "log2_FC",  
  cluster_columns = FALSE,  
  show_row_names = FALSE,  
  show_column_names = FALSE,  
  top_annotation = HeatmapAnnotation(  
    df = colData(rna)[ , 'group', drop = FALSE]  
  ),  
  width = unit(4, "cm"),  
  height = unit(8, "cm"),  
  heatmap_legend_param = list(  
    at = scales::pretty_breaks(n = 3)(  
      range(counts_fc, na.rm = TRUE)),  
    title = "log2(FC)" # fancier display name  
  )  
)
```



7.6 Question 2

Similar to `EnrichedHeatmap`, we can also split RNA-seq data based on `logFC`. Can you try to do this?

Just as we split our spatial ChIP-seq heatmaps by genomic regions (Promoter vs Intergenic), we can split our RNA-seq expression heatmap by gene behavior. The most logical way to do this is to physically separate the genes based on their statistical fold-change (`logFC`).

We will extract the pre-calculated statistical `logFC` from the `rowData(rna)` metadata. Because `logFC` is a continuous number, we must convert it into distinct, discrete groups (e.g., “Up-regulated” and “Down-regulated”) so the software knows how to draw the split. We then simply pass this new grouping vector directly into the `row_split` argument of our `Heatmap()` function.

```
# Extract the statistical logFC for all genes
# Remind rowRanges stores the physical genomic coordinates,
# while rowData stores the statistical metadata as a DataFrame
```

```
logFC <- rowData(rna)[, "logFC"]

summary(logFC)
```

```
      Min.   1st Qu.   Median     Mean  3rd Qu.     Max.
-6.94492 -0.60775   0.05993   0.31056   0.98024   8.98301
```

```
# Categorize the genes (here we split strictly at 0), but we can
# change the 0 to 1 or 2 if we want to be more strict
gene_status <- ifelse(logFC > 0, "Up-regulated", "Down-regulated")
```

We can rename the factor levels in the `rna` metadata to the corresponding biological names before we draw the heatmap.

```
colData(rna)$group <- ifelse(
  colData(rna)$group == "11half", "E11.5", "E15.5")
```

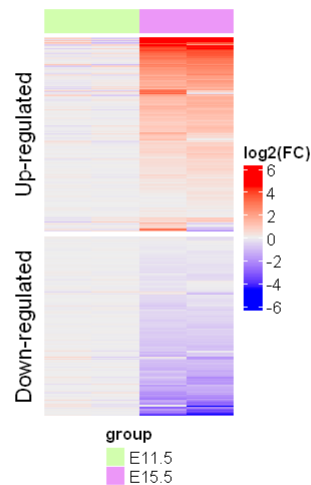
By default, heatmaps are rendered automatically upon creation. To gain precise control over the layout of our various legends, we first save the heatmap to an object and then explicitly render it using the `draw()` function. During this construction phase, we also apply a critical aesthetic trick: by passing `show_annotation_name = FALSE` deep inside the `HeatmapAnnotation()` function, we force the engine to hide the redundant column grouping label. Without this specific override, `ComplexHeatmap` would forcefully print the word “group” on the far right edge of the plot, creating an ugly, illegible text collision overlapping directly on top of our heatmap colors.

```
set.seed(100)

ht <- Heatmap(
  matrix = counts_fc,
  row_split = gene_status,
  cluster_columns = FALSE,
  show_row_names = FALSE,
  show_column_names = FALSE,
  show_row_dend = FALSE,
  width = unit(4, "cm"), height = unit(8, "cm"),
  top_annotation = HeatmapAnnotation(
    df = colData(rna)[,2, drop = FALSE],
    show_annotation_name = FALSE # Remove 'group' annotation name
  ),
```

```
heatmap_legend_param = list(title = "log2(FC)")
)

draw(ht, annotation_legend_side = "bottom", heatmap_legend_side = "right")
```



7.6.1 A note on visualizing expression changes

By manually centering the matrix, we preserve all 4 individual columns (samples). This allows us to visually verify replicate consistency, observe the underlying biological noise, and see exactly how each sample behaves within its group. In contrast, the true $\log_2\text{FC}$ is a single, aggregated statistical value per gene. Plotting it directly would result in a 1-column heatmap that completely hides sample-level variability.

There are many valid ways to visualize RNA-seq data, and we will cover other powerful methods for displaying these statistical results later in the course. For a deeper dive into standard RNA-seq workflows, differential expression statistics, and visualizations, you can refer to my previous [RNA-seq course on GitHub](#).

Part III

III. Multi-Omics Integration

8 Overlapping all the datasets

The core challenge of multi-omics integration is linking datasets that operate on completely different coordinate systems. For example, we might want to link the statistical fold-change of a gene (from RNA-seq) to the physical location of an open chromatin enhancer (from ATAC-seq).

8.1 Learning objectives

- Load and explore genomic data using Bioconductor classes such as `GRangesList` and `RangedSummarizedExperiment`.
- Subset and filter genomic features based on annotation or genomic coordinates.
- Perform basic overlap and distance-based queries between genomic intervals.
- Integrate multi-assay data (e.g., chromatin marks and gene expression) using shared genomic coordinates.

8.2 Initial setup and data loading

Before we begin calculating the genomic overlaps, we need to set up our environment by loading the necessary R packages, importing our processed data, and defining a custom helper function.

8.2.1 Loading required libraries

```
suppressPackageStartupMessages(suppressWarnings({  
  # Core/utility packages  
  library(parallel)  
  library(R.utils)  
  
  # Bioconductor  
  library(SummarizedExperiment)  
  library(rtracklayer)
```

```

library(gUtils)

# Plotting/visualization
library(ComplexHeatmap)
library(EnrichedHeatmap)
library(circlize)

# Data wrangling/Tidyverse
# (MASKING! Load last so its verbs never get overwritten)
library(data.table)
library(dplyr)
library(tibble)
}))

```

8.2.2 Loading the SummarizedExperiment objects

```

atac <- readRDS("data/atac_se.rds")
rna <- readRDS("data/rna_se.rds")
h3k4me3 <- readRDS("data/h3k4me3_se.rds")
h3k4me1 <- readRDS("data/h3k4me1_se.rds")
h3k27me3 <- readRDS("data/h3k27me3_se.rds")
h3k27ac <- readRDS("data/h3k27ac_se.rds")

```

8.2.3 Defining a custom function for multi-omics integration

Unfortunately, standard Bioconductor packages do not have a simple, built-in “left-join” function for genomic coordinates. To solve this, we define a custom helper function called `metaGR()`.

This function takes a primary, target set of genomic regions (`gr1`) and looks for any physical overlaps with a secondary set of regions (`gr2`). Wherever a physical overlap occurs, it extracts the statistical metadata columns (like `logFC` or `q-value`) from `gr2` and seamlessly attaches them as new columns to the overlapping region in `gr1`. If a region in `gr1` doesn’t overlap with anything, it safely fills those new columns with `NA`.

Why a minimum overlap of 10? By default, the R overlap engine registers a “hit” even if two genomic features only touch by a single base pair. Because sequencing data always has a tiny bit of mapping noise at the very edges of peaks, 1-bp or 2-bp overlaps are often accidental false positives. Setting the default minimum overlap to 10 base pairs (which is roughly one full turn of the DNA double helix)

ensures that the physical overlap is robust and represents a genuine biological co-localization!

```
# Merges metadata from gr2 onto gr1 based on genomic overlaps
metaGR <- function(gr1, gr2, minOverlap = 10) {
  # 1. Save the original metadata from our primary target object (gr1)
  original_mcols <- mcols(gr1)

  # 2. Find where the two genomic objects physically overlap
  # gr1 is the "query", gr2 is the "subject"
  hits <- findOverlaps(
    query = gr1,
    subject = gr2,
    minoverlap = minOverlap)

  # Extract the row indices of the overlapping regions
  idx_gr1 <- queryHits(hits) # Which regions in gr1 had a match?
  idx_gr2 <- subjectHits(hits) # Which regions in gr2 did they match with?

  # 3. Create a blank metadata table to hold the incoming gr2 data
  # It must be the exact same length as gr1, filled with NAs (for non-overlaps)
  new_mcols <- DataFrame(
    matrix(NA, nrow = length(gr1), ncol = ncol(mcols(gr2))))
  colnames(new_mcols) <- colnames(mcols(gr2))

  # 4. Inject the gr2 metadata exactly into the rows where an overlap occurred
  new_mcols[idx_gr1, ] <- mcols(gr2)[idx_gr2, , drop = FALSE]

  # 5. Bind the original gr1 metadata and the newly acquired gr2 metadata together
  mcols(gr1) <- cbind(original_mcols, new_mcols)

  # Return the fully annotated primary object
  return(gr1)
}
```

8.3 Constructing the multi-omics Overlap Matrix

We will systematically extract the genomic coordinates (`rowRanges`) from our ATAC, RNA, and ChIP-seq objects. Rather than trying to overlap the entire genome, we will apply a strict statistical filter to each dataset to isolate only the most highly significant, dynamic features (specifically requiring an **absolute** `logFC` ≥ 0.5 and a `qvalue` ≤ 0.1). Furthermore, since

this training course focuses strictly on Chromosomes 1 and 2, we will proactively filter our datasets by `seqnames` at this early stage to drastically reduce memory usage and computation time.

To prevent confusing naming collisions when we stitch these datasets together (since they all contain columns identically named “logFC” and “qvalue”), we will prepend a unique identifier prefix (like `ATAC_` or `RNA_`) to the metadata of each object. Finally, we will use our `metaGR()` function to project all of these significant RNA and ChIP-seq features directly onto our foundational ATAC-seq peaks, creating a single, unified Overlap Matrix.

8.3.1 Processing and merging datasets

The values selected for the filtering is a very common **baseline heuristic**, but it is not a hard law. A `qvalue <= 0.1` means we are accepting a 10% False Discovery Rate (FDR); and `abs(logFC) >= 0.5` translates to a fold-change of $\sim 1.41\times$. In RNA-seq, people often use a stricter `logFC >= 1` (a true doubling of expression) and `q <= 0.05`. However, epigenetic marks (like ChIP-seq) are often much “noisier” and have lower dynamic ranges than RNA. Because a massive 2-fold change in a histone mark is rare, relaxing the threshold to 0.5 is standard practice to avoid throwing away real biology.

```
# Extract rowRanges and filter peaks with strong signal in chr1 and chr2
rd_atac <- rowRanges(atac)
rd_atac_filtered <- subset(
  rd_atac,
  abs(logFC) >= .5 &
  qvalue <= .1 &
  seqnames %in% c("chr1", "chr2"))

colnames(elementMetadata(rd_atac_filtered)) <- paste(
  "ATAC", colnames(elementMetadata(rd_atac_filtered)), sep = "_")

seqlevels(rd_atac_filtered)
```

```
1. 'chr1' 2. 'chr2' 3. 'chr3' 4. 'chr4' 5. 'chr5' 6. 'chr6' 7. 'chr7' 8. 'chr8' 9. 'chr9' 10. 'chr10'
11. 'chr11' 12. 'chr12' 13. 'chr13' 14. 'chr14' 15. 'chr15' 16. 'chr16' 17. 'chr17' 18. 'chr18'
19. 'chr19' 20. 'chrX' 21. 'chrY'
```

*Note: in Bioconductor, the chromosomes (`seqnames`) are stored internally as an R **factor**. When we subset an object in R to remove data (like removing “chr3”, “chr4”, etc.), R successfully deletes the physical data, but it **never automatically drops the factor levels!** The object intentionally retains a “memory” of the original chromosome list (the `seqinfo` metadata), even though the actual count for those chromosomes is now exactly zero. To force the **GRanges** object to forget the empty chromosomes, we just need to tell it to explicitly drop the unused levels.*

```
seqlevelsInUse(rd_atac_filtered)
```

1. 'chr1' 2. 'chr2'

```
# Filter differentially expressed genes from chr1 and chr2
rd_rna <- rowRanges(rna)
rd_rna_filtered <- subset(
  rd_rna,
  abs(logFC) >= .5 & qvalue <= .1 & seqnames %in% c("chr1", "chr2"))

colnames(elementMetadata(rd_rna_filtered)) <- paste(
  "RNA", colnames(elementMetadata(rd_rna_filtered)), sep = "_")
```

In a real-world scenario, the optimal `logFC` and `qvalue` thresholds are chosen based on the specific biological question, often by first visualizing the overall data distribution using a **Volcano Plot**. If we are designing an expensive downstream lab experiment (such as a CRISPR knockout screen), we would typically apply strict thresholds (e.g., `logFC` ≥ 1 and `qvalue` ≤ 0.01) to isolate only the most undeniable, bulletproof targets. Conversely, a more permissive approach (such as dropping the `logFC` filter entirely and relying only on `qvalue` ≤ 0.05) is better suited for detecting subtle, systemic shifts. For instance, a transcription factor might only boost its target genes by 20% (`logFC` = 0.26), but if it does so consistently across a massive network of genes, that subtle shift remains highly biologically significant.

```
# Initialize master object: project RNA data onto ATAC peaks
overlap <- metaGR(gr1 = rd_atac_filtered, gr2 = rd_rna_filtered)
```

When we run `metaGR()` for the very first time, we are officially initializing our master multi-omics object. We use the filtered ATAC-seq peaks (`rd_atac_filtered`) as our foundational genomic coordinates (`gr1`), and project the significant RNA-seq data onto them. Once this master `overlap` object is created, we simply feed it back into the function as the new `gr1` target to keep stacking the rest of the histone marks onto it.

```
# Filter differentially enriched H3K4me3 peaks in chr1 and chr2
rd_h3k4me3 <- rowRanges(h3k4me3)
rd_h3k4me3_filtered <- subset(
  rd_h3k4me3,
  abs(logFC) >= .5 & qvalue <= .1 & seqnames %in% c("chr1", "chr2"))

colnames(elementMetadata(rd_h3k4me3_filtered)) <- paste(
  "H3K4me3", colnames(elementMetadata(rd_h3k4me3_filtered)), sep = "_")

overlap <- metaGR(gr1 = overlap, gr2 = rd_h3k4me3_filtered)
```

Because we filtered the datasets *before* we fed them into `metaGR()`, `metaGR()` is completely blind to any genes or histone marks that failed the significance test. If an ATAC-seq peak physically overlaps with a gene, but that gene didn't change enough to pass our filter, `metaGR()` simply injects NA into the ATAC peak's metadata for that RNA or histone modification column. It means any row in our final Overlap Matrix that actually has numbers in it represents a *statistically significant, dynamic biological event*. If it's just background noise, it's safely masked as NA.

```
# Filter differentially enriched H3K4me1 peaks in chr1 and chr2
rd_h3k4me1 <- rowRanges(h3k4me1)
rd_h3k4me1_filtered <- subset(
  rd_h3k4me1,
  abs(logFC) >= .5 & qvalue <= .1 & seqnames %in% c("chr1", "chr2"))

colnames(elementMetadata(rd_h3k4me1_filtered)) <- paste(
  "H3K4me1", colnames(elementMetadata(rd_h3k4me1_filtered)), sep = "_")

overlap <- metaGR(gr1 = overlap, gr2 = rd_h3k4me1_filtered)
```

Warning message in `.merge_two_Seqinfo_objects(x, y)`:

"Each of the 2 combined objects has sequence levels not in the other:

- in 'x': chrY

- in 'y': chr4_GL456216_random, chr4_JH584295_random, chrX_GL456233_random, chrUn_JH584304

Make sure to always combine/compare objects based on the same reference genome (use `suppressWarnings()` to suppress this warning)."

```
# Filter differentially enriched H3K27me3 peaks in chr 1 and chr2
rd_h3k27me3 <- rowRanges(h3k27me3)
rd_h3k27me3_filtered <- subset(
  rd_h3k27me3,
  abs(logFC) >= .5 & qvalue <= .1 & seqnames %in% c("chr1", "chr2"))

colnames(elementMetadata(rd_h3k27me3_filtered)) <- paste(
  "H3K27me3", colnames(elementMetadata(rd_h3k27me3_filtered)), sep = "_")

overlap <- metaGR(gr1 = overlap, gr2 = rd_h3k27me3_filtered)
```

```
# Filter differentially enriched H3K27ac peaks in chr1 and chr2
rd_h3k27ac <- rowRanges(h3k27ac)
rd_h3k27ac_filtered <- subset(
  rd_h3k27ac,
```

```
abs(logFC) >= .5 & qvalue <= .1 & seqnames %in% c("chr1", "chr2"))

colnames(elementMetadata(rd_h3k27ac_filtered)) <- paste(
  "H3K27ac", colnames(elementMetadata(rd_h3k27ac_filtered)), sep = "_")

overlap <- metaGR(gr1 = overlap, gr2 = rd_h3k27ac_filtered)
```

8.3.2 Finalizing and saving the integrated object

Now that all of our multi-omics metadata is stitched together onto our foundational ATAC-seq peaks, we assign a unique ID (`name`) to every peak. This ensures we can easily track specific genomic regions during downstream analysis. Finally, we save the completed `GRanges` object to disk so it can be instantly loaded in future notebooks.

```
# Add a unique name field to the metadata and GRanges names
overlap$name <- paste("peak", 1:length(overlap), sep = "_")
names(overlap) <- overlap$name

# Create an output directory (if it doesn't exist) and save the object
dir.create("output", showWarnings = FALSE)
saveRDS(object = overlap, file = "output/overlap_data.rds")
```

8.3.3 Inspecting the integrated object

Before we move on, let's verify that our multi-omics integration was successful. Our `overlap` object is still fundamentally an ATAC-seq `GRanges` object (meaning its physical coordinates represent open chromatin peaks), but its metadata now contains the statistical results from every other dataset we processed.

Note: Keep in mind that `overlap` is now a pure `GRanges` object, not a `SummarizedExperiment` (SE) container. Because of this, SE-specific functions like `rowRanges()` and `colData()` will no longer work. Instead, we must use standard `GRanges` functions to interact with the object, such as `length()` (to count the peaks), `mcols()` (to access the metadata), and `genome()`.

```
# Verify all the multi-omics metadata columns are present
print(colnames(mcols(overlap)))
```

[1] "ATAC_annotation"	"ATAC_geneChr"	"ATAC_geneStart"
[4] "ATAC_geneEnd"	"ATAC_geneLength"	"ATAC_geneStrand"
[7] "ATAC_geneId"	"ATAC_transcriptId"	"ATAC_distanceToTSS"

[10]	"ATAC_ENSEMBL"	"ATAC_SYMBOL"	"ATAC_GENENAME"
[13]	"ATAC_anno"	"ATAC_logFC"	"ATAC_logCPM"
[16]	"ATAC_F"	"ATAC_PValue"	"ATAC_qvalue"
[19]	"RNA_Row.names"	"RNA_gene_id"	"RNA_SYMBOL"
[22]	"RNA_GENENAME"	"RNA_ENSEMBL"	"RNA_geneLength"
[25]	"RNA_geneChr"	"RNA_geneStart"	"RNA_geneEnd"
[28]	"RNA_geneStrand"	"RNA_geneId"	"RNA_transcriptId"
[31]	"RNA_anno"	"RNA_logFC"	"RNA_logCPM"
[34]	"RNA_F"	"RNA_PValue"	"RNA_qvalue"
[37]	"H3K4me3_annotation"	"H3K4me3_geneChr"	"H3K4me3_geneStart"
[40]	"H3K4me3_geneEnd"	"H3K4me3_geneLength"	"H3K4me3_geneStrand"
[43]	"H3K4me3_geneId"	"H3K4me3_transcriptId"	"H3K4me3_distanceToTSS"
[46]	"H3K4me3_ENSEMBL"	"H3K4me3_SYMBOL"	"H3K4me3_GENENAME"
[49]	"H3K4me3_anno"	"H3K4me3_logFC"	"H3K4me3_logCPM"
[52]	"H3K4me3_F"	"H3K4me3_PValue"	"H3K4me3_qvalue"
[55]	"H3K4me1_annotation"	"H3K4me1_geneChr"	"H3K4me1_geneStart"
[58]	"H3K4me1_geneEnd"	"H3K4me1_geneLength"	"H3K4me1_geneStrand"
[61]	"H3K4me1_geneId"	"H3K4me1_transcriptId"	"H3K4me1_distanceToTSS"
[64]	"H3K4me1_ENSEMBL"	"H3K4me1_SYMBOL"	"H3K4me1_GENENAME"
[67]	"H3K4me1_anno"	"H3K4me1_logFC"	"H3K4me1_logCPM"
[70]	"H3K4me1_F"	"H3K4me1_PValue"	"H3K4me1_qvalue"
[73]	"H3K27me3_annotation"	"H3K27me3_geneChr"	"H3K27me3_geneStart"
[76]	"H3K27me3_geneEnd"	"H3K27me3_geneLength"	"H3K27me3_geneStrand"
[79]	"H3K27me3_geneId"	"H3K27me3_transcriptId"	"H3K27me3_distanceToTSS"
[82]	"H3K27me3_ENSEMBL"	"H3K27me3_SYMBOL"	"H3K27me3_GENENAME"
[85]	"H3K27me3_anno"	"H3K27me3_logFC"	"H3K27me3_logCPM"
[88]	"H3K27me3_F"	"H3K27me3_PValue"	"H3K27me3_qvalue"
[91]	"H3K27ac_annotation"	"H3K27ac_geneChr"	"H3K27ac_geneStart"
[94]	"H3K27ac_geneEnd"	"H3K27ac_geneLength"	"H3K27ac_geneStrand"
[97]	"H3K27ac_geneId"	"H3K27ac_transcriptId"	"H3K27ac_distanceToTSS"
[100]	"H3K27ac_ENSEMBL"	"H3K27ac_SYMBOL"	"H3K27ac_GENENAME"
[103]	"H3K27ac_anno"	"H3K27ac_logFC"	"H3K27ac_logCPM"
[106]	"H3K27ac_F"	"H3K27ac_PValue"	"H3K27ac_qvalue"
[109]	"name"		

```
# Chromosomes included in the overlap matrix
print(seqlevelsInUse(overlap))
```

```
[1] "chr1" "chr2"
```

8.3.4 Processing ATAC and ChIP-seq continuous signals

To visualize the continuous signal of our epigenetic marks (using `.bw` files), we first need to decide how wide of a genomic window we want to look at. Because different histone modifications have entirely different biological footprints (some are sharp, some are massive and sprawling), the [ENCODE Project Consortium](#) and the current literature provide standard recommendations:

Assays	Typical Window Size	Reason / Notes
ATAC-seq	± 1 kb	Narrow peaks, accessibility centered on summit
H3K4me3	± 1 kb to ± 2 kb	Promoter mark, sharp and narrow peaks
H3K4me1	± 2 kb to ± 5 kb	Enhancer-associated, moderately broad
H3K27me3	± 5 kb to ± 10 kb	Broad repressive domains
H3K27ac	± 2 kb to ± 5 kb	Active enhancers/promoters, broader peaks

8.3.5 Verifying our dataset's peak widths

Let's now check the actual physical widths of the significant peaks we just filtered. Remind that we already did some preliminary visual exploration of this in the [previous notebook](#), where we plotted width histograms for all four histone marks.

Here, we will leverage R's built-in `summary()` function to efficiently calculate the complete statistical profile of our peak widths (Min, 1st Quartile, Median, Mean, 3rd Quartile, and Max) in a single, clean table. This allows us to quantitatively verify that our data follows the expected biological behavior (e.g., proving that our H3K27me3 peaks are physically much broader than our sharp ATAC peaks).

```
# Bundle the filtered objects into a named list (same as before)
gr_list <- list(
  "ATAC-seq" = rd_atac_filtered,
  "H3K4me3"  = rd_h3k4me3_filtered,
  "H3K4me1"  = rd_h3k4me1_filtered,
  "H3K27me3" = rd_h3k27me3_filtered,
  "H3K27ac"  = rd_h3k27ac_filtered
)

# Use summary() to calculate all stats, which outputs a matrix
stats_matrix <- sapply(gr_list, function(gr) summary(width(gr)))

# Transpose (t) the matrix so Assays are rows, and convert to a clean tibble
summary_table <- as_tibble(t(stats_matrix), rownames = "Assay")
```

```
print(summary_table)
```

```
# A tibble: 5 × 7
  Assay      Min. `1st Qu.` Median  Mean `3rd Qu.`  Max.
  <chr>    <dbl>    <dbl>  <dbl> <dbl>    <dbl> <dbl>
1 ATAC-seq   151      640    932  956.    1236  3490
2 H3K4me3    191      352    749 1316.    1684. 18723
3 H3K4me1    171      349    563  791.     975 12590
4 H3K27me3   176      367    568 1170.    1108. 23571
5 H3K27ac    171      334    618  914.    1216. 22276
```

Notice that we used `sapply()` here instead of `lapply()`.

- **lapply** (List apply) always returns a basic **list**. If we used it here, we would get a clunky list containing 5 separated summary objects, which is annoying to stitch together.
- **sapply** (Simplified apply) does the exact same loop, but attempts to **simplify** the final output. Because `summary()` returns exactly 6 numbers for every assay, **sapply** is smart enough to automatically bind them together into a ready-to-use mathematical matrix!

8.3.6 Unifying the coordinate system (centering on ATAC)

As seen in the summary table, our peaks have highly variable widths. Plotting them using their native sizes would result in misaligned heatmaps. To create a unified coordinate system, we will use the exact center (`mid`) of the ATAC-seq peaks as a universal anchor point, since open chromatin serves as the regulatory hub for these regions.

Next, we must determine a standardized window size for our analysis. While a few outlier peaks stretch for many kilobases, the core biological signal is much tighter. The median widths for most marks sit between 500bp and 950bp, and over 75% of our peaks (the 3rd Quartile) are smaller than 1.6kb.

It is important to remember a key structural detail about how we built our `overlap` object: because we used ATAC-seq as our foundational `gr1` target during the `metaGR()` joining process, the physical coordinates of the `overlap` object **are** the exact ATAC-seq peaks.

```
# Returns ONLY the pure physical coordinates
granges(overlap)
```

GRanges object with 1933 ranges and 0 metadata columns:

	seqnames	ranges	strand
	<Rle>	<IRanges>	<Rle>
peak_1	chr1	3670547-3672665	*
peak_2	chr1	4332510-4332710	*
peak_3	chr1	4491755-4492573	*
peak_4	chr1	4571186-4572423	*
peak_5	chr1	4785062-4786325	*
...	...	...	...
peak_1929	chr2	181713633-181714134	*
peak_1930	chr2	181714476-181715931	*
peak_1931	chr2	181762779-181764180	*
peak_1932	chr2	181764664-181764891	*
peak_1933	chr2	181837244-181838655	*

seqinfo: 21 sequences from an unspecified genome; no seqlengths

We can thus simply apply the `resize()` function directly to our master `overlap` object, and it will instantly collapse every peak in our table down to the exact center point of the open chromatin. Moreover, each unique peak is already perfectly identified in its own row.

```
# Extract midpoints of peaks to create windows for visualization
# mid_peaks <- gr.mid(overlap)
mid_peaks <- resize(overlap, width = 1, fix = "center")
```

Because the vast majority of our dynamic biological signal fits within a 2,000 base-pair footprint, we will define our universal extraction regions (`heatmap_windows`) as a $\pm 1\text{kb}$ window anchored directly on the ATAC-seq `mid_peaks`. This standardizes our coordinate system and captures the relevant signal of every mark while minimizing empty background noise.

```
# Define the actual 2000 bp windows we need for the import step
heatmap_windows <- resize(mid_peaks, width = 2000, fix = "center")
```

```
print(seqlevelsInUse(heatmap_windows))
```

```
[1] "chr1" "chr2"
```

8.3.7 Extracting and normalizing bigWig signals

As we saw in the previous notebooks, to process the [ATAC](#) and [ChIP-seq](#) `.bw` files efficiently, we use a clean `lapply()` loop for each dataset package. Within these loops, we use

`rtracklayer::import()` and pass our specific 2kb windows to the `which` argument. This forces R to dynamically read the continuous signal coverage **only** within our exact targeted regions directly from the hard drive, saving RAM and computation time.

Once the targeted signal is read, the exact same loop immediately feeds it into `normalizeToMatrix()`, outputting clean lists of normalized matrices that we save as `.rds` files for downstream visualization.

```
# -- Process ATAC ---
# (See notebook 02-atac_heatmaps.ipynb for all the details)
atac_files <- list.files("data", pattern = "ATAC", full.names = TRUE)
names(atac_files) <- gsub(pattern = "\\\\.bw", replacement = "", x = basename(atac_files))

atac_bw <- lapply(atac_files, function(x){
  a <- rtracklayer::import(x, which = heatmap_windows)
  return(a)})

mat_AS <- lapply(atac_bw, FUN = function(x) {
  normalizeToMatrix(
    x, mid_peaks, mean_mode = "w0", extend = 1000, value_column = "score",
    include_target = TRUE, w = 20, smooth = T, background = 0)})

saveRDS(mat_AS, file = "output/mat_atac.rds")
```

Warning message:

"Width of `target` are all 1, `include\_target` is set to `FALSE`."

Warning message:

"Width of `target` are all 1, `include\_target` is set to `FALSE`."

```
# -- Process ChIP ---
# (See notebook 03-chip_rna_visualization.ipynb for all the details)
chip_bw_files <- list.files("data", pattern = "^H3K.*\\.bw$", full.names = TRUE)
names(chip_bw_files) <- gsub("\\\\.bw$", "", basename(chip_bw_files))

chip_matrices <- lapply(chip_bw_files, function(file_path) {

  signal <- rtracklayer::import(file_path, which = heatmap_windows)
  mat <- normalizeToMatrix(
    signal,
    target = mid_peaks,
    value_column = "score",
    extend = 1000,
```

```

    w = 20,
    smooth = TRUE,
    background = 0)
return(mat)}

```

Because we grouped all 8 matrices into a single named list (`chip_matrices`), the most efficient practice is to save the entire list as a single `.rds` file. However, to ensure perfect compatibility with the downstream code of the original course (which expects individual matrix files), we can also use a quick loop to dynamically save them as separate files based on their names.

```

# Best Practice: Save the entire master list as a single object
saveRDS(chip_matrices, file = "output/chip_matrices.rds")

# Group matrices by histone mark and save each mark's list (11half & 15half) as an individual
marks <- unique(gsub("_.*", "", names(chip_matrices))) # Extracts just "H3K4me3", "H3K27ac",

# Wrapping in invisible() hides the NULLs from printing to the screen
invisible(lapply(marks, function(mark) {
  # Subset the list to grab both timepoints for this specific mark
  mark_list <- chip_matrices[grepl(mark, names(chip_matrices))]

  # Create a clean lowercase filename like: "output/mat_h3k4me3.rds"
  file_name <- paste0("output/mat_", tolower(mark), ".rds")
  saveRDS(mark_list, file = file_name)
}))

```

8.4 Processing RNA-seq discrete signals

Remind that the RNA-seq data behaves a bit differently than the ATAC/ChIP bigWig files because RNA-seq data is discrete (counts mapped to a gene) rather than continuous genomic coverage. Because of this, we don't use `normalizeToMatrix()`. Instead, we just grab the relative fold-changes, just like we did in [Notebook 03](#), and use a standard `merge()` to map those gene scores directly onto our ATAC peak names.

As a reminder, rather than plotting raw expression counts, we calculate the relative fold-change by taking the `logCPM` matrix and subtracting the baseline mean of our E11.5 control samples. We will apply that exact same math here to generate those centered expression scores. Then, using a standard `merge()`, we will map those gene scores directly onto our corresponding ATAC peak names to complete our RNA-seq matrix.

```

# -- Process RNA ---
# Create an index mapping the overlapping ATAC peak names to their RNA-seq gene names
# peak_gene_map <- elementMetadata(overlap)[,c("RNA_Row.names", "name")]
peak_gene_map <- mcols(overlap)[, c("RNA_Row.names", "name")]

# (See notebook 03-chip_rna_visualization.ipynb for all the details)
log_cpm <- as.matrix(assay(rna, "logCPM"))
idx_11half <- grep("11half", colnames(log_cpm))
counts_fc <- log_cpm - rowMeans(log_cpm[, idx_11half])

# Merge the gene expression fold-changes with our ATAC peak names using the map
mat_RNA <- merge(
  peak_gene_map,
  counts_fc,
  by.x = "RNA_Row.names",
  by.y = "row.names",
  all.x = TRUE)

rownames(mat_RNA) <- mat_RNA$name

# Safely drop the text ID columns by name so the matrix is purely numeric
# mat_RNA <- mat_RNA[, -c(1:2)]
mat_RNA$RNA_Row.names <- NULL
mat_RNA$name <- NULL

# Clean up column names by removing the file extension
colnames(mat_RNA) <- gsub(
  pattern = "\\..tsv\\.gz", replacement = "", x = colnames(mat_RNA))

# Cast back to a true numeric matrix
mat_RNA <- data.matrix(mat_RNA)
# Strictly reorder to match the master overlap object!
mat_RNA <- mat_RNA[names(overlap), ]

saveRDS(mat_RNA, file = "output/mat_rna.rds")

# Print the first 5 rows of the RNA matrix
print(mat_RNA[1:5,])

```

	RNA_11half_1	RNA_11half_2	RNA_15half_1	RNA_15half_2
peak_1	-0.05267984	0.05267984	1.173054	1.538339
peak_2	NA	NA	NA	NA

peak_3	NA	NA	NA	NA
peak_4	NA	NA	NA	NA
peak_5	NA	NA	NA	NA

When merging the RNA data onto our ATAC peaks, the `all.x = TRUE` argument performs a **Left Join**. This forces R to keep every single ATAC peak in our master list (`x`), even if it doesn't have a matching significant RNA gene (it simply fills those empty RNA rows with `NA`). If we forgot this argument, R would default to an "Inner Join" and instantly delete all intergenic ATAC peaks from our matrix.

Because we mapped the RNA data onto our ATAC peaks using `metaGR()`, and our ATAC peaks were already restricted to `chr1` and `chr2`.

8.5 Processing WGBS (DNA Methylation) signals

8.5.1 Reading the WGBS coverage files

Whole Genome Bisulfite Sequencing (WGBS) measures DNA methylation at CpG dinucleotides. Unlike our ATAC-seq and ChIP-seq data (which is stored in binary `.bw` files), WGBS data is provided as compressed text `.bed.gz` files. These files contain explicit columns for read coverage (`cov`), Methylated counts (`Me`), Unmethylated counts (`Un`), and the final methylation ratio (`meth`).

```
print(dir("data/"))
```

```
[1] "ATAC_11half.bw"      "ATAC_15half.bw"      "atac_se.rds"
[4] "H3K27ac_11half.bw"   "H3K27ac_15half.bw"   "h3k27ac_se.rds"
[7] "H3K27me3_11half.bw"  "H3K27me3_15half.bw"  "h3k27me3_se.rds"
[10] "H3K4me1_11half.bw"   "H3K4me1_15half.bw"   "h3k4me1_se.rds"
[13] "H3K4me3_11half.bw"   "H3K4me3_15half.bw"   "h3k4me3_se.rds"
[16] "RNA_11half_1.tsv.gz" "RNA_11half_2.tsv.gz" "RNA_15half_1.tsv.gz"
[19] "RNA_15half_2.tsv.gz" "rna_se.rds"           "WGBS_11half.bed.gz"
[22] "WGBS_15half.bed.gz"
```

Because these are text files, we will use the lightning-fast `data.table::fread()` function to read them into R, convert them into `GRanges` objects, and then feed them into `normalizeToMatrix()`.

```

# Bisulfite seq coverage files after methylation call
bs_files <- list.files("data", pattern = "WGBS", full.names = TRUE)
names(bs_files) <- gsub(
  pattern = "\\..bed.gz", replacement = "", x = basename(bs_files))

# Read the fast text files and convert to GRanges
bs_cov <- lapply(bs_files, function(x) {
  gr <- GRanges(
    fread(
      input = x,
      sep = " ", quote = F, stringsAsFactors = F, data.table = FALSE,
      nThread = parallel::detectCores(), showProgress = F,
      col.names = c("seqnames", "start", "end", "cov", "Me", "Un", "meth")
    )
  )

  # Filter for only chr1 and chr2
  gr <- subset(gr, seqnames %in% c("chr1", "chr2"))

  return(gr)
})

```

8.5.2 Handling high-resolution WGBS data

We didn't use `metaGR()` to pull the WGBS methylation data into our master `overlap` table alongside the ATAC, ChIP, and RNA-seq data because of the specific **genomic resolution**.

- **ATAC, ChIP, and RNA** are all “Region-level” data. When an ATAC peak (~1kb) overlaps a gene or a ChIP peak, it is generally a simple 1-to-1 relationship, which `metaGR()` handles perfectly.
- **WGBS data** operates at **Single Base-Pair Resolution** (individual CpG sites).

A single 1kb ATAC-seq peak might contain **50 different CpG sites**. If we used `metaGR()` to join them, the spatial join engine would blindly grab the methylation data of the very *first* CpG it touched and throw the other 49 away! This would result in completely inaccurate methylation profiling for that regulatory region.

Instead of forcing single-bp statistics into our master table, we handle WGBS data entirely during the **Continuous Signal** extraction phase.

By passing our WGBS `GRanges` objects directly into `normalizeToMatrix()`, the software properly reads all 50 CpGs inside the 2kb window and smoothly averages them across the ATAC peak, allowing us to visualize the true methylation burden correctly.

Because we are measuring a methylation *percentage* instead of standard read pileups, we must change two critical arguments in `normalizeToMatrix()`: `* value_column = "meth"`: We explicitly extract the methylation ratio rather than the read coverage depth. `* mean_mode = "absolute"`: By default, the software calculates a weighted mean ("`w0`"), which weights the signal based on how physically wide the overlapping genomic regions are (e.g., broad ChIP-seq peaks). Because our WGBS data operates at the single-nucleotide level (where every CpG is exactly 1 base pair wide), weighting by width is unnecessary. We override this to "`absolute`" to tell the function to simply take the straight, arithmetic mean of the methylation scores inside each window bin. `* background = N`: In ChIP-seq, if there are no reads, the signal is 0. But in WGBS, if there are no reads covering a CpG, we simply don't know the methylation status! Therefore, the background *must* be NA (unknown). If we left it as 0, we would mathematically trick R into thinking those unsequenced regions were completely unmethylated!

```
# Normalized matrices
mat_bs <- lapply(bs_cov, FUN = function(x) {
  normalizeToMatrix(
    x, mid_peaks,
    value_column = "meth",
    mean_mode = "absolute",
    extend = 1000,
    # include_target = TRUE,
    smooth = TRUE,
    background = NA)})

saveRDS(mat_bs, file = "output/mat_bs.rds")
```

Warning message:

"Smoothing is failed for one row because there are very few signals overlapped to it. Please use ``failed_rows(mat)`` to get the index of the failed row and consider to remove it."

All signal values are within [0, 1], so we assume it is methylation signal. Automatically set limit [0, 1] to the smoothed values. If this is not the case, set argument ``limit = NA`` in the function to remove the limits. Set ``verbose = FALSE`` to turn off this message.

Warning message:

"Smoothing is failed for one row because there are very few signals overlapped to it. Please use ``failed_rows(mat)`` to get the index of the failed row and consider to remove it."

```
# Print a few rows and columns from the first WGBS matrix
print(mat_bs$WGBS_11half[1:5, 45:50])
```

```
      u45      u46      u47      u48      u49      u50
peak_1 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000 0.0000000
peak_2 0.0625000 0.0625000 0.0625000 0.0625000 0.0625000 0.0625000
peak_3 0.1956267 0.1890559 0.1824142 0.1757514 0.1691174 0.1625621
peak_4 0.2356774 0.2323634 0.2276826 0.2218883 0.2152339 0.2079729
peak_5 0.2162887 0.2130513 0.2100990 0.2082533 0.2083358 0.2095957
```

8.5.3 Question 1

Make an **EnrichedHeatmap** of methylation data.

To create our composite **EnrichedHeatmap** for the methylation data, we can take full advantage of the highly modular code architecture we developed in earlier notebooks.

First, we need to upgrade our custom plotting function (**make_EH_custom**). Because our WGBS matrices contain NA values (to represent unsequenced background), our original color-scaling math would instantly crash. We simply need to update the **quantile()** math inside our custom function to include **na.rm = TRUE**, allowing it to safely ignore missing data.

Furthermore, we need to upgrade the function's color-mapping capabilities. The **colorRamp2** function strictly requires the exact same number of numerical anchor points as there are colors in the provided palette. To make our custom function truly universal, i.e., it can dynamically handle a 2-color gradient, a 3-color diverging map (like blue-white-red), or even a massive 11-color continuous palette, we will use the **seq()** function. This automatically generates a perfectly spaced mathematical sequence of anchor points from **vmin** to **vmax** that matches the exact length of any color palette we pass to it.

```
make_EH_custom_na <- function(
  norm_mat, hm_name, split_rows = NULL,
  width = unit(1, "null"), height = NULL, top_anno_height = grid::unit(1.5, "cm"),
  pal = RColorBrewer::brewer.pal(n = 9, name = "RdBu"),
  col_fill = "white", line_colors = NULL) {

  # Added na.rm = TRUE to handle WGBS missing values
  vmin <- as.numeric(quantile(norm_mat, c(0.01), na.rm = TRUE))
  vmax <- as.numeric(quantile(norm_mat, c(0.99), na.rm = TRUE))
  vmid <- (vmin + vmax) / 2
  legend_ticks <- c(vmin, vmid, vmax)
```

```

# Generate exactly enough anchor points to match the palette size
color_breaks <- seq(
  from = vmin, to = vmax, length.out = length(pal))

col_fun <- colorRamp2(
  breaks = color_breaks,
  colors = pal)

EnrichedHeatmap(
  mat = norm_mat, name = hm_name, row_split = split_rows,
  width = width, height = height,
  row_title = NULL, col = col_fun, column_title = hm_name,
  column_title_gp = gpar(fontsize = 10, fill = col_fill),
  axis_name = c("-1kb", "mid", "1kb"),
  heatmap_legend_param = list(
    at = legend_ticks, labels = round(legend_ticks, digits = 1),
    title_gp = gpar(fontsize = 8), labels_gp = gpar(fontsize = 7)),
  top_annotation = HeatmapAnnotation(
    lines = anno_enriched(
      height = top_anno_height,
      axis_param = list(facing = "inside"),
      gp = gpar(col = line_colors))))}

```

Notice that inside our `make_EH_custom_na` function, the x-axis labels are strictly hardcoded as `axis_name = c("-1kb", "mid", "1kb")`. This works perfectly for our current analysis because we explicitly chose a 2kb total window earlier.

However, if we decide to experiment and change our `extend` window size in the future (e.g., expanding to $\pm 5\text{kb}$ for broad marks), these visual labels will be mathematically incorrect! For now, we will leave it as-is, but we will look at ways to make these labels dynamic and fully “upgrade” this function later in the course.

Next, we will extract the `ATAC_anno` metadata directly from our master `overlap` object to dynamically build the left-side color bar (which splits our peaks into “Promoter”, “Intron”, and intergenic groups).

```

split_anno <- mcols(overlap)$ATAC_anno
names(split_anno) <- names(overlap)

anno_colors <- RColorBrewer::brewer.pal(
  n = length(unique(split_anno)), name = "Set1")
names(anno_colors) <- sort(unique(split_anno))

```

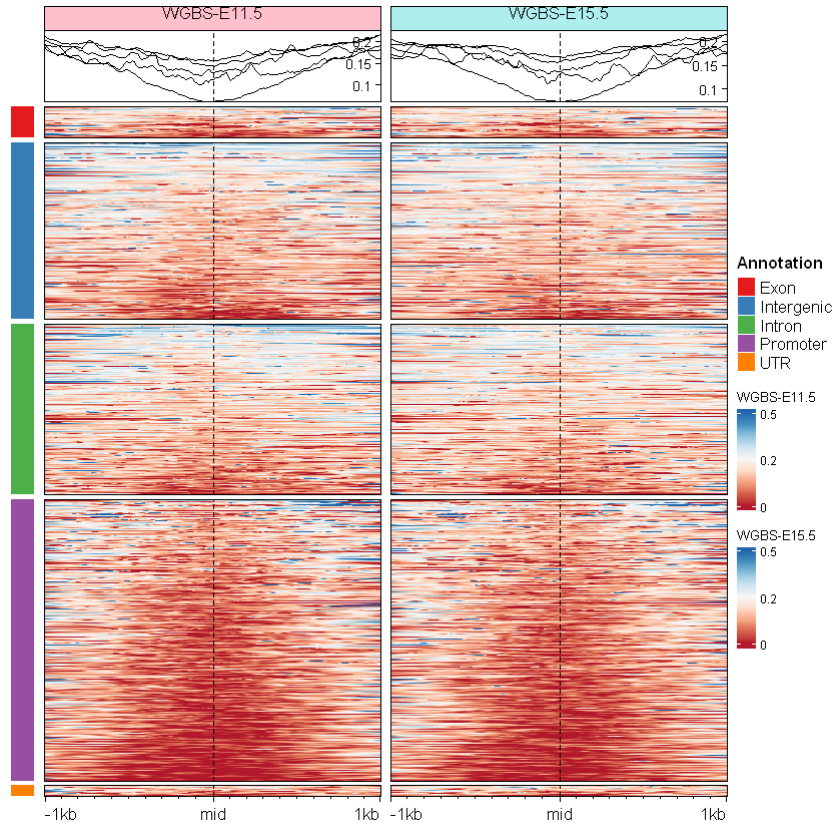
```
eh_annotations <- rowAnnotation(
  df = data.frame(Annotation = split_anno),
  col = list(Annotation = anno_colors),
  show_annotation_name = FALSE)
```

Finally, we will pass our two WGBS normalized matrices (E11.5 and E15.5) through our upgraded custom function, link them together with the annotation bar (+), and globally split the rows using the `draw()` function.

```
eh_wgbs_11h <- make_EH_custom_na(
  norm_mat = mat_bs[["WGBS_11half"]],
  hm_name = 'WGBS-E11.5',
  col_fill = 'pink')

eh_wgbs_15h <- make_EH_custom_na(
  norm_mat = mat_bs[["WGBS_15half"]],
  hm_name = 'WGBS-E15.5',
  col_fill = 'paleturquoise')

draw(
  eh_annotations + eh_wgbs_11h + eh_wgbs_15h,
  split = split_anno)
```



8.5.4 Biological interpretation of DNA methylation

Looking at our generated heatmaps, we can draw several immediate observational conclusions:

1. **No glaring global differences:** By naked eye, the global methylation profiles (Blue = 0% unmethylated, Red = 50% methylated) between E11.5 and E15.5 look virtually identical. There is no massive, genome-wide shift in methylation at these open chromatin regions between the two developmental stages.
2. **Promoters are unmethylated:** Thanks to our annotation splitting, we can clearly see that **Promoter** regions are intensely red (unmethylated). Furthermore, there is a distinct “dip” of unmethylated DNA exactly at the **mid** point of the ATAC peak. This makes perfect biological sense: active regulatory hubs must remain unmethylated to keep the DNA physically open for transcription factors.
3. **Introns and Intergenic regions are more methylated:** In contrast, the **Intron** and **Intergenic** blocks are noticeably more blue, indicating a higher level of basal methylation compared to the promoters.

While these visual **stratifications** provide fantastic biological context, we cannot definitively say if specific, individual peaks significantly gained or lost methylation between the two stages. To prove that mathematically, we would need to extract the raw **Me** and **Un** counts and run a heavy **statistical modeling** package (like DSS or methylKit) to generate formal **p-values** and **logFC** scores.

9 Integrating ATAC and RNA sequencing data

9.1 Learning objectives

- Load and manipulate `SummarizedExperiment` objects containing ATAC-seq and RNA-seq data.
- Understand how to categorize genomic regions based on combined accessibility (ATAC) and expression (RNA) changes.
- Annotate genomic regions with meaningful biological categories such as “Active”, “Repressed”, “Silent”, and others.
- Use the `ComplexHeatmap` and `EnrichedHeatmap` packages to visualize multi-omics data with customizable heatmaps.
- Implement and modify heatmap functions to integrate and compare ATAC and RNA signal patterns across samples.
- Adjust factor levels to control the order of categories in heatmaps for clear biological interpretation.
- Combine multiple heatmaps and annotations for integrative visualization of chromatin accessibility and gene expression.

9.2 Defining functional genomic states

Integrating chromatin accessibility (ATAC-seq) and gene expression (RNA-seq) data allows us to map the genome’s dynamic regulatory landscape. For this course, we will use a simplified, fold-change-driven approach to categorize these **functional states**. Because our datasets were already pre-filtered for **statistical significance**, we will use basic logical conditions on the `logFC` values to assign categories. This keeps the coding pipeline straightforward while effectively illustrating the core biological concepts.

Category	ATAC-RNA-seq		Biological interpretation	Analysis pipeline logic
Active	↑	↑	Open chromatin and high gene expression; marks an actively transcribed gene or highly active regulatory element.	ATAC_logFC > 0 & RNA_logFC > 0
Repressed	↓	↓	Closed chromatin and low/no gene expression; indicates a stably silenced gene or inactive regulatory element.	ATAC_logFC < 0 & RNA_logFC < 0
Silencer Activation	↑	↓	Chromatin becomes accessible while expression drops; indicates a repressor or silencer element becoming active.	ATAC_logFC > 0 & RNA_logFC < 0
Incongruent	↓	↑	Closed chromatin but high gene expression; suggests very efficient transcription or regulation from a highly distant active element.	ATAC_logFC < 0 & RNA_logFC > 0
Increased Accessibility	↑	NA (~ No change)	Chromatin becomes more accessible but expression remains unchanged; suggests a “primed” region prepared for future activation.	ATAC_logFC > 0 & is.na(RNA_logFC)
Decreased Accessibility	↓	NA (~ No change)	Chromatin becomes less accessible but expression remains unchanged; indicates a localized regulatory shift that does not impact overall transcripts.	ATAC_logFC < 0 & is.na(RNA_logFC)

Note: More details can be found in the [comprehensive epigenetic analysis reference guide](#).

In our [previous preparation steps](#), we explicitly filtered our ATAC and RNA datasets to only include features with statistically significant changes (`abs(logFC) >= .5` and `qvalue <= .1`). This pre-filtering ensures that categories requiring dual changes (like **Active** or **Repressed**) are highly robust.

However, because we removed non-significant genes prior to merging, an NA in our RNA column simply means a region didn’t overlap a *significantly changing* gene. To keep this course module straightforward, our pipeline maps these NA values to the “Increased/Decreased Accessibility” categories.

This limitation also highlights the challenge of distinguishing a genuinely ‘Silent’ state, characterized by decreased chromatin accessibility and an absolute lack of gene expression, from a ‘Repressed’ state. Attempting to programmatically assign a ‘Silent’ category using only missing

`logFC` values (`NA`) is inaccurate, as it conflates genes that are completely turned off with genes that are actively expressed but simply unchanged. For this reason, the ‘Silent’ category has been intentionally omitted from our simplified logic.

In a more advanced bioinformatics pipeline, we would merge the full, unfiltered datasets first. This would allow us to evaluate baseline raw counts (`baseMean`) to mathematically distinguish a genuinely **Silent** region (zero baseline transcripts) from a primed region where the gene is actively expressed but statistically unchanged.

9.3 Initial setup and data loading

Before integrating the data, we need to load our required R packages and import the processed `SummarizedExperiment` objects and normalized matrices that we generated in previous notebooks. Because all the heavy preprocessing and signal extraction is already complete, we no longer need to load intensive data-wrangling packages, and can focus entirely on visualization and integration.

```
suppressPackageStartupMessages(suppressWarnings({
  library(SummarizedExperiment)
  library(ComplexHeatmap)
  library(EnrichedHeatmap)
  library(circlize)
  library(dplyr) # used for case_when()
}))
```

9.3.1 Importing processed datasets

Here we load our master `overlap` annotation object and the normalized signal matrices.

Note that we are *not* loading the genome-wide `atac_se.rds` and `rna_se.rds` `SummarizedExperiment` objects in this notebook because we are focusing entirely on visualizing the highly significant, pre-filtered regions we extracted in the [previous notebook](#).

As a brief reminder, here are the core steps we took previously to generate these highly refined files:

- **Master overlap annotation object:** Generated by mapping physical overlaps between our highly significant ATAC-seq peak regions and RNA-seq gene annotations (using our custom `metaGR()` function), seamlessly joining their statistical metadata together into a single `GRanges` object.

- **Normalized signal matrices:** Created using `normalizeToMatrix()` to extract continuous signal coverage from bigWig files across fixed genomic windows (e.g., +/- 1 kb around peak centers), formatted as mathematical grids for immediate heatmap visualization.

```
overlap <- readRDS("output/overlap_data.rds")
mat_ATAC <- readRDS("output/mat_atac.rds")
mat_RNA <- readRDS("output/mat_rna.rds")
```

9.4 Categorizing genomic regions by multi-omics behavior

As we established in the biological framework during our introduction, combining ATAC-seq (chromatin accessibility) with RNA-seq (gene expression) allows us to mathematically infer the functional state of these regulatory elements.

Using the `logFC` directions from our master `overlap` object, we will explicitly annotate every peak into one of these core biological states:

- **Active:** Both accessibility and expression increase (e.g., chromatin opening to allow transcription machinery to bind).
- **Repressed:** Accessibility increases, but expression decreases (e.g., chromatin opening specifically to allow a transcriptional *repressor* to dock).
- **Silenced:** Both accessibility and expression decrease (e.g., chromatin physically tightening to shut down a gene).
- **Incongruent:** Accessibility decreases while expression increases (often driven by complex distal interactions or alternate promoters).
- **Inc. Acc. / Dec. Acc.:** Accessibility changes, but there is no corresponding RNA-seq gene data (e.g., distant intergenic regulatory regions).

```
# Use case_when to dynamically calculate the biological state in one elegant sweep
overlap$Regulatory_State <- case_when(
  overlap$ATAC_logFC > 0 & overlap$RNA_logFC > 0 ~ "Active",
  overlap$ATAC_logFC < 0 & overlap$RNA_logFC < 0 ~ "Silenced",
  overlap$ATAC_logFC > 0 & overlap$RNA_logFC < 0 ~ "Repressed",
  overlap$ATAC_logFC < 0 & overlap$RNA_logFC > 0 ~ "Incongruent",

  # For intergenic regions where there is no matching RNA gene
  overlap$ATAC_logFC > 0 & is.na(overlap$RNA_logFC) ~ "Inc. Acc.",
  overlap$ATAC_logFC < 0 & is.na(overlap$RNA_logFC) ~ "Dec. Acc.",

  # Default fallback if anything slips through (NA is not NULL!)
  # Also note that dtype is CHAR, so it cannot be NA
  TRUE ~ NA_character_
```

```
)
```

```
saveRDS(object = overlap, file = "output/overlap_regulatory_status.rds")
```

```
overlap
```

GRanges object with 1933 ranges and 110 metadata columns:

	seqnames	ranges	strand	ATAC_annotation
	<Rle>	<IRanges>	<Rle>	<character>
peak_1	chr1	3670547-3672665	*	Promoter (<=1kb)
peak_2	chr1	4332510-4332710	*	Intron (ENSMUST000000..
peak_3	chr1	4491755-4492573	*	Promoter (1-2kb)
peak_4	chr1	4571186-4572423	*	Distal Intergenic
peak_5	chr1	4785062-4786325	*	Promoter (<=1kb)
...	...	...	...	...
peak_1929	chr2	181713633-181714134	*	Promoter (<=1kb)
peak_1930	chr2	181714476-181715931	*	Promoter (<=1kb)
peak_1931	chr2	181762779-181764180	*	Promoter (<=1kb)
peak_1932	chr2	181764664-181764891	*	Promoter (1-2kb)
peak_1933	chr2	181837244-181838655	*	Promoter (<=1kb)

	ATAC_geneChr	ATAC_geneStart	ATAC_geneEnd	ATAC_geneLength
	<integer>	<integer>	<integer>	<integer>
peak_1	1	3214482	3671498	457017
peak_2	1	4344146	4360314	16169
peak_3	1	4492465	4493735	1271
peak_4	1	4491390	4497354	5965
peak_5	1	4773206	4785710	12505
...	...	...	...	...
peak_1929	2	181715016	181720985	5970
peak_1930	2	181715016	181720985	5970
peak_1931	2	181763332	181827797	64466
peak_1932	2	181763332	181827797	64466
peak_1933	2	181837854	181857461	19608

	ATAC_geneStrand	ATAC_geneId	ATAC_transcriptId
	<integer>	<character>	<character>
peak_1	2	497097	ENSMUST000000070533.4
peak_2	2	19888	ENSMUST000000027032.5
peak_3	2	20671	ENSMUST000000191939.1
peak_4	2	20671	ENSMUST000000192650.5
peak_5	2	27395	ENSMUST000000130201.7
...	...	...	...

peak_1929	1	18389	ENSMUST000000071585.9		
peak_1930	1	18389	ENSMUST000000071585.9		
peak_1931	1	17932	ENSMUST000000081125.10		
peak_1932	1	17932	ENSMUST000000081125.10		
peak_1933	1	245867	ENSMUST000000029116.13		
	ATAC_distanceToTSS	ATAC_ENSEMBL	ATAC_SYMBOL		
	<numeric>	<character>	<character>		
peak_1	0	ENSMUSG000000051951	Xkr4		
peak_2	27604	ENSMUSG000000025900	Rp1		
peak_3	1162	ENSMUSG000000025902	Sox17		
peak_4	-73832	ENSMUSG000000025902	Sox17		
peak_5	0	ENSMUSG000000033845	Mrpl15		
...	...	...	...		
peak_1929	-882	ENSMUSG000000027584	Oprl1		
peak_1930	0	ENSMUSG000000027584	Oprl1		
peak_1931	0	ENSMUSG000000010505	Myt1		
peak_1932	1332	ENSMUSG000000010505	Myt1		
peak_1933	0	ENSMUSG000000027589	Pcmdt2		
	ATAC_GENENAME	ATAC_anno	ATAC_logFC	ATAC_logCPM	ATAC_F
	<character>	<character>	<numeric>	<numeric>	<numeric>
peak_1	X-linked Kx blood gr..	Promoter	0.602984	10.45378	41.78446
peak_2	retinitis pigmentosa..	Intron	0.855748	5.18424	3.68371
peak_3	SRY (sex determining..	Promoter	-0.620926	7.13469	9.14522
peak_4	SRY (sex determining..	Intergenic	-0.511046	8.90885	15.15688
peak_5	mitochondrial riboso..	Promoter	0.540908	8.63469	15.08892
...	...	...	...	...	...
peak_1929	opioid receptor-like 1	Promoter	1.922716	6.37607	40.6570
peak_1930	opioid receptor-like 1	Promoter	1.213349	7.47748	37.2944
peak_1931	myelin transcription..	Promoter	-2.192918	7.94745	146.4501
peak_1932	myelin transcription..	Promoter	-1.275275	5.54135	10.4487
peak_1933	protein-L-isoasparta..	Promoter	0.689247	7.92175	16.6788
	ATAC_PValue	ATAC_qvalue	RNA_Row.names	RNA_gene_id	RNA_SYMBOL
	<numeric>	<numeric>	<character>	<character>	<character>
peak_1	6.58453e-08	4.81983e-07	ENSMUSG000000051951	497097	Xkr4
peak_2	6.13517e-02	9.28785e-02	<NA>	<NA>	<NA>
peak_3	4.12368e-03	8.38540e-03	<NA>	<NA>	<NA>
peak_4	3.27158e-04	8.67115e-04	<NA>	<NA>	<NA>
peak_5	3.36058e-04	8.82634e-04	<NA>	<NA>	<NA>
...	...	...	...	...	...
peak_1929	8.88771e-08	6.28500e-07	<NA>	<NA>	<NA>
peak_1930	2.22741e-07	1.40677e-06	ENSMUSG000000027584	18389	Oprl1
peak_1931	1.07660e-15	1.40573e-13	ENSMUSG000000010505	17932	Myt1
peak_1932	2.30922e-03	4.97558e-03	ENSMUSG000000010505	17932	Myt1

peak_1933	1.81039e-04	5.14680e-04		<NA>	<NA>	<NA>
	RNA_GENENAME		RNA_ENSEMBL	RNA_geneLength		
	<character>		<character>	<integer>		
peak_1	X-linked Kx blood gr..	ENSMUSG000000051951		457017		
peak_2	<NA>	<NA>		<NA>		
peak_3	<NA>	<NA>		<NA>		
peak_4	<NA>	<NA>		<NA>		
peak_5	<NA>	<NA>		<NA>		
...	...	...		...		
peak_1929	<NA>	<NA>		<NA>		
peak_1930	opioid receptor-like 1	ENSMUSG000000027584		5970		
peak_1931	myelin transcription..	ENSMUSG000000010505		64466		
peak_1932	myelin transcription..	ENSMUSG000000010505		64466		
peak_1933	<NA>	<NA>		<NA>		
	RNA_geneChr	RNA_geneStart	RNA_geneEnd	RNA_geneStrand	RNA_geneId	
	<character>	<integer>	<integer>	<character>	<character>	
peak_1	chr1	3214482	3671498	-	497097	
peak_2	<NA>	<NA>	<NA>	<NA>	<NA>	
peak_3	<NA>	<NA>	<NA>	<NA>	<NA>	
peak_4	<NA>	<NA>	<NA>	<NA>	<NA>	
peak_5	<NA>	<NA>	<NA>	<NA>	<NA>	
...	...	...	...	...	...	
peak_1929	<NA>	<NA>	<NA>	<NA>	<NA>	
peak_1930	chr2	181715016	181720985	+	18389	
peak_1931	chr2	181763332	181827797	+	17932	
peak_1932	chr2	181763332	181827797	+	17932	
peak_1933	<NA>	<NA>	<NA>	<NA>	<NA>	
	RNA_transcriptId	RNA_anno	RNA_logFC	RNA_logCPM	RNA_F	
	<character>	<character>	<numeric>	<numeric>	<numeric>	
peak_1	ENSMUST000000070533.4	Gene	1.37498	5.62452	33.7077	
peak_2	<NA>	<NA>	NA	NA	NA	
peak_3	<NA>	<NA>	NA	NA	NA	
peak_4	<NA>	<NA>	NA	NA	NA	
peak_5	<NA>	<NA>	NA	NA	NA	
...	...	...	...	...	...	
peak_1929	<NA>	<NA>	NA	NA	NA	
peak_1930	ENSMUST000000071585.9	Gene	1.87383	9.02372	145.077	
peak_1931	ENSMUST000000081125.10	Gene	-1.76404	8.78746	142.012	
peak_1932	ENSMUST000000081125.10	Gene	-1.76404	8.78746	142.012	
peak_1933	<NA>	<NA>	NA	NA	NA	
	RNA_PValue	RNA_qvalue	H3K4me3_annotation	H3K4me3_geneChr		
	<numeric>	<numeric>	<character>	<integer>		
peak_1	0.00224273	0.00628871	Promoter (<=1kb)	1		

peak_2	NA	NA	<NA>	<NA>
peak_3	NA	NA	<NA>	<NA>
peak_4	NA	NA	Distal Intergenic	1
peak_5	NA	NA	<NA>	<NA>
...	...	...	...	...
peak_1929	NA	NA	<NA>	<NA>
peak_1930	7.64282e-05	0.000816625	Promoter (<=1kb)	2
peak_1931	8.04360e-05	0.000822152	Promoter (<=1kb)	2
peak_1932	8.04360e-05	0.000822152	Promoter (<=1kb)	2
peak_1933	NA	NA	<NA>	<NA>

	H3K4me3_geneStart	H3K4me3_geneEnd	H3K4me3_geneLength
	<integer>	<integer>	<integer>
peak_1	3214482	3671498	457017
peak_2	<NA>	<NA>	<NA>
peak_3	<NA>	<NA>	<NA>
peak_4	4491390	4497354	5965
peak_5	<NA>	<NA>	<NA>
...	...	...	...
peak_1929	<NA>	<NA>	<NA>
peak_1930	181715016	181720985	5970
peak_1931	181763332	181827797	64466
peak_1932	181763332	181827797	64466
peak_1933	<NA>	<NA>	<NA>

	H3K4me3_geneStrand	H3K4me3_geneId	H3K4me3_transcriptId
	<integer>	<character>	<character>
peak_1	2	497097	ENSMUST00000070533.4
peak_2	<NA>	<NA>	<NA>
peak_3	<NA>	<NA>	<NA>
peak_4	2	20671	ENSMUST000000192650.5
peak_5	<NA>	<NA>	<NA>
...	...	...	...
peak_1929	<NA>	<NA>	<NA>
peak_1930	1	18389	ENSMUST000000071585.9
peak_1931	1	17932	ENSMUST000000081125.10
peak_1932	1	17932	ENSMUST000000081125.10
peak_1933	<NA>	<NA>	<NA>

	H3K4me3_distanceToTSS	H3K4me3_ENSEMBL	H3K4me3_SYMBOL
	<numeric>	<character>	<character>
peak_1	0	ENSMUSG000000051951	Xkr4
peak_2	NA	<NA>	<NA>
peak_3	NA	<NA>	<NA>
peak_4	-73878	ENSMUSG000000025902	Sox17
peak_5	NA	<NA>	<NA>

...	...	...	...
peak_1929	NA	<NA>	<NA>
peak_1930	0 ENSMUSG000000027584		Oprl1
peak_1931	0 ENSMUSG000000010505		Myt1
peak_1932	0 ENSMUSG000000010505		Myt1
peak_1933	NA	<NA>	<NA>
	H3K4me3_GENENAME	H3K4me3_anno	H3K4me3_logFC
	<character>	<character>	<numeric>
peak_1	X-linked Kx blood gr..	Promoter	0.607114
peak_2	<NA>	<NA>	NA
peak_3	<NA>	<NA>	NA
peak_4	SRY (sex determining..	Intergenic	0.567068
peak_5	<NA>	<NA>	NA
...	...	...	...
peak_1929	<NA>	<NA>	NA
peak_1930	opioid receptor-like 1	Promoter	1.97861
peak_1931	myelin transcription..	Promoter	-2.26675
peak_1932	myelin transcription..	Promoter	-2.26675
peak_1933	<NA>	<NA>	NA
	H3K4me3_F	H3K4me3_PValue	H3K4me3_qvalue
	<numeric>	<numeric>	<numeric>
peak_1	135.4429	4.65079e-31	1.28968e-29
peak_2	NA	NA	NA
peak_3	NA	NA	NA
peak_4	33.3348	8.04296e-09	6.25198e-08
peak_5	NA	NA	NA
...	...	...	...
peak_1929	NA	NA	NA
peak_1930	325.324	2.42880e-71	2.10325e-69
peak_1931	316.946	1.38385e-69	1.13845e-67
peak_1932	316.946	1.38385e-69	1.13845e-67
peak_1933	NA	NA	NA
	H3K4me1_geneChr	H3K4me1_geneStart	H3K4me1_geneEnd
	<integer>	<integer>	<integer>
peak_1	<NA>	<NA>	<NA>
peak_2	<NA>	<NA>	<NA>
peak_3	<NA>	<NA>	<NA>
peak_4	<NA>	<NA>	<NA>
peak_5	1	4783572	4785692
...	...	...	...
peak_1929	<NA>	<NA>	<NA>
peak_1930	2	181715703	181719372
peak_1931	<NA>	<NA>	<NA>

peak_1932	<NA>	<NA>	<NA>
peak_1933	2	181837913	181857446
	H3K4me1_geneLength	H3K4me1_geneStrand	H3K4me1_geneId
	<integer>	<integer>	<character>
peak_1	<NA>	<NA>	<NA>
peak_2	<NA>	<NA>	<NA>
peak_3	<NA>	<NA>	<NA>
peak_4	<NA>	<NA>	<NA>
peak_5	2121	2	27395
...	...	...	...
peak_1929	<NA>	<NA>	<NA>
peak_1930	3670	1	18389
peak_1931	<NA>	<NA>	<NA>
peak_1932	<NA>	<NA>	<NA>
peak_1933	19534	1	245867
	H3K4me1_transcriptId	H3K4me1_distanceToTSS	H3K4me1_ENSEMBL
	<character>	<numeric>	<character>
peak_1	<NA>	NA	<NA>
peak_2	<NA>	NA	<NA>
peak_3	<NA>	NA	<NA>
peak_4	<NA>	NA	<NA>
peak_5	ENSMUST00000132625.1	412	ENSMUSG000000033845
...	...	...	...
peak_1929	<NA>	NA	<NA>
peak_1930	ENSMUST00000184127.1	38	ENSMUSG000000027584
peak_1931	<NA>	NA	<NA>
peak_1932	<NA>	NA	<NA>
peak_1933	ENSMUST00000108754.7	318	ENSMUSG000000027589
	H3K4me1_SYMBOL	H3K4me1_GENENAME	H3K4me1_anno H3K4me1_logFC
	<character>	<character>	<character> <numeric>
peak_1	<NA>	<NA>	<NA> NA
peak_2	<NA>	<NA>	<NA> NA
peak_3	<NA>	<NA>	<NA> NA
peak_4	<NA>	<NA>	<NA> NA
peak_5	Mrpl15 mitochondrial riboso..	Promoter	0.674968
...	...	...	...
peak_1929	<NA>	<NA>	<NA> NA
peak_1930	Oprl1 opioid receptor-like 1	Promoter	1.086359
peak_1931	<NA>	<NA>	<NA> NA
peak_1932	<NA>	<NA>	<NA> NA
peak_1933	Pcmt2 protein-L-isoasparta..	Promoter	0.767955
	H3K4me1_logCPM	H3K4me1_F	H3K4me1_PValue H3K4me1_qvalue
	<numeric>	<numeric>	<numeric> <numeric>

peak_1	NA	NA	NA	NA
peak_2	NA	NA	NA	NA
peak_3	NA	NA	NA	NA
peak_4	NA	NA	NA	NA
peak_5	5.83337	12.1017	0.00050642	0.00802852
...	...	...	...	...
peak_1929	NA	NA	NA	NA
peak_1930	6.13490	36.7670	1.39101e-09	1.90365e-07
peak_1931	NA	NA	NA	NA
peak_1932	NA	NA	NA	NA
peak_1933	6.07534	18.4909	1.72723e-05	5.94889e-04

	H3K27me3_annotation	H3K27me3_geneChr	H3K27me3_geneStart
	<character>	<integer>	<integer>
peak_1	<NA>	<NA>	<NA>
peak_2	<NA>	<NA>	<NA>
peak_3	Promoter (≤1kb)	1	4492465
peak_4	<NA>	<NA>	<NA>
peak_5	<NA>	<NA>	<NA>
...	...	...	...
peak_1929	<NA>	<NA>	<NA>
peak_1930	<NA>	<NA>	<NA>
peak_1931	<NA>	<NA>	<NA>
peak_1932	<NA>	<NA>	<NA>
peak_1933	<NA>	<NA>	<NA>

	H3K27me3_geneEnd	H3K27me3_geneLength	H3K27me3_geneStrand
	<integer>	<integer>	<integer>
peak_1	<NA>	<NA>	<NA>
peak_2	<NA>	<NA>	<NA>
peak_3	4493735	1271	2
peak_4	<NA>	<NA>	<NA>
peak_5	<NA>	<NA>	<NA>
...	...	...	...
peak_1929	<NA>	<NA>	<NA>
peak_1930	<NA>	<NA>	<NA>
peak_1931	<NA>	<NA>	<NA>
peak_1932	<NA>	<NA>	<NA>
peak_1933	<NA>	<NA>	<NA>

	H3K27me3_geneId	H3K27me3_transcriptId	H3K27me3_distanceToTSS
	<character>	<character>	<numeric>
peak_1	<NA>	<NA>	NA
peak_2	<NA>	<NA>	NA
peak_3	20671	ENSMUST00000191939.1	0
peak_4	<NA>	<NA>	NA

peak_5	<NA>	<NA>	NA
...	...	...	...
peak_1929	<NA>	<NA>	NA
peak_1930	<NA>	<NA>	NA
peak_1931	<NA>	<NA>	NA
peak_1932	<NA>	<NA>	NA
peak_1933	<NA>	<NA>	NA

	H3K27me3_ENSEMBL	H3K27me3_SYMBOL	H3K27me3_GENENAME
	<character>	<character>	<character>
peak_1	<NA>	<NA>	<NA>
peak_2	<NA>	<NA>	<NA>
peak_3	ENSMUSG00000025902	Sox17	SRY (sex determining..
peak_4	<NA>	<NA>	<NA>
peak_5	<NA>	<NA>	<NA>
...	...	...	...
peak_1929	<NA>	<NA>	<NA>
peak_1930	<NA>	<NA>	<NA>
peak_1931	<NA>	<NA>	<NA>
peak_1932	<NA>	<NA>	<NA>
peak_1933	<NA>	<NA>	<NA>

	H3K27me3_anno	H3K27me3_logFC	H3K27me3_logCPM	H3K27me3_F
	<character>	<numeric>	<numeric>	<numeric>
peak_1	<NA>	NA	NA	NA
peak_2	<NA>	NA	NA	NA
peak_3	Promoter	-1.0597	9.04668	87.0802
peak_4	<NA>	NA	NA	NA
peak_5	<NA>	NA	NA	NA
...	...	...	...	...
peak_1929	<NA>	NA	NA	NA
peak_1930	<NA>	NA	NA	NA
peak_1931	<NA>	NA	NA	NA
peak_1932	<NA>	NA	NA	NA
peak_1933	<NA>	NA	NA	NA

	H3K27me3_PValue	H3K27me3_qvalue	H3K27ac_annotation	H3K27ac_geneChr
	<numeric>	<numeric>	<character>	<integer>
peak_1	NA	NA	Promoter (<=1kb)	1
peak_2	NA	NA	<NA>	<NA>
peak_3	1.32026e-20	5.28858e-18	<NA>	<NA>
peak_4	NA	NA	<NA>	<NA>
peak_5	NA	NA	Promoter (<=1kb)	1
...	...	...	...	...
peak_1929	NA	NA	Promoter (<=1kb)	2
peak_1930	NA	NA	Promoter (<=1kb)	2

peak_1931	NA	NA	Promoter (<=1kb)	2
peak_1932	NA	NA	Promoter (<=1kb)	2
peak_1933	NA	NA	Promoter (<=1kb)	2
	H3K27ac_geneStart	H3K27ac_geneEnd	H3K27ac_geneLength	
	<integer>	<integer>	<integer>	
peak_1	3214482	3671498	457017	
peak_2	<NA>	<NA>	<NA>	
peak_3	<NA>	<NA>	<NA>	
peak_4	<NA>	<NA>	<NA>	
peak_5	4783572	4785692	2121	
...	...	...	...	
peak_1929	181715016	181720985	5970	
peak_1930	181715016	181720985	5970	
peak_1931	181763332	181827797	64466	
peak_1932	181763332	181827797	64466	
peak_1933	181837854	181857461	19608	
	H3K27ac_geneStrand	H3K27ac_geneId	H3K27ac_transcriptId	
	<integer>	<character>	<character>	
peak_1	2	497097	ENSMUST00000070533.4	
peak_2	<NA>	<NA>	<NA>	
peak_3	<NA>	<NA>	<NA>	
peak_4	<NA>	<NA>	<NA>	
peak_5	2	27395	ENSMUST00000132625.1	
...	...	...	...	
peak_1929	1	18389	ENSMUST00000071585.9	
peak_1930	1	18389	ENSMUST00000071585.9	
peak_1931	1	17932	ENSMUST00000081125.10	
peak_1932	1	17932	ENSMUST00000081125.10	
peak_1933	1	245867	ENSMUST00000029116.13	
	H3K27ac_distanceToTSS	H3K27ac_ENSEMBL	H3K27ac_SYMBOL	
	<numeric>	<character>	<character>	
peak_1	0	ENSMUSG000000051951	Xkr4	
peak_2	NA	<NA>	<NA>	
peak_3	NA	<NA>	<NA>	
peak_4	NA	<NA>	<NA>	
peak_5	0	ENSMUSG000000033845	Mrpl15	
...	...	...	...	
peak_1929	0	ENSMUSG000000027584	Oprl1	
peak_1930	0	ENSMUSG000000027584	Oprl1	
peak_1931	0	ENSMUSG000000010505	Myt1	
peak_1932	0	ENSMUSG000000010505	Myt1	
peak_1933	0	ENSMUSG000000027589	Pcmdt2	
	H3K27ac_GENENAME	H3K27ac_anno	H3K27ac_logFC	H3K27ac_logCPM

	<character>	<character>	<numeric>	<numeric>
peak_1	X-linked Kx blood gr..	Promoter	1.25269	8.81735
peak_2	<NA>	<NA>	NA	NA
peak_3	<NA>	<NA>	NA	NA
peak_4	<NA>	<NA>	NA	NA
peak_5	mitochondrial riboso..	Promoter	1.07555	6.41736
...	...	...	...	...
peak_1929	opioid receptor-like 1	Promoter	1.363849	7.84616
peak_1930	opioid receptor-like 1	Promoter	1.363849	7.84616
peak_1931	myelin transcription..	Promoter	-0.640374	9.23253
peak_1932	myelin transcription..	Promoter	-0.640374	9.23253
peak_1933	protein-L-isoasparta..	Promoter	0.791927	6.70742
	H3K27ac_F	H3K27ac_PValue	H3K27ac_qvalue	name
	<numeric>	<numeric>	<numeric>	<character>
peak_1	151.4959	1.67395e-34	4.08130e-32	peak_1
peak_2	NA	NA	NA	peak_2
peak_3	NA	NA	NA	peak_3
peak_4	NA	NA	NA	peak_4
peak_5	26.5442	2.63728e-07	5.98352e-06	peak_5
...	...	...	...	...
peak_1929	104.9298	1.79151e-24	2.34334e-22	peak_1929
peak_1930	104.9298	1.79151e-24	2.34334e-22	peak_1930
peak_1931	53.0025	3.64753e-13	1.90481e-11	peak_1931
peak_1932	53.0025	3.64753e-13	1.90481e-11	peak_1932
peak_1933	18.4367	1.77717e-05	2.82934e-04	peak_1933
	Regulatory_State			
	<character>			
peak_1	Active			
peak_2	Inc. Acc.			
peak_3	Dec. Acc.			
peak_4	Dec. Acc.			
peak_5	Inc. Acc.			
...	...			
peak_1929	Inc. Acc.			
peak_1930	Active			
peak_1931	Silenced			
peak_1932	Silenced			
peak_1933	Inc. Acc.			

seqinfo: 21 sequences from an unspecified genome; no seqlengths

```
print(overlap$Regulatory_State[1:5])
```

```
[1] "Active"      "Inc. Acc." "Dec. Acc." "Dec. Acc." "Inc. Acc."
```

9.5 Visualizing multi-omics integration

To visually prove the direct relationship between chromatin accessibility and gene expression, we will leverage the modular architecture we built using the `EnrichedHeatmap` and `ComplexHeatmap` packages.

Our integration strategy will follow three clear steps:

1. **The ATAC-seq Heatmap:** We will reuse our robust `make_EH_custom_na()` wrapper function (which we successfully used for our ChIP-seq and WGBS data) to plot the continuous ATAC-seq signal over our regulatory regions. Crucially, we will split the rows of this heatmap by the newly calculated `Regulatory_State` categories we just defined.
2. **The RNA-seq Heatmap:** Because RNA-seq data is discrete (a single fold-change value per gene) rather than continuous genomic coverage, we cannot use `EnrichedHeatmap`. Instead, we will construct a standard `ComplexHeatmap` block to visualize the corresponding gene expression changes.
3. **Multi-omics Assembly:** Finally, we will stitch the continuous ATAC-seq block and the discrete RNA-seq block together side-by-side. This will allow us to visually validate our biological categories (e.g., verifying that the “Active” cluster truly has high accessibility and high expression).

9.5.1 Recycling our custom plotting function

To ensure our color scaling and formatting remains mathematically consistent across our entire course, we need to reuse the heatmap plotting function we perfected in previous notebooks (`custom_enrichedheatmap_plotting`).

Rather than redundantly copy-pasting this large block of code across every notebook, we have saved it into a dedicated, external R script. We can instantly load this tool into our environment using a single `source()` command.

```
# Load our custom multi-omics plotting tool from the external script
# (Note that the R kernel environment was launched from the root of the main project folder)
source("scripts/custom_enrichedheatmap_plotting.R")
```

While our custom wrapper perfectly automates the most critical layout settings, there are hundreds of other parameters available to tweak almost every single pixel of our plots! If we ever need to heavily customize specific fonts, borders, grid gaps, or annotation orientations, we should explore the incredibly comprehensive [official ComplexHeatmap documentation](#) and [EnrichedHeatmap documentation](#).

9.5.2 ATAC-seq signal heatmaps

We want to stratify the E11.5 and E15.5 ATAC-seq data using our newly calculated `Regulatory_State` categories. For that, we define an annotation vector (`split_anno`) to physically split the rows of the heatmap into functional clusters, and generate a narrow, adjacent color strip (`anno_hm`) to visually label each of these distinct biological states (Active, Silenced, Repressed, etc.), just as we did previously using genomic annotation.

```
split_state <- overlap$Regulatory_State
names(split_state) <- names(overlap)

print(head(split_state))
```

```
      peak_1      peak_2      peak_3      peak_4      peak_5      peak_6
"Active" "Inc. Acc." "Dec. Acc." "Dec. Acc." "Inc. Acc." "Dec. Acc."
```

Just as we did in the previous notebooks when we labeled our peaks by their physical location (Promoter vs. Intron) or genomic annotations (Exon, Intron, etc.), we will dynamically assign a distinct color from the `Set1` palette to each of our newly defined biological categories (Active, Silenced, Repressed, etc.).

```
# Dynamically grab enough colors from Set1 for our distinct biological states
state_names <- na.omit(sort(unique(split_state)))
anno_colors <- RColorBrewer::brewer.pal(n = length(state_names), name = "Set1")

names(anno_colors) <- state_names
```

Notice that we wrap our unique categories in `na.omit()`. Because our `case_when()` logic earlier included a fallback `NA` for any peaks that didn't perfectly match our mathematical criteria, the `unique()` function would accidentally count `NA` as a distinct biological category and try to assign a color to it! `na.omit()` safely purges any missing values from the list before we build our color palette.

```
eh_state_anno <- rowAnnotation(
  df = data.frame(Regulatory_State = split_state),
  col = list(Regulatory_State = anno_colors),
  show_annotation_name = FALSE,
  # show_legend = FALSE, # Hides the stubborn right-side legend
  annotation_legend_param = list(Regulatory_State = list(title = "Regulatory State")))
```

When assembling complex multi-omics figures, the **ComplexHeatmap** layout engine can force row annotation legends (like our **Regulatory State** colors) to strictly follow the continuous heatmaps on the right side of the plot. If we want to physically separate them and move our categorical metadata to the bottom while keeping the continuous heatmaps on the right, we have to bypass the automatic layout engine.

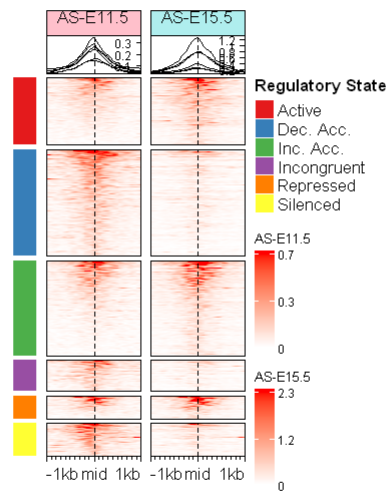
To achieve this structural layout, we first disable the automatic legend generation inside our **eh_state_anno** object (by passing **show_legend = FALSE**). We then use the advanced **Legend()** function to manually reconstruct it as a standalone object (**lgd_state**), formatting it to print horizontally. We can thus inject this custom legend directly into our **draw()** function via the **annotation_legend_list** argument, giving us absolute architectural control over where it is ultimately placed.

```
eh_atac_11h <- custom_enrichedheatmap_plotting(
  norm_mat = mat_ATAC[["ATAC_11half"]],
  hm_name = "AS-E11.5",
  hm_width = unit(2, "cm"), hm_height = unit(8, "cm"),
  top_anno_height = unit(.8, "cm"),
  col_fill = "pink"
)

eh_atac_15h <- custom_enrichedheatmap_plotting(
  norm_mat = mat_ATAC[["ATAC_15half"]],
  hm_name = "AS-E15.5",
  hm_width = unit(2, "cm"), hm_height = unit(8, "cm"),
  top_anno_height = unit(.8, "cm"),
  col_fill = "paleturquoise"
)

# Manually build the Regulatory State legend so we can force it to the bottom
# lgd_state <- Legend(
#   labels = names(anno_colors),
#   legend_gp = gpar(fill = anno_colors),
#   title = "Regulatory State",
#   direction = "horizontal")
```

```
draw(
  eh_state_anno + eh_atac_11h + eh_atac_15h,
  # annotation_legend_list = lgd_state,
  split = split_state,
)
```



9.5.3 Interpreting ATAC-seq accessibility clusters

As expected, the categories defined by decreasing accessibility (**Dec. Acc.**, **Incongruent**, and **Silenced**) show a *clear, global loss of ATAC signal* as we transition from E11.5 to E15.5. The chromatin in these regions is physically *shutting down* during development. Conversely, the categories defined by increasing accessibility (**Active**, **Inc. Acc.**, and **Repressed**) clearly gain intensity in E15.5 (the E15.5 heatmap scales up to 2.3, whereas E11.5 only goes to 0.7).

Counter-intuitively, the raw ATAC signal in the **Dec. Acc.** cluster (at E11.5) appears significantly stronger and physically *wider* than the signal seen in the **Inc. Acc.** cluster (at E15.5). This suggests that the open regulatory regions driving early developmental states (E11.5) might

exist in massive, highly accessible “super-enhancer” clusters, whereas the newly opening regions required later in development (E15.5) are much more tightly and narrowly focused.

We often assume that regions driving active, increasing gene expression (the red **Active** cluster) would have the most blindingly intense ATAC signal. However, looking at the heatmap, the signal intensity is surprisingly moderate compared to the **Dec. Acc.** group. This confirms that a regulatory element doesn’t need to be a massive, gaping hole in the chromatin to successfully drive gene expression; it only needs to be open *just enough* to let the specific transcription machinery dock.

9.5.4 RNA-seq expression heatmap

Our RNA-seq data consists of discrete, normalized expression values (one mathematical value per gene, per sample). Therefore, just as we did in [Notebook 3](#), we will map this data onto a standard **ComplexHeatmap** block rather than an **EnrichedHeatmap**. To keep our analysis biologically consistent, we will continue to stratify these rows using the exact same **split_state** categories we just applied to the ATAC-seq data, allowing us to observe how gene expression behaves within each of our defined regulatory states.

```
print(colnames(mat_RNA))
```

```
[1] "RNA_11half_1" "RNA_11half_2" "RNA_15half_1" "RNA_15half_2"
```

```
set.seed(100)

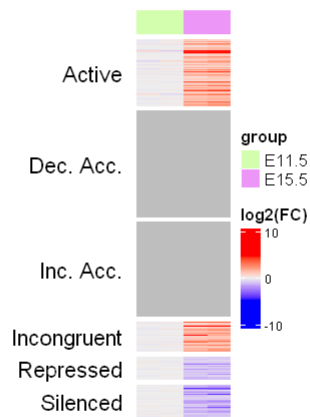
# Refactor/rename the RNA matrix column headers
rna_groups <- ifelse(grepl("11half", colnames(mat_RNA)), "E11.5", "E15.5")

rna_hm <- Heatmap(
  matrix = mat_RNA,
  row_split = split_state, # Stratify rows by biological category
  cluster_columns = FALSE,
  cluster_rows = FALSE, # CRITICAL: prevents clustering from crashing on NA values!
  show_row_names = FALSE,
  show_column_names = FALSE,
  row_title_rot = 0, # Forces the split category labels to print horizontally
  show_row_dend = FALSE,
  na_col = "grey", # Grey out regions with no RNA data
  width = unit(2, "cm"), height = unit(8, "cm"),
  top_annotation = HeatmapAnnotation(
    group = rna_groups,
```

```

    show_annotation_name = FALSE),
  heatmap_legend_param = list(
    at = scales::pretty_breaks(n = 2)(
      range(mat_RNA, na.rm = TRUE)),
    labels_gp = gpar(fontsize = 7),
    title = "log2(FC)")
draw(rna_hm, merge_legend = TRUE)

```



Notice that we add `merge_legend = TRUE` to our final `draw()` command. While this parameter is typically used to deduplicate mathematically identical colorbars, it also serves as a brilliant layout trick. Because our new composite figure mixes three disparate types of legends (the main heatmap scales, a row annotation for biological state, and a column annotation for the RNA stages), the default layout algorithm will often scatter them side-by-side into different zones on the page. This command acts as a strict architectural override, forcing the engine to collect every single legend and pack them into one beautifully stacked vertical column on the right.

9.5.5 RNA-seq expression patterns

Looking specifically at the RNA-seq heatmap block, we can clearly visualize the distinct transcriptional fates of our genes during development:

1. **The Upregulated Genes (Red):** The **Active** and **Incongruent** clusters display a striking shift from a pale E11.5 baseline to a deep red E15.5 state. This visually confirms that the genes linked to these regulatory regions are strongly upregulated and actively turning on as embryonic development progresses.
2. **The Downregulated Genes (Blue):** Conversely, the **Silenced** and **Repressed** clusters transition from the E11.5 baseline into a deep blue state in E15.5. This vividly demonstrates transcriptional shutdown, confirming that these specific genes are being deactivated in the later embryonic stage.
3. **The Transcriptional “Grey Zones”:** Notice that the massive **Dec. Acc.** and **Inc. Acc.** blocks are completely grey (NA). This perfectly visualizes our mathematical filtering: these specific categories represent dynamic regions that lack any significant corresponding gene expression data. They remain transcriptionally silent or statistically insignificant in our RNA analysis.

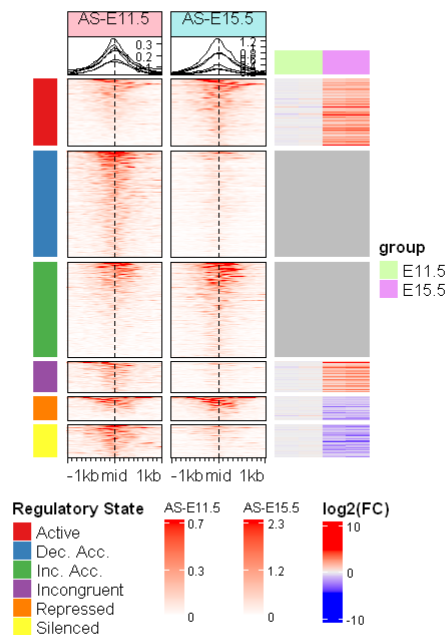
9.5.6 Constructing the composite multi-omics heatmap

Using the `+` operator provided by `ComplexHeatmap`, we can now concatenate our colored biological annotation bar, the continuous ATAC-seq accessibility heatmaps, and the discrete RNA-seq expression heatmap into a single, highly informative composite figure.

By passing our `split_state` categories directly into the global `draw()` engine (which **overrides** individual settings), the entire composite visualization is perfectly stratified, allowing us to seamlessly validate the functional relationship between chromatin state and gene expression in a single glance.

```
composite_ht <- eh_state_anno + eh_atac_11h + eh_atac_15h + rna_hm

draw(
  composite_ht,
  split = split_state, # global splitting
  # annotation_legend_list = lgd_state,
  # annotation_legend_side = "right",
  heatmap_legend_side = "bottom"
)
```



9.5.7 Reordering biological categories

Exercise: Can you change the order of the heatmap regions to follow this exact biological hierarchy? 1. Active 2. Repressed 3. Silent 4. Incongruent 5. Dec. Acc. 6. Inc. Acc.

Hint: The `split` argument in the `draw()` function automatically orders the heatmap splits based on the factor levels of the provided grouping variable (`split_anno`).

To reorder the layout, we convert our `split_state` text vector into a formal R **factor**, explicitly defining the **levels** in our exact desired biological hierarchy.

Because **ComplexHeatmap** is highly optimized, its layout engine will automatically detect these new factor levels and physically reorder the rows to match. The best part of this object-oriented design is that we **do not** need to recompute any of the heavy ATAC or RNA heatmaps! We simply re-instantiate the lightweight `eh_state_anno` color bar so it learns the new order, stitch it right back onto our existing heatmap objects, and effortlessly render the plot.

```
print(unique(split_state))
```

```
[1] "Active"      "Inc. Acc."   "Dec. Acc."   "Incongruent" "Silenced"
[6] "Repressed"
```

```
# Convert to a strict factor with the desired hierarchical order
split_state_reordered <- factor(
  split_state,
  levels = c(
    "Active", "Repressed", "Silenced", "Incongruent", "Dec. Acc.", "Inc. Acc.")

# Re-instantiate the row annotation ONLY so it learns the new order
eh_state_anno <- rowAnnotation(
  df = data.frame(Regulatory_State = split_state_reordered),
  col = list(Regulatory_State = anno_colors),
  show_annotation_name = FALSE,
  annotation_legend_param = list(Regulatory_State = list(title = "Regulatory State")))

# 3Redraw the composite plot
composite_ht_2 <- eh_state_anno + eh_atac_11h + eh_atac_15h + rna_hm
```

9.5.8 Global vsualization settings

Instead of manually tweaking the borders, title padding, and legend spacing for every single heatmap we create, **ComplexHeatmap** provides a powerful global options engine called `ht_opt`. By configuring these aesthetic rules just once per R session, they will automatically apply to every subsequent plot, guaranteeing a polished, publication-ready look with minimal code.

```
# Shrinks the empty space between the main heatmap and its title (e.g., "ATAC-E11.5")
ht_opt$TITLE_PADDING <- unit(1, "mm")

# Controls the spacing between multiple different legend boxes so they don't overlap
ht_opt$legend_gap <- unit(3, "mm")

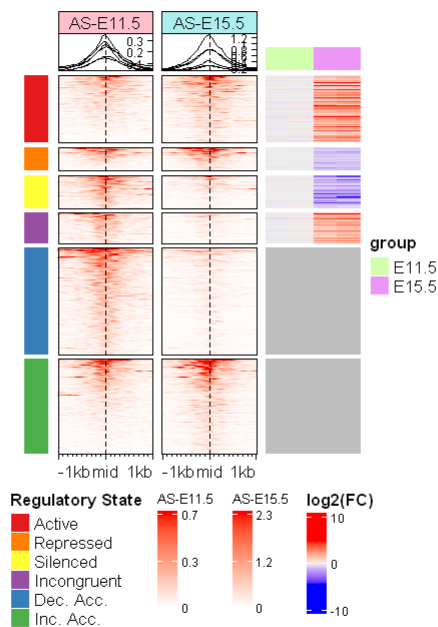
# Shrinks the physical height of the colored squares/bars inside the legend
ht_opt$legend_grid_height <- unit(2, "mm")

# Shrinks the physical width of the colored squares/bars inside the legend
ht_opt$legend_grid_width <- unit(2, "mm")

# Squeezes the empty gap between the main heatmap body and the legend itself
ht_opt$HEATMAP_LEGEND_PADDING <- unit(1, "mm")
```

```
# Automatically draws a clean, sharp black border around every heatmap matrix!
ht_opt$heatmap_border <- TRUE
```

```
draw(composite_ht_2, split = split_state_reordered, heatmap_legend_side = "bottom")
```



9.5.9 Decoding the Multi-Omics landscape

By perfectly aligning continuous chromatin accessibility (ATAC-seq) with discrete transcriptional output (RNA-seq), this composite heatmap reveals the stunning complexity of epigenetic regulation during embryonic development. We can visually validate several profound biological paradigms in a single glance:

9.5.9.1 The Classic Paradigms (Active & Silenced)

The **Active** (Red) and **Silenced** (Yellow) clusters clearly demonstrate the classic “central dogma” of epigenetics: - **Active**: We see the ATAC signal physically widening and intensifying

in E15.5, allowing the transcription machinery to dock. This structurally drives the massive surge in gene expression visually confirmed in the red RNA-seq block. - **Silenced:** Conversely, we see the chromatin physically condensing and shutting down (loss of ATAC signal in E15.5), which directly results in the severe deactivation of the corresponding genes (blue RNA-seq block).

9.5.9.2 The Repressive Elements (Repressed)

Biology is rarely perfectly linear. In the **Repressed** (Orange) cluster, the chromatin actually *gains* massive accessibility in E15.5. However, the corresponding RNA-seq block turns deep blue, indicating the gene is strongly shut off. This visually proves that newly opened DNA is not always an activator; these specific regions are likely acting as docking sites for powerful **transcriptional repressors**.

9.5.9.3 The Decoupled Elements (Incongruent)

The **Incongruent** (Purple) cluster shows the exact opposite paradox: the local chromatin is physically closing down (loss of ATAC signal), yet the gene strongly turns on (red RNA). This implies that these genes are no longer dependent on this specific local element, and are likely being driven by a different, distant “super-enhancer” looping in from elsewhere in the genome.

9.5.9.4 The Structural DNA (Dec. Acc. & Inc. Acc.)

Finally, the massive bottom blocks represent regions undergoing massive structural changes (violent opening or closing of chromatin) without any corresponding significant gene expression (grey RNA). This perfectly illustrates a critical lesson: **chromatin accessibility does not guarantee transcription**. Many of these regions are structural elements, distant intergenic enhancers preparing for a later developmental stage, or regions driving non-coding RNAs not captured in our matrix.

10 Integrating ATAC, ChIP, and RNA sequencing data

10.1 Learning objectives

- Understand the biological significance of different assays such as ATAC-seq, ChIP-seq for histone marks, DNA methylation, and RNA-seq, and how they relate to gene regulation.
- Process and integrate multi-omics data using SummarizedExperiment objects and annotation data.
- Identify and classify regulatory genomic regions (e.g., gained/lost enhancers, activated/repressed promoters) based on changes in chromatin accessibility, histone modifications, and gene expression.
- Visualize complex multi-omics data by generating heatmaps with the EnrichedHeatmap and ComplexHeatmap R packages, highlighting changes across developmental stages.
- Interpret integrated heatmaps to explore relationships between chromatin state, DNA methylation, and transcriptional output.
- Use custom functions to automate visualization workflows, adapting color scales and annotations for effective data presentation.

10.2 Defining multi-omics regulatory states

In the [previous notebook](#), we mapped the regulatory landscape using only chromatin accessibility and gene expression. Now, we will increase the biological resolution by integrating **histone modification (ChIP-seq)** data.

By overlapping ATAC-seq, RNA-seq, and specific histone marks, we can transition from generic “Active” states to highly precise biological categories, such as distinguishing a proximal active promoter from a distal active enhancer.

Just as before, we will use a simplified, fold-change-driven approach to categorize these complex **multi-omics states**. Because our datasets were already pre-filtered for statistical significance, we will use basic logical conditions across all our different modalities, combined with **genomic distance metrics**, to assign these categories.

Cate- gory	ATAC	H3K4me3	H3K27ac	H3K4me1	H3K27me3	RNA	Dist. to TSS	Biological interpretation	Analysis pipeline logic
Gained En- hancers	↑	↑	↑	none	-	↑	>2500 bp	Activation of a distal regulatory element driving gene expression.	ATAC > 0 & K4me1 > 0 & K27ac > 0 & is.na(K4me3) & RNA > 0 & dist > 2500
Lost En- hancers	↓	↓	↓	none	-	↓	>2500 bp	Deactivation of a distal enhancer, resulting in transcriptional shutdown.	ATAC < 0 & K4me1 < 0 & K27ac < 0 & is.na(K4me3) & RNA < 0 & dist > 2500
Active Pro- mot- ers	↑	-	↑	↑	-	↑	<2500 bp	Activation of a proximal promoter directly launching transcription.	ATAC > 0 & K27ac > 0 & K4me3 > 0 & RNA > 0 & dist < 2500
Re- pressed Pro- mot- ers	↓	-	↓	↓	↑	↓	<2500 bp	A promoter being actively shut down via repressive Polycomb histone marks.	ATAC < 0 & K27ac < 0 & K4me3 < 0 & K27me3 > 0 & RNA < 0 & dist < 2500

*Note: In this table, **none** indicates the strict absence of the mark (which will appear as **NA** in our overlap matrix), while - indicates the mark's status is flexible or not strictly required for the classification.*

To successfully interpret these complex multi-omic states, we must first understand the distinct biological language of each individual sequencing assay. As a quick refresher, here is exactly what each modality in our matrix measures:

Assay	What It Measures	Biological Insight
ATAC-seq	Chromatin accessibility	Identifies open chromatin regions , such as promoters and enhancers.
ChIP-seq (H3K4me3)	Active promoter mark	Marks active promoters ; often found at TSS (Transcription Start Sites).
ChIP-seq (H3K27ac)	Active enhancer/promoter	Marks active regulatory regions , especially enhancers.
ChIP-seq (H3K4me1)	Primed or poised enhancers	Marks enhancers that may be active or poised , not promoters.

Assay	What It Measures	Biological Insight
ChIP-seq (H3K27me3)	Repressive regions	Marks silenced genes or developmental repression (Polycomb repression).
DNAme (WGBS)	DNA methylation (CpG sites)	Typically associated with gene silencing and heterochromatin .
RNA-seq	Gene expression	Final transcriptional output ; mRNA abundance.

Note: More details can be found in the [comprehensive epigenetic analysis reference guide](#).

Because *DNA methylation* can be a highly variable or “lagging” epigenetic mark, forcing it into rigid threshold rules can prematurely exclude interesting mechanistic. Instead of using it to define categories, we will plot the WGBS data alongside our final heatmaps to independently observe and validate how the DNA methylation landscape naturally responds to these distinct regulatory states.

10.3 Loading packages and pre-computed data

Just as in our previous analysis, we will keep our pipeline fast and memory-efficient by strictly utilizing pre-computed datasets.

By completely bypassing the massive, genome-wide raw `SummarizedExperiment (.rds)` objects, we can immediately load the exact data we need for filtering and visualization:

1. **overlap_regulatory_status.rds:** Our upgraded master annotation object. We will explicitly load the exact file we saved at the very end of the [previous notebook](#). This is critical because it seamlessly carries over our categorized `Regulatory_State` metadata (Active, Repressed, Silenced, etc.) alongside the `logFC` stats for all our new ChIP-seq modalities.
2. **Normalized Signal Matrices (mat_...rds):** The pre-calculated continuous signal grids perfectly formatted for `EnrichedHeatmap` visualization.

To keep our environment clean, we won’t load the fragmented individual histone mark files. Instead, we will directly load the single, consolidated `chip_matrices.rds` [master list we generated previously](#), which contains all our normalized matrices for every mark and timepoint in one perfectly organized object!

```
suppressPackageStartupMessages(suppressWarnings({
  library(SummarizedExperiment)
  library(magick) # Required for high-quality ComplexHeatmap rasterization
  library(ComplexHeatmap)
```

```
library(EnrichedHeatmap)
library(circlize)
library(dplyr) # %>%, case_when(), mutate()
library(tidyr) # replace_na()
}))
```

```
overlap <- readRDS("output/overlap_regulatory_status.rds")
```

```
mat_ATAC <- readRDS("output/mat_atac.rds")
```

```
mat_RNA <- readRDS("output/mat_rna.rds")
```

```
# Instead of loading 4 fragmented individual files...
```

```
# mat_H3K4me3 <- readRDS("output/mat_h3k4me3.rds")
```

```
# mat_H3K4me1 <- readRDS("output/mat_h3k4me1.rds")
```

```
# mat_H3K27me3 <- readRDS("output/mat_h3k27me3.rds")
```

```
# mat_H3K27ac <- readRDS("output/mat_h3k27ac.rds")
```

```
# ...we can just load the entire consolidated master list
```

```
# (see 04-genomic_overlap_matrices.ipynb)
```

```
chip_matrices <- readRDS("output/chip_matrices.rds")
```

```
mat_BS <- readRDS("output/mat_bs.rds")
```

```
# Shows just the top-level list items without printing the massive hidden attributes
str(chip_matrices, give.attr = FALSE)
```

List of 8

```
$ H3K27ac_11half : 'normalizedMatrix' num [1:1933, 1:100] 0.2324 0.0266 0.0239 0.0307 0.036
$ H3K27ac_15half : 'normalizedMatrix' num [1:1933, 1:100] 0.48311 0.01757 0.00016 0.03229 0
$ H3K27me3_11half: 'normalizedMatrix' num [1:1933, 1:100] 0.0603 0.0167 0.0423 0.0352 0.031
$ H3K27me3_15half: 'normalizedMatrix' num [1:1933, 1:100] 0.03311 0.01926 0.02749 0.08658 0
$ H3K4me1_11half : 'normalizedMatrix' num [1:1933, 1:100] 0.0344 0.0305 0 0.0573 0.0882 ...
$ H3K4me1_15half : 'normalizedMatrix' num [1:1933, 1:100] 0.12863 0.028996 0.000604 0.05359
$ H3K4me3_11half : 'normalizedMatrix' num [1:1933, 1:100] 0.7152 0 0.0556 0 0.0751 ...
$ H3K4me3_15half : 'normalizedMatrix' num [1:1933, 1:100] 2.0519 0.0119 0.0217 0.0397 0.051
```

10.4 Categorizing genomic regions by multi-omics behavior

As we established in the biological framework during our introduction, integrating histone modifications alongside chromatin accessibility and gene expression allows us to vastly increase the precision of our regulatory mapping.

In our previous preparation steps, we explicitly generated overlap matrices mapping all these different sequencing modalities back to our master ATAC-seq peak atlas. Because our raw input files were already strictly pre-filtered to only include statistically significant changes ($\text{abs}(\text{logFC}) \geq .5$ and $\text{qvalue} \leq .1$), we can confidently use simple directional math (> 0 or < 0) alongside the structural distance to the Transcription Start Site (distanceToTSS) to define our complex biological signatures.

10.4.1 Mathematical cleanup (Handling missing values)

Before we assign these multi-omics categories, we must apply a highly targeted mathematical cleanup. In R, evaluating $\text{NA} > 0$ returns NA rather than FALSE , which can silently corrupt complex filtering chains. To prevent this, any missing NA values *specifically within our logFC columns* will be safely converted to 0, ensuring our directional math executes flawlessly. We deliberately leave the `annotation` columns untouched so we can still use `is.na()` to formally check for the strict absence of specific physical overlaps (such as ruling out the H3K4me3 promoter mark when defining distal enhancers).

```
# # Identify all column indices that contain the string "logFC"
# logFC_cols <- grep("logFC", names(mcols(overlap)))

# # Replace all NA values with 0 in those specific columns
# mcols(overlap)[logFC_cols] <- lapply(mcols(overlap)[logFC_cols], function(x) {
#   replace(x, is.na(x), 0)
# })

# Extract metadata, clean it with dplyr, and assign it right back to the GRanges object
mcols(overlap) <- as.data.frame(mcols(overlap)) %>%
  mutate(across(contains("logFC"), ~ replace_na(.x, 0)))

# First few rows of the pre-processed GRanges object
print(head(overlap[, c("ATAC_logFC", "RNA_logFC")], 10))
```

GRanges object with 10 ranges and 2 metadata columns:

	seqnames	ranges	strand	ATAC_logFC	RNA_logFC
	<Rle>	<IRanges>	<Rle>	<numeric>	<numeric>
peak_1	chr1	3670547-3672665	*	0.602984	1.37498
peak_2	chr1	4332510-4332710	*	0.855748	0.00000
peak_3	chr1	4491755-4492573	*	-0.620926	0.00000
peak_4	chr1	4571186-4572423	*	-0.511046	0.00000
peak_5	chr1	4785062-4786325	*	0.540908	0.00000
peak_6	chr1	5232000-5233386	*	-2.501780	0.00000

peak_7	chr1	5587862-5589288	*		-0.596064	2.81715
peak_8	chr1	6358864-6359863	*		-0.913905	0.00000
peak_9	chr1	6734526-6735054	*		-0.918568	-2.16727
peak_10	chr1	7313422-7313895	*		-3.295645	0.00000

seqinfo: 21 sequences from an unspecified genome; no seqlengths

10.4.2 Defining strict biological states

We then execute an advanced filtering logic to strictly annotate every peak into one of four precise states:

- **Gained Enhancers:** Distal elements ($> 2.5\text{kb}$ from a TSS) that gain chromatin accessibility (ATAC is higher in E15.5 than E11.5), gain activating enhancer marks (H3K4me1, H3K27ac), strictly lack promoter marks (H3K4me3), and successfully drive increased gene expression.
- **Lost Enhancers:** Distal elements that lose both accessibility (ATAC is lower in E15.5 than E11.5) and activating histone marks, resulting in the transcriptional shutdown of their associated genes.
- **Active Promoters:** Proximal elements ($< 2.5\text{kb}$ from a TSS) that gain accessibility (ATAC is higher in E15.5 than E11.5) and powerful active promoter marks (H3K4me3, H3K27ac) to directly launch transcription.
- **Repressed Promoters:** Proximal elements that lose accessibility (ATAC is lower in E15.5 than E11.5) and active marks, while explicitly gaining the Polycomb repressive mark (H3K27me3) to securely and stably silence transcription.

To do this, we will use `dplyr::case_when()` to create a pure, strict `Multiomics_State` column. Any peak that does not mathematically meet these rigorous criteria will default to "Unclassified".

```
mcols(overlap) <- as.data.frame(mcols(overlap)) %>%
  mutate(
    # Create a NEW column (preserving the original Regulatory_State column)
    Multiomics_State = case_when(

      # Gained Enhancers
      ATAC_logFC > 0 &
        ATAC_distanceToTSS > 2500 &
        H3K4me1_logFC > 0 &
        H3K27ac_logFC > 0 &
        is.na(H3K4me3_annotation) &
        RNA_logFC > 0
```

```

~ "Gained Enhancers",

# Lost Enhancers
ATAC_logFC < 0 &
  ATAC_distanceToTSS > 2500 &
  H3K4me1_logFC < 0 &
  H3K27ac_logFC < 0 &
  is.na(H3K4me3_annotation) &
  RNA_logFC < 0
~ "Lost Enhancers",

# Active Promoters
ATAC_logFC > 0 &
  ATAC_distanceToTSS < 2500 &
  H3K4me3_logFC > 0 &
  H3K27ac_logFC > 0 &
  RNA_logFC > 0
~ "Active Promoters",

# Repressed Promoters
ATAC_logFC < 0 &
  ATAC_distanceToTSS < 2500 &
  H3K4me3_logFC < 0 &
  H3K27ac_logFC < 0 &
  H3K27me3_logFC > 0 &
  RNA_logFC < 0
~ "Repressed Promoters",

# If a region doesn't match our strict rules, leave it unclassified
.default = "Unclassified"
)
)

```

```

# First few rows of the final overlap object
print(overlap[1:15, c("Regulatory_State", "Multiomics_State")])

```

GRanges object with 15 ranges and 2 metadata columns:

	seqnames	ranges	strand	Regulatory_State	Multiomics_State
	<Rle>	<IRanges>	<Rle>	<character>	<character>
peak_1	chr1	3670547-3672665	*	Active	Active Promoters
peak_2	chr1	4332510-4332710	*	Inc. Acc.	Unclassified
peak_3	chr1	4491755-4492573	*	Dec. Acc.	Unclassified

peak_4	chr1	4571186-4572423	*		Dec. Acc.	Unclassified
peak_5	chr1	4785062-4786325	*		Inc. Acc.	Unclassified
...	...	...	...	.	...	...
peak_11	chr1	8410409-8410976	*		Incongruent	Unclassified
peak_12	chr1	8913622-8914181	*		Incongruent	Unclassified
peak_13	chr1	8948294-8948530	*		Incongruent	Unclassified
peak_14	chr1	9298449-9300284	*		Active	Active Promoters
peak_15	chr1	9564437-9564960	*		Active	Unclassified

seqinfo: 21 sequences from an unspecified genome; no seqlengths

10.4.3 The reality of multi-omics strictness

If we look at the results above, we see that the vast majority of the peaks are labeled "Unclassified". This is exactly what we expect biologically!

Our rules are very strict; for a region to be an **Active Promoter**, it must simultaneously gain ATAC, RNA, H3K4me3, *and* H3K27ac. If even one of those marks fails to pass our significance thresholds, the entire chain breaks. This perfectly highlights a core biological reality: **most of the genome does not undergo perfectly coordinated changes across 5 different sequencing assays simultaneously**. The relatively few regions that do survive this filtering are the absolute “gold standard” regulatory elements driving development.

10.4.4 Hierarchy collapsing for visualization

When we plot our final massive heatmap, we want a single, clean layout that pulls out our elite Promoters and Enhancers at the top, while still displaying the generically “Active” or “Repressed” genomic background below them.

If we ask **ComplexHeatmap** to split the rows using our current column, it will group the majority of the genome into one massive, uninformative "Unclassified" block, and we will lose all the ATAC/RNA patterns we discovered in the previous notebook. Conversely, if we try to feed the heatmap *both* columns separately, it will mathematically multiply them into 20+ visually fractured micro-slices.

To solve this, we decide to execute a data science trick: **Hierarchy Collapsing**. We will run our logic one more time, but this time, if a peak fails the strict ChIP-seq test, we will tell it to dynamically inherit its old **Regulatory_State** label.

```
# Execute the logic again, but this time using the fallback **Hierarchy Collapsing**
mcols(overlap) <- as.data.frame(mcols(overlap)) %>%
  mutate(
    # Create a NEW column (preserving the original Regulatory_State column)
```

```

Multiomics_State = case_when(

  # Gained Enhancers
  ATAC_logFC > 0 &
    ATAC_distanceToTSS > 2500 &
    H3K4me1_logFC > 0 &
    H3K27ac_logFC > 0 &
    is.na(H3K4me3_annotation) &
    RNA_logFC > 0
  ~ "Gained Enhancers",

  # Lost Enhancers
  ATAC_logFC < 0 &
    ATAC_distanceToTSS > 2500 &
    H3K4me1_logFC < 0 &
    H3K27ac_logFC < 0 &
    is.na(H3K4me3_annotation) &
    RNA_logFC < 0
  ~ "Lost Enhancers",

  # Active Promoters
  ATAC_logFC > 0 &
    ATAC_distanceToTSS < 2500 &
    H3K4me3_logFC > 0 &
    H3K27ac_logFC > 0 &
    RNA_logFC > 0
  ~ "Active Promoters",

  # Repressed Promoters
  ATAC_logFC < 0 &
    ATAC_distanceToTSS < 2500 &
    H3K4me3_logFC < 0 &
    H3K27ac_logFC < 0 &
    H3K27me3_logFC > 0 &
    RNA_logFC < 0
  ~ "Repressed Promoters",

  # FALLBACK: inherit the generic label instead of "Unclassified"
  .default = Regulatory_State
)
)

```

```
# First few rows of the updated final overlap object
print(overlap[1:15, c("Regulatory_State", "Multiomics_State")])
```

GRanges object with 15 ranges and 2 metadata columns:

	seqnames	ranges	strand	Regulatory_State	Multiomics_State
	<Rle>	<IRanges>	<Rle>	<character>	<character>
peak_1	chr1	3670547-3672665	*	Active	Active Promoters
peak_2	chr1	4332510-4332710	*	Inc. Acc.	Inc. Acc.
peak_3	chr1	4491755-4492573	*	Dec. Acc.	Dec. Acc.
peak_4	chr1	4571186-4572423	*	Dec. Acc.	Dec. Acc.
peak_5	chr1	4785062-4786325	*	Inc. Acc.	Inc. Acc.
...	...	...	...	...	...
peak_11	chr1	8410409-8410976	*	Incongruent	Incongruent
peak_12	chr1	8913622-8914181	*	Incongruent	Incongruent
peak_13	chr1	8948294-8948530	*	Incongruent	Incongruent
peak_14	chr1	9298449-9300284	*	Active	Active Promoters
peak_15	chr1	9564437-9564960	*	Active	Active

seqinfo: 21 sequences from an unspecified genome; no seqlengths

```
# Saving the whole final project overlap SE object
saveRDS(object = overlap, file = "output/overlap_multiomics_state.rds")
```

10.5 Visualizing the complete multi-omics landscape

10.5.1 Preparing the continuous ATAC, ChIP and WGBS block

Now that every peak in our dataset has been assigned a precise, hierarchical multi-omics state, we can render the final composite figure. To do this, we will aggressively reuse the `custom_enrichedheatmap_plotting()` wrapper function we built previously.

We will simply pass it our newly loaded `chip_matrices` and `mat_bs` objects to rapidly generate heatmaps for all **12 continuous sequencing modalities** (ATAC, H3K4me3, H3K4me1, H3K27me3, H3K27ac, and DNA Methylation). To maintain strict visual consistency across the entire course, we will also explicitly reuse the exact same track colors (e.g., Red for ATAC, Green for H3K4me3) and layout parameters we established for these assays in our earlier notebooks!

```
# Load our custom multi-omics plotting tool from the external script
source("scripts/custom_enrichedheatmap_plotting.R")
```

Because we are plotting a massive 14-column figure in this notebook, we need to strictly constrain the physical dimensions of every single heatmap to fit on the page (e.g., width = 2 cm, height = 8 cm, and top annotation height = 1 cm).

While we *could* permanently change the default arguments inside our `custom_enrichedheatmap_plotting.R` script, doing so is generally considered bad practice. That script is a shared utility used across our entire project (including Notebooks 03 and 05). If we hardcode fixed centimeter dimensions into the master script now, we risk breaking the layouts of our older notebooks, which rely on flexible proportional scaling (`unit(1, "null")`).

Instead, we will use a **local wrapper**; we will define a tiny, temporary function in the current notebook that passes our data down to the master script, but “bakes in” our specific dimensions. This allows us to rapidly generate all 12 ChIP/ATAC/WGBS heatmaps without copying and pasting the dimension arguments 12 times.

```
# Create a local wrapper to safely lock in our specific layout dimensions
make_EH <- function(...) {
  custom_enrichedheatmap_plotting(
    ..., # Passes all the unique arguments (e.g., norm_mat, hm_name) straight through
    hm_width = unit(2, "cm"),
    hm_height = unit(8, "cm"),
    top_anno_height = unit(1, "cm"),

    # Aesthetic fine-tuning
    title_fontsize = 8,
    axis_fontsize = 7,
    axis_lwd = 0.4
  )
}
```

```
# ATAC heatmaps
# (See notebook 05-atac_rna_integration.ipynb)
eh_atac_11h <- make_EH(
  norm_mat = mat_ATAC$ATAC_11half,
  hm_name = "AS-E11.5", col_fill = "pink")

eh_atac_15h <- make_EH(
  norm_mat = mat_ATAC$ATAC_15half,
  hm_name = "AS-E15.5", col_fill = "paleturquoise")
```

```
# H3K4me3 heatmaps
# (See notebook 03-chip_rna_visualization.ipynb)
eh_h3k4me3_11h <- make_EH(
```

```

    norm_mat = chip_matrices$H3K4me3_11half,
    hm_name = 'K4me3-E11.5', col_fill = 'pink',
    heatmap_cols = c('white', 'darkgreen'))

eh_h3k4me3_15h <- make_EH(
  norm_mat = chip_matrices$H3K4me3_15half,
  hm_name = 'K4me3-E15.5', col_fill = 'paleturquoise',
  heatmap_cols = c('white', 'darkgreen'))

```

```

# H3K4me1 heatmaps
# (See notebook 03-chip_rna_visualization.ipynb)
eh_h3k4me1_11h <- make_EH(
  norm_mat = chip_matrices$H3K4me1_11half,
  hm_name = 'K4me1-E11.5', col_fill = 'pink',
  heatmap_cols = c('white', 'darkblue'))

eh_h3k4me1_15h <- make_EH(
  norm_mat = chip_matrices$H3K4me1_15half,
  hm_name = 'K4me1-E15.5', col_fill = 'paleturquoise',
  heatmap_cols = c('white', 'darkblue'))

```

```

# H3K27me3 heatmaps
# (See notebook 03-chip_rna_visualization.ipynb)
eh_h3k27me3_11h <- make_EH(
  norm_mat = chip_matrices$H3K27me3_11half,
  hm_name = 'K27me3-E11.5', col_fill = 'pink',
  heatmap_cols = c('white', 'darkorchid'))

eh_h3k27me3_15h <- make_EH(
  norm_mat = chip_matrices$H3K27me3_15half,
  hm_name = 'K27me3-E15.5', col_fill = 'paleturquoise',
  heatmap_cols = c('white', 'darkorchid'))

```

```

# H3K27ac heatmaps
# (See notebook 03-chip_rna_visualization.ipynb)
eh_h3k27ac_11h <- make_EH(
  norm_mat = chip_matrices$H3K27ac_11half,
  hm_name = 'K27ac-E11.5', col_fill = 'pink',
  heatmap_cols = c('white', 'darkred'))

eh_h3k27ac_15h <- make_EH(

```

```

norm_mat = chip_matrices$H3K27ac_15half,
hm_name = 'K27ac-E15.5', col_fill = 'paleturquoise',
heatmap_cols = c('white', 'darkred'))

# WGBS heatmaps
# (See notebook 04-genomic_overlap_matrices.ipynb)
eh_wgbs_11h <- make_EH(
  norm_mat = mat_BS$WGBS_11half,
  hm_name = 'WGBS-E11.5', col_fill = 'pink',
  heatmap_cols = c("red", "white", "blue"))

eh_wgbs_15h <- make_EH(
  norm_mat = mat_BS$WGBS_15half,
  hm_name = 'WGBS-E15.5', col_fill = 'paleturquoise',
  heatmap_cols = c("red", "white", "blue"))

```

10.5.2 Preparing the discrete RNA-seq block

Because RNA-seq is measured in discrete counts per gene rather than continuous genomic coverage, it bypasses our `EnrichedHeatmap` wrapper. We simply construct it as a standard `ComplexHeatmap` block, setting `cluster_rows = FALSE` so it perfectly maps onto the ATAC peak coordinates.

```

# RNA heatmap (ComplexHeatmap)
# (See notebook 05-atac_rna_integration.ipynb)
set.seed(100)

rna_groups <- ifelse(grepl("11half", colnames(mat_RNA)), "E11.5", "E15.5")

rna_hm <- Heatmap(
  matrix = mat_RNA,
  cluster_columns = FALSE,
  cluster_rows = FALSE,
  show_row_names = FALSE,
  show_column_names = FALSE,
  show_row_dend = FALSE,
  na_col = "#c0c0c0",
  width = unit(2, "cm"), height = unit(18, "cm"),
  top_annotation = HeatmapAnnotation(
    group = rna_groups,
    col = list(group = c("E11.5" = "#ff00ff", "E15.5" = "#adff2f")),

```

```

    show_annotation_name = FALSE),
  heatmap_legend_param = list(
    at = scales::pretty_breaks(n = 2)(
      range(mat_RNA, na.rm = TRUE)),
    labels_gp = gpar(fontsize = 7), # To match those of other EnrichedHeatmaps
    title = "log2(FC)")

```

10.5.3 Preparing the hierarchical row splits

To divide our massive heatmap, we will extract our newly merged `Multiomics_State` column. We will convert it into a formal R `factor`, which allows us to explicitly define the top-to-bottom visual rendering order. This ensures our elite “Active Promoters” and “Gained Enhancers” sit prominently at the top of the figure.

```

# Extract our new merged hierarchical column
split_multiomics <- overlap$Multiomics_State

# Explicitly order the biological categories for the visual split
# (Ensure the names exactly match the strings in the Multiomics_State column!)
biological_order <- c(
  "Active Promoters", "Gained Enhancers", "Active", "Inc. Acc.",
  "Repressed Promoters", "Lost Enhancers", "Repressed", "Dec. Acc.",
  "Silenced", "Incongruent")

split_multiomics <- factor(split_multiomics, levels = biological_order)

# Sample from splitting vector
print(head(split_multiomics, 3))

```

```

[1] Active Promoters Inc. Acc.      Dec. Acc.
10 Levels: Active Promoters Gained Enhancers Active ... Incongruent

```

10.5.4 Annotation bar

To visually communicate the relationship between our categories, we will map our colors based on their **biological trajectories**:

- **The “Active” Pathway (Reds):** Dark red represents massive multi-omics evidence (*Active Promoters*), fading down to light pink for basic ATAC changes (*Inc. Acc.*).

- **The “Repressed” Pathway (Blues):** Dark blue represents confirmed *Repressed Promoters*, fading to light blue for basic ATAC closure (*Dec. Acc.*).
- **The Exceptions:** Distinct Grey and Purple colors separate our isolated *Silenced* and *Incongruent* regions.

This gradient approach allows us to instantly visualize the biological hierarchy of our data at a single glance!

```
# Extract 4-tone gradients; we ask for 5 colors and take 2:5,
# because the 1st color in Brewer palettes is usually too close to white)
active_tones <- RColorBrewer::brewer.pal(n = 5, name = "Reds")[2:5]
repressed_tones <- RColorBrewer::brewer.pal(n = 5, name = "Blues")[2:5]

# Map the tones based on biological severity (Darkest = Most confident state)
cols_an <- c(
  # The "Active" Trajectory (Dark Red to Light Pink)
  "Active Promoters"      = active_tones[4],
  "Gained Enhancers"     = active_tones[3],
  "Active"                = active_tones[2],
  "Inc. Acc."             = active_tones[1],

  # The "Repressed" Trajectory (Dark Blue to Light Blue)
  "Repressed Promoters" = repressed_tones[4],
  "Lost Enhancers"      = repressed_tones[3],
  "Repressed"           = repressed_tones[2],
  "Dec. Acc."           = repressed_tones[1],

  # The Exceptions
  "Silenced"             = "#525252", # Dark Grey
  "Incongruent"          = "#984ea3"  # Purple
)
```

```
eh_state_anno <- rowAnnotation(
  df = data.frame(Multiomics_State = split_multiomics),
  col = list(Multiomics_State = cols_an),
  show_annotation_name = FALSE,
  simple_anno_size = unit(2, "mm"), # Keeps the annotation bar nice and thin
  annotation_legend_param = list(
    Multiomics_State = list(title = "Regulatory Hierarchy")))

```

10.5.5 Rendering the composite figure

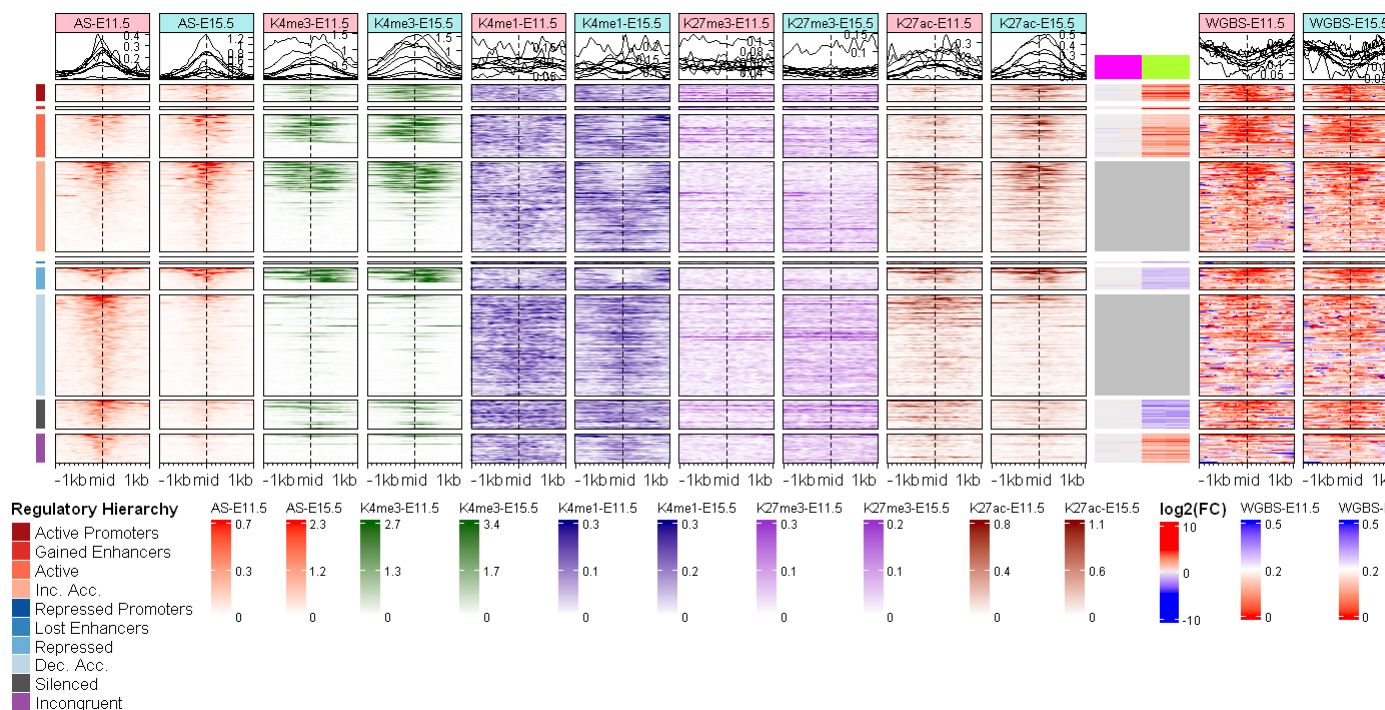
We are finally ready to stitch all 14 objects together (`anno_hm + 12 EnrichedHeatmaps + rna_hm`) using the simple `+` operator.

```
composite_ht <- eh_state_anno +  
  eh_atac_11h + eh_atac_15h +  
  eh_h3k4me3_11h + eh_h3k4me3_15h +  
  eh_h3k4me1_11h + eh_h3k4me1_15h +  
  eh_h3k27me3_11h + eh_h3k27me3_15h +  
  eh_h3k27ac_11h + eh_h3k27ac_15h +  
  rna_hm +  
  eh_wgbs_11h + eh_wgbs_15h
```

```
options(repr.plot.width = 13, repr.plot.height = 6)
```

```
ht_opt$TITLE_PADDING <- unit(1, "mm")  
ht_opt$legend_gap <- unit(3, "mm")  
ht_opt$legend_grid_height <- unit(2, "mm")  
ht_opt$legend_grid_width <- unit(2, "mm")  
ht_opt$HEATMAP_LEGEND_PADDING <- unit(1, "mm")  
ht_opt$heatmap_border <- TRUE
```

```
draw(composite_ht, split = split_multiomics, heatmap_legend_side = "bottom", merge_legend = 1)
```



10.5.6 Exporting the composite figure

We will execute this rendering process twice to generate two different file types for two specific professional use cases:

1. **PDF Export:** A vector graphic that scales infinitely without ever losing resolution. This is the absolute gold standard for scientific publications.
2. **High-Res PNG Export:** A rasterized image (drawn at 330 dpi). Because massive multi-omics PDFs contain millions of individual vector data points, they can sometimes be slow to open or crash presentation software. Rendering a crisp PNG is the perfect, lightweight alternative for PowerPoint slides and fast web viewing. (*Notice we slightly increase the canvas dimensions for the PNG to ensure the rasterized text stays perfectly legible*).

Because a 14-column figure is incredibly dense, standard block borders will look too thick and cluttered. To fix this, we will use a clever **grid graphics** trick: `pushViewport(viewport(gp = gpar(lwd = 0.5)))`. This forces the global line width (`lwd`) of the entire drawing canvas to be razor-thin (0.5) *before* the heatmaps are drawn, giving our final exported figure a perfectly sharp, professional aesthetic.

```

# Open a PDF graphics device with explicit dimensions (in inches)
pdf(
  file = "output/final_heatmap.pdf",
  width = 13, height = 6,
)

# Create a fresh, blank canvas to draw on
grid.newpage()

# THE VIEWPORT TRICK: Force the global line width (lwd) to be razor-thin (0.5)
# This prevents the borders of our 14 dense columns from looking cluttered
pushViewport(viewport(gp = gpar(lwd = 0.5)))

# Draw the final composite figure
draw(
  composite_ht, split = split_multiomics, heatmap_legend_side = "bottom", merge_legend = FALSE)

# Close the viewport (returning the global line width back to its normal default)
popViewport()

# Close the PDF device, which formally saves the rendered file to the hard drive
dev.off()

```

agg___record___23a4252c779c: 2

```

# Open a PNG graphics device with tightly wrapped dimensions (in inches)
# We set res = 330 (dots per inch) to guarantee the raster image is sharp
png(
  filename = "output/final_heatmap.png",
  width = 15, height = 7, units = "in",
  res = 330)

# Create a fresh, blank canvas
grid.newpage()

# THE VIEWPORT TRICK: Force razor-thin line widths globally (lwd = 0.5)
pushViewport(viewport(gp = gpar(lwd = 0.5)))

# Draw the final composite figure
draw(
  composite_ht, split = split_multiomics, heatmap_legend_side = "bottom", merge_legend = FALSE)

```

```
# Close the viewport (returning line widths back to normal)
popViewport()

# Close the PNG device, formally saving the image to the hard drive
dev.off()
```

```
agg\__record\__23a443b54ff6: 2
```

10.6 Biological interpretation

10.6.1 Validating “Active Promoters”

By looking horizontally across the top row of our massive composite figure, we can confidently confirm that our **Active Promoters** category is biologically accurate. The heatmap reveals a textbook multi-omics signature for active transcription start sites:

1. **Open Chromatin (ATAC):** A sharp, intense signal exactly at the center (0), representing the nucleosome-free region where transcription machinery is actively bound.
2. **The Promoter Hallmark (H3K4me3):** Massive, tightly centered enrichment. This is the universal, defining histone modification of an active promoter.
3. **Active Acetylation (H3K27ac):** High signal, often showing a slight “bimodal” dip at the very center (where the nucleosome is physically missing), flanking the ATAC peak.
4. **Absence of Repression (H3K27me3):** The track is completely blank (depleted), confirming that Polycomb repressive complexes are not silencing these genes.
5. **DNA Hypomethylation (WGBS):** The exact center of the peak is heavily depleted of methylation. The genome must keep these promoter regions completely unmethylated to maintain active transcription!
6. **Gene Expression (RNA-seq):** The RNA heatmap shows intense, positive expression, confirming that this open, actively modified chromatin structure successfully translates into transcribed genes.

This perfect, physical alignment across all 6 distinct biological assays proves that our `dplyr::case_when()` boolean logic successfully isolated true active promoters from the massive complexity of the genome.

10.6.2 Validating “Repressed Promoters”

If we look at the dark blue **Repressed Promoters** track, we see the exact opposite biological signature:

1. **Polycomb Silencing (H3K27me3):** We see massive, broad enrichment of H3K27me3. This is the universal hallmark of Polycomb-mediated gene silencing.
2. **Chromatin Closure:** The ATAC-seq, H3K4me3, and H3K27ac tracks are almost completely barren. The chromatin is physically compacted and stripped of active modifications.
3. **Gene Silencing:** The RNA-seq track is distinctly blue (negative log2FC), proving that this heavy repressive chromatin structure effectively shuts down transcription.

10.6.3 Validating “Lost Enhancers”

Enhancers have a completely different epigenetic signature than promoters. If we look at the light blue **Lost Enhancers** track, we can clearly see the dynamic process of an enhancer being decommissioned over developmental time:

1. **The Enhancer Hallmark (H3K4me1):** Unlike promoters, these regions are strongly enriched for H3K4me1 while being completely devoid of H3K4me3. This exact signature (High K4me1 + Low K4me3) is the classic mathematical rule used to separate enhancers from promoters!
2. **Dynamic Loss of Activity:** At E11.5, these regions have strong ATAC accessibility and H3K27ac (active acetylation). However, by E15.5, both the ATAC signal and the H3K27ac signal fade away as the enhancer is decommissioned.
3. **Target Gene Repression:** The RNA-seq track is blue, suggesting that as these specific enhancers lose their active state over time, the downstream genes they regulate are simultaneously downregulated.

10.6.4 Validating the “Silenced” state (Hypermethylation)

Looking at the dark grey **Silenced** track near the bottom, we notice it is completely blank for ATAC, H3K4me3, and H3K27ac, and the gene expression is completely dead. But interestingly, it is *also* missing the Polycomb H3K27me3 repressive mark! So how are these regions being kept shut?

If we look at the far right, the **WGBS tracks are solid Red (hypermethylated)** across the entire 2kb window. This perfectly demonstrates that permanent silencing (heterochromatin) in the genome is often driven by massive DNA methylation rather than histone modifications.

10.6.5 The Mystery of “Incongruent” elements

The purple **Incongruent** row at the very bottom represents regions where chromatin accessibility and gene expression move in *opposite* directions (e.g., the chromatin physically opens up, but the gene suddenly turns off).

The most important takeaway here is how incredibly **thin** this row is compared to the main trajectories. This visually proves us that in biology, chromatin state and transcription are overwhelmingly tightly coupled. When they *do* decouple, it usually indicates complex secondary biology, such as a transcriptional repressor protein physically binding to that newly opened ATAC region, or the mRNA being degraded by microRNAs.

11 Functional enrichment of genomic regions

In the previous notebooks, we successfully integrated ATAC-seq, ChIP-seq, RNA-seq, and WGBS data to classify thousands of genomic regions into distinct biological states (e.g., *Active Promoters*, *Lost Enhancers*, *Silenced*). However, assigning a biological label to a genomic coordinate is only half the battle. To truly understand the developmental biology of our system (E11.5 to E15.5), we need to ask: **What biological pathways do these specific regulatory regions actually control?**

In this notebook, we will perform **Functional Enrichment Analysis**. Unlike standard RNA-seq analysis (which relies on isolated lists of genes), we will use **GREAT** (Genomic Regions Enrichment of Annotations Tool) to mathematically map our physical, non-coding regulatory coordinates directly to biological functions.

11.1 Learning objectives

- Use the **rGREAT** package to map cis-regulatory elements directly to biological pathways.
- Generate Volcano plots and region-gene association graphs to diagnose enrichment significance.
- Use the **simplifyEnrichment** package to collapse hundreds of highly redundant Gene Ontology (GO) terms into clustered functional groups.
- Interpret the actual biological meaning behind the pathways enriched in our *Active Promoters*.

11.2 Environment setup & data loading

Because we are mathematically mapping raw genomic coordinates to biological meaning, we need to load massive **Annotation Databases** that contain the physical coordinates and pathway annotations for our specific organism (Mouse).

11.2.1 Loading the required libraries

```

suppressPackageStartupMessages(suppressWarnings({
  library(dplyr)

  # Core analytical packages for region enrichment and clustering
  library(magick)
  library(rGREAT)
  library(simplifyEnrichment)
  library(flexclust)

  # Transcript Database containing the exact physical coordinates
  # for every known gene in the Mouse `mm10` genome
  library(TxDb.Mmusculus.UCSC.mm10.knownGene)

  # Organism-level database that links mouse gene IDs to human-readable GO terms
  library(org.Mm.eg.db)
}))

```

11.2.2 Importing the multi-omics overlap data

We will start by loading the final, fully-annotated `GRanges` object that we generated in the previous notebooks. This object contains all of our ATAC-seq peaks, cross-referenced against RNA-seq, and tagged with their final biological regulatory category.

```

# Load the annotated overlap object
overlap <- readRDS("output/overlap_multiomics_state.rds")

# Contingency table to view the categories we can test
table(overlap$Multiomics_State)

```

Active	Active Promoters	Dec. Acc.	Gained Enhancers
248	99	580	18
Inc. Acc.	Incongruent	Lost Enhancers	Repressed
516	166	9	126
Repressed Promoters	Silenced		
5	166		

11.2.3 Exploring the “Silenced” Regions

We just saw that our **Active Promoters** are strongly enriched for synapse formation and neurodevelopment. However, as cells differentiate, they must also permanently repress genes from earlier progenitor stages or alternative cell lineages.

To investigate this repression, we will analyze the **Silenced** category. As we saw in our contingency table, this cluster contains a robust **166 regions**, providing excellent statistical power for the rGREAT algorithm. More importantly, these regions share a highly specific multi-omics signature: a severe decrease in chromatin accessibility (ATAC-seq) and active acetylation (H3K27ac), a massive gain in Polycomb-mediated repression (H3K27me3), and a clear downregulation in target gene expression (RNA-seq).

Let’s quickly re-run our enrichment pipeline on these 166 regions to see exactly what biological pathways the developing brain is actively shutting down!

11.3 Genomic region enrichment via rGREAT

Now that we have selected our target category, we need to extract those specific physical coordinates from our massive overlapping dataset and feed them into the **GREAT** algorithm.

11.3.1 Subsetting the Genomic Ranges

First, we use standard R indexing to extract only the 99 peaks classified as **Active Promoters**.

```
# Extract only the Active Promoters using the cleaner subset() method
gr <- subset(overlap, Multiomics_State == "Active Promoters")

# Drop empty chromosome levels using the official Bioconductor method
seqlevels(gr) <- seqlevelsInUse(gr)

# Print the isolated object to verify we have exactly 99 ranges
gr
```

GRanges object with 99 ranges and 111 metadata columns:

	seqnames	ranges	strand	ATAC_annotation
	<Rle>	<IRanges>	<Rle>	<character>
peak_1	chr1	3670547-3672665	*	Promoter (<=1kb)
peak_14	chr1	9298449-9300284	*	Promoter (<=1kb)
peak_69	chr1	25228208-25229714	*	Promoter (<=1kb)

peak_70	chr1	25504015-25505076	*	Exon (ENSMUST0000014..
peak_83	chr1	34578812-34580363	*	Promoter (<=1kb)
...	...	...	...	...
peak_1902	chr2	178175169-178175887	*	Promoter (<=1kb)
peak_1922	chr2	181134082-181135787	*	Promoter (<=1kb)
peak_1923	chr2	181156149-181157204	*	Promoter (<=1kb)
peak_1925	chr2	181313272-181314836	*	Promoter (<=1kb)
peak_1930	chr2	181714476-181715931	*	Promoter (<=1kb)

	ATAC_geneChr	ATAC_geneStart	ATAC_geneEnd	ATAC_geneLength
	<integer>	<integer>	<integer>	<integer>
peak_1	1	3214482	3671498	457017
peak_14	1	8361475	9298573	937099
peak_69	1	25067476	25228826	161351
peak_70	1	25276404	25277954	1551
peak_83	1	34579657	34590944	11288
...	...	...	...	...
peak_1902	2	178175349	178336491	161143
peak_1922	2	181075579	181135214	59636
peak_1923	2	181147653	181157014	9362
peak_1925	2	181306459	181314500	8042
peak_1930	2	181715016	181720985	5970

	ATAC_geneStrand	ATAC_geneId	ATAC_transcriptId
	<integer>	<character>	<character>
peak_1	2	497097	ENSMUST00000070533.4
peak_14	2	71096	ENSMUST00000132064.7
peak_69	2	210933	ENSMUST00000126626.7
peak_70	2	100504323	ENSMUST00000193138.1
peak_83	1	211383	ENSMUST00000052670.10
...	...	...	...
peak_1902	1	74189	ENSMUST00000108915.7
peak_1922	2	16536	ENSMUST00000149964.8
peak_1923	2	13628	ENSMUST00000055990.7
peak_1925	2	20262	ENSMUST00000103045.3
peak_1930	1	18389	ENSMUST00000071585.9

	ATAC_distanceToTSS	ATAC_ENSEMBL	ATAC_SYMBOL
	<numeric>	<character>	<character>
peak_1	0	ENSMUSG00000051951	Xkr4
peak_14	0	ENSMUSG00000025909	Sntg1
peak_69	0	ENSMUSG00000033569	Adgrb3
peak_70	-226061	ENSMUSG00000104257	Gm20172
peak_83	0	ENSMUSG00000045174	Amer3
...	...	...	...
peak_1902	0	ENSMUSG00000027525	Phactr3

peak_1922	0	ENSMUSG00000016346	Kcnq2		
peak_1923	0	ENSMUSG00000016349	Eef1a2		
peak_1925	0	ENSMUSG00000027581	Stmn3		
peak_1930	0	ENSMUSG00000027584	Oprl1		
	ATAC_GENENAME	ATAC_anno	ATAC_logFC	ATAC_logCPM	ATAC_F
	<character>	<character>	<numeric>	<numeric>	<numeric>
peak_1	X-linked Kx blood gr..	Promoter	0.602984	10.45378	41.78446
peak_14	syntrophin, gamma 1	Promoter	0.960798	8.76438	46.46418
peak_69	adhesion G protein-c..	Promoter	0.986539	8.25070	40.79772
peak_70	predicted gene, 20172	Exon	0.802232	6.45404	8.85783
peak_83	APC membrane recruit..	Promoter	1.376255	8.69661	86.70410
...	...	...	...	...	...
peak_1902	phosphatase and acti..	Promoter	0.714177	6.72118	8.78683
peak_1922	potassium voltage-ga..	Promoter	0.912713	8.10930	32.69152
peak_1923	eukaryotic translati..	Promoter	1.338590	7.82461	54.99971
peak_1925	stathmin-like 3	Promoter	0.892601	8.12151	30.64904
peak_1930	opioid receptor-like 1	Promoter	1.213349	7.47748	37.29439
	ATAC_PValue	ATAC_qvalue	RNA_Row.names	RNA_gene_id	RNA_SYMBOL
	<numeric>	<numeric>	<character>	<character>	<character>
peak_1	6.58453e-08	4.81983e-07	ENSMUSG000000051951	497097	Xkr4
peak_14	1.97965e-08	1.70296e-07	ENSMUSG000000025909	71096	Sntg1
peak_69	8.55724e-08	6.07481e-07	ENSMUSG000000033569	210933	Adgrb3
peak_70	4.69864e-03	9.38854e-03	ENSMUSG000000033569	210933	Adgrb3
peak_83	4.98564e-12	1.57133e-10	ENSMUSG000000045174	211383	Amer3
...	...	...	...	...	...
peak_1902	4.85335e-03	9.67494e-03	ENSMUSG000000027525	74189	Phactr3
peak_1922	8.37488e-07	4.46335e-06	ENSMUSG000000016346	16536	Kcnq2
peak_1923	2.58482e-09	2.97631e-08	ENSMUSG000000016349	13628	Eef1a2
peak_1925	1.54857e-06	7.67150e-06	ENSMUSG000000027581	20262	Stmn3
peak_1930	2.22741e-07	1.40677e-06	ENSMUSG000000027584	18389	Oprl1
	RNA_GENENAME	RNA_ENSEMBL	RNA_geneLength		
	<character>	<character>	<integer>		
peak_1	X-linked Kx blood gr..	ENSMUSG000000051951	457017		
peak_14	syntrophin, gamma 1	ENSMUSG000000025909	938404		
peak_69	adhesion G protein-c..	ENSMUSG000000033569	762232		
peak_70	adhesion G protein-c..	ENSMUSG000000033569	762232		
peak_83	APC membrane recruit..	ENSMUSG000000045174	11288		
...	...	...	...		
peak_1902	phosphatase and acti..	ENSMUSG000000027525	219518		
peak_1922	potassium voltage-ga..	ENSMUSG000000016346	59722		
peak_1923	eukaryotic translati..	ENSMUSG000000016349	9362		
peak_1925	stathmin-like 3	ENSMUSG000000027581	8042		
peak_1930	opioid receptor-like 1	ENSMUSG000000027584	5970		

	RNA_geneChr	RNA_geneStart	RNA_geneEnd	RNA_geneStrand	RNA_geneId
	<character>	<integer>	<integer>	<character>	<character>
peak_1	chr1	3214482	3671498	-	497097
peak_14	chr1	8361475	9299878	-	71096
peak_69	chr1	25067476	25829707	-	210933
peak_70	chr1	25067476	25829707	-	210933
peak_83	chr1	34579657	34590944	+	211383
...	...	...	...	...	...
peak_1902	chr2	178118975	178338492	+	74189
peak_1922	chr2	181075579	181135300	-	16536
peak_1923	chr2	181147653	181157014	-	13628
peak_1925	chr2	181306459	181314500	-	20262
peak_1930	chr2	181715016	181720985	+	18389
	RNA_transcriptId	RNA_anno	RNA_logFC	RNA_logCPM	RNA_F
	<character>	<character>	<numeric>	<numeric>	<numeric>
peak_1	ENSMUST00000070533.4	Gene	1.37498	5.62452	33.7077
peak_14	ENSMUST000000132064.7	Gene	4.03266	5.98830	192.7039
peak_69	ENSMUST000000126626.7	Gene	1.96492	8.73183	107.5339
peak_70	ENSMUST000000126626.7	Gene	1.96492	8.73183	107.5339
peak_83	ENSMUST000000052670.10	Gene	2.29007	8.17011	191.9172
...	...	...	...	...	...
peak_1902	ENSMUST000000108917.7	Gene	1.75721	7.97049	103.8623
peak_1922	ENSMUST000000149964.8	Gene	1.08069	9.08113	43.8401
peak_1923	ENSMUST000000055990.7	Gene	4.06536	6.16441	278.0522
peak_1925	ENSMUST000000103045.3	Gene	1.87268	12.05944	135.2780
peak_1930	ENSMUST000000071585.9	Gene	1.87383	9.02372	145.0766
	RNA_PValue	RNA_qvalue	H3K4me3_annotation	H3K4me3_geneChr	
	<numeric>	<numeric>	<character>	<integer>	
peak_1	2.24273e-03	0.006288707	Promoter (<=1kb)	1	
peak_14	3.88128e-05	0.000618058	Promoter (<=1kb)	1	
peak_69	1.56060e-04	0.001091928	Promoter (<=1kb)	1	
peak_70	1.56060e-04	0.001091928	Exon (ENSMUST00000014..	1	
peak_83	3.90118e-05	0.000618058	Promoter (<=1kb)	1	
...	...	...	...	...	...
peak_1902	1.69461e-04	0.001152960	Promoter (<=1kb)	2	
peak_1922	1.25004e-03	0.004047574	Promoter (<=1kb)	2	
peak_1923	1.59710e-05	0.000490337	Promoter (<=1kb)	2	
peak_1925	9.03474e-05	0.000834961	Promoter (<=1kb)	2	
peak_1930	7.64282e-05	0.000816625	Promoter (<=1kb)	2	
	H3K4me3_geneStart	H3K4me3_geneEnd	H3K4me3_geneLength		
	<integer>	<integer>	<integer>		
peak_1	3214482	3671498	457017		
peak_14	8361475	9298573	937099		

peak_69	25067476	25228826	161351
peak_70	25276404	25277954	1551
peak_83	34579657	34590944	11288
...	...	...	...
peak_1902	178175349	178336491	161143
peak_1922	181075579	181135214	59636
peak_1923	181147653	181157014	9362
peak_1925	181306459	181314500	8042
peak_1930	181715016	181720985	5970
H3K4me3_geneStrand H3K4me3_geneId H3K4me3_transcriptId			
	<integer>	<character>	<character>
peak_1	2	497097	ENSMUST00000070533.4
peak_14	2	71096	ENSMUST00000132064.7
peak_69	2	210933	ENSMUST00000126626.7
peak_70	2	100504323	ENSMUST00000193138.1
peak_83	1	211383	ENSMUST00000052670.10
...	...	...	...
peak_1902	1	74189	ENSMUST00000108915.7
peak_1922	2	16536	ENSMUST00000149964.8
peak_1923	2	13628	ENSMUST00000055990.7
peak_1925	2	20262	ENSMUST00000103045.3
peak_1930	1	18389	ENSMUST00000071585.9
H3K4me3_distanceToTSS H3K4me3_ENSEMBL H3K4me3_SYMBOL			
	<numeric>	<character>	<character>
peak_1	0	ENSMUSG00000051951	Xkr4
peak_14	0	ENSMUSG00000025909	Sntg1
peak_69	0	ENSMUSG00000033569	Adgrb3
peak_70	-226252	ENSMUSG00000104257	Gm20172
peak_83	0	ENSMUSG00000045174	Amer3
...	...	...	...
peak_1902	56	ENSMUSG00000027525	Phactr3
peak_1922	0	ENSMUSG00000016346	Kcnq2
peak_1923	0	ENSMUSG00000016349	Eef1a2
peak_1925	0	ENSMUSG00000027581	Stmn3
peak_1930	0	ENSMUSG00000027584	Oprl1
H3K4me3_GENENAME H3K4me3_anno H3K4me3_logFC H3K4me3_logCPM			
	<character>	<character>	<numeric>
peak_1	X-linked Kx blood gr..	Promoter	0.607114
peak_14	syntrophin, gamma 1	Promoter	0.787977
peak_69	adhesion G protein-c..	Promoter	1.205708
peak_70	predicted gene, 20172	Exon	2.657731
peak_83	APC membrane recruit..	Promoter	1.158232
...	...	...	...

peak_1902	phosphatase and acti..	Promoter	1.655324	3.89074
peak_1922	potassium voltage-ga..	Promoter	0.750262	8.80515
peak_1923	eukaryotic translati..	Promoter	1.819209	7.73711
peak_1925	stathmin-like 3	Promoter	0.615019	8.78460
peak_1930	opioid receptor-like 1	Promoter	1.978607	7.54088
	H3K4me3_F	H3K4me3_PValue	H3K4me3_qvalue	H3K4me1_annotation
	<numeric>	<numeric>	<numeric>	<character>
peak_1	135.44293	4.65079e-31	1.28968e-29	<NA>
peak_14	149.35508	4.76345e-34	1.46952e-32	<NA>
peak_69	146.70663	1.76407e-33	5.27723e-32	<NA>
peak_70	6.77423	9.26510e-03	2.58814e-02	<NA>
peak_83	203.92865	1.03211e-45	4.76122e-44	<NA>
...	...	...	...	...
peak_1902	14.963	1.10494e-04	4.90024e-04	<NA>
peak_1922	151.440	1.69973e-34	5.34388e-33	<NA>
peak_1923	332.565	7.40439e-73	6.76816e-71	Promoter (<=1kb)
peak_1925	103.048	4.54823e-24	9.80353e-23	Promoter (<=1kb)
peak_1930	325.324	2.42880e-71	2.10325e-69	Promoter (<=1kb)
	H3K4me1_geneChr	H3K4me1_geneStart	H3K4me1_geneEnd	
	<integer>	<integer>	<integer>	
peak_1	<NA>	<NA>	<NA>	
peak_14	<NA>	<NA>	<NA>	
peak_69	<NA>	<NA>	<NA>	
peak_70	<NA>	<NA>	<NA>	
peak_83	<NA>	<NA>	<NA>	
...	...	...	...	
peak_1902	<NA>	<NA>	<NA>	
peak_1922	<NA>	<NA>	<NA>	
peak_1923	2	181147653	181157014	
peak_1925	2	181306459	181314500	
peak_1930	2	181715703	181719372	
	H3K4me1_geneLength	H3K4me1_geneStrand	H3K4me1_geneId	
	<integer>	<integer>	<character>	
peak_1	<NA>	<NA>	<NA>	
peak_14	<NA>	<NA>	<NA>	
peak_69	<NA>	<NA>	<NA>	
peak_70	<NA>	<NA>	<NA>	
peak_83	<NA>	<NA>	<NA>	
...	...	...	...	
peak_1902	<NA>	<NA>	<NA>	
peak_1922	<NA>	<NA>	<NA>	
peak_1923	9362	2	13628	
peak_1925	8042	2	20262	

peak_1930	3670	1	18389	
	H3K4me1_transcriptId	H3K4me1_distanceToTSS	H3K4me1_ENSEMBL	
	<character>	<numeric>	<character>	
peak_1	<NA>	NA	<NA>	
peak_14	<NA>	NA	<NA>	
peak_69	<NA>	NA	<NA>	
peak_70	<NA>	NA	<NA>	
peak_83	<NA>	NA	<NA>	
...	...	...	...	
peak_1902	<NA>	NA	<NA>	
peak_1922	<NA>	NA	<NA>	
peak_1923	ENSMUST00000055990.7	-64	ENSMUSG00000016349	
peak_1925	ENSMUST00000103045.3	911	ENSMUSG00000027581	
peak_1930	ENSMUST00000184127.1	38	ENSMUSG00000027584	
	H3K4me1_SYMBOL	H3K4me1_GENENAME	H3K4me1_anno	H3K4me1_logFC
	<character>	<character>	<character>	<numeric>
peak_1	<NA>	<NA>	<NA>	0
peak_14	<NA>	<NA>	<NA>	0
peak_69	<NA>	<NA>	<NA>	0
peak_70	<NA>	<NA>	<NA>	0
peak_83	<NA>	<NA>	<NA>	0
...	...	...	...	...
peak_1902	<NA>	<NA>	<NA>	0.000000
peak_1922	<NA>	<NA>	<NA>	0.000000
peak_1923	Eef1a2	eukaryotic translati..	Promoter	0.932761
peak_1925	Stmn3	stathmin-like 3	Promoter	0.612051
peak_1930	Oprl1	opioid receptor-like 1	Promoter	1.086359
	H3K4me1_logCPM	H3K4me1_F	H3K4me1_PValue	H3K4me1_qvalue
	<numeric>	<numeric>	<numeric>	<numeric>
peak_1	NA	NA	NA	NA
peak_14	NA	NA	NA	NA
peak_69	NA	NA	NA	NA
peak_70	NA	NA	NA	NA
peak_83	NA	NA	NA	NA
...	...	...	...	...
peak_1902	NA	NA	NA	NA
peak_1922	NA	NA	NA	NA
peak_1923	5.74097	20.7805	5.22563e-06	2.25916e-04
peak_1925	6.38866	15.0376	1.06225e-04	2.45541e-03
peak_1930	6.13490	36.7670	1.39101e-09	1.90365e-07
	H3K27me3_annotation	H3K27me3_geneChr	H3K27me3_geneStart	
	<character>	<integer>	<integer>	
peak_1	<NA>	<NA>	<NA>	

peak_14	Promoter (<=1kb)	1	8361475
peak_69	<NA>	<NA>	<NA>
peak_70	<NA>	<NA>	<NA>
peak_83	Promoter (<=1kb)	1	34579657
...	...	...	...
peak_1902	<NA>	<NA>	<NA>
peak_1922	Promoter (<=1kb)	2	181080503
peak_1923	Promoter (<=1kb)	2	181147653
peak_1925	Promoter (<=1kb)	2	181306459
peak_1930	<NA>	<NA>	<NA>
H3K27me3_geneEnd H3K27me3_geneLength H3K27me3_geneStrand			
	<integer>	<integer>	<integer>
peak_1	<NA>	<NA>	<NA>
peak_14	9299238	937764	2
peak_69	<NA>	<NA>	<NA>
peak_70	<NA>	<NA>	<NA>
peak_83	34590944	11288	1
...	...	...	...
peak_1902	<NA>	<NA>	<NA>
peak_1922	181134922	54420	2
peak_1923	181157014	9362	2
peak_1925	181314500	8042	2
peak_1930	<NA>	<NA>	<NA>
H3K27me3_geneId H3K27me3_transcriptId H3K27me3_distanceToTSS			
	<character>	<character>	<numeric>
peak_1	<NA>	<NA>	NA
peak_14	71096	ENSMUST00000140295.7	0
peak_69	<NA>	<NA>	NA
peak_70	<NA>	<NA>	NA
peak_83	211383	ENSMUST00000052670.10	-733
...	...	...	...
peak_1902	<NA>	<NA>	NA
peak_1922	16536	ENSMUST00000103049.9	0
peak_1923	13628	ENSMUST00000055990.7	0
peak_1925	20262	ENSMUST00000103045.3	-125
peak_1930	<NA>	<NA>	NA
H3K27me3_ENSEMBL H3K27me3_SYMBOL H3K27me3_GENENAME			
	<character>	<character>	<character>
peak_1	<NA>	<NA>	<NA>
peak_14	ENSMUSG00000025909	Sntg1	syntrophin, gamma 1
peak_69	<NA>	<NA>	<NA>
peak_70	<NA>	<NA>	<NA>
peak_83	ENSMUSG00000045174	Amer3	APC membrane recruit..

...	...	...	...
peak_1902	<NA>	<NA>	<NA>
peak_1922	ENSMUSG000000016346	Kcnq2	potassium voltage-ga..
peak_1923	ENSMUSG000000016349	Eef1a2	eukaryotic translati..
peak_1925	ENSMUSG000000027581	Stmn3	stathmin-like 3
peak_1930	<NA>	<NA>	<NA>
	H3K27me3_anno	H3K27me3_logFC	H3K27me3_logCPM
	<character>	<numeric>	<numeric>
peak_1	<NA>	0.000000	NA
peak_14	Promoter	-0.578443	7.07069
peak_69	<NA>	0.000000	NA
peak_70	<NA>	0.000000	NA
peak_83	Promoter	-0.926854	5.73161
...	...	...	...
peak_1902	<NA>	0.000000	NA
peak_1922	Promoter	-1.01484	9.08503
peak_1923	Promoter	-1.18230	8.57170
peak_1925	Promoter	-1.14450	6.24934
peak_1930	<NA>	0.000000	NA
	H3K27me3_PValue	H3K27me3_qvalue	H3K27ac_annotation
	<numeric>	<numeric>	<character>
peak_1	NA	NA	Promoter (<=1kb)
peak_14	0.00426473	0.0616407	Promoter (<=1kb)
peak_69	NA	NA	Promoter (<=1kb)
peak_70	NA	NA	Exon (ENSMUST0000014..
peak_83	0.00762819	0.0899976	Promoter (<=1kb)
...	...	...	...
peak_1902	NA	NA	Promoter (<=1kb)
peak_1922	2.09500e-19	7.17349e-17	Promoter (<=1kb)
peak_1923	5.65224e-20	2.06724e-17	Promoter (<=1kb)
peak_1925	5.11572e-05	1.99691e-03	Promoter (<=1kb)
peak_1930	NA	NA	Promoter (<=1kb)
	H3K27ac_geneChr	H3K27ac_geneStart	H3K27ac_geneEnd
	<integer>	<integer>	<integer>
peak_1	1	3214482	3671498
peak_14	1	8361475	9298573
peak_69	1	25067476	25228826
peak_70	1	25276404	25277954
peak_83	1	34579657	34590944
...	...	...	...
peak_1902	2	178175349	178336491
peak_1922	2	181080503	181134922
peak_1923	2	181147653	181157014

peak_1925	2	181306459	181314500
peak_1930	2	181715016	181720985
	H3K27ac_geneLength	H3K27ac_geneStrand	H3K27ac_geneId
	<integer>	<integer>	<character>
peak_1	457017	2	497097
peak_14	937099	2	71096
peak_69	161351	2	210933
peak_70	1551	2	100504323
peak_83	11288	1	211383
...	...	...	...
peak_1902	161143	1	74189
peak_1922	54420	2	16536
peak_1923	9362	2	13628
peak_1925	8042	2	20262
peak_1930	5970	1	18389
	H3K27ac_transcriptId	H3K27ac_distanceToTSS	H3K27ac_ENSEMBL
	<character>	<numeric>	<character>
peak_1	ENSMUST00000070533.4	0	ENSMUSG000000051951
peak_14	ENSMUST000000132064.7	0	ENSMUSG000000025909
peak_69	ENSMUST000000126626.7	0	ENSMUSG000000033569
peak_70	ENSMUST000000193138.1	-226214	ENSMUSG000000104257
peak_83	ENSMUST000000052670.10	0	ENSMUSG000000045174
...	...	...	...
peak_1902	ENSMUST000000108915.7	0	ENSMUSG000000027525
peak_1922	ENSMUST000000103049.9	0	ENSMUSG000000016346
peak_1923	ENSMUST000000055990.7	0	ENSMUSG000000016349
peak_1925	ENSMUST000000103045.3	0	ENSMUSG000000027581
peak_1930	ENSMUST000000071585.9	0	ENSMUSG000000027584
	H3K27ac_SYMBOL	H3K27ac_GENENAME	H3K27ac_anno H3K27ac_logFC
	<character>	<character>	<character> <numeric>
peak_1	Xkr4 X-linked Kx blood gr..	Promoter	1.25269
peak_14	Sntg1 syntrophin, gamma 1	Promoter	2.25132
peak_69	Adgrb3 adhesion G protein-c..	Promoter	1.71288
peak_70	Gm20172 predicted gene, 20172	Exon	2.45135
peak_83	Amer3 APC membrane recruit..	Promoter	1.44331
...	...	...	...
peak_1902	Phactr3 phosphatase and acti..	Promoter	1.382198
peak_1922	Kcnq2 potassium voltage-ga..	Promoter	0.909119
peak_1923	Eef1a2 eukaryotic translati..	Promoter	1.426917
peak_1925	Stmn3 stathmin-like 3	Promoter	1.337649
peak_1930	Oprl1 opioid receptor-like 1	Promoter	1.363849
	H3K27ac_logCPM	H3K27ac_F	H3K27ac_PValue H3K27ac_qvalue name
	<numeric>	<numeric>	<numeric> <numeric> <character>

peak_1	8.81735	151.4959	1.67395e-34	4.08130e-32	peak_1
peak_14	7.44136	188.6314	1.91288e-42	6.62156e-40	peak_14
peak_69	7.75798	148.3754	7.82086e-34	1.80483e-31	peak_69
peak_70	5.04358	34.5321	4.36085e-09	1.34796e-07	peak_70
peak_83	7.35920	86.0818	2.18441e-20	2.14293e-18	peak_83
...	...	...	...	...	...
peak_1902	6.22622	35.9816	2.07855e-09	6.74235e-08	peak_1902
peak_1922	8.31643	64.4153	1.15104e-15	7.72601e-14	peak_1922
peak_1923	7.39512	86.2073	2.05151e-20	2.03730e-18	peak_1923
peak_1925	8.00741	111.4177	7.08864e-26	1.02853e-23	peak_1925
peak_1930	7.84616	104.9298	1.79151e-24	2.34334e-22	peak_1930
Regulatory_State Multiomics_State					
<character> <character>					
peak_1	Active	Active	Promoters		
peak_14	Active	Active	Promoters		
peak_69	Active	Active	Promoters		
peak_70	Active	Active	Promoters		
peak_83	Active	Active	Promoters		
...	...	...	...		
peak_1902	Active	Active	Promoters		
peak_1922	Active	Active	Promoters		
peak_1923	Active	Active	Promoters		
peak_1925	Active	Active	Promoters		
peak_1930	Active	Active	Promoters		

seqinfo: 2 sequences from an unspecified genome; no seqlengths

Notice that we re-assign the `seqlevels`. As discussed in [a previous notebook](#), when we subset a `GRanges` object, it remembers *all* the original chromosomes, even the ones that now have 0 peaks. Dropping these “empty” chromosomes is a critical sanitization step to prevent downstream algorithms from crashing or miscalculating genome background sizes.

11.3.2 Executing the GREAT algorithm

Now we pass our isolated coordinates directly to the `great()` function.

Unlike standard RNA-seq enrichment that relies on isolated gene names, GREAT maps **physical genome space**. Because our ATAC-seq peaks are mostly non-coding regulatory elements (like distal enhancers), they rarely sit directly *inside* the genes they regulate.

To solve this, GREAT calculates a “**regulatory domain**” for every known gene in the mouse genome by starting at the Transcription Start Site (TSS): 1. **Basal Domain**: It first assigns a core regulatory region immediately surrounding the TSS (typically 5kb upstream and 1kb

downstream). 2. **Distal Extension:** It then extends this domain outward in both directions along the chromosome until it hits the regulatory domain of the nearest neighboring gene (up to a massive maximum of 1,000 kb!).

GREAT then maps our 99 ATAC-seq peaks to see which of these extended regulatory domains they physically fall into. This allows us to accurately link distal regulatory DNA to the specific biological pathways they control.

```
# Run the Genomic Regions Enrichment of Annotations Tool
```

```
res <- great(  
  gr = gr,  
  gene_sets = "BP", # testing against the **Biological Process** GO  
  tss_source = "mm10" # use the Mouse mm10 database to locate the TSS  
)
```

```
* TSS source: TxDb.
```

```
* check whether TxDb package 'TxDb.Mmusculus.UCSC.mm10.knownGene' is installed.
```

```
* gene ID type in the extended TSS is 'Entrez Gene ID'.
```

```
* restrict chromosomes to 'chr1, chr2, chr3, chr4, chr5, chr6, chr7, chr8, chr9, chr10,  
  chr11, chr12, chr13, chr14, chr15, chr16, chr17, chr18, chr19, chrX, chrY, chrM'.
```

```
* 20598/24515 protein-coding genes left.
```

```
* update seqinfo to the selected chromosomes.
```

```
* TSS extension mode is 'basalPlusExt'.
```

```
* construct the basal domains by extending 5000bp to upstream and 1000bp to downstream of TSS.
```

```
* calculate distances to neighbour regions.
```

```
* extend to both sides until reaching the neighbour genes or to the maximal extension.
```

```
* use GO:BP ontology with 14957 gene sets (source: org.Mm.eg.db).
```

```
* check gene ID type in `gene_sets` and in `extended_tss`.
```

```
* use whole genome as background.
```

```
* remove excluded regions from background.
```

```

* overlap `gr` to background regions (based on midpoint).

* in total 99 `gr`.

* overlap extended TSS to background regions.

* check which genes are in the gene sets.

* only take gene sets with size >= 5.

* in total 9288 gene sets.

* overlap `gr` to every extended TSS.

* perform binomial test for each biological term.

```

Notice that it says `* restrict chromosomes to chr1, chr2....` `rGREAT` is actually smart enough to automatically filter out weird unplaced contigs based on its internal TSS database. However, applying `seqlevelsInUse()` immediately after subsetting is still a crucial Bioconductor habit to learn, as many other downstream algorithms will crash or miscalculate background statistics if they detect empty chromosomes sitting in our object's metadata.

11.3.3 QA/QC: verifying the GREAT mapping

Before we jump into the visualizations, simply printing the `res` object provides a critical Quality Control check.

It tells us exactly how many of our 99 physical ATAC-seq peaks successfully intersected with the mathematical regulatory domain of a known gene. If the vast majority of our peaks mapped successfully, we have the green light to proceed to pathway visualization.

```

# Print a summary of the GREAT object
print(res)

```

```

99 regions are associated to 113 genes' extended TSSs.
  TSS source: TxDb.Mmusculus.UCSC.mm10.knownGene
  Genome: mm10
  OrgDb: org.Mm.eg.db
  Gene sets: BP
  Background: whole genome excluding gaps
  Mode: Basal plus extension

```

Proximal: 5000 bp upstream, 1000 bp downstream,
plus Distal: up to 1000000 bp

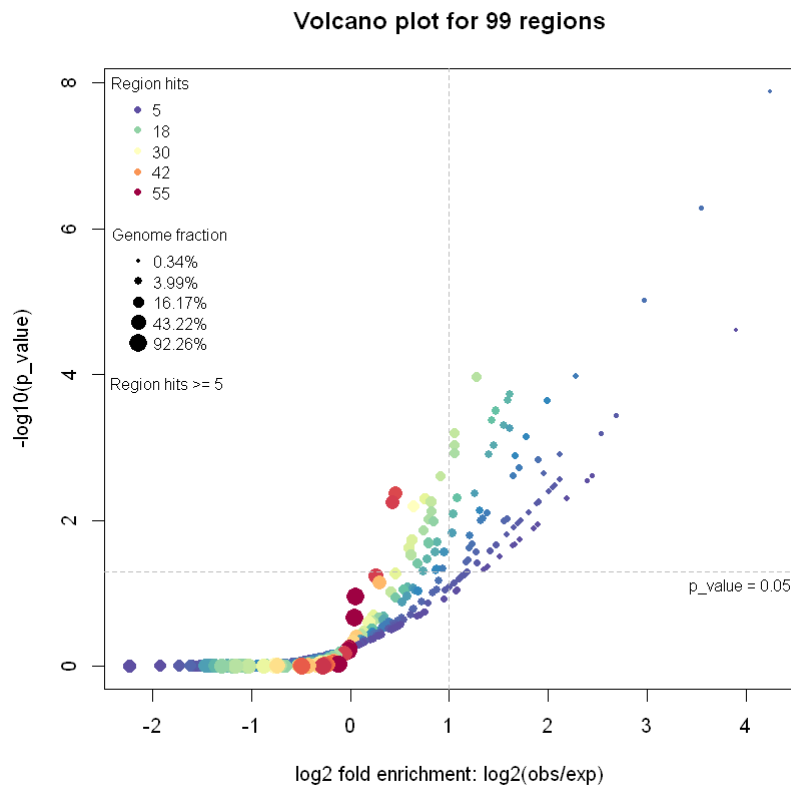
11.4 Visualizing and interpreting enrichment results

Now that the GREAT algorithm has successfully mapped our ATAC-seq peaks to biological pathways, we need to extract biological meaning from the statistics. We will explore and refine our results using four distinct approaches.

11.4.1 Statistical significance: the Volcano plot

Because `rGREAT` is built specifically for genomic regions, its native `plotVolcano()` function operates a little differently than a standard RNA-seq volcano plot. Instead of plotting individual genes, each dot on this plot represents an entire **Biological Pathway**.

```
# Generate a Volcano plot directly from the GREAT results object.  
# GREAT actually runs two mathematical tests simultaneously, so this will  
# plot the Fold Enrichment vs P-value for BOTH the Binomial test (genomic space)  
# and the Hypergeometric test (gene count).  
plotVolcano(res)
```



Each dot represents an entire **Biological Pathway**. To find the most meaningful networks, we must read the plot across three dimensions:

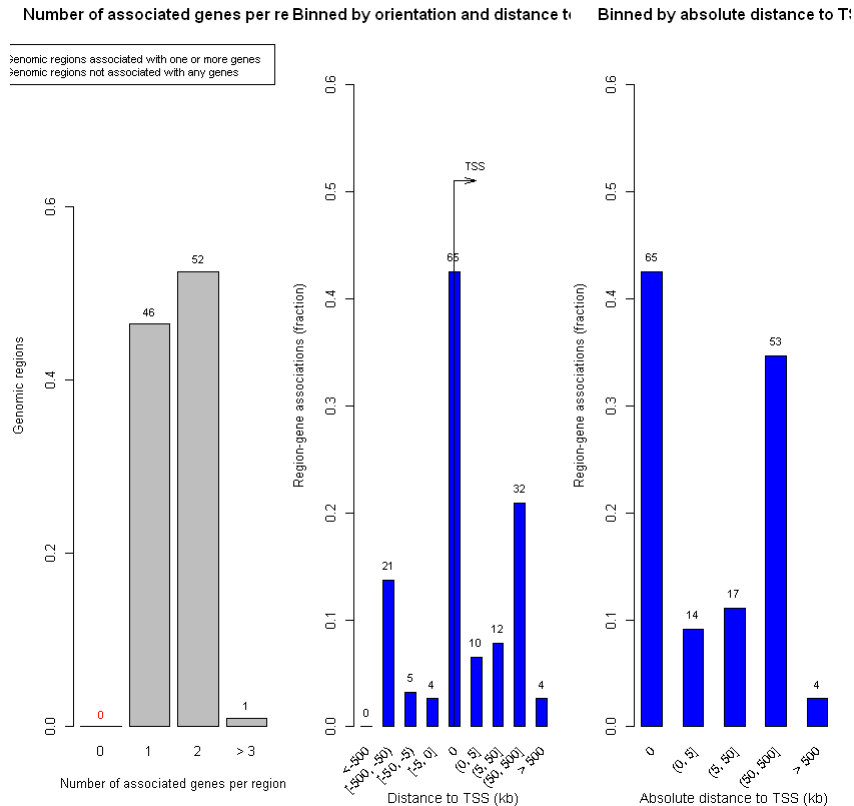
- **Location (Significance & Enrichment):** The true “winners” fly into the **top-right quadrant** (above the $p = 0.05$ significance threshold and $> 2\times$ fold enrichment).
- **Dot Size (Pathway Size):** Massive dark red dots near the bottom have many hits (~55 peaks), but because they are huge, generic pathways, hitting them is expected by pure random chance.
- **Color (Absolute Hits):** The small blue dots in the top-right only have ~5 peak hits. However, because those pathways are physically tiny (small dot size), getting even 5 hits is incredibly rare. These represent highly specific, tightly coordinated biological networks.

11.4.2 Physical mapping: region-gene associations

Now that we know which biological pathways are enriched, we need to inspect the physical mapping that generated those statistics. How exactly did GREAT assign our 99 ATAC-seq peaks to target genes? Did they map to local promoters, or long-range distal regions?

```
# options(repr.plot.width = 12, repr.plot.height = 6)

# Generate three diagnostic bar plots showing exactly how many genes
# each ATAC-seq peak was mapped to, and the physical distance to the TSS.
plotRegionGeneAssociations(res)
```



These three plots perfectly validate the multi-omics classification pipeline we built in our previous notebook:

- **Plot 1 (Left - Genes per Region):** We see 0 red hits, meaning 100% of our peaks successfully mapped to at least one gene! Most peaks map to 1 or 2 genes (because GREAT’s “extension” rule often hits immediate neighbors).
- **Plots 2 & 3 (Distance to TSS):** Because we specifically isolated the **Active Promoters** cluster for this analysis, we biologically expect these peaks to sit directly on top of genes. The massive spike exactly at 0 kb physically proves that our multi-omics logic successfully isolated true promoters!

(Note: The smaller bars further out at 50-500kb simply reflect that genes are packed tightly together, and one gene’s promoter might physically overlap with the extended regulatory domain

of a distant neighbor).

11.4.3 Exploring the raw data

While the Volcano plot gives us a great high-level view of our enriched pathways, we now need to dig deeper into the actual biology. To do this, we must extract the raw data so we can systematically sort the pathways by significance and identify the exact names of the target genes our ATAC-seq peaks hit.

*While **rGREAT** does come with a built-in interactive dashboard (`shinyReport(res)`), **Shiny applications do not play nicely with Jupyter Notebooks**. Launching a Shiny app will hijack the R kernel. In many Jupyter environments, attempting to stop the Shiny app will completely crash the R session, forcing to restart the kernel and run all previous cells from the beginning.*

```
# (Optional) Launch the interactive GREAT dashboard.
# Warning: This will freeze the Jupyter kernel until manually interrupted!

# shinyReport(res)
```

Instead of relying on a clunky dashboard, we can extract the raw mathematical results into a standard R dataframe.

```
# Extract the full enrichment table into a standard dataframe
tb <- getEnrichmentTable(res)

# View the top 6 most significantly enriched pathways
print(head(tb))
```

	id		description	
1	GO:0099509	regulation of presynaptic cytosolic calcium ion concentration		
3	GO:0070050		neuron cellular homeostasis	
4	GO:0051480	regulation of cytosolic calcium ion concentration		
5	GO:1901385	regulation of voltage-gated calcium channel activity		
10	GO:0019226		transmission of nerve impulse	
11	GO:0098655	monoatomic cation transmembrane transport		
	genome_fraction	observed_region_hits	fold_enrichment	p_value
1	0.004255723	8	18.988097	1.305671e-08
3	0.006914366	8	11.686983	5.113900e-07
4	0.010302148	8	7.843809	9.447899e-06
5	0.003398694	5	14.860134	2.486941e-05

10	0.018695488		9	4.862622	1.062248e-04
11	0.087208298		21	2.432351	1.102352e-04
	p_adjust	mean_tss_dist	observed_gene_hits	gene_set_size	
1	5.222684e-06	120058	5		29
3	1.022780e-04	120058	5		69
4	1.259720e-03	120058	5		82
5	2.486941e-03	128355	3		27
10	7.349016e-03	81545	7		119
11	7.349016e-03	93654	19		868
	fold_enrichment_hyper	p_value_hyper	p_adjust_hyper		
1	31.42814	4.859233e-07	4.859233e-05		
3	13.20893	3.862111e-05	5.327049e-04		
4	11.11483	8.860743e-05	1.042440e-03		
5	20.25369	4.271773e-04	3.797131e-03		
10	10.72254	4.188185e-06	1.047046e-04		
11	3.99007	2.437046e-07	3.249395e-05		

11.4.4 Downstream filtering of biological pathways

Because `getEnrichmentTable()` returns a standard R dataframe, we are no longer trapped inside a “black box” bioinformatics tool. We can use the exact same `dplyr` data wrangling skills we learned in previous notebooks to aggressively mine this table for biological meaning.

Here are three highly practical examples of how we can filter our pathway results both mathematically and biologically.

```
# 1. Filter Mathematically: Keep only highly significant pathways (FDR < 0.05)
sig_pathways <- tb %>% filter(p_adjust < 0.05)

# 2. Filter by Evidence: Keep pathways hit by at least 5 of our ATAC peaks
# (GREAT outputs specific columns tracking region hits)
robust_pathways <- sig_pathways %>% filter(observed_region_hits >= 5)

# 3. Filter Biologically: Search for specific pathway keywords (e.g., 'calcium' or 'synap')
calcium_pathways <- tb %>% filter(grepl("calcium|synap", description, ignore.case = TRUE))

# View our biologically targeted pathways
print(head(calcium_pathways))
```

	id	description
1	G0:0099509	regulation of presynaptic cytosolic calcium ion concentration
2	G0:0051480	regulation of cytosolic calcium ion concentration

3	GO:1901385	regulation of voltage-gated calcium channel activity
4	GO:0006874	intracellular calcium ion homeostasis
5	GO:2000300	regulation of synaptic vesicle exocytosis
6	GO:0070588	calcium ion transmembrane transport

	genome_fraction	observed_region_hits	fold_enrichment	p_value
1	0.004255723	8	18.988097	1.305671e-08
2	0.010302148	8	7.843809	9.447899e-06
3	0.003398694	5	14.860134	2.486941e-05
4	0.025446934	10	3.969441	2.254245e-04
5	0.009403625	6	6.444968	3.672851e-04
6	0.039552568	12	3.064583	5.528246e-04

	p_adjust	mean_tss_dist	observed_gene_hits	gene_set_size
1	5.222684e-06	120058	5	29
2	1.259720e-03	120058	5	82
3	2.486941e-03	128355	3	27
4	1.001887e-02	96075	7	229
5	1.224284e-02	92670	4	71
6	1.474199e-02	80239	9	350

	fold_enrichment_hyper	p_value_hyper	p_adjust_hyper
1	31.428135	4.859233e-07	4.859233e-05
2	11.114828	8.860743e-05	1.042440e-03
3	20.253687	4.271773e-04	3.797131e-03
4	5.571975	2.725825e-04	2.659341e-03
5	10.269475	6.287080e-04	4.699258e-03
6	4.687282	1.348578e-04	1.419555e-03

11.4.5 Clustering redundant pathways (semantic similarity)

As we just saw in our filtered data table above, the Gene Ontology (GO) database is notoriously redundant. Because our active promoters are enriched for calcium signaling, the database returned *regulation of presynaptic cytosolic calcium*, *intracellular calcium ion homeostasis*, and *calcium ion transmembrane transport* as completely separate pathways.

If we don't mathematically compress this data, we will be forced to manually read through hundreds of nearly identical terms. To make sense of this massive, repetitive list, we can use the `simplifyEnrichment` package. This algorithm calculates the **Semantic Similarity** between all of our significant GO terms (how closely related they are in the biological tree) and physically clusters them into clean, high-level functional blocks.

```
# Extract the exact GO IDs for our statistically significant pathways
sig_go_ids <- tb$id[tb$p_adjust < 0.05]
```

```
# Calculate the semantic similarity matrix and cluster the redundant terms
# (Note: This might take a minute as it mathematically compares hundreds of GO terms!)
cl <- simplifyGO(sig_go_ids)
```

You haven't provided value for `ont`, guess it as `BP`.

relations: is_a, part_of, regulates, negatively_regulates, positively_regulates

IC_method: IC_annotation

term_sim_method: Sim_XGraSM_2013

IC_method: IC_annotation

Cluster 41 terms by 'binary_cut'...

4 clusters, used 0.2807021 secs.

Perform keywords enrichment for 4 GO lists...

1 GO terms clustered by 'binary_cut'



This visualization condenses the repetitive results. The red diagonal matrix mathematically groups all the 41 GO terms into distinct clusters based on their similarity. Instead of forcing us to read dozens of nearly identical pathway names, the algorithm generates clean word clouds summarizing the overarching biological function of each block: calcium homeostasis, synaptic transport, membrane potential, and learning behavior.

```
# Generate a heatmap summarizing our high-level biological clusters
summarizeGO(
  go_id = sig_go_ids,
  -log10(tb$p_adjust[tb$p_adjust < 0.05]),
  axis_label = "average -log10(p.adjust)"
)
```

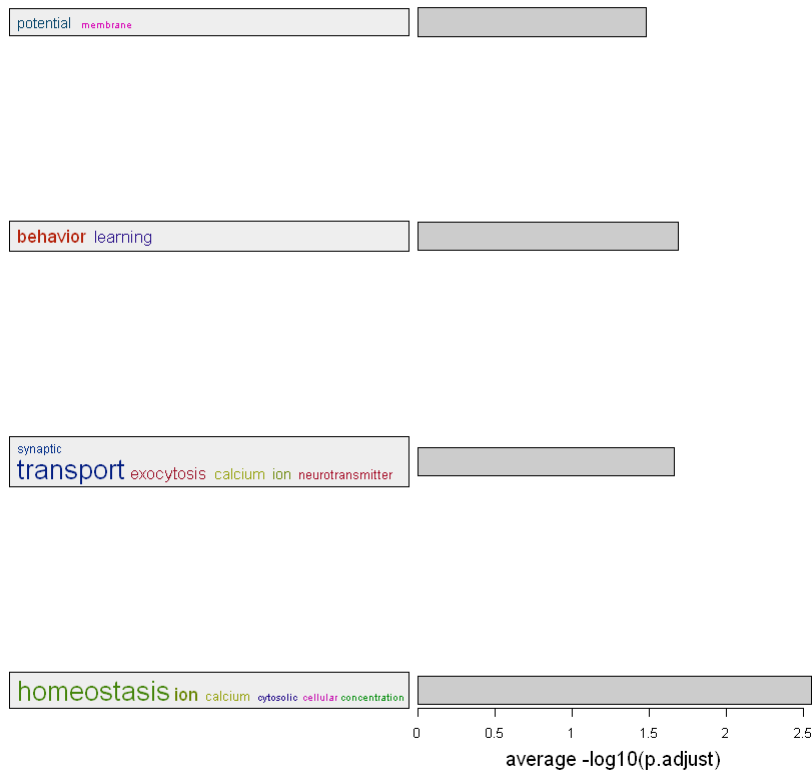
You haven't provided value for `ont`, guess it as `BP`.

term_sim_method: Sim_XGraSM_2013

IC_method: IC_annotation

Cluster 41 terms by 'binary_cut'...
4 clusters, used 0.2557878 secs.

Perform keywords enrichment for 4 GO lists...



This bar chart summarizes our data by showing the average statistical significance of each functional block. The “calcium homeostasis” cluster is by far the most statistically robust group (the longest bar), indicating that calcium signaling is a primary molecular driver in these cells.

Biologically, this makes perfect sense; E11.5 to E15.5 is a critical embryonic window for neural development, involving rapid neuronal differentiation and synapse formation. The fact that our mathematically isolated *Active Promoters* are overwhelmingly driving ion transport, synaptic transmission, and learning circuitry perfectly aligns with early neural maturation. Our multi-omics integration successfully captured the exact developmental window where the embryonic brain begins to wire itself.

11.4.6 Exploring the “Silenced” regions

We just saw that our **Active Promoters** are strongly enriched for synapse formation and neurodevelopment. However, as cells differentiate, they must also permanently repress genes from earlier progenitor stages or alternative cell lineages.

To investigate this repression, we will analyze the **Silenced** category. As we saw in our contingency table, this cluster contains a robust **166 regions**, providing excellent statistical power for the rGREAT algorithm. More importantly, these regions share a highly specific multi-omics signature: a severe decrease in chromatin accessibility (ATAC-seq) and active acetylation (H3K27ac), a massive gain in Polycomb-mediated repression (H3K27me3), and a clear downregulation in target gene expression (RNA-seq).

Let’s quickly re-run our enrichment pipeline on these 166 regions to see exactly what biological pathways the developing brain is actively shutting down.

```
gr_silenced <- subset(overlap, Multiomics_State == "Silenced")
seqlevels(gr_silenced) <- seqlevelsInUse(gr_silenced)

res_silenced <- great(
  gr = gr_silenced,
  gene_sets = "BP",
  tss_source = "mm10"
)
```

```
* TSS source: TxDb.

* extended_tss is already cached, directly use it.

* use GO:BP ontology with 14957 gene sets (source: org.Mm.eg.db).

* check gene ID type in `gene_sets` and in `extended_tss`.

* use whole genome as background.

* remove excluded regions from background.

* overlap `gr` to background regions (based on midpoint).

* in total 166 `gr`.

* overlap extended TSS to background regions.
```

```

* check which genes are in the gene sets.

* only take gene sets with size >= 5.

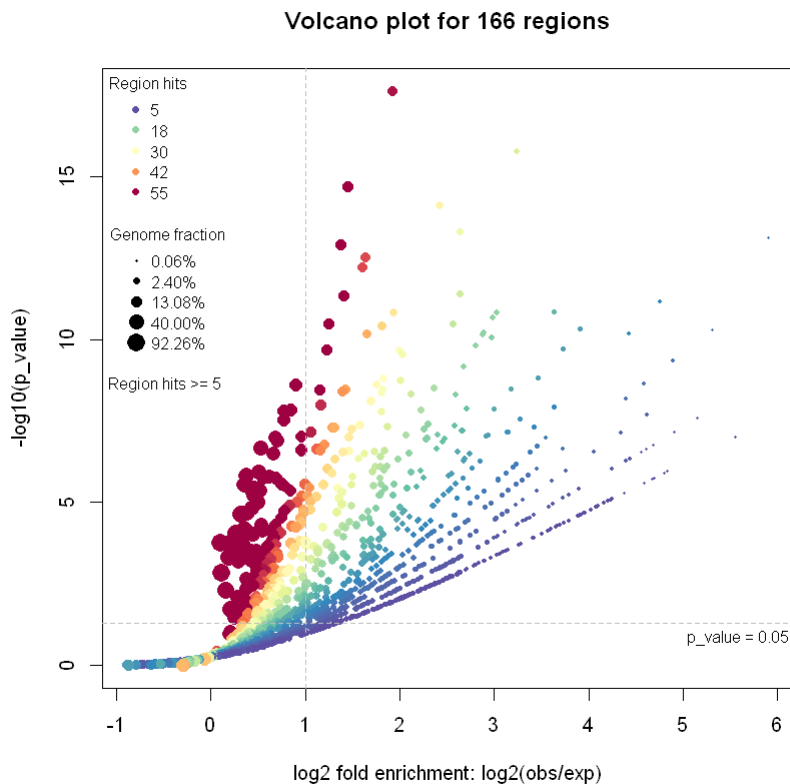
* in total 9288 gene sets.

* overlap `gr` to every extended TSS.

* perform binomial test for each biological term.

# Generate the Volcano plot for the silenced regions
plotVolcano(res_silenced)

```



Notice how different this Volcano plot is compared to our **Active Promoters**. The significance scores reach massively higher ($p < 10^{-17}$), and there is an absolute swarm of pathways flying into the top-right quadrant.

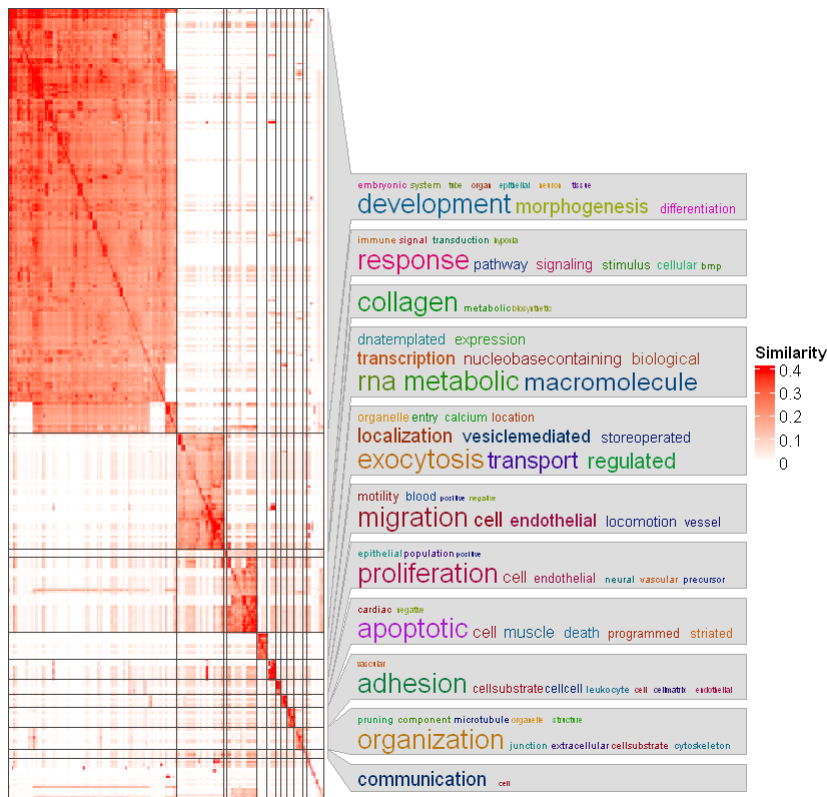
While a differentiating cell only needs to activate a small, highly targeted set of pathways to become a neuron, it must simultaneously repress *massive, heavily populated* gene networks

associated with every other possible cell fate. Because Polycomb (H3K27me3) coordinates the shutdown of these massive networks all at once, the statistical enrichment for repression is mathematically off the charts.

```
tb_silenced <- getEnrichmentTable(res_silenced)
sig_go_ids_silenced <- tb_silenced$id[tb_silenced$p_adjust < 0.05]

cl_silenced <- suppressMessages(
  simplifyGO(sig_go_ids_silenced))
```

19 GO terms clustered by 'binary_cut'



While our **Active Promoters** enriched for just 41 pathways, the **Silenced** regions enriched for **hundreds**.

The semantic similarity matrix above mathematically groups this massive list into distinct functional blocks. Looking closely at the word clouds on the right, we can see the algorithm successfully clustered terms related to **Collagen** (connective tissue), **Endothelial** (blood vessels), and **Striated Muscle**.

```
suppressMessages(  
  summarizeGO(  
    go_id = sig_go_ids_silenced,  
    -log10(tb_silenced$p_adjust[tb_silenced$p_adjust < 0.05]),  
    axis_label = "average -log10(p.adjust)")
```



This bar chart highlights the average statistical significance of our repressed functional blocks. Notice that the significance scale is much higher than what we saw in the Active Promoters. The “collagen” (connective tissue) cluster is by far the most statistically *repressed* group, closely followed by general embryonic morphogenesis, endothelial cell migration, and cellular proliferation.

Biologically, this validates the E11.5 to E15.5 developmental timeline. As the embryonic brain rapidly develops and neurons terminally differentiate, the cells must make permanent fate decisions. While we saw our **Active Promoters** turning *on* synapses and calcium signaling, this plot reveals the exact opposite side of the development process. The developing neural tissue is actively and permanently crushing alternative mesoderm lineages—like collagen, endothelial cells, and striated muscle, using Polycomb-mediated silencing.

Furthermore, the strong repression of the “proliferation” cluster perfectly aligns with the biology of early neurogenesis, as mature neurons must become post-mitotic and stop dividing. By mathematically integrating ATAC, ChIP, and RNA-seq, our multi-omics pipeline has successfully decoded the entire developmental decision-making process of the cell!